

NtPlugin

Generated by Doxygen 1.14.0

1 NtPlugin	1
1.1 Target audience	1
1.2 Features	1
1.2.1 The UI	1
1.3 Getting started	2
1.3.1 Install needed software	2
1.3.1.1 Linux dependencies	2
1.3.2 Try it out	2
1.4 Installation from source	2
1.5 Usage	3
1.5.1 Caveats	3
1.5.2 Making it work with VsCode	4
1.6 NtPlugin CLI	5
1.7 Collaborations	5
1.8 Mono plugins	5
1.9 Testing your plugin	5
1.9.1 Setting it up	5
1.9.2 Add your own test	6
1.9.3 Test with Pluginval	6
1.10 The NTfx library	7
1.10.1 signal_t	7
1.10.2 The Audio class	7
1.10.3 The Component class	7
1.10.4 The Plugin class	7
1.10.4.1 UI Specification	8
1.10.4.2 Parameters	8
1.10.4.3 Side chain input	9
1.10.4.4 Tempo	9
1.10.5 The Glider class	9
1.10.6 The FirstOrder class	9
1.10.7 The Biquad class	9
1.10.8 The PeakSensor class	9
1.10.9 The RmsSensor class	9
1.10.10 The SampleRateConverter class	10
1.10.11 SideChain	10
1.10.12 SoftClip	10
1.10.13 Transformer	10
2 NtPlugin License	11
2.1 Using Wrappers	11
2.2 GNU GENERAL PUBLIC LICENSE	11
2.2.1 Preamble	12

2.2.2 TERMS AND CONDITIONS	12
2.2.2.1 0. Definitions.	12
2.2.2.2 1. Source Code.	13
2.2.2.3 2. Basic Permissions.	14
2.2.2.4 3. Protecting Users' Legal Rights From Anti-Circumvention Law.	14
2.2.2.5 4. Conveying Verbatim Copies.	14
2.2.2.6 5. Conveying Modified Source Versions.	14
2.2.2.7 6. Conveying Non-Source Forms.	15
2.2.2.8 7. Additional Terms.	16
2.2.2.9 8. Termination.	17
2.2.2.10 9. Acceptance Not Required for Having Copies.	17
2.2.2.11 10. Automatic Licensing of Downstream Recipients.	17
2.2.2.12 11. Patents.	18
2.2.2.13 12. No Surrender of Others' Freedom.	18
2.2.2.14 13. Use with the GNU Affero General Public License.	19
2.2.2.15 14. Revised Versions of this License.	19
2.2.2.16 15. Disclaimer of Warranty.	19
2.2.2.17 16. Limitation of Liability.	19
2.2.2.18 17. Interpretation of Sections 15 and 16.	19
2.2.3 How to Apply These Terms to Your New Programs	20
3 Namespace Index	21
3.1 Namespace List	21
4 Hierarchical Index	23
4.1 Class Hierarchy	23
5 Class Index	27
5.1 Class List	27
6 File Index	31
6.1 File List	31
7 Namespace Documentation	33
7.1 NtFx Namespace Reference	33
7.1.1 Detailed Description	35
7.1.2 Function Documentation	35
7.1.2.1 _calculateSoftClipCoeffs()	35
7.1.3 Variable Documentation	35
7.1.3.1 _coeffsFifth	35
7.1.3.2 _coeffsSeventh	35
7.1.3.3 _coeffsThird	36
7.1.3.4 nMetersMax	36
7.2 ntPlugin Namespace Reference	36

7.2.1 Detailed Description	37
7.2.2 Function Documentation	37
7.2.2.1 addNewPluginId()	37
7.2.2.2 build()	37
7.2.2.3 configure()	38
7.2.2.4 createParser()	38
7.2.2.5 main()	38
7.2.2.6 newPlugin()	38
7.2.2.7 newPluginTest()	39
7.2.2.8 process()	39
7.2.2.9 readPluginIds()	39
7.2.2.10 runAuVal()	39
7.2.2.11 runCTest()	39
7.2.2.12 storeArtifacts()	40
7.2.3 Variable Documentation	40
7.2.3.1 CATEGORY_MAP	40
7.3 package Namespace Reference	40
7.3.1 Detailed Description	41
7.3.2 Function Documentation	41
7.3.2.1 createParser()	41
7.3.2.2 main()	42
7.3.2.3 makeDistributionXml()	42
7.3.2.4 makeInstallerPkg()	42
7.3.2.5 makePluginPkg()	42
7.3.2.6 makePluginPkgs()	43
7.3.2.7 notarize()	43
7.3.2.8 notarizePlugins()	43
7.3.2.9 readPlugins()	44
7.3.2.10 sign()	44
7.3.2.11 staple()	44
7.3.2.12 staplePlugins()	44
7.3.2.13 validateInstaller()	45
7.3.2.14 zipPackage()	45
7.3.3 Variable Documentation	45
7.3.3.1 SECRETS	45
7.3.3.2 TARGET_EXT_MAP	45
7.4 test Namespace Reference	46
7.4.1 Detailed Description	47
7.4.2 Function Documentation	47
7.4.2.1 acceptLatestResult()	47
7.4.2.2 clean()	47
7.4.2.3 createParser()	47

7.4.2.4 generateBurstSine()	48
7.4.2.5 generateImpulse()	48
7.4.2.6 generateLinearSweep()	48
7.4.2.7 generateSyncSweep()	49
7.4.2.8 generateTestVectors()	49
7.4.2.9 main()	49
7.4.2.10 plotDynamic()	50
7.4.2.11 plotFrequencyDomain()	50
7.4.2.12 plotImpulse()	50
7.4.2.13 plotSpectrum()	51
7.4.2.14 plotSweeps()	51
7.4.2.15 plotTimeDomain()	51
7.4.2.16 run()	52
7.4.2.17 runTests()	52
7.4.3 Variable Documentation	52
7.4.3.1 STIMULI	52
8 Class Documentation	53
8.1 NtFx::Biquad::Biquad6 Struct Reference	53
8.2 NtFx::Src::Coeffs Struct Reference	53
8.2.1 Detailed Description	53
8.3 NtFx::Biquad::Coeffs5 Struct Reference	54
8.4 NtFx::Biquad::Coeffs6 Struct Reference	54
8.5 NtFx::ComponentBase< T > Struct Template Reference	54
8.5.1 Detailed Description	55
8.5.2 Member Function Documentation	55
8.5.2.1 process()	55
8.5.2.2 reset()	55
8.5.2.3 update()	56
8.6 NtFx::ComponentTest Struct Reference	56
8.6.1 Detailed Description	57
8.6.2 Constructor & Destructor Documentation	57
8.6.2.1 ComponentTest()	57
8.6.3 Member Function Documentation	57
8.6.3.1 run()	57
8.7 NtFx::ComponentTestSet Struct Reference	58
8.7.1 Detailed Description	58
8.7.2 Constructor & Destructor Documentation	59
8.7.2.1 ComponentTestSet()	59
8.7.3 Member Function Documentation	59
8.7.3.1 addTest()	59
8.7.3.2 getResults()	59

8.7.3.3 runAllTests()	60
8.8 NtFx::Biquad::EqBand6Mono Struct Reference	60
8.8.1 Member Function Documentation	60
8.8.1.1 process()	60
8.8.1.2 reset()	61
8.8.1.3 update()	61
8.9 NtFx::Biquad::EqBand6Stereo Struct Reference	61
8.9.1 Member Function Documentation	62
8.9.1.1 process()	62
8.9.1.2 reset()	62
8.9.1.3 update()	63
8.10 NtFx::Biquad::EqBandMono Struct Reference	63
8.10.1 Member Function Documentation	63
8.10.1.1 process()	63
8.10.1.2 reset()	64
8.10.1.3 update()	64
8.11 NtFx::Biquad::EqBandStereo Struct Reference	64
8.11.1 Member Function Documentation	65
8.11.1.1 process()	65
8.11.1.2 reset()	65
8.11.1.3 update()	66
8.12 NtFx::ExpGlider Struct Reference	66
8.12.1 Constructor & Destructor Documentation	66
8.12.1.1 ExpGlider()	66
8.12.2 Member Function Documentation	67
8.12.2.1 process()	67
8.12.2.2 update()	67
8.13 NtFx::FirstOrder::Filter< shape > Struct Template Reference	67
8.13.1 Detailed Description	68
8.13.2 Member Function Documentation	68
8.13.2.1 process()	68
8.13.2.2 reset()	69
8.13.2.3 update()	69
8.14 gainExample Struct Reference	69
8.14.1 Member Function Documentation	71
8.14.1.1 process()	71
8.14.1.2 reset()	71
8.14.1.3 update()	72
8.15 NtFx::KnobLookAndFeel Struct Reference	72
8.16 NtFx::KnobSpec Struct Reference	72
8.16.1 Detailed Description	73
8.16.2 Member Function Documentation	73

8.16.2.1 setLogScale()	73
8.16.3 Member Data Documentation	73
8.16.3.1 name	73
8.17 NtFx::LinGlider Struct Reference	74
8.17.1 Detailed Description	74
8.17.2 Constructor & Destructor Documentation	74
8.17.2.1 LinGlider()	74
8.17.3 Member Function Documentation	75
8.17.3.1 process()	75
8.17.3.2 update()	75
8.18 NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen > Struct Template Reference	75
8.18.1 Detailed Description	76
8.18.2 Member Function Documentation	77
8.18.2.1 getRms()	77
8.18.2.2 process()	77
8.18.2.3 processDelayLine()	77
8.18.2.4 reset()	78
8.18.2.5 setT_ms()	78
8.18.2.6 update()	78
8.19 NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen > Struct Template Reference	79
8.19.1 Detailed Description	79
8.19.2 Member Function Documentation	80
8.19.2.1 getRms()	80
8.19.2.2 setT_ms()	80
8.20 NtFx::Meter Struct Reference	80
8.21 NtFx::MeterGroup Struct Reference	81
8.22 NtFx::MeterScale Struct Reference	82
8.23 NtFx::MeterSpec Struct Reference	83
8.23.1 Detailed Description	83
8.23.2 Member Data Documentation	83
8.23.2.1 addRms	83
8.24 NtFx::Mono< signal_t > Struct Template Reference	84
8.24.1 Member Function Documentation	84
8.24.1.1 abs()	84
8.24.1.2 absMax()	85
8.24.1.3 absMin()	85
8.24.1.4 avgSquared()	85
8.25 NtFx::MonoComponent< signal_t, component_t > Struct Template Reference	85
8.25.1 Detailed Description	86
8.25.2 Member Function Documentation	86
8.25.2.1 process()	86
8.25.2.2 reset()	87

8.25.2.3 update()	87
8.26 NtFx::Generator::Noise Struct Reference	87
8.26.1 Member Function Documentation	88
8.26.1.1 process()	88
8.27 ntCompressor Struct Reference	88
8.27.1 Member Function Documentation	90
8.27.1.1 process()	90
8.27.1.2 reset()	91
8.27.1.3 update()	91
8.28 ntDeEsser Struct Reference	91
8.28.1 Member Function Documentation	93
8.28.1.1 process()	93
8.28.1.2 reset()	93
8.28.1.3 update()	94
8.29 ntFilters Struct Reference	94
8.29.1 Member Function Documentation	96
8.29.1.1 process()	96
8.29.1.2 reset()	96
8.29.1.3 update()	97
8.30 ntGate Struct Reference	97
8.30.1 Member Function Documentation	99
8.30.1.1 process()	99
8.30.1.2 reset()	100
8.30.1.3 update()	100
8.31 ntMultiband3 Struct Reference	100
8.31.1 Member Function Documentation	102
8.31.1.1 process()	102
8.31.1.2 reset()	103
8.31.1.3 update()	103
8.32 NtFx::NtPlugin Struct Reference	103
8.32.1 Detailed Description	105
8.32.2 Member Function Documentation	105
8.32.2.1 getAndResetPeakLevel()	105
8.32.2.2 getDropDownValuePtr()	105
8.32.2.3 getKnobValuePtr()	106
8.32.2.4 getRms()	106
8.32.2.5 getToggleValuePtr()	106
8.32.2.6 onTempoChanged()	106
8.32.2.7 updatePeakLevel()	106
8.32.3 Member Data Documentation	107
8.32.3.1 meters	107
8.33 NtPluginAudioProcessor Struct Reference	107

8.34 NtPluginAudioProcessorEditor Struct Reference	108
8.35 ntRmsMeter Struct Reference	110
8.35.1 Member Function Documentation	112
8.35.1.1 process()	112
8.35.1.2 reset()	112
8.35.1.3 update()	113
8.36 ntTapeEcho Struct Reference	113
8.36.1 Member Function Documentation	115
8.36.1.1 onTempoChanged()	115
8.36.1.2 process()	115
8.36.1.3 reset()	115
8.36.1.4 update()	116
8.37 ntTransformer Struct Reference	116
8.37.1 Member Function Documentation	118
8.37.1.1 process()	118
8.37.1.2 reset()	118
8.37.1.3 update()	119
8.38 NtFx::OptionsSpec Struct Reference	119
8.38.1 Detailed Description	119
8.39 NtFx::PeakHoldSensor< delayLineLength > Struct Template Reference	119
8.39.1 Detailed Description	120
8.39.2 Member Function Documentation	121
8.39.2.1 process()	121
8.39.2.2 update()	121
8.40 NtFx::PeakHoldSensorStereo< delayLineLength > Struct Template Reference	121
8.40.1 Member Function Documentation	122
8.40.1.1 setT_ms()	122
8.40.1.2 setTHold_ms()	122
8.41 NtFx::PeakSensor Struct Reference	123
8.41.1 Detailed Description	124
8.41.2 Member Function Documentation	124
8.41.2.1 _peakSensor()	124
8.41.2.2 process()	124
8.41.2.3 reset()	125
8.41.2.4 update()	125
8.41.3 Member Data Documentation	125
8.41.3.1 tPeak_ms	125
8.42 NtFx::PeakSensorStereo Struct Reference	126
8.42.1 Detailed Description	126
8.42.2 Member Function Documentation	127
8.42.2.1 setT_ms()	127
8.43 NtFx::Comp::PeakSideChainDb Struct Reference	127

8.43.1 Detailed Description	128
8.43.2 Constructor & Destructor Documentation	128
8.43.2.1 PeakSideChainDb()	128
8.43.3 Member Function Documentation	129
8.43.3.1 _gainComputer_db()	129
8.43.3.2 process()	129
8.43.3.3 reset()	129
8.43.3.4 update()	130
8.44 NtFx::Comp::PeakSideChainLinear Struct Reference	130
8.44.1 Detailed Description	131
8.44.2 Constructor & Destructor Documentation	131
8.44.2.1 PeakSideChainLinear()	131
8.44.3 Member Function Documentation	132
8.44.3.1 _gainComputer_lin()	132
8.44.3.2 process()	132
8.44.3.3 update()	133
8.45 NtFx::RadioButtonSet Struct Reference	133
8.45.1 Member Function Documentation	134
8.45.1.1 updateToggleStates()	134
8.46 NtFx::Comp::RmsSideChainDb Struct Reference	134
8.46.1 Detailed Description	135
8.46.2 Member Function Documentation	135
8.46.2.1 process()	135
8.46.2.2 reset()	136
8.46.2.3 update()	136
8.47 NtFx::Comp::RmsSideChainLinear Struct Reference	136
8.47.1 Detailed Description	138
8.47.2 Constructor & Destructor Documentation	138
8.47.2.1 RmsSideChainLinear()	138
8.47.3 Member Function Documentation	138
8.47.3.1 process()	138
8.47.3.2 reset()	138
8.47.3.3 update()	139
8.48 NtFx::Src::SampleRateConverter Struct Reference	139
8.48.1 Detailed Description	139
8.48.2 Constructor & Destructor Documentation	140
8.48.2.1 SampleRateConverter()	140
8.48.3 Member Function Documentation	140
8.48.3.1 process()	140
8.48.3.2 reset()	140
8.49 NtFx::Gate::Sc Struct Reference	141
8.49.1 Member Function Documentation	141

8.49.1.1 process()	141
8.49.1.2 reset()	142
8.49.1.3 update()	142
8.50 NtFx::Comp::ScSettings Struct Reference	142
8.50.1 Detailed Description	143
8.51 NtFx::Gate::ScSettings Struct Reference	143
8.52 NtFx::Biquad::Settings Struct Reference	144
8.53 NtFx::DynamicFilter::Shelf Struct Reference	144
8.53.1 Detailed Description	145
8.53.2 Member Function Documentation	145
8.53.2.1 process()	145
8.53.2.2 reset()	145
8.53.2.3 update()	146
8.53.3 Member Data Documentation	146
8.53.3.1 gain_lin	146
8.53.3.2 q2	146
8.54 NtFx::SoftClip3 Class Reference	146
8.54.1 Detailed Description	147
8.55 NtFx::SoftClip5 Class Reference	147
8.55.1 Detailed Description	148
8.56 NtFx::Biquad::State Struct Reference	148
8.57 NtFx::Src::State Struct Reference	148
8.57.1 Detailed Description	148
8.58 NtFx::Stereo< signal_t > Struct Template Reference	149
8.58.1 Detailed Description	150
8.58.2 Member Function Documentation	150
8.58.2.1 abs()	150
8.58.2.2 absMax()	150
8.58.2.3 absMin()	151
8.58.2.4 avgSquared()	151
8.59 NtFx::StereoComponent< signal_t, component_t > Struct Template Reference	151
8.59.1 Detailed Description	152
8.59.2 Member Function Documentation	152
8.59.2.1 process()	152
8.59.2.2 reset()	152
8.59.2.3 update()	153
8.60 NtFx::FirstOrder::StereoFilter< shape > Struct Template Reference	153
8.60.1 Detailed Description	154
8.60.2 Member Function Documentation	154
8.60.2.1 process()	154
8.60.2.2 reset()	154
8.60.2.3 update()	155

8.61 NtFx::StereoMeter Struct Reference	155
8.62 NtFx::Biquad::StereoState Struct Reference	156
8.63 NtFx::TitleBarLookAndFeel Struct Reference	156
8.64 NtFx::TitleBarSpec Struct Reference	156
8.64.1 Detailed Description	157
8.64.2 Member Data Documentation	157
8.64.2.1 dropdowns	157
8.65 NtFx::Toggle Class Reference	157
8.66 NtFx::ToggleSet Struct Reference	158
8.67 NtFx::ToggleSetBase Struct Reference	159
8.68 NtFx::ToggleSetSpec Struct Reference	159
8.69 NtFx::ToggleSpec Struct Reference	159
8.69.1 Detailed Description	160
8.69.2 Member Data Documentation	160
8.69.2.1 name	160
8.70 NtFx::Transformer Struct Reference	160
8.70.1 Member Function Documentation	161
8.70.1.1 process()	161
8.70.1.2 reset()	161
8.70.1.3 update()	161
8.71 NtFx::UiSpec Struct Reference	162
8.71.1 Detailed Description	163
9 File Documentation	165
9.1 LookAndFeel.h	165
9.2 Meter.h	166
9.3 PluginEditor.h	170
9.4 PluginProcessor.h	172
9.5 RadioButtons.h	174
9.6 Toggle.h	175
9.7 Audio.h	176
9.8 Biquad.h	181
9.9 Comp.h	186
9.10 lib/Component.h File Reference	188
9.10.1 Detailed Description	189
9.11 Component.h	190
9.12 lib/ComponentTest.h File Reference	190
9.12.1 Detailed Description	192
9.12.2 Macro Definition Documentation	192
9.12.2.1 _NTFX_ADD_TEST_IMPL	192
9.12.2.2 _NTFX_EXPAND_AND_QUOTE	192
9.12.2.3 _NTFX_QUOTE	193

9.12.2.4 _NTFX_TEST_BEGIN_IMPL	193
9.12.2.5 _NTFX_TEST_IMPL	193
9.12.2.6 NTFX_ADD_TEST	193
9.12.2.7 NTFX_RUN_TESTS	193
9.12.2.8 NTFX_TEST	194
9.13 ComponentTest.h	194
9.14 lib/DynamicFilter.h File Reference	196
9.14.1 Detailed Description	197
9.15 DynamicFilter.h	198
9.16 lib/FirstOrder.h File Reference	198
9.16.1 Detailed Description	199
9.17 FirstOrder.h	199
9.18 lib/GateSc.h File Reference	200
9.18.1 Detailed Description	201
9.19 GateSc.h	201
9.20 Generator.h	202
9.21 lib/Glider.h File Reference	203
9.21.1 Detailed Description	203
9.22 Glider.h	204
9.23 lib/PeakSensor.h File Reference	204
9.23.1 Detailed Description	205
9.24 PeakSensor.h	206
9.25 lib/Plugin.h File Reference	207
9.25.1 Detailed Description	207
9.26 Plugin.h	208
9.27 lib/RmsSensor.h File Reference	210
9.27.1 Detailed Description	211
9.28 RmsSensor.h	211
9.29 lib/SampleRateConverter.h File Reference	212
9.29.1 Detailed Description	213
9.30 SampleRateConverter.h	214
9.31 lib/SoftClip.h File Reference	216
9.31.1 Detailed Description	216
9.32 SoftClip.h	217
9.33 Transformer.h	217
9.34 lib/UiSpec.h File Reference	218
9.34.1 Detailed Description	219
9.35 UiSpec.h	220
9.36 utils.h	221
9.37 lib/windowFunctions.h File Reference	222
9.37.1 Detailed Description	223
9.38 windowFunctions.h	223

9.39 gainExample.h	229
9.40 ntCompressor.h	231
9.41 ntDeEsser.h	234
9.42 ntFilters.h	235
9.43 plugins/ntGate.h File Reference	237
9.43.1 Detailed Description	238
9.44 ntGate.h	238
9.45 ntMultiband3.h	241
9.46 ntRmsMeter.h	244
9.47 ntTapeEcho.h	245
9.48 ntTransformer.h	248
Index	251

Chapter 1

NtPlugin

A framework for fast and easy audio plugin development. Includes a number of plugins that can be used as is or as examples.

1.1 Target audience

The target audience of this project fall in two groups. One is musicians and producers who want high quality plugins free of charge and with a consistent UI look and feel. The other is hobbyists, students and professionals who needs to try out an idea quickly, without having to build a custom GUI for it. Users will be able to focus on the algorithms and not all the details of using JUCE or another framework.

1.2 Features

- Write audio plugins in plain C++ without the need for knowledge of the JUCE framework.
- Automatic layout of UI including metering and UI scaling. Just list your parametes, and the framework takes care of the rest.
- Oversampling available by default.
- Possibility for wrapping plugins for other targets.

1.2.1 The UI

The autogenerated UI is devided into six sections. The **main grid** displays the user defined primary knobs in the order they are defined. The grid is automatically laid out for minimal empty space in the UI. Maximum rows and columns are user customizable and defaults to 6 and 3. Only the main grid is not optional. The remaining areas can be disabled by the user. The **secondary knobs** are for fine control or less important parameters. The **toggles/dropdowns area** is at the bottom of the UI for boolean paramters and drop downs. To the left is the **meters area**, which displays peak and RMS level. Both normal and inverted (gain reduction) meters are available. RMS is only available for the first two meters, which are assumed to measure in- and output. To the right, the **radio buttons area** displays all the sets of radio buttons that the user defines. Finally, the **title bar** at the top contains the default options such as colour theme, oversampling setting and UI scaling. This can be disabled by the user, but the parameters are still there in the background.

1.3 Getting started

1.3.1 Install needed software

- Install `git`.
- Install `Cmake`.
- Install `Visual Studio` (Windows) or `XCode` (Mac) or the Linux dependencies (Linux).
- Git clone with `--recurse-submodules` flag. This will add the JUCE framework to the download along with other libraries.

1.3.1.1 Linux dependencies

Run the following command to install dependencies on Linux.

1.3.1.1.1 Ubuntu

```
sudo apt update
sudo apt install -y git cmake libasound2-dev libjack-jackd2-dev ladspa-sdk libcurl4-openssl-dev
libfreetype-dev libfontconfig1-dev libx11-dev libxcomposite-dev libxcursor-dev libxinerama-dev
libxext-dev libxrandr-dev libxrender-dev libwebkit2gtk-4.1-dev libglu1-mesa-dev mesa-common-dev
python3.12-venv
```

1.3.1.1.2 Fedora

```
sudo dnf update
sudo dnf install -y g++ git cmake alsa-lib-devel jack-audio-connection-kit-devel.aarch64 libcurl-devel
freetype-devel libX11-devel libXcomposite-devel libXcursor-devel libXinerama-devel freeglut-devel
gnutils-devel libstdc++-static ladspa-devel webkit2gtk4.0-devel.aarch64 gtk3-devel mesa-libGL-devel
```

1.3.2 Try it out

- To test the install, type

```
cmake -B build -S JuceWrapper -DNTEFX_PLUGIN=gainExample
cmake --build build
```

in the terminal. This should build and install the simple gain knob example plugin.

1.4 Installation from source

If you just want to install the included plugins, this can be done from source. The included `ntPlugin.sh` script will do this for Mac and Linux once the needed software is installed. `ntPlugin.bat` is available for build on Windows. At the time of writing, it does not install anything because of Windows security features. The easiest way to use the plugins after this build is to add `[repo dir]\artifacts\VST3` to you DAW's plugin search path. Alternatively, you can copy and paste the contents of that folder to your plugin install folder. In order to perform the build type:

```
bash ./ntPlugin.sh build all # Linux / MacOS terminal.
'''
or
'''sh
cmd /c ./ntPlugin.bat build all # Windows PowerShell for VS.
```

1.5 Usage

- Create your plugin as `plugins/[name of your plugin].h` in the project directory.
- Write a class that inherits from `NtFx::NtPlugin`. The name of your class must be the same as the file name.
- The base class requires you to implement the methods `process`, `update` and `reset`. `process` runs for every sample, `update` is called every time a parameter changes and `reset` (re)initializes the plugin and sets the samplerate. `reset` should always call `update`.
- The base class contains a number of empty vectors, that the user can add parameters to in the constructor of the plugin. `primaryKnobs` contains specifications for the main knobs, which are laid out in a grid automatically, `secondaryKnobs` can be used to add a single row of smaller knobs below main grid for fine tuning or utility controls. `toggles` contain the boolean parameters. The constructor should call `updateDefaults` before returning.
- Similarly, the base class contain a spec for the UI named `uiSpec`, which can be modified for customization of the UI.
- Use the following commands to configure and build your project:

```
cmake -B build -S JuceWrapper -DNTFX_PLUGIN=[name of your plugin]
cmake --build build
```

- Once the project is configured, only the build-command is needed for incremental builds.
- On Mac and Linux, the plugin is automatically installed on the system. Launch your DAW and test your plugin.

Have fun coding.

1.5.1 Caveats

As with all code projects, there are secrets to know.

- If you want to make AAX plugins for Pro Tools, you need to get the AAX SDK from avid and put it in the JuceWrapper directory.
- Plugin IDs are selected at random. This means that when ever Cmake is reconfigured, it's a new plugin and you'll need to reinsert the plugin in the host/DAW. If you need the plugin to have the same ID after reconfiguration, you can add it manually by adding `-DNTFX_ID=[Original ID]` to the configure command.
- On windows, the system won't be able to install outputs, so the easiest way to test your plugin is to add `/path/to/repo/build/[your plugin]\_artefacts/Debug/VST3` to the list of VST search paths in your DAW, so you don't need to copy it to `C:\Program Files\Common Files\VST3` for every build.

1.5.2 Making it work with VsCode

All of these things are specific to JUCE and not the NtPlugin framework, and they should work on and JUCE plugin project.

NOTE: VsCode on ARM Windows is an absolute mess. Not only does it not work, it also breaks your tools some magical way. Debugging works, but don't try to use Cmake.

- Install the Cmake extension.
- Add this to `.vscode/settings.json`:

```
"cmake.configureArgs": [
  // Insert your plugin name. REMEMBER to update when switching plugins or
  // one plugin will overwrite another under a wrong name and you're fucked.
  "-DNTFX_PLUGIN=gainExample"
],
```

- Add thing to `.vscode/c_cpp_properties.json`. The rest of the configuration depends on your OS and local setup, but is the same as any project:

```
"configurations": [
  {
    "name": "JUCE",
    "includePath": [
      "${workspaceFolder}/lib",
      "${workspaceFolder}/plugins",
      "${workspaceFolder}/JuceWrapper",
      "${workspaceFolder}/JuceWrapper/JUCE/modules"
    ],
    // These don't have to be changed when switching plugins.
    // They are only there to keep intellisense happy.
    "defines": [
      "JucePlugin_Name=\"gainExample\"",
      "NTFX_PLUGIN=gainExample",
      "NTFX_PLUGIN_FILE=\"plugins/gainExample.h\""
    ],
  },
],
"version": 4
```

- Add this to `.vscode/launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "REAPER",
      "type": "lldb",
      // Or "cppvsgdb" on Windows or whatever your favorite debugger is.
      // lldb seems to work well on Mac.
      "cwd": "${workspaceFolder}",
      "request": "launch",
      "program": "/Applications/REAPER.app/Contents/MacOS/REAPER",
      // Or the path to your favorite DAW. Tested to work against REAPER. Pro
      // Tools not so much.
      // On Windows:
      "program": "C:\\Program Files\\REAPER (arm64)\\reaper.exe"
    }
  ]
}
```

- Now, you can hit `cmd-shift-p` and type `Cmake: Configure` followed by hitting `F5` and the debugger should start.

1.6 NtPlugin CLI

In order to automate a number of general workflows, a CLI tool name `ntPlugin.py` is available. This will build all available plugins, run tests and create new plugins. Results of this program are stored in `artifacts`, separate from `build` so that the build dir can be cleaned out without deleting any artifacts. Files in `artifacts` are sorted according to plugin type and a text file named `pluginIds.txt` contains the ID and category of each plugin so that those persist across builds. Type `python ntPlugin.py -h` for more info.

1.7 Collaborations

Collaborators are most welcome. Feel free to make bug reports, feature requests and pull requests. If you manage to make a nice plugin, add wish to share with the world, it can become part of the repo through a pull request. A wrapper for the JUCE framework is available, but since plugins are written in platform-agnostic code, targeting more platforms in the future is possible. If anyone wishes to collaborate, adding a wrapper for eg. ESP32 or an ADI device would be an option for a project.

1.8 Mono plugins

Plugins are stereo by default. If you want to build a plugin as mono, simply add `#define NTFX_MONO` to your plugin file **before** including any `NtFx` library headers. If you want your plugin to build in both mono and stereo versions, add a new plugin, which set the `NTFX_MONO` define, includes your plugin and typedefs your original plugin to the new plugin file name. This snippet is the mono plugin file in it's totality.

```
#pragma once
#define NTFX_MONO
#include "[stereo plugin name].h"
template <typename signal_t>
using [mono plugin name] = [stereo plugin name]<signal_t>;
```

1.9 Testing your plugin

The `testWrapper` can be used to test plugins. The individual tests are stored in `testWrapper/tests` and are individual C++ programs. A Python script is used to run the tests. It will generate test input files, build the test program, run it and plot the results. Plots are placed in `testWrapper/img`. If the results are accepted, the script can approve them so that the test will succeed on next run. Type `python ntPlugin.py test -h` for more information.

1.9.1 Setting it up

First, we need the normal python envirement stuff:

```
python3 -m venv .venv
. ./venv/bin/activate
pip install -r requirements.txt
```

Now run the existing tests:

```
python testWrapper/test.py run
```

This should run all existing tests, which should all succeed. `testWrapper/img` should now contain a number of plots.

1.9.2 Add your own test

In order to test a plugin you must create a file named `[plugin name]_test.cpp`, include `lib/ComponentTest.h` and implement a main function, where you instantiate you plugin, make some settings, and add the object to the test class using `NTFX_ADD_TEST`. Then you retrun `runAllTests()`. Example:

```
#include "lib/ComponentTest.h"
#include "plugins/[your plugin].h"
NTFX_TEST_BEGIN // Macro to instantiate statics.

NTFX_TEST() { // Make a main function.
    [plugin name]<double> plug; // Instantiate plugin.
    plug.[some variable] = 2; // Make setting you wanna test.
    NTFX_ADD_TEST(plug, "impulse") // Add test to testWrapper framework.
    return NTFX_RUN_TESTS(); // Run all test tests.
}
```

`NTFX_ADD_TEST` takes a string argument that selects the stimulus to test against. The options are "impulse", "syncSweep", "linearSweep" and "dynamic_matched" and "dynamic_alternating". Different plots are made based on the selected stimulus.

Save as `test/[plugin name]_test.cpp` and run

```
python testWrapper/test.py run [plugin name]
```

Alternatively, the path of the test cpp file can be used as argument.

This should generate plots of frequency and phase response ("impulse"), spectrogram ("linearSweep") and dynamic response using a stimulus of a 10 kHz sine stepping between -12 dB and 0 dB level ("dynamic_matched" for the same on both channels, "dynamic_alternating" for alternating so the left is at 0 dB when right is at -12 dB and vice versa.) . "syncSweep" does not generate a plot at the time of writing. Both left and right channels are displayed in all plots to make differences or interactions visible. If the plots are good and the plugin is seen as passing test, the results are approved with the command:

```
python testWrapper/test.py approve [test object names]
```

Note that the argument is not the name of the class, but the name of the object instance in the test program. This is so some tests can be approved, while others may not.

1.9.3 Test with Pluginval

If you want to use pluginval you will need to do the following:

1. Make sure that set your cmake configuration to either Debug or Release.
2. That you set your preferred test strictness ind the file `JuceWrapper/tests/CMakeLists.txt`

The difference in strictness levels can be read inside the folder `JuceWrapper/pluginval/README.md`, but by default it is set to 10, which will test everything on the plugin for both AU and VST. You are able to add another test in `JuceWrapper/tests/CMakeLists.txt`, if you were to make a plugin as Standalone.

Once you have set the cmake configuration to Debug in VSC, and called:

```
cmake --build build
```

You can then do the following commands to call the pluginval test in order to validate your plugins:

```
ctest --test-dir build
```

After calling `ctest` it will then start the validation of your plugin, and this will then prompt in the terminal:

```
Start 1: ValidatePluginAU
1/2 Test #1: ValidatePluginAU ..... Passed   19.66 sec
Start 2: ValidatePluginVST3
2/2 Test #2: ValidatePluginVST3 ..... Passed   17.05 sec

100% tests passed, 0 tests failed out of 2
```

1.10 The NTfx library

The `lib` directory contains a library with a number of audio components, that can be used for plugins. This is not dependant on any JUCE code, and only uses the GCEM library for comiletime math. This ensures that the various features in the library can be used in any wrapper that might get added to this framework in the future. All the components in the library are templates and takes `typename signal_t` as template paramters. This is so that we can swap the underlying signal datatype if need be. We might want to implement a fractional datatype in the future for use on platforms without an FPU, or just switch to double precision for JUCE plugins.

1.10.1 `signal_t`

All classes and free functions in the library are templates of `signal_t`, which is the main signal datatype. At the time of writing, this always resolves to a float, but it is a template parameter in order to change that at a later point without having to touch all the code.

1.10.2 The Audio class

The Audio class wraps two `signal_t` values in a single object and allows the user to write the dsp code once for both channels. If `NTFX_MONO` is set, it will wrap a single value.

1.10.3 The Component class

The `NtFx::Component` class sits at the heart of the framework. This is not to be confused with the `juce::Component` class, which is for UI components. It is a true virtual base class, which enforces the interface for audio components in the system. All stateful audio components in the library inherits this class, including the `NtFx::NtPlugin` class, with either `signal_t` or `Audio<signal_t>` as template parameter denoting whether the component process stereo or mono signals.

As stated earlier, `NtFx::Component` has the following true virtual methods. These must be implemented by all components in order to adhere to the interface.

- `reset` Is to be called in the reset method of the plugin in order to propagate the sample rate and to where it's needed and empty any buffers and/or filter state.
- `update` Should be called if a component is affected by any parameters, in the update method of the plugin in order to update the filter coefficients this. If not, reset should be enough.
- `process` Processes the signal pr sample and should be called in the process method of the plugin.

The only member of the `Component` class is `float fs`, which is the oversampled (high) sample rate.

1.10.4 The Plugin class

Each plugin design is controlled by settings in `uiSpec` and a number of vectors, which are all members inherited from the `NtPlugin` class. Most of the vectors are empty, and can be populated in order to add features to the UI.

1.10.4.1 UI Specification

The member `uiSpec` of `NtPlugin` contains a number of settings that can be set by the user in the constructor for the plugin. This is where the user sets things like window width, number of rows and columns and default sizes of elements in the UI. Some sizes are dynamic and use the set size as a max.

1.10.4.2 Parameters

UI elements are added by pushing objects to the vectors that the user plugin inherits from the `NtPlugin` base plugin. The available vectors are:

- `primaryKnobs` contains the big knobs to be rendered in the main grid of the plugin. These are laid out on a grid automatically in the order they are defined. Adding an empty knob to `primaryKnob` like this: `this->primaryKnobs.push_back({});` leaves an empty space in the grid, in order to place the knobs where they are wanted. `uiSpec.maxRows` and `uiSpec.maxColumns` can be used to control the number of rows and columns. `p_val` of primary knobs must point to a `signal_t` value. If not, the parameter is seen as a dummy parameter.
- `secondaryKnobs` are placed below the main grid on a single row, that clips if it gets too long. The width of the UI can be set to accommodate more controls if needed. `p_val` of secondary knobs must point to a `signal_t` value. If not, the parameter is seen as a dummy parameter.
- `toggles`
- `dropdowns` are placed to the left of toggles in the bottom row. The field `options` can be used to name the available options. `p_val` of dropdowns must point to an `int` value.
- `radioButtons` are placed in an area to the right of the main grid below each other in the order they are defined. They have the same underlying datatype as dropdowns and thus share the same members. `Valued` is an `int`.
- `toggleSets` is a vector of toggles grouped together in with a name. These are placed in the right side column along with the radio buttons and look the same. Only difference is that more than one on no toggle can be enabled at any one time. Stored as a bunch of `bools`.
- `meters` are placed to the left. By default, an input and an output meter is available. More meters can be added using `push_back` and if no meters are wanted, they be deleted using the `clear`-method in the `std::vector` class.

Knobs can be added to the vectors `primaryKnobs` or `secondaryKnobs` like this:

```
this->primaryKnobs.push_back({
    .p_val = &this->mix_percent,
    .name  = "Mix",
    .suffix = " %",
    .minVal = 0.0,
    .maxVal = 100.0,
});
```

The remaining parameters types follow a similar patterns with a pointer to a value, a name and some members specific to the type of parameter.

- `p_val` is a pointer to the value for the parameter, which should be a member of the plugin. Type must be `signal_t` for knobs, `bool` for toggles and `int` for dropdowns and radiobuttons.
- `name` is the display name, used as an identifier for DAW.
- `suffix` is added at the end of the text box for knobs.
- Dropdowns and radiobuttons have `options`, which is a vector of strings for the options to be displayed in the UI.

1.10.4.3 Side chain input

The side chain input is a mono input, which is updated for each sample at the base sample rate and stored as the member `xSc` in the `NtPlugin` base class. This can be used in the `process` method of the user plugin.

1.10.4.4 Tempo

`tempo` is another member of the `NtPlugin` base class that can be used for coefficient calculation in the user plugin. It is updated per buffer if the host supports it. If not, the tempo is 0, so the user should always check it before using it.

1.10.5 The Glider class

Under some circumstances, changing parameters will cause clicks and pops in audio output. In order to mitigate this, `gliders` can be used to glide changes in parameters into place slowly. The value facing the UI side of the plugin is called `ui` and this should be set when the parameter changes, and the processing facing value is named `pr`. This is what you use in the processing code. Both linear and exponential gliders are available, and both follow the same pattern. The exponential glider is asymptotic and reaches 90% of the target value as the glide time expires, whereas the linear glider will reach the target exactly when the time expires. The `Glider` class does follow the `Component` interface, since `process` doesn't take an input and there is no `reset` method. Instead is sample rate and the main setting, the glide time, set in the call to `update`. `Glider::process` should only be called once per call to `process` in the user plugin. The `ntTapeEcho` plugin in `plugins` contains an example of usage.

1.10.6 The FirstOrder class

The `FirstOrder` class inherits the `Component` class and thus follows that pattern. It delivers a first order high filter and low pass filters, with and without zero at the Nyquist frequency. The version with a zero at the Nyquist frequency performs better in the high frequencies but the one without sums perfectly when the filter is used for crossovers. The shape of first order filters is set as a template parameter and thus is fixed at runtime.

1.10.7 The Biquad class

Second order filters are encapsulated in the `Biquad` namespace. Both five and six coefficient versions are available along with coefficient calculation for both. An `EqBand` class pairs two five-coefficient biquads for left and right channel with settings and can be used to add second order filters to a plugin. The `Shape` enum holds the possible shapes and `none` can be used to bypass the filter without branching if needed.

1.10.8 The PeakSensor class

The peak sensor can be used to detect the peak level of a signal with a user controllable release. This is used for dynamics side chains and peak level meters.

1.10.9 The RmsSensor class

Rms sensor with variable averaging time in milliseconds. Used for RMS dynamics and metering.

1.10.10 The SampleRateConverter class

The sample rate converter is a part of the plugin by default and the user doesn't need to do anything in order to utilize it. It's a sincos converter that takes a reference to an `NtPlugin`, who's `process`-method will be called per sample at the upsampled rate.

1.10.11 SideChain

The `SideChain` namespace contains a number of compressor side chains, including peak- and RMS-sensing in dB and linear domains.

1.10.12 SoftClip

Since `SoftClip` does not hold state, it's not class. Classes are for storing state, not for wrapping functions in a namespace (Java, I'm looking at you!). The coefficients are calculated compile time and used in free functions for third, fifth and seventh order soft clipping.

1.10.13 Transformer

The `Transformer` class is an experiment to make a simplified transformer model. It's only modelling the low frequency part of the transformer behavior.

Chapter 2

NtPlugin License

This project is licensed under the [AGPLv3](#).

Copyright (C) 2026 Niels Thøgersen

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

2.1 Using Wrappers

Since this framework depends on wrappers, it must adhere to the licensing model of each wrapper when that wrapper is in use. Look into the individual wrappers for their respective licenses. If the JUCE wrapper is in use, the AGPLv3 license is required for open source projects and the commercial [JUCE licence](#) is required for closed source/commercial projects.

2.2 GNU GENERAL PUBLIC LICENSE

Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

2.2.1 Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program—to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

2.2.2 TERMS AND CONDITIONS

2.2.2.1 0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

2.2.2.2 1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2.2.2.3 2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

2.2.2.4 3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

2.2.2.5 4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

2.2.2.6 5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

- a) The work must carry prominent notices stating that you modified it, and giving a relevant date.
- b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".
- c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

- d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

2.2.2.7 6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

- a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.
- b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.
- c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.
- d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.
- e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

2.2.2.8 7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

- a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or
- b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or
- c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or
- d) Limiting the use for publicity purposes of names of licensors or authors of the material; or
- e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or
- f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

2.2.2.9 8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

2.2.2.10 9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

2.2.2.11 10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

2.2.2.12 11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

2.2.2.13 12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

2.2.2.14 13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

2.2.2.15 14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

2.2.2.16 15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

2.2.2.17 16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

2.2.2.18 17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

2.2.3 How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

```
<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>

This program is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program. If not, see <https://www.gnu.org/licenses/>.
```

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

```
<program> Copyright (C) <year> <name of author>
This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
This is free software, and you are welcome to redistribute it
under certain conditions; type `show c' for details.
```

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see <https://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <https://www.gnu.org/licenses/why-not-lgpl.html>.

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

NtFx	An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side	33
ntPlugin		36
package		40
test		46

Chapter 4

Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

juce::AudioProcessor	
NtPluginAudioProcessor	107
juce::AudioProcessorEditor	
NtPluginAudioProcessorEditor	108
NtFx::Biquad::Biquad6	53
juce::ChangeBroadcaster	
NtFx::ToggleSetBase	159
NtFx::RadioButtonSet	133
NtFx::ToggleSet	158
juce::ChangeListener	
NtPluginAudioProcessorEditor	108
NtFx::Src::Coeffs	53
NtFx::Biquad::Coeffs5	54
NtFx::Biquad::Coeffs6	54
juce::Component	
NtFx::Meter	80
NtFx::MeterGroup	81
NtFx::MeterScale	82
NtFx::StereoMeter	155
NtFx::ToggleSetBase	159
NtFx::ComponentBase< T >	54
NtFx::ComponentBase< Audio >	54
NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::lpf >	153
NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::hpf >	153
NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::lpfZero >	153
NtFx::Biquad::EqBand6Mono	60
NtFx::Biquad::EqBand6Stereo	61
NtFx::Biquad::EqBandMono	63
NtFx::Biquad::EqBandStereo	64
NtFx::Comp::PeakSideChainDb	127
NtFx::Comp::PeakSideChainLinear	130
NtFx::Comp::RmsSideChainLinear	136
NtFx::Comp::RmsSideChainDb	134
NtFx::DynamicFilter::Shelf	144
NtFx::FirstOrder::StereoFilter< shape >	153

NtFx::Gate::Sc	141
NtFx::Generator::Noise	87
NtFx::NtPlugin	103
gainExample	69
ntCompressor	88
ntDeEsser	91
ntFilters	94
ntGate	97
ntMultiband3	100
ntRmsMeter	110
ntTapeEcho	113
ntTransformer	116
NtFx::SoftClip3	146
NtFx::SoftClip5	147
NtFx::Transformer	160
NtFx::ComponentBase< Mono< signal_t > >	54
NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >	75
NtFx::MonoComponent< signal_t, component_t >	85
NtFx::ComponentBase< signal_t >	54
NtFx::FirstOrder::Filter< shape >	67
NtFx::PeakSensor	123
NtFx::PeakHoldSensor< delayLineLength >	119
NtFx::ComponentBase< Stereo< signal_t > >	54
NtFx::StereoComponent< signal_t, component_t >	151
NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >	79
NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >	79
NtFx::PeakHoldSensorStereo< delayLineLength >	121
NtFx::PeakSensorStereo	126
NtFx::ComponentTest	56
NtFx::ComponentTestSet	58
NtFx::ExpGlider	66
NtFx::KnobSpec	72
NtFx::LinGlider	74
juce::ComboBox::Listener	
NtPluginAudioProcessorEditor	108
juce::Slider::Listener	
NtPluginAudioProcessorEditor	108
juce::ToggleButton::Listener	
NtPluginAudioProcessorEditor	108
juce::LookAndFeel_V4	
NtFx::KnobLookAndFeel	72
NtFx::TitleBarLookAndFeel	156
NtFx::MeterSpec	83
NtFx::Mono< signal_t >	84
NtFx::OptionsSpec	119
NtFx::Src::SampleRateConverter	139
NtFx::Comp::ScSettings	142
NtFx::Gate::ScSettings	143
NtFx::Biquad::Settings	144
NtFx::Biquad::State	148
NtFx::Src::State	148
NtFx::Stereo< signal_t >	149
NtFx::Biquad::StereoState	156
juce::TextButton	
NtFx::Toggle	157
juce::Timer	
NtPluginAudioProcessorEditor	108

NtFx::TitleBarSpec	156
NtFx::ToggleSetSpec	159
NtFx::ToggleSpec	159
NtFx::UiSpec	162

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

NtFx::Biquad::Biquad6	53
NtFx::Src::Coeffs	
Coefficients structure for sample rate converter	53
NtFx::Biquad::Coeffs5	54
NtFx::Biquad::Coeffs6	54
NtFx::ComponentBase< T >	
Base class for audio components in NtPlugin . All processors adhere to this interface	54
NtFx::ComponentTest	
Processes a vector of test data with audio component and compares to expected results. Total number of tests and number of succesful test are stored as statics in order to make statistics	56
NtFx::ComponentTestSet	
Set of a number of tests. Represents a full test file and this a test of a Component in the library or an NtPlugin	58
NtFx::Biquad::EqBand6Mono	60
NtFx::Biquad::EqBand6Stereo	61
NtFx::Biquad::EqBandMono	63
NtFx::Biquad::EqBandStereo	64
NtFx::ExpGlider	66
NtFx::FirstOrder::Filter< shape >	
A single channel first order filter	67
gainExample	69
NtFx::KnobLookAndFeel	72
NtFx::KnobSpec	
Specification for a knob	72
NtFx::LinGlider	
Glides member 'pr' linearly towards 'ui'	74
NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >	
RMS (Root Mean Square) sensor component for audio signal processing	75
NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >	
Stereo RMS sensor component	79
NtFx::Meter	80
NtFx::MeterGroup	81
NtFx::MeterScale	82
NtFx::MeterSpec	
Specification for meters	83

NtFx::Mono< signal_t >	84
NtFx::MonoComponent< signal_t, component_t >	
Wraps two objects of type Component<signal_t> as a Component<Audio> and calls through the both objects	85
NtFx::Generator::Noise	87
ntCompressor	88
ntDeEsser	91
ntFilters	94
ntGate	97
ntMultiband3	100
NtFx::NtPlugin	
True virtual base class for NtPlugin . Inherit from this in order to make a stereo audio plugin with auto-generated UI	103
NtPluginAudioProcessor	107
NtPluginAudioProcessorEditor	108
ntRmsMeter	110
ntTapeEcho	113
ntTransformer	116
NtFx::OptionsSpec	
Specification for drop down in UI	119
NtFx::PeakHoldSensor< delayLineLength >	
Peak sensor with hold. Output will remain at the max value for tHold_ms milliseconds when after peak and the curve will be delayed by the same amount time when going down	119
NtFx::PeakHoldSensorStereo< delayLineLength >	121
NtFx::PeakSensor	
Peak sensor component for audio processing	123
NtFx::PeakSensorStereo	
Stereo peak sensor component for audio processing	126
NtFx::Comp::PeakSideChainDb	
Peak-based side chain processor using dB domain	127
NtFx::Comp::PeakSideChainLinear	
Peak-based side chain processor using linear domain	130
NtFx::RadioButtonSet	133
NtFx::Comp::RmsSideChainDb	
RMS-based side chain processor using dB domain	134
NtFx::Comp::RmsSideChainLinear	
RMS-based side chain processor using linear domain	136
NtFx::Src::SampleRateConverter	
Sample rate converter class	139
NtFx::Gate::Sc	141
NtFx::Comp::ScSettings	
Side chain settings structure	142
NtFx::Gate::ScSettings	143
NtFx::Biquad::Settings	144
NtFx::DynamicFilter::Shelf	
An optimized dynamic shelving filter, which exposes both Q values. Feedback coeffs are calculated in update, while feed forward is calculated in process, so the gain of the filter can be updated while running, making the filter usefull for dynamic processing. Gain is in the linear domain for the same reason	144
NtFx::SoftClip3	
Third order soft clipper wrapped in a Component	146
NtFx::SoftClip5	
Fifth order soft clipper wrapped in a Component	147
NtFx::Biquad::State	148
NtFx::Src::State	
State structure for sample rate converter	148

NtFx::Stereo< signal_t >	
Wraps two values as a stereo signal.This allows us to do DSP math on both channels at the same time. The class overloads all the normal math functions. Please note that comparison operators are different for stereo signals and that a < b not necessarily equals b > a since '<' compares the minimum values while '>' compares maximums	149
NtFx::StereoComponent< signal_t, component_t >	
Wraps two objects of type Component<signal_t> as a Component<Audio> and calls through the both objects	151
NtFx::FirstOrder::StereoFilter< shape >	
Stereo pair of first order filters	153
NtFx::StereoMeter	155
NtFx::Biquad::StereoState	156
NtFx::TitleBarLookAndFeel	156
NtFx::TitleBarSpec	
Specifies the layout of the title bar	156
NtFx::Toggle	157
NtFx::ToggleSet	158
NtFx::ToggleSetBase	159
NtFx::ToggleSetSpec	159
NtFx::ToggleSpec	
Specification for toggles in UI	159
NtFx::Transformer	160
NtFx::UiSpec	
General settings for to UI	162

Chapter 6

File Index

6.1 File List

Here is a list of all documented files with brief descriptions:

JuceWrapper/ LookAndFeel.h	165
JuceWrapper/ Meter.h	166
JuceWrapper/ PluginEditor.h	170
JuceWrapper/ PluginProcessor.h	172
JuceWrapper/ RadioButtons.h	174
JuceWrapper/ Toggle.h	175
lib/ Audio.h	176
lib/ Biquad.h	181
lib/ Comp.h	186
lib/ Component.h	
Base class for Audio components. Defines the general interface of the NTplugin framework	188
lib/ ComponentTest.h	
Test of audio components	190
lib/ DynamicFilter.h	
Dynamic biquad filter	196
lib/ FirstOrder.h	
First order high- and low pass filters	198
lib/ GateSc.h	
Gate side chain	200
lib/ Generator.h	202
lib/ Glider.h	
Glider smooths out changes in parameters	203
lib/ PeakSensor.h	
Peak sensor component for audio processing. Detect and tracks the peak amplitude of an audio signal. The peak	204
lib/ Plugin.h	
Base class for NTplugin framework	207
lib/ RmsSensor.h	
RMS sensor for audio. Accumulates in multiple stages, with a one millisecond accumulator followed by a user controllable accumulator. This means that the output has a real world sample rate of 1 kHz	210
lib/ SampleRateConverter.h	
Sample rate converter using FIR filtering for oversampling	212
lib/ SoftClip.h	
Soft clippers for audio processing. Calculates coeffs at compile time and applies polynomials to signals	216

lib/Transformer.h	217
lib/UiSpec.h	
Specification for plugin UI	218
lib/utils.h	221
lib/windowFunctions.h	
Various usefull math functions. Mainly concerning FIR fitlers. This code is modified from https://github.com/sidneycadot/WindowFunctions.git	222
plugins/gainExample.h	229
plugins/ntCompressor.h	231
plugins/ntDeEsser.h	234
plugins/ntFilters.h	235
plugins/ntGate.h	
Niose gate	237
plugins/ntMultiband3.h	241
plugins/ntRmsMeter.h	244
plugins/ntTapeEcho.h	245
plugins/ntTransformer.h	248

Chapter 7

Namespace Documentation

7.1 NtFx Namespace Reference

An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Classes

- struct [ComponentBase](#)
Base class for audio components in [NtPlugin](#). All processors adhere to this interface.
- struct [ComponentTest](#)
Processes a vector of test data with audio component and compares to expected results. Total number of tests and number of succesful test are stored as statics in order to make statistics.
- struct [ComponentTestSet](#)
Set of a number of tests. Represents a full test file and this a test of a Component in the library or an [NtPlugin](#).
- struct [ExpGlider](#)
- struct [KnobLookAndFeel](#)
- struct [KnobSpec](#)
Specification for a knob.
- struct [LinGlider](#)
Glides member 'pr' linearly towards 'ui'.
- struct [LongRmsSensorMono](#)
RMS (Root Mean Square) sensor component for audio signal processing.
- struct [LongRmsSensorStereo](#)
Stereo RMS sensor component.
- struct [Meter](#)
- struct [MeterGroup](#)
- struct [MeterScale](#)
- struct [MeterSpec](#)
Specification for meters.
- struct [Mono](#)
- struct [MonoComponent](#)
Wraps two objects of type `Component<signal_t>` as a `Component<Audio>` and calls through the both objects.
- struct [NtPlugin](#)
True virtual base class for [NtPlugin](#). Inherit from this in order to make a stereo audio plugin with auto-genrated UI.
- struct [OptionsSpec](#)

- Specification for drop down in UI.*

 - struct [PeakHoldSensor](#)

Peak sensor with hold. Output will remain at the max value for tHold_ms milliseconds when after peak and the curve will be delayed by the same amount time when going down.
 - struct [PeakHoldSensorStereo](#)
 - struct [PeakSensor](#)

Peak sensor component for audio processing.
 - struct [PeakSensorStereo](#)

Stereo peak sensor component for audio processing.
 - struct [RadioButtonSet](#)
 - class [SoftClip3](#)

Third order soft clipper wrapped in a Component.
 - class [SoftClip5](#)

Fifth order soft clipper wrapped in a Component.
 - struct [Stereo](#)

Wraps two values as a stereo signal. This allows us to do DSP math on both channels at the same time. The class overloads all the normal math functions. Please note that comparison operators are different for stereo signals and that $a < b$ not necessarily equals $b > a$ since ' $<$ ' compares the minimum values while ' $>$ ' compares maximums.
 - struct [StereoComponent](#)

Wraps two objects of type Component<signal_t> as a Component<Audio> and calls through the both objects.
 - struct [StereoMeter](#)
 - struct [TitleBarLookAndFeel](#)
 - struct [TitleBarSpec](#)

Specifies the layout of the title bar.
 - class [Toggle](#)
 - struct [ToggleSet](#)
 - struct [ToggleSetBase](#)
 - struct [ToggleSetSpec](#)
 - struct [ToggleSpec](#)

Specification for toggles in UI.
 - struct [Transformer](#)
 - struct [UiSpec](#)

General settings for to UI.

Typedefs

- template<typename signal_t, typename component_t>
using **AudioComponent** = [StereoComponent](#)<signal_t, component_t>
- template<int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
using **LongRmsSensor** = [LongRmsSensorStereo](#)<maxT_ms, maxSampleDLineLen>
- typedef [OptionsSpec](#) **DropDownSpec**
- typedef [OptionsSpec](#) **RadioButtonSetSpec**

Functions

- template<typename signal_t, size_t N>
constexpr std::array< signal_t, N+1 > [_calculateSoftClipCoeffs](#) () noexcept
Calculates coefficients for symmetrical soft clipper at compile time.

Variables

- constexpr char **SEPARATOR** = '.'
- constexpr int **defaultPeakSensorDelayLineLength** = 192 * 10 * 8
- constexpr std::array< signal_t, 4 > **_coeffsSeventh**
- constexpr std::array< signal_t, 3 > **_coeffsFifth**
- constexpr std::array< signal_t, 2 > **_coeffsThird**
- constexpr int **nMetersMax** = 8

7.1.1 Detailed Description

An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

7.1.2 Function Documentation

7.1.2.1 _calculateSoftClipCoeffs()

```
template<typename signal_t, size_t N>
std::array< signal_t, N+1 > NtFx::_calculateSoftClipCoeffs () [inline], [constexpr], [noexcept]
```

Calculates coefficients for symmetrical soft clipper at compile time.

Template Parameters

<i>signal_t</i>	Audio signal type.
<i>N</i> .	Determines order. Order = 2 * N + 1.

Returns

constexpr array coefficients. Length N + 1;

7.1.3 Variable Documentation

7.1.3.1 _coeffsFifth

```
std::array<signal_t, 3> NtFx::_coeffsFifth [constexpr]
```

Initial value:

```
=
    _calculateSoftClipCoeffs<signal_t, 2>()
```

7.1.3.2 _coeffsSeventh

```
std::array<signal_t, 4> NtFx::_coeffsSeventh [constexpr]
```

Initial value:

```
=
    _calculateSoftClipCoeffs<signal_t, 3>()
```

7.1.3.3 `_coeffsThird`

```
std::array<signal_t, 2> NtFx::_coeffsThird [constexpr]
```

Initial value:

```
=  
    _calculateSoftClipCoeffs<signal_t, 1>()
```

7.1.3.4 `nMetersMax`

```
int NtFx::nMetersMax = 8 [constexpr]
```

Size of meter peak level array and thus max number of meters available.

7.2 ntPlugin Namespace Reference

Functions

- bool `_artefactExists` (str plugin, str target)
- None `_openInVscode` (str path)
- bool `_writeFile` (str path, str content)
- bool `newPlugin` (str name)
- `newPluginTest` (str name)
- dict[str, list[str]] `readPluginIds` ()
- bool `configure` (str plugin, dict[str, list[str]] pluginIds, str category="", str version="", str company="")
- bool `addNewPluginId` (str plugin, str cmakeOut, str category="")
- bool `_writePluginId` (str plugin, str newPluginId, str category)
- bool `_overwriteIdFile` (dict[str, list[str]] pluginIds)
- bool `updatePluginId` (str plugin, str category)
- bool `build` ()
- bool `runCTest` ()
- bool `storeArtifacts` (str plugin)
- bool `runAuVal` ()
- bool `process` (dict args)
- argparse.ArgumentParser `createParser` ()
- bool `main` (dict args)

Variables

- `REPO_BASE_DIR` = os.path.realpath(os.path.dirname(__file__))
- `BUILD_DIR` = os.path.realpath(f"{REPO_BASE_DIR}/build")
- `ARTIFACTS_DIR` = os.path.realpath(f"{REPO_BASE_DIR}/artifacts")
- `JUCE_WRAPPER_DIR` = os.path.realpath(f"{REPO_BASE_DIR}/JuceWrapper")
- `TEST_SCRIPT_DIR` = os.path.realpath(f"{REPO_BASE_DIR}/testWrapper")
- `ID_FILE` = os.path.realpath(f"{JUCE_WRAPPER_DIR}/pluginIds.txt")
- list `SECRETS` = ["devId", "email", "password", "team", "installerId", "company"]
- int `ID` = 0
- int `VST3_CAT` = 1
- int `AAX_CAT` = 2
- dict `CATEGORY_MAP`

7.2.1 Detailed Description

```
@file ntPlugin.py
@author Niels Thøgersen (niels.thoegersen@gmail.com)
@brief Top level CLI for working with the NTplugin framework.
@version 0.1
```

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <<https://www.gnu.org/licenses/>>.

7.2.2 Function Documentation

7.2.2.1 addNewPluginId()

```
bool ntPlugin.addNewPluginId (
    str plugin,
    str cmakeOut,
    str category = "")
```

Adds a new plugin ID along with AAX and VST3 categories to plugin ID file.

Args:

```
plugin (str): Name of plugin.
cmakeOut (str): Stdout from Cmake configure command. Used to deduce plugin ID.
category (str): Category to add to record in ID file in AAX format.
```

Returns:

```
bool: True on success.
```

7.2.2.2 build()

```
bool ntPlugin.build ()
```

Builds plugin.

Returns:

```
bool: True on success.
```

7.2.2.3 configure()

```
bool ntPlugin.configure (  
    str plugin,  
    dict[str, list[str]] pluginIds,  
    str category = "",  
    str version = "",  
    str company = "")
```

Configures Cmake for build

Args:

plugin (str): Name of plugin to build.
pluginIds (dict[str, list[str]]): Information about plugins. See 'readPluginIds' for more.
category (str): AAX format category for plugin. Overrides categories in 'pluginIds' if set.

Returns:

bool: True on success.

7.2.2.4 createParser()

```
argparse.ArgumentParser ntPlugin.createParser ()
```

Creates argument parser for ntPlugin CLI.

Returns:

argparse.ArgumentParser: New parser.

7.2.2.5 main()

```
bool ntPlugin.main (  
    dict args)
```

Main function for ntPlugin CLI.

Returns:

bool: True on success.

7.2.2.6 newPlugin()

```
bool ntPlugin.newPlugin (  
    str name)
```

Creates a new plugin. If Vscode is installed, opens the file.

Args:

name: Name of new plugin.

Returns:

bool: True on success.

7.2.2.7 newPluginTest()

```
ntPlugin.newPluginTest (  
    str name)
```

Creates a new test file for a plugin plugin. If Vscode is installed, opens the file.

Args:
 name: Name of new plugin.

Returns:
 bool: True on success.

7.2.2.8 process()

```
bool ntPlugin.process (  
    dict args)
```

Configures, builds and tests all plugins.

Args:
 args (dict): CLI args.

Returns:
 bool: True on success.

7.2.2.9 readPluginIds()

```
dict[str, list[str]] ntPlugin.readPluginIds ()
```

Reads cached plugin ids along with categories from file and returns as a dict with plugin name as keys and list of strings as values. Order of values is ID, VST3 category, AAX category.

7.2.2.10 runAuVal()

```
bool ntPlugin.runAuVal ()
```

Runs Apple's AU validation utility for all plugins made by the 'NTfx' vendor.

Returns:
 bool: True on success.

7.2.2.11 runCtest()

```
bool ntPlugin.runCtest ()
```

Runs Ctest.

Returns:
 bool: True on success.

7.2.2.12 storeArtifacts()

```
bool ntPlugin.storeArtifacts (
    str plugin)
```

Stores output from build to 'artifacts' folder. Sorts according to plugin type, that is 'AAX', 'VST3', 'AU' and 'Standalone'.

Args:

plugin (str): Name of plugin to find and store artifacts for.

Returns:

bool: True on success.

7.2.3 Variable Documentation

7.2.3.1 CATEGORY_MAP

```
dict ntPlugin.CATEGORY_MAP
```

Initial value:

```
00001 = {
00002     "Effect": "Fx",
00003     "EQ": "EQ",
00004     "Dynamics": "Dynamics",
00005     "Reverb": "Reverb",
00006     "Delay": "Delay",
00007     "PitchShift": "Pitch Shift",
00008     "Modulation": "Modulation",
00009     "Harmonic": "Distortion",
00010     "NoiseReduction": "Restoration",
00011     "SoundField": "Spatial",
00012 }
```

7.3 package Namespace Reference

Functions

- `_pluginPath` (str plugin, str target)
- `list[str] readPlugins` ()
- `bool storeSecrets` (dict[str, str] secrets)
- `dict[str, str] loadSecrets` ()
- `bool sign` (list[str] plugins, list[str] targets, str devId)
- `bool notarizePlugins` (list[str] plugins, list[str] targets, str email, str password, str teamId)
- `bool notarize` (str path, str email, str password, str teamId)
- `bool validateInstaller` (str path)
- `bool staplePlugins` (list[str] plugins, list[str] targets)
- `bool staple` (str path)
- `bool zipPackage` (list[str] plugins, list[str] targets, str version)
- `bool makeDistributionXml` (list[str] plugins, list[str] targets, str company, str version="0.1.0")
- `bool makeInstallerPkg` (str installerId, str path)
- `bool makePluginPkgs` (list[str] plugins, list[str] targets, str company)
- `bool makePluginPkg` (str plugin, str target, str company)
- `bool secretsAreValid` (dict secrets)
- `bool verisonIsNewer` (str new, str old)
- `str incrementMinorMonirVersion` (str old)
- `str findVersion` (dict args, dict secrets)
- `bool main` (dict args)
- `argparse.ArgumentParser createParser` ()

Variables

- str **BLACK** = "\033[0m"
- str **RED** = "\033[31m"
- str **GREEN** = "\033[32m"
- str **YELLOW** = "\033[33m"
- str **BLUE** = "\033[34m"
- **REPO_BASE_DIR** = os.path.realpath(f"{os.path.dirname(__file__)}/..")
- **PLUGINS_DIR** = os.path.realpath(f"{REPO_BASE_DIR}/plugins")
- **BUILD_DIR** = os.path.realpath(f"{REPO_BASE_DIR}/build")
- **ARTIFACTS_DIR** = os.path.realpath(f"{REPO_BASE_DIR}/artifacts")
- **JUCE_WRAPPER_DIR** = os.path.realpath(f"{REPO_BASE_DIR}/JuceWrapper")
- **TEST_SCRIPT_DIR** = os.path.realpath(f"{REPO_BASE_DIR}/testWrapper")
- **SECRETS_FILE** = os.path.realpath(f"{JUCE_WRAPPER_DIR}/.secrets.txt")
- **PACKAGING_DIR** = os.path.realpath(f"{BUILD_DIR}/packaging")
- str **PACKAGE_ARTIFACT** = "ntPlugin"
- list **SECRETS**
- dict **TARGET_EXT_MAP**

7.3.1 Detailed Description

```
@file package.py
@author Niels Thøgersen (niels.thoegersen@gmail.com)
@brief Packaging of plugins for MacOS.
@version 0.1
```

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <<https://www.gnu.org/licenses/>>.

7.3.2 Function Documentation

7.3.2.1 createParser()

```
argparse.ArgumentParser package.createParser ()
```

Creates an argument parser for packaging CLI features.

Returns:

```
    argparse.ArgumentParser: Parser.
```

7.3.2.2 main()

```
bool package.main (  
    dict args)
```

Packages selected plugins for MacOS.

Args:
args (dict): CLI arguments.

Returns:
bool: True on success.

7.3.2.3 makeDistributionXml()

```
bool package.makeDistributionXml (  
    list[str] plugins,  
    list[str] targets,  
    str company,  
    str version = "0.1.0")
```

Generates a distribution XML file.

Args:
plugins (list[str]): Names of plugins to add to installer.
company (str): Vendor/company name.
version (str, optional): Version string. Defaults to "0.1.0".

Returns:
bool: True on success.

7.3.2.4 makeInstallerPkg()

```
bool package.makeInstallerPkg (  
    str installerId,  
    str path)
```

Makes an installer package from individual plugin packages.

Args:
installerId (str): Apple Developer ID Installer to sign with.
path (str): Output path.

Returns:
bool: True on success.

7.3.2.5 makePluginPkg()

```
bool package.makePluginPkg (  
    str plugin,  
    str target,  
    str company)
```

Makes a package file for a single plugin.

Args:
plugin (str): Name of plugin.
target (str): Target format.
company (str): Company/vendor.

Returns:
bool: True on success.

7.3.2.6 makePluginPkgs()

```
bool package.makePluginPkgs (  
    list[str] plugins,  
    list[str] targets,  
    str company)
```

Makes a pkg file for each plugin.

Args:

- `plugins (list[str]):` List of names of plugins.
- `targets (list[str]):` List of target formats (AU, VST3 ect.).
- `company (str):` Name of vendor/company.

Returns:

- `bool:` True on success.

7.3.2.7 notarize()

```
bool package.notarize (  
    str path,  
    str email,  
    str password,  
    str teamId)
```

Notarizes an ininstaller or plugin with Apple's servers.

Args:

- `path (str):` Path of file to me notarized.
- `email (str):` E-mail address of Apple Developer Account.
- `password (str):` Password of Apple Developer Account.
- `teamId (str):` Apple Developer team ID.

Returns:

- `bool:` True on success.

7.3.2.8 notarizePlugins()

```
bool package.notarizePlugins (  
    list[str] plugins,  
    list[str] targets,  
    str email,  
    str password,  
    str teamId)
```

Notarizes plugins.

Args:

- `plugins (list[str]):` Names of plugins to notarize.
- `targets (list[str]):` Names of targeted plugin formats.
- `email (str):` Email address/user name for Apple ID.
- `password (str):` Password for Apple Developer ID. Must be app-specific.
- `teamId (str):` Team ID from Apple Developer.

Returns:

- `bool:` True on success.

7.3.2.9 readPlugins()

```
list[str] package.readPlugins ()
```

Returns a list of all plugins found in folder 'plugins'.

7.3.2.10 sign()

```
bool package.sign (  
    list[str] plugins,  
    list[str] targets,  
    str devId)
```

Applies code signing on Mac.

Args:
 plugins (list[str]): Names of plugins to sign.
 targets (list[str]): Names of targeted plugin formats.
 devId (str): Developer Application ID as generated at
 developer.apple.com or using Xcode.

Returns:
 bool: True on success.

7.3.2.11 staple()

```
bool package.staple (  
    str path)
```

Staples selected file with signature from Apple.

Args:
 path (str): Path to file to staple.

Returns:
 bool: True on success.

7.3.2.12 staplePlugins()

```
bool package.staplePlugins (  
    list[str] plugins,  
    list[str] targets)
```

Staple plugins with approval. Depends on plugins being notarized.

Args:
 plugins (list[str]): Plugins to staple.

Returns:
 bool: True on success.

7.3.2.13 validateInstaller()

```
bool package.validateInstaller (
    str path)
```

Returns True if installer is signed, notarized and stapled.

Args:
path (str): Path of installer.

Returns:
bool: True if valid.

7.3.2.14 zipPackage()

```
bool package.zipPackage (
    list[str] plugins,
    list[str] targets,
    str version)
```

Zips selected plugins in a single package for 'manual install' distribution.

Args:
plugins (list[str]): Plugins to package.
targets (list[str]): Target formats to package.

Returns:
bool: True on success.

7.3.3 Variable Documentation

7.3.3.1 SECRETS

```
list package.SECRETS
```

Initial value:

```
00001 = [
00002     "devId",
00003     "email",
00004     "password",
00005     "team",
00006     "installerId",
00007     "company",
00008     "version",
00009 ]
```

7.3.3.2 TARGET_EXT_MAP

```
dict package.TARGET_EXT_MAP
```

Initial value:

```
00001 = {
00002     "VST3": "vst3",
00003     "AU": "component",
00004     "AAX": "aaxplugin",
00005     "Standalone": "app",
00006 }
```

7.4 test Namespace Reference

Functions

- np.ndarray [generateImpulse](#) (int n)
- np.ndarray [generateBurstSine](#) (float fs, float t, float f=10e3, int nBursts=2, float lowLevel=0.25, float highLevel=1)
- np.ndarray [generateSyncSweep](#) (float f_start=10, float f_stop=24e3, float fs=48e3, int k=2)
- np.ndarray [generateLinearSweep](#) (float fs, float t)
- [_storeMonoTestVectorAsStereo](#) (np.ndarray x, str filename)
- [_storeStereoTestVector](#) (np.ndarray x, str filename)
- tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray] [generateTestVectors](#) (float t, float fs)
- bool [_buildTestProg](#) (str cppPath)
- int [_runTestProg](#) ()
- list[str] [_findAllTests](#) ()
- str [_idxToLineStyle](#) (int i)
- [plotImpulse](#) (np.ndarray x, float fs, str filename, list|None legends=None)
- None [plotTimeDomain](#) (np.ndarray x, float fs, str filename, list|None legends=None, list|None ylim=None, str|None ylabel=None)
- None [plotFrequencyDomain](#) (np.ndarray x, float fs, str filename, list|None legends=None, list|None ylim=None, str|None ylabel=None)
- None [plotSpectrum](#) (np.ndarray x, float fs, str filename)
- None [plotDynamic](#) (np.ndarray x, float fs, str filename, list|None legends=None, int zoom=1)
- [plotSweeps](#) (list[np.ndarray] results, dict[str, list[str]] legends, str testFileName, float fs, str objectName)
- [_parseFiles](#) (list[str] files, list[str] exceptedFiles)
- np.ndarray|None [_readResult](#) (str path)
- [_plotResults](#) (dict[str, list[np.ndarray]] results, dict[str, list[str]] legends, str testFileName, float fs)
- [_plotDynamic](#) (dict[str, list[np.ndarray]] results, str name, str testFileName, dict[str, list[str]] legends, float fs)
- [_readAndPlotTestResults](#) (str testFileName, float fs)
- bool [acceptLatestResult](#) (list[str] files, list[str] objects)
- bool [clean](#) ()
- bool [runTests](#) (str path, float fs)
- dict[str, int] [_readAggregateResults](#) ()
- [run](#) (dict args)
- argparse.ArgumentParser [createParser](#) ()
- bool [main](#) (dict args)

Variables

- str **SEPARATOR** = "."
- str **EXPECTED_DIR** = "in"
- str **TMP_DIR** = "out"
- list [STIMULI](#)
- **FILE_DIR** = os.path.dirname(__file__)

7.4.1 Detailed Description

```
@file ntPlugin.py
@author Niels Thøgersen (niels.thoegersen@gmail.com)
@brief Genarates test vectors for unittest of NTplugin components and analyzes
and plots results.
@version 0.1
```

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <<https://www.gnu.org/licenses/>>.

7.4.2 Function Documentation

7.4.2.1 acceptLatestResult()

```
bool test.acceptLatestResult (
    list[str] files,
    list[str] objects)
```

Approves the latest results that match a specific test program file and objects therein. Both args accept ["all"], which will match all files or all objects within a file.

Args:

- files (list[str]): Test program files. Typically named after class under test.
- objects (list[str]): Names of objects under test.

Returns:

- bool: True on success.

7.4.2.2 clean()

```
bool test.clean ()
```

Cleans all previous test outputs.

Returns:

- bool: True on success.

7.4.2.3 createParser()

```
argparse.ArgumentParser test.createParser ()
```

Creates an argument parser for test program.

Returns:

- argparse.ArgumentParser: New argument parser.

7.4.2.4 generateBurstSine()

```
np.ndarray test.generateBurstSine (  
    float fs,  
    float t,  
    float f = 10e3,  
    int nBursts = 2,  
    float lowLevel = 0.25,  
    float highLevel = 1)
```

Generates a sine wave at a 'f' Hz, whose level steps between 'lowLevel' and 'highLevel'.

Args:

fs (float): Sample rate.
t (float): Length of result in seconds.
f (float, optional): Frequency of result. Defaults to 10e3.
nBursts (int, optional): Number of bursts. Defaults to 2.
lowLevel (float, optional): Low level between bursts. Defaults to 0.25.
highLevel (float, optional): Level during burst. Defaults to 1.

Returns:

np.ndarray: Generated, bursting sine wave.

7.4.2.5 generateImpulse()

```
np.ndarray test.generateImpulse (  
    int n)
```

Generates an impulse stimulus.

Args:

n (int): Length of result

Returns:

np.ndarray: Impulse.

7.4.2.6 generateLinearSweep()

```
np.ndarray test.generateLinearSweep (  
    float fs,  
    float t)
```

Generates a linear sweep.

Args:

fs (float): Sample rate.
t (float): Length in seconds.

Returns:

np.ndarray: Linear sweep.

7.4.2.7 generateSyncSweep()

```
np.ndarray test.generateSyncSweep (  
    float  f_start = 10,  
    float  f_stop = 24e3,  
    float  fs = 48e3,  
    int    k = 2)
```

Generates a synchronized sine sweep.

Args:

f_start (float, optional): Starting frequency. Defaults to 10.
f_stop (float, optional): Ending frequency. Defaults to 24e3.
fs (float, optional): Sample rate. Defaults to 48e3.
k (int, optional): Konstant the length is derived from. Defaults to 2.

Returns:

np.ndarray: Synchronized chirp.

7.4.2.8 generateTestVectors()

```
tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray] test.generateTestVectors (  
    float t,  
    float fs)
```

Generates and stores all the test vectors.

Args:

t (float): Length of results.
fs (float): Sample rate.

Returns:

tuple: The generated test vectors.

7.4.2.9 main()

```
bool test.main (  
    dict args)
```

Main function for test program.

Args:

args (dict): Arguments as passed in from the command line.

Returns:

bool: True on success.

7.4.2.10 plotDynamic()

```
None test.plotDynamic (  
    np.ndarray x,  
    float fs,  
    str filename,  
    list | None legends = None,  
    int zoom = 1)
```

Plots a dynamic response.

Args:

```
x (np.ndarray): Data to plot.  
fs (float): Sample rate.  
filename (str): Output file name.  
legends (list | None, optional): Legends of plots. Defaults to None.  
zoom (int, optional): Zoom level. 1 for whole plot. Defaults to 1.
```

7.4.2.11 plotFrequencyDomain()

```
None test.plotFrequencyDomain (  
    np.ndarray x,  
    float fs,  
    str filename,  
    list | None legends = None,  
    list | None ylim = None,  
    str | None ylabel = None)
```

Plots signal in frequency domain.

Args:

```
x (np.ndarray): Data to plot.  
fs (float): Sample rate.  
filename (str): Output file name.  
legends (list | None, optional): Legends to add to plot. Defaults to None.  
ylim (list | None, optional): Y axis limits. Defaults to None.  
ylabel (str | None, optional): Label for Y axis. Defaults to None.
```

7.4.2.12 plotImpulse()

```
test.plotImpulse (  
    np.ndarray x,  
    float fs,  
    str filename,  
    list | None legends = None)
```

Plots the results of an impulse response an frequency amplitude and phase responses.

Args:

```
x (np.ndarray): Data to plot.  
fs (float): Sample rate.  
filename (str): Name of output file.  
legends (list | None, optional): Legends of plots. Defaults to None.
```

7.4.2.13 plotSpectrum()

```
None test.plotSpectrum (  
    np.ndarray x,  
    float fs,  
    str filename)
```

Plots a spectrum of a sweep result.

Args:

- x* (np.ndarray): Sweep to plot.
- fs* (float): Sample rate.
- filename* (str): Path to store at.

7.4.2.14 plotSweeps()

```
test.plotSweeps (  
    list[np.ndarray] results,  
    dict[str, list[str]] legends,  
    str testFileName,  
    float fs,  
    str objectName)
```

Plots a spectrum for each sweep in 'results'. File name will be generated from legends and objectName.

Args:

- results* (dict[str, list[np.ndarray]]): Data to plot.
- legends* (dict[str, list[str]]): Legends to add.
- testFileName* (str): Name of the test program the data came from.
- fs* (float): Sample rate.
- objectName* (str): Name of the specific test.

7.4.2.15 plotTimeDomain()

```
None test.plotTimeDomain (  
    np.ndarray x,  
    float fs,  
    str filename,  
    list | None legends = None,  
    list | None ylim = None,  
    str | None ylabel = None)
```

Plots signal in time domain.

Args:

- x* (np.ndarray): Data to plot.
- fs* (float): Sample rate.
- filename* (str): Output file name.
- legends* (list | None, optional): Legends to add to plot. Defaults to None.
- ylim* (list | None, optional): Y axis limits. Defaults to None.
- ylabel* (str | None, optional): Label for Y axis. Defaults to None.

7.4.2.16 run()

```
test.run (  
    dict args)
```

Runs test application.

Args:
args (dict): Used to get 'files' and 'fs' args from command line.

Returns:
bool: True on success.

7.4.2.17 runTests()

```
bool test.runTests (  
    str path,  
    float fs)
```

Runs tests for a specific test program. Build the program, runs it and collects results.

Args:
path (str): _description_
fs (float): _description_

Returns:
bool: _description_

7.4.3 Variable Documentation

7.4.3.1 STIMULI

```
list test.STIMULI
```

Initial value:

```
00001 = [  
00002     "impulse",  
00003     "linearSweep",  
00004     "syncSweep",  
00005     "dynamic_alternating",  
00006     "dynamic_matched",  
00007 ]
```

Chapter 8

Class Documentation

8.1 NtFx::Biquad::Biquad6 Struct Reference

Public Member Functions

- **Biquad6** ([Coeffs6](#) &coeffs)
- `signal_t process` (`signal_t x`)

Public Attributes

- [Coeffs6](#) & **coeffs**
- [State](#) **state**

The documentation for this struct was generated from the following file:

- `lib/Biquad.h`

8.2 NtFx::Src::Coeffs Struct Reference

Coefficients structure for sample rate converter.

```
#include <SampleRateConverter.h>
```

Public Attributes

- `bool disable` = false
Flag to disable sample rate conversion.
- `size_t osFactor` = 1
Oversampling factor.
- `size_t osFirLenMult` = 12
FIR filter length multiplier.
- `size_t n` = 12
FIR filter length.
- `signal_t fsHi` = 48000
High sampling rate.
- `std::array< signal_t, nDelayLine > b`
FIR filter coefficients.

8.2.1 Detailed Description

Coefficients structure for sample rate converter.

Template Parameters

<i>signal</i> ↔ _t	Type of signal samples
-----------------------	------------------------

The documentation for this struct was generated from the following file:

- lib/[SampleRateConverter.h](#)

8.3 NtFx::Biquad::Coeffs5 Struct Reference

Public Attributes

- signal_t **b** [3] { 1, 0, 0 }
- signal_t **a** [2] { 0, 0 }

The documentation for this struct was generated from the following file:

- lib/Biquad.h

8.4 NtFx::Biquad::Coeffs6 Struct Reference

Public Attributes

- signal_t **b** [3] { 1, 0, 0 }
- signal_t **a** [3] { 1, 0, 0 }

The documentation for this struct was generated from the following file:

- lib/Biquad.h

8.5 NtFx::ComponentBase< T > Struct Template Reference

Base class for audio components in [NtPlugin](#). All processors adhere to this interface.

```
#include <Component.h>
```

Public Member Functions

- virtual T [process](#) (T x) noexcept=0
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float fs) noexcept
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- float **fs** { 48e3 }
Sample rate of component.

8.5.1 Detailed Description

template<typename T>
struct NtFx::ComponentBase< T >

Base class for audio components in [NtPlugin](#). All processors adhere to this interface.

Template Parameters

<i>signal</i> ↔ _t	Datatype for audio signals.
-----------------------	-----------------------------

8.5.2 Member Function Documentation

8.5.2.1 process()

```
template<typename T>
virtual T NtFx::ComponentBase< T >::process (
    T x) [pure virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

x	T Input
---	---------

Returns

T output.

Implemented in [gainExample](#), [ntCompressor](#), [ntDeEsser](#), [ntFilters](#), [NtFx::Biquad::EqBand6Mono](#), [NtFx::Biquad::EqBand6Stereo](#), [NtFx::Biquad::EqBandMono](#), [NtFx::Biquad::EqBandStereo](#), [NtFx::Comp::PeakSideChainDb](#), [NtFx::Comp::PeakSideChainLinear](#), [NtFx::Comp::RmsSideChainDb](#), [NtFx::Comp::RmsSideChainLinear](#), [NtFx::DynamicFilter::Shelf](#), [NtFx::FirstOrder::Filter< shape >](#), [NtFx::FirstOrder::StereoFilter< shape >](#), [NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::hpf >](#), [NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::lpfZero >](#), [NtFx::Gate::Sc](#), [NtFx::Generator::Noise](#), [NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >](#), [NtFx::MonoComponent< signal_t, component_t >](#), [NtFx::PeakHoldSensor< delayLineLength >](#), [NtFx::PeakSensor](#), [NtFx::StereoComponent< signal_t, component_t >](#), [NtFx::Transformer](#), [ntGate](#), [ntMultiband3](#), [ntRmsMeter](#), [ntTapeEcho](#), and [ntTransformer](#).

8.5.2.2 reset()

```
template<typename T>
virtual void NtFx::ComponentBase< T >::reset (
    float fs) [inline], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented in [gainExample](#), [ntCompressor](#), [ntDeEsser](#), [ntFilters](#), [NtFx::Biquad::EqBand6Mono](#), [NtFx::Biquad::EqBand6Stereo](#), [NtFx::Biquad::EqBandMono](#), [NtFx::Biquad::EqBandStereo](#), [NtFx::Comp::PeakSideChainDb](#), [NtFx::Comp::RmsSideChainDb](#), [NtFx::Comp::RmsSideChainLinear](#), [NtFx::DynamicFilter::Shelf](#), [NtFx::FirstOrder::Filter< shape >](#), [NtFx::FirstOrder::StereoFilter< shape >](#), [NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::hpf >](#), [NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::lpf >](#), [NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::lpfZero >](#), [NtFx::Gate::Sc](#), [NtFx::LongRmsSensorMono< signal_t, maxT_m >](#), [NtFx::MonoComponent< signal_t, component_t >](#), [NtFx::PeakSensor](#), [NtFx::StereoComponent< signal_t, component_t >](#), [NtFx::Transformer](#), [ntGate](#), [ntMultiband3](#), [ntRmsMeter](#), [ntTapeEcho](#), and [ntTransformer](#).

8.5.2.3 update()

```
template<typename T>
virtual void NtFx::ComponentBase< T >::update () [inline], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

Reimplemented in [gainExample](#), [ntCompressor](#), [ntDeEsser](#), [ntFilters](#), [NtFx::Biquad::EqBand6Mono](#), [NtFx::Biquad::EqBand6Stereo](#), [NtFx::Biquad::EqBandMono](#), [NtFx::Biquad::EqBandStereo](#), [NtFx::Comp::PeakSideChainDb](#), [NtFx::Comp::PeakSideChainLinear](#), [NtFx::Comp::RmsSideChainDb](#), [NtFx::Comp::RmsSideChainLinear](#), [NtFx::DynamicFilter::Shelf](#), [NtFx::FirstOrder::Filter< shape >](#), [NtFx::FirstOrder::StereoFilter< shape >](#), [NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::hpf >](#), [NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::lpfZero >](#), [NtFx::Gate::Sc](#), [NtFx::LongRmsSensorMono< signal_t, maxT_m >](#), [NtFx::MonoComponent< signal_t, component_t >](#), [NtFx::PeakHoldSensor< delayLineLength >](#), [NtFx::PeakSensor](#), [NtFx::StereoComponent< signal_t, component_t >](#), [NtFx::Transformer](#), [ntGate](#), [ntMultiband3](#), [ntRmsMeter](#), [ntTapeEcho](#), and [ntTransformer](#).

The documentation for this struct was generated from the following file:

- [lib/Component.h](#)

8.6 NtFx::ComponentTest Struct Reference

Processes a vector of test data with audio component and compares to expected results. Total number of tests and number of succesful test are stored as statics in order to make statistics.

```
#include <ComponentTest.h>
```

Public Member Functions

- [ComponentTest](#) ([ComponentTestSet](#) &[owner](#), std::string [objName](#), [ComponentBase](#)< [Stereo](#)< [signal_t](#) > > &[cut](#), std::vector< std::string > [stimuli](#)={ })
Construct a new Component Test object.
- bool [run](#) (std::string [stimulus](#))
Processes the object under test with 'stimulus', writes out results to a file and compares to expected results if found.
- std::vector< [Stereo](#)< [signal_t](#) > > [_readInput](#) (std::string [stimulus](#))
- bool [_stimulusIsActive](#) (std::string [stimulus](#))
- std::vector< [Stereo](#)< [signal_t](#) > > [_readExpected](#) (std::string [stimulus](#))
- bool [_compareExpected](#) (std::string [stimulus](#), const std::vector< [Stereo](#)< [signal_t](#) > > &[y](#))

Public Attributes

- [ComponentTestSet](#) & **owner**
Set this test belongs to.
- const std::string **objName**
Object to be tested.
- [ComponentBase](#)< [Stereo](#)< signal_t > > & **cut**
Component under test.
- std::vector< std::string > **activeStimuli** = STIMULI_NAMES
Tests to run.

8.6.1 Detailed Description

Processes a vector of test data with audio component and compares to expected results. Total number of tests and number of succesful test are stored as statics in order to make statistics.

8.6.2 Constructor & Destructor Documentation

8.6.2.1 ComponentTest()

```
NtFx::ComponentTest::ComponentTest (
    ComponentTestSet & owner,
    std::string objName,
    ComponentBase< Stereo< signal_t > > & cut,
    std::vector< std::string > stimuli = { }) [inline]
```

Construct a new Component Test object.

Parameters

<i>owner</i>	Test set this test belongs to.
<i>objName</i>	A name for the current object under test.
<i>cut</i>	Reference to the object under test.
<i>stimuli</i>	Vector of strings naming stimuli to test object against. Accepted values are defined in STIMULI_NAMES. Defaults to an empty vector, selecting all knows stimuli as defined in STIMULI_NAMES.

8.6.3 Member Function Documentation

8.6.3.1 run()

```
bool NtFx::ComponentTest::run (
    std::string stimulus) [inline]
```

Processes the object under test with 'stimulus', writes out results to a file and compares to expected results if found.

Parameters

<i>stimulus</i>	Name of stimulus to process with object under test.
-----------------	---

Returns

true
false

The documentation for this struct was generated from the following file:

- lib/[ComponentTest.h](#)

8.7 NtFx::ComponentTestSet Struct Reference

Set of a number of tests. Represents a full test file and this a test of a Component in the library or an [NtPlugin](#).

```
#include <ComponentTest.h>
```

Public Member Functions

- [ComponentTestSet](#) (std::string [name](#))
Construct a new Component Test Set object.
- bool [getResults](#) ()
Get and print the results of all tests.
- bool [addTest](#) ([ComponentBase](#)< [Stereo](#)< signal_t > > &componentObj, std::string objName, std::vector< std::string > stimuli)
Adds a new test to set.
- int [runAllTests](#) ()
Runs all tests added to test set.

Public Attributes

- int **nTests**
Total tests run.
- int **nSuccessful**
Number of successful tests.
- std::vector< std::unique_ptr< [ComponentTest](#) > > **tests**
Vector of tests to run.
- std::string **name**
Name of set of tests.

8.7.1 Detailed Description

Set of a number of tests. Represents a full test file and this a test of a Component in the library or an [NtPlugin](#).

Template Parameters

<i>signal</i> ↔ _t	Audio datatype
-----------------------	----------------

8.7.2 Constructor & Destructor Documentation

8.7.2.1 ComponentTestSet()

```
NtFx::ComponentTestSet::ComponentTestSet (
    std::string name) [inline]
```

Construct a new Component Test Set object.

Parameters

<i>name</i>	Name of component under test.
-------------	-------------------------------

8.7.3 Member Function Documentation

8.7.3.1 addTest()

```
bool NtFx::ComponentTestSet::addTest (
    ComponentBase< Stereo< signal_t > > & componentObj,
    std::string objName,
    std::vector< std::string > stimuli)
```

Adds a new test to set.

Parameters

<i>componentObj</i>	Reference to object under test.
<i>objName</i>	Name of object under test.
<i>stimuli</i>	Vector of stimuli to test against.

Returns

true if tests pass.
false if tests fail.

8.7.3.2 getResults()

```
bool NtFx::ComponentTestSet::getResults () [inline]
```

Get and print the results of all tests.

Returns

true If all tests passed.
false If any test failed. Missing expected vector is a failure.

8.7.3.3 runAllTests()

```
int NtFx::ComponentTestSet::runAllTests ()
```

Runs all tests added to test set.

Returns

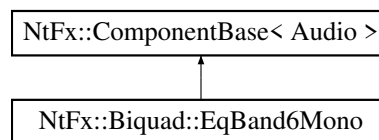
int 0 on success, 1 on failure.

The documentation for this struct was generated from the following file:

- lib/[ComponentTest.h](#)

8.8 NtFx::Biquad::EqBand6Mono Struct Reference

Inheritance diagram for NtFx::Biquad::EqBand6Mono:



Public Member Functions

- virtual [Audio](#) [process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- [Biquad6](#) l
- [Settings](#) settings
- [Coeffs6](#) coeffs

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float [fs](#)
Sample rate of component.

8.8.1 Member Function Documentation

8.8.1.1 process()

```
virtual Audio NtFx::Biquad::EqBand6Mono::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<i>x</i>	T Input
----------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.8.1.2 reset()

```
virtual void NtFx::Biquad::EqBand6Mono::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.8.1.3 update()

```
virtual void NtFx::Biquad::EqBand6Mono::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

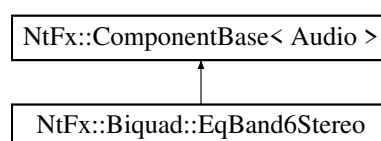
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- lib/Biquad.h

8.9 NtFx::Biquad::EqBand6Stereo Struct Reference

Inheritance diagram for NtFx::Biquad::EqBand6Stereo:



Public Member Functions

- virtual [Audio process](#) ([Audio x](#)) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- [Biquad6 l](#)
- [Biquad6 r](#)
- [Settings settings](#)
- [Coeffs6 coeffs](#)

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float [fs](#)
Sample rate of component.

8.9.1 Member Function Documentation

8.9.1.1 process()

```
virtual Audio NtFx::Biquad::EqBand6Stereo::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

x	T Input
-------------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.9.1.2 reset()

```
virtual void NtFx::Biquad::EqBand6Stereo::reset (  
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.9.1.3 update()

```
virtual void NtFx::Biquad::EqBand6Stereo::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

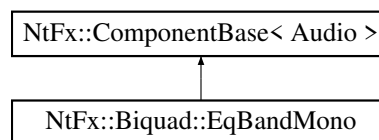
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- lib/Biquad.h

8.10 NtFx::Biquad::EqBandMono Struct Reference

Inheritance diagram for NtFx::Biquad::EqBandMono:



Public Member Functions

- virtual [Audio process](#) ([Audio x](#)) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- [Settings settings](#)
- [Coeffs5 coeffs](#)
- [State state](#)

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float [fs](#)
Sample rate of component.

8.10.1 Member Function Documentation

8.10.1.1 process()

```
virtual Audio NtFx::Biquad::EqBandMono::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

x	T Input
-----	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.10.1.2 reset()

```
virtual void NtFx::Biquad::EqBandMono::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

fs	Sample rate. If you need it for calculateCoeffs, store it.
------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.10.1.3 update()

```
virtual void NtFx::Biquad::EqBandMono::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

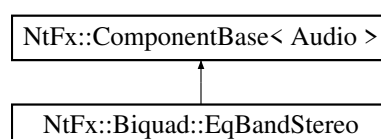
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- lib/Biquad.h

8.11 NtFx::Biquad::EqBandStereo Struct Reference

Inheritance diagram for NtFx::Biquad::EqBandStereo:



Public Member Functions

- virtual [Audio process](#) ([Audio x](#)) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- [Settings settings](#)
- [Coeffs5 coeffs](#)
- [State stateL](#)
- [State stateR](#)

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float [fs](#)
Sample rate of component.

8.11.1 Member Function Documentation

8.11.1.1 process()

```
virtual Audio NtFx::Biquad::EqBandStereo::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

x	T Input
-------------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.11.1.2 reset()

```
virtual void NtFx::Biquad::EqBandStereo::reset (  
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.11.1.3 update()

```
virtual void NtFx::Biquad::EqBandStereo::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- lib/Biquad.h

8.12 NtFx::ExpGlider Struct Reference

Public Member Functions

- [ExpGlider](#) (signal_t def=0)
Construct a new Exp Glider object.
- signal_t [process](#) () noexcept
Processes glider. To be called every sample.
- void [update](#) (signal_t fs, signal_t tSmooth) noexcept
Update the glider coefficient.

Public Attributes

- signal_t **ui** = 0
UI side input value.
- signal_t **pr** = 0
Processing side output of glider.
- signal_t **_a** = 0

8.12.1 Constructor & Destructor Documentation

8.12.1.1 ExpGlider()

```
NtFx::ExpGlider::ExpGlider (
    signal_t def = 0) [inline]
```

Construct a new Exp Glider object.

Parameters

<i>def</i>	Default value the glider should start at.
------------	---

8.12.2 Member Function Documentation

8.12.2.1 process()

```
signal_t NtFx::ExpGlider::process () [inline], [noexcept]
```

Processes glider. To be called every sample.

Returns

signal_t Audio datatype.

8.12.2.2 update()

```
void NtFx::ExpGlider::update (
    signal_t fs,
    signal_t tSmooth) [inline], [noexcept]
```

Update the glider coefficient.

Parameters

<i>fs</i>	Sample rate
<i>tSmooth</i>	Smoothing time constant.

The documentation for this struct was generated from the following file:

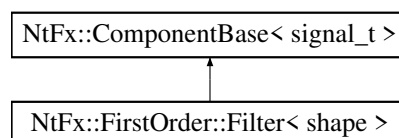
- lib/[Glider.h](#)

8.13 NtFx::FirstOrder::Filter< shape > Struct Template Reference

A single channel first order filter.

```
#include <FirstOrder.h>
```

Inheritance diagram for NtFx::FirstOrder::Filter< shape >:



Public Member Functions

- virtual `signal_t process` (`signal_t x`) noexcept override
Called for every sample as audio is processed.
- virtual void `update` () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void `reset` (`float fs`) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- `signal_t fc_hz` = 1000
- `signal_t _a` = 0
- `signal_t _yn1` = 0
- `signal_t _xn1` = 0

Public Attributes inherited from `NtFx::ComponentBase< signal_t >`

- `float fs`
Sample rate of component.

8.13.1 Detailed Description

```
template<Shape shape>
struct NtFx::FirstOrder::Filter< shape >
```

A single channel first order filter.

Template Parameters

<code>signal_t</code>	Audio datatype.
<code>shape</code>	Type of filter.

8.13.2 Member Function Documentation

8.13.2.1 `process()`

```
template<Shape shape>
virtual signal_t NtFx::FirstOrder::Filter< shape >::process (
    signal_t x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements `NtFx::ComponentBase< signal_t >`.

8.13.2.2 reset()

```
template<Shape shape>
virtual void NtFx::FirstOrder::Filter< shape >::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< signal_t >](#).

8.13.2.3 update()

```
template<Shape shape>
virtual void NtFx::FirstOrder::Filter< shape >::update () [inline], [override], [virtual],
[noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

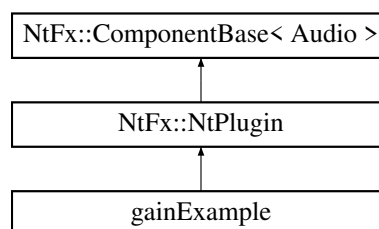
Reimplemented from [NtFx::ComponentBase< signal_t >](#).

The documentation for this struct was generated from the following file:

- lib/[FirstOrder.h](#)

8.14 gainExample Struct Reference

Inheritance diagram for gainExample:



Public Member Functions

- [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Member Functions inherited from `NtFx::NtPlugin`

- virtual void `onTempoChanged` () noexcept
Called by the wrapper whenever the tempo changes.
- signal_t * `getKnobValuePtr` (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- bool * `getToggleValuePtr` (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- int * `getDropDownValuePtr` (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void `setParameterActive` (std::string name, bool val)
- void `deactivateParameter` (std::string name)
- void `activateParameter` (std::string name)
- constexpr void `updateDefaults` () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggles. Should always be called as the last thing in the constructor of the derived plugin class.
- template<size_t idx, bool invert = false>
`Audio updatePeakLevel` (Audio val) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- `Audio getAndResetPeakLevel` (size_t idx) noexcept
Get and reset peak level for specified meter.
- `Audio getRms` (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Attributes

- signal_t `gain_db` { 0 }
- signal_t `gain_lin` { 1 }

Public Attributes inherited from `NtFx::NtPlugin`

- std::array< `LongRmsSensor`< 250, 192 >, 2 > `xRms`
Array of rms sensors for RMS meter.
- std::array< `Audio`, `nMetersMax` > `peakLevels`
Peak level to be displayed in the meters.
- `UiSpec` `uiSpec`
Specification for UI. Modify this to change the look of your plugin.
- std::vector< `KnobSpec` > `primaryKnobs`
vector of primary knobs. Add your number paramters to this to display them in the UI.
- std::vector< `KnobSpec` > `secondaryKnobs`
vector of secondary knobs.
- std::vector< `DropDownSpec` > `dropdowns`
vector of DropDowns to be displayed
- std::vector< `ToggleSpec` > `toggles`
vector of toggles to be displayed at the bottom of the UI.
- std::vector< `ToggleSetSpec` > `toggleSets`
vector of groups of toggles to be placed in the right side area.
- std::vector< `RadioButtonSetSpec` > `radioButtons`
vector of radioButtons to be displayed at the right of the UI.
- std::vector< `MeterSpec` > `meters`

List of all meters to be displayed in the UI.

- `signal_t tempo` = 0
Session tempo. Automatically updated by wrapper.
- `signal_t xSc`
Updated at base sample rate and can be used in [process\(\)](#) for external side chain input.
- `bool uiNeedsUpdate` = false
Set this to true if you want to force an update of the UI.
- `bool isInitialized` = false
True if plugin is fully initialized. Must be set by child plugin.

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- `float fs`
Sample rate of component.

8.14.1 Member Function Documentation

8.14.1.1 `process()`

```
Audio gainExample::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.14.1.2 `reset()`

```
void gainExample::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<code>fs</code>	Sample rate. If you need it for calculateCoeffs, store it.
-----------------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.14.1.3 update()

```
void gainExample::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

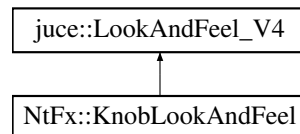
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- plugins/gainExample.h

8.15 NtFx::KnobLookAndFeel Struct Reference

Inheritance diagram for NtFx::KnobLookAndFeel:



Public Member Functions

- juce::Font **getLabelFont** (juce::Label &l) override
- void **drawRotarySlider** (juce::Graphics &g, int x, int y, int width, int height, float sliderPos, const float rotaryStartAngle, const float rotaryEndAngle, juce::Slider &slider) override

Public Attributes

- float **fontSize** = 1
- float **uiScale** = 1
- uint32_t **backgroundColour** { 0xFFD3D3D3 }
- uint32_t **foregroundColour** { 0xFF000000 }

The documentation for this struct was generated from the following file:

- JuceWrapper/LookAndFeel.h

8.16 NtFx::KnobSpec Struct Reference

Specification for a knob.

```
#include <UiSpec.h>
```

Public Member Functions

- void [setLogScale](#) ()

Public Attributes

- signal_t * **p_val** { nullptr }
Pointer to value the knob represents.
- std::string **name** { "" }
- std::string **suffix** { "" }
Added to the end of value label for knob.
- signal_t **minVal** { 0.0 }
Starting level of knob.
- signal_t **maxVal** { 1.0 }
End level of knob.
- bool **logScale** { false }
Call setLogScale at construction.
- bool **isActive** { true }
Gray out knob and make it unresponsive.
- signal_t **midPoint** { 0.0 }
Sets the middle of the knob. 0 for don't care.
- signal_t **_defaultVal**
Default value. Set by updateDefaults().

8.16.1 Detailed Description

Specification for a knob.

Template Parameters

<i>signal</i> ↔	
_t	

8.16.2 Member Function Documentation

8.16.2.1 setLogScale()

```
void NtFx::KnobSpec::setLogScale () [inline]
```

< Sets midPoint for logarithmic scale.

8.16.3 Member Data Documentation

8.16.3.1 name

```
std::string NtFx::KnobSpec::name { "" }
```

Name of knob, used for IDs and label in UI. Can't contain ' '. '_' is replaced with ' ' in UI labels.

The documentation for this struct was generated from the following file:

- lib/[UiSpec.h](#)

8.17 NtFx::LinGlider Struct Reference

Glides member 'pr' linearly towards 'ui'.

```
#include <Glider.h>
```

Public Member Functions

- [LinGlider](#) (signal_t def=0)
Construct a new Lin Glider object.
- signal_t [process](#) () noexcept
Processes glider. Should be called every sample.
- void [update](#) (signal_t fs, signal_t tSmooth)
Updates glider coeff.

Public Attributes

- signal_t **ui** = 0.0
- signal_t **pr** = 0.0
- signal_t **s** = 0.0

8.17.1 Detailed Description

Glides member 'pr' linearly towards 'ui'.

Template Parameters

<i>signal</i> ↔	
<i>_t</i>	

8.17.2 Constructor & Destructor Documentation

8.17.2.1 LinGlider()

```
NtFx::LinGlider::LinGlider (
    signal_t def = 0) [inline]
```

Construct a new Lin Glider object.

Parameters

<i>def</i>	Default value to start glider at.
------------	-----------------------------------

8.17.3 Member Function Documentation

8.17.3.1 process()

```
signal_t NtFx::LinGlider::process () [inline], [noexcept]
```

Processes glider. Should be called every sample.

Returns

signal_t Audio datatype.

8.17.3.2 update()

```
void NtFx::LinGlider::update (
    signal_t fs,
    signal_t tSmooth) [inline]
```

Updates glider coeff.

Parameters

<i>fs</i>	Sample rate.
<i>tSmooth</i>	Time to glide 'pr' to 'ui'.

The documentation for this struct was generated from the following file:

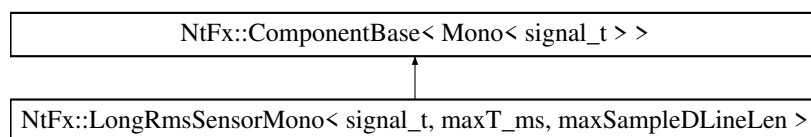
- lib/[Glider.h](#)

8.18 NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen > Struct Template Reference

RMS (Root Mean Square) sensor component for audio signal processing.

```
#include <RmsSensor.h>
```

Inheritance diagram for NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >:



Public Member Functions

- virtual [Mono](#)< [signal_t](#) > [process](#) ([Mono](#)< [signal_t](#) > x) noexcept override
Process the input signal and update RMS calculation.
- void [processDelayLine](#) ([signal_t](#) x) noexcept
Processes accumulators and update delay lines without calculating result. To be called each sample if 'process' is not used. This way, you can update accumulators in the processing thread while calculating rms values somewhere else.
- virtual void [update](#) () noexcept override
Update the component state.
- virtual void [reset](#) (float [fs](#)) noexcept override
Reset the component with a new sample rate.
- [signal_t](#) [getRms](#) () const noexcept
Get the current RMS value.
- void [setT_ms](#) (int t_ms)
Set the time window for RMS calculation.

Public Attributes

- [std::array](#)< [signal_t](#), [maxSampleDLineLen](#) > **[sampleDLine](#)**
Sample delay line.
- [std::array](#)< [signal_t](#), [maxT_ms](#) > **[msDLine](#)**
Millisecond delay line.
- [signal_t](#) **[sampleAccum](#)** { 0 }
Accumulator for current sample values.
- [signal_t](#) **[msAccum](#)** { 0 }
Accumulator for millisecond-level values.
- int **[sampleIdx](#)** { 0 }
Current index in the sample delay line.
- int **[msIdx](#)** { 0 }
Current index in the millisecond delay line.
- bool **[resetAccums](#)** { false }
Flag to reset accumulators.
- int **[msDLineLen](#)** { [maxT_ms](#) }
Current time window in milliseconds.
- int **[sampleDLineLen](#)** { 48 }
Length of sample delay line.

Public Attributes inherited from [NtFx::ComponentBase](#)< [Mono](#)< [signal_t](#) > >

- float **[fs](#)**
Sample rate of component.

8.18.1 Detailed Description

```
template<typename signal\_t, int maxT\_ms = 1000, int maxSampleDLineLen = 192 * 8>
struct NtFx::LongRmsSensorMono< signal\_t, maxT\_ms, maxSampleDLineLen >
```

RMS (Root Mean Square) sensor component for audio signal processing.

This component calculates the RMS value of an audio signal over a specified time period. It maintains two delay lines: one for sample-level accumulation and one for millisecond-level accumulation, allowing for efficient RMS calculation.

Template Parameters

<i>signal_t</i>	The type of the audio signal (e.g., float, double)
<i>maxT_ms</i>	Maximum time window in milliseconds for RMS calculation (default: 1000)
<i>maxSampleDLineLen</i>	Maximum length of the sample delay line (default: 192 * 8)

8.18.2 Member Function Documentation

8.18.2.1 getRms()

```
template<typename signal_t, int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
signal_t NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >::getRms () const
[inline], [noexcept]
```

Get the current RMS value.

This method calculates and returns the current RMS value based on the accumulated signal values.

Returns

The current RMS value

8.18.2.2 process()

```
template<typename signal_t, int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
virtual Mono< signal_t > NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >::process (
    Mono< signal_t > x) [inline], [override], [virtual], [noexcept]
```

Process the input signal and update RMS calculation.

This method processes the input signal, calculates its square value, and updates the RMS calculation using a delay line approach.

Parameters

<i>x</i>	The input signal value
----------	------------------------

Returns

The current RMS value

Implements [NtFx::ComponentBase< Mono< signal_t > >](#).

8.18.2.3 processDelayLine()

```
template<typename signal_t, int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
void NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >::processDelayLine (
    signal_t x) [inline], [noexcept]
```

Processes accumulators and update delay lines without calculating result. To be called each sample if 'process' is not used. This way, you can update accumulators in the processing thread while calculating rms values somewhere else.

Parameters

<i>x</i>	Inout signal to calculate RMS level for.
----------	--

8.18.2.4 reset()

```
template<typename signal_t, int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
virtual void NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Reset the component with a new sample rate.

This method updates the sample rate and recalculates the sample delay line length based on the new rate.

Parameters

<i>fs</i>	The new sample rate in Hz
-----------	---------------------------

Reimplemented from [NtFx::ComponentBase< Mono< signal_t > >](#).

8.18.2.5 setT_ms()

```
template<typename signal_t, int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
void NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >::setT_ms (
    int t_ms) [inline]
```

Set the time window for RMS calculation.

This method updates the time window in milliseconds and resets the accumulators.

Parameters

<i>t_ms</i>	The new time window in milliseconds
-------------	-------------------------------------

8.18.2.6 update()

```
template<typename signal_t, int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
virtual void NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >::update () [inline],
[override], [virtual], [noexcept]
```

Update the component state.

This method resets the accumulators and delay lines if the resetAccums flag is set.

Reimplemented from [NtFx::ComponentBase< Mono< signal_t > >](#).

The documentation for this struct was generated from the following file:

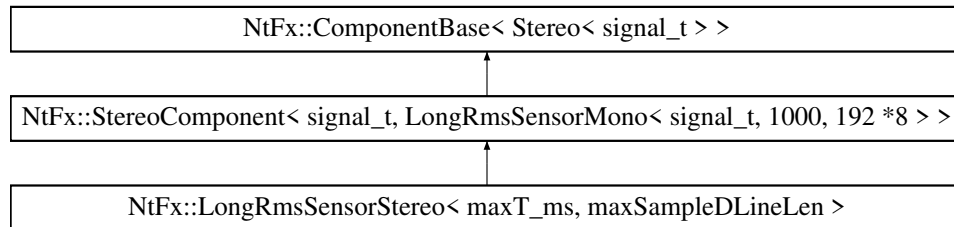
- [lib/RmsSensor.h](#)

8.19 NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen > Struct Template Reference

[Stereo](#) RMS sensor component.

```
#include <RmsSensor.h>
```

Inheritance diagram for NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >:



Public Member Functions

- void [setT_ms](#) (int t_ms)
Set the time window for RMS calculation.
- [Audio getRms](#) () const noexcept
Get the current RMS values for both channels.

Public Member Functions inherited from

[NtFx::StereoComponent< signal_t, component_t >](#)

- virtual [Stereo< signal_t > process](#) ([Stereo< signal_t > x](#)) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float fs) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Additional Inherited Members

Public Attributes inherited from [NtFx::StereoComponent< signal_t, component_t >](#)

- component_t l
- component_t r

Public Attributes inherited from [NtFx::ComponentBase< Stereo< signal_t > >](#)

- float fs
Sample rate of component.

8.19.1 Detailed Description

```
template<int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
struct NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >
```

[Stereo](#) RMS sensor component.

This component provides RMS calculation for stereo audio signals by maintaining two RmsSensor instances (one for each channel).

Template Parameters

<i>signal_t</i>	The type of the audio signal (e.g., float, double)
<i>maxT_ms</i>	Maximum time window in milliseconds for RMS calculation (default: 1000)
<i>maxSampleDLineLen</i>	Maximum length of the sample delay line (default: 192 * 8)

8.19.2 Member Function Documentation

8.19.2.1 getRms()

```
template<int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
Audio NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >::getRms () const [inline],
[noexcept]
```

Get the current RMS values for both channels.

This method returns the RMS values for both left and right channels as a [Stereo](#) structure.

Returns

The current RMS values for both channels

8.19.2.2 setT_ms()

```
template<int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
void NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >::setT_ms (
    int t_ms) [inline]
```

Set the time window for RMS calculation.

This method updates the time window for both left and right channels.

Parameters

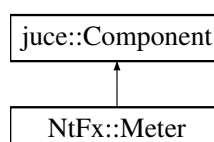
<i>t_ms</i>	The new time window in milliseconds
-------------	-------------------------------------

The documentation for this struct was generated from the following file:

- lib/[RmsSensor.h](#)

8.20 NtFx::Meter Struct Reference

Inheritance diagram for NtFx::Meter:



Public Member Functions

- **Meter** ([MeterSpec](#) &meterSpec, [UiSpec](#) &uiSpec)
- void **paint** (juce::Graphics &g) override
- void **resized** () override
- void **refresh** (float xPeak, float xRms)
- void **refresh** (bool repaint=true)
- void **updateRelease** (float fs, float t_ms, float tHold_ms=100)
- int **calcActiveDots** (float peak_db)
- void **refreshPeakHold** (float peak_db)

Public Attributes

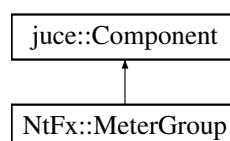
- [PeakHoldSensor](#) **peakSensor**
- [MeterSpec](#) & **meterSpec**
- [UiSpec](#) & **uiSpec**
- std::string **label** { "" }
- int **pad** { 10 }
- int **dotDiameter** { 0 }
- int **dotDist** { 0 }
- int **nDots** { 14 }
- int **nActiveDotsPeak** { 0 }
- float **fractPeak** { 0 }
- int **nActiveDotsRms** { 0 }
- float **peakVal_lin** { 0 }
- float **rmsVal_lin** { 0 }
- float **dbPrDot** { 0 }
- float **opacity** { 0.7f }
- bool **isInitialized** { false }
- int **fontSize** { 20 }
- int **nHold_frames** { 0 }
- int **holdCounter_frames** { 0 }
- float **holdVal_db** { 0 }
- int **iHoldDot** { 0 }
- bool **hasScale** { false }

The documentation for this struct was generated from the following file:

- JuceWrapper/Meter.h

8.21 NtFx::MeterGroup Struct Reference

Inheritance diagram for NtFx::MeterGroup:



Public Member Functions

- void **resized** () override
- virtual void **setFontSize** (int size)
- **MeterGroup** ([UiSpec](#) &uiSpec, std::vector< [MeterSpec](#) > &meterSpecs)
- void **refresh** (size_t idx, [Audio](#) xPeak, [Audio](#) xRms)
- void **updateUi** () noexcept
- void **setUiScale** (float uiScale)
- void **setOnlyShowLeft** (bool onlyShowLeft)
- void **updateRelease** (float fs)
- float **getMinimalWidth** () const noexcept
- float **getMinimalHeight** () const noexcept

Public Attributes

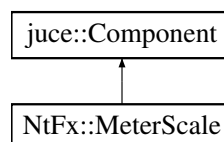
- std::vector< std::unique_ptr< [StereoMeter](#) > > **meters**
- std::vector< std::unique_ptr< [MeterScale](#) > > **scales**
- int **nChs** { 2 }

The documentation for this struct was generated from the following file:

- JuceWrapper/Meter.h

8.22 NtFx::MeterScale Struct Reference

Inheritance diagram for NtFx::MeterScale:



Public Member Functions

- **MeterScale** ([Meter](#) &m)
- void **paint** (juce::Graphics &g) override
- void **resized** () override

Public Attributes

- [Meter](#) & **meter**

The documentation for this struct was generated from the following file:

- JuceWrapper/Meter.h

8.23 NtFx::MeterSpec Struct Reference

Specification for meters.

```
#include <UiSpec.h>
```

Public Attributes

- `std::string name { "" }`
Name of meter, used for label and id.
- `float maxVal_db { 0.0 }`
Top value of meter.
- `float minVal_db { -36.0 }`
Bottom value of meter.
- `bool invert { false }`
When true, the meter lights up top down.
- `bool hasScale { false }`
Adds a text scale to the right of the meter.
- `float decay_s { 0.25 }`
Decay of meter fall off in seconds.
- `float hold_s { 2 }`
[Meter](#) Hold time in seconds.
- `bool addRms = false`

8.23.1 Detailed Description

Specification for meters.

8.23.2 Member Data Documentation

8.23.2.1 addRms

```
bool NtFx::MeterSpec::addRms = false
```

Adds RMS to meter. Currently works for the first two meters. RMS of input will be added to meter 0 and RMS of output will be added to meter 1.

The documentation for this struct was generated from the following file:

- [lib/UiSpec.h](#)

8.24 NtFx::Mono< signal_t > Struct Template Reference

Public Member Functions

- **Mono** (signal_t v)
- **Mono** (signal_t l, signal_t r)
- **Mono** ([Stereo](#)< signal_t > x)
- [Mono](#)< signal_t > & **operator=** (const [Mono](#)< signal_t > &x) noexcept
- [Mono](#)< signal_t > & **operator=** (const signal_t &x) noexcept
- bool **operator==** (const [Mono](#)< signal_t > &x) const noexcept
- bool **operator<** (const [Mono](#)< signal_t > &x) const noexcept
- bool **operator>** (const [Mono](#)< signal_t > &x) const noexcept
- bool **operator<=** (const [Mono](#)< signal_t > &x) const noexcept
- bool **operator>=** (const [Mono](#)< signal_t > &x) const noexcept
- bool **operator<** (const signal_t &x) const noexcept
- bool **operator>** (const signal_t &x) const noexcept
- bool **operator<=** (const signal_t &x) const noexcept
- bool **operator>=** (const signal_t &x) const noexcept
- [Mono](#)< signal_t > & **operator*=** (const signal_t x) noexcept
- [Mono](#)< signal_t > & **operator*=** (const [Mono](#)< signal_t > x) noexcept
- [Mono](#)< signal_t > & **operator+=** (const signal_t x) noexcept
- [Mono](#)< signal_t > & **operator+=** (const [Mono](#)< signal_t > x) noexcept
- [Mono](#)< signal_t > **operator-** () const
- signal_t [avgSquared](#) () const noexcept
- Square left and right channels and take the average.*
- signal_t [absMax](#) () const noexcept
- Returns the larges of left and right disregarding sign.*
- signal_t [absMin](#) () const noexcept
- Returns the smallest of left and right disregarding sign.*
- [Mono](#)< signal_t > [abs](#) () const noexcept
- Returns the abs of both channels.*

Public Attributes

- signal_t l { 0 }

8.24.1 Member Function Documentation

8.24.1.1 abs()

```
template<typename signal_t>
Mono< signal_t > NtFx::Mono< signal_t >::abs () const [inline], [noexcept]
```

Returns the abs of both channels.

Returns

[Mono](#)<signal_t>

8.24.1.2 absMax()

```
template<typename signal_t>
signal_t NtFx::Mono< signal_t >::absMax () const [inline], [noexcept]
```

Returns the larges of left and right disregarding sign.

Returns

signal_t

8.24.1.3 absMin()

```
template<typename signal_t>
signal_t NtFx::Mono< signal_t >::absMin () const [inline], [noexcept]
```

Returns the smallest of left and right disregarding sign.

Returns

signal_t

8.24.1.4 avgSquared()

```
template<typename signal_t>
signal_t NtFx::Mono< signal_t >::avgSquared () const [inline], [noexcept]
```

Square left and right channels and take the average.

Returns

signal_t Result.

The documentation for this struct was generated from the following file:

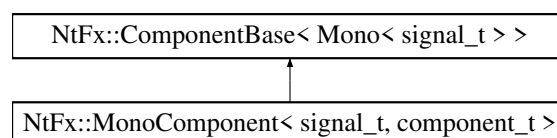
- lib/Audio.h

8.25 NtFx::MonoComponent< signal_t, component_t > Struct Template Reference

Wraps two objects of type Component<signal_t> as a Component<Audio> and calls through the both objects.

```
#include <Component.h>
```

Inheritance diagram for NtFx::MonoComponent< signal_t, component_t >:



Public Member Functions

- virtual [Mono](#)< signal_t > [process](#) ([Mono](#)< signal_t > x) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float fs) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- component_t l

Public Attributes inherited from [NtFx::ComponentBase](#)< [Mono](#)< signal_t > >

- float fs
Sample rate of component.

8.25.1 Detailed Description

```
template<typename signal_t, typename component_t>
struct NtFx::MonoComponent< signal_t, component_t >
```

Wraps two objects of type [Component](#)<signal_t> as a [Component](#)<Audio> and calls through the both objects.

Template Parameters

<i>signal_t</i>	Datatype for signal. Must be a floating point type.
<i>component_t</i>	Type of Component to wrap.

8.25.2 Member Function Documentation

8.25.2.1 process()

```
template<typename signal_t, typename component_t>
virtual Mono< signal_t > NtFx::MonoComponent< signal_t, component_t >::process (
    Mono< signal_t > x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

x	T Input
---	---------

Returns

T output.

Implements [NtFx::ComponentBase](#)< [Mono](#)< signal_t > >.

8.25.2.2 reset()

```
template<typename signal_t, typename component_t>
virtual void NtFx::MonoComponent< signal_t, component_t >::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Mono< signal_t > >](#).

8.25.2.3 update()

```
template<typename signal_t, typename component_t>
virtual void NtFx::MonoComponent< signal_t, component_t >::update () [inline], [override],
[virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

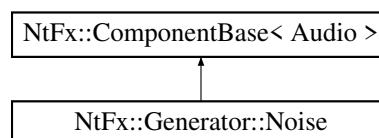
Reimplemented from [NtFx::ComponentBase< Mono< signal_t > >](#).

The documentation for this struct was generated from the following file:

- [lib/Component.h](#)

8.26 NtFx::Generator::Noise Struct Reference

Inheritance diagram for NtFx::Generator::Noise:



Public Member Functions

- virtual [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.

Public Member Functions inherited from [NtFx::ComponentBase< Audio >](#)

- virtual void [update](#) () noexcept
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept
Called when the plugin loads and when ever the samplerate or buffer size changes.

Additional Inherited Members

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float **fs**
Sample rate of component.

8.26.1 Member Function Documentation

8.26.1.1 process()

```
virtual Audio NtFx::Generator::Noise::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

x	T Input
---	---------

Returns

T output.

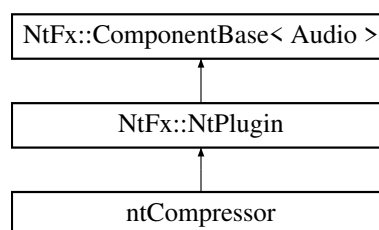
Implements [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- lib/Generator.h

8.27 ntCompressor Struct Reference

Inheritance diagram for ntCompressor:



Public Member Functions

- [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- void [reset](#) (float **fs**) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Member Functions inherited from `NtFx::NtPlugin`

- virtual void `onTempoChanged` () noexcept
Called by the wrapper whenever the tempo changes.
- `signal_t * getKnobValuePtr` (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- `bool * getToggleValuePtr` (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- `int * getDropDownValuePtr` (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void `setParameterActive` (std::string name, bool val)
- void `deactivateParameter` (std::string name)
- void `activateParameter` (std::string name)
- constexpr void `updateDefaults` () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggels. Should always be called as the last thing in the constructor of the derived plugin class.
- template<size_t idx, bool invert = false>
`Audio updatePeakLevel` (Audio val) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- `Audio getAndResetPeakLevel` (size_t idx) noexcept
Get and reset peak level for specified meter.
- `Audio getRms` (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Attributes

- `NtFx::Comp::ScSettings` **scSettings**
- `NtFx::Comp::PeakSideChainDb` **peakScDb**
- `NtFx::Comp::PeakSideChainLinear` **peakScLin**
- `NtFx::Comp::RmsSideChainDb` **rmsScDb**
- `NtFx::Comp::RmsSideChainLinear` **rmsScLin**
- `NtFx::Biquad::EqBand` **hpf**
- `NtFx::Biquad::EqBand` **boost**
- `signal_t` **makeup_db** { `signal_t(0.0)` }
- `signal_t` **mix_percent** { `signal_t(100.0)` }
- `int` **linEnable** { 0 }
- `int` **rmsEnable** { 0 }
- `int` **scMode** { `scMode::feedForward` }
- `bool` **scListenEnable** { false }
- `bool` **clip** { true }
- `bool` **bypassEnable** { false }
- `signal_t` **mix_lin** { `signal_t(1.0)` }
- `signal_t` **makeup_lin** { `signal_t(1.0)` }
- `Audio` **fbState** { `signal_t(0.0)` }
- `bool` **dummy1**
- `bool` **dummy2**
- `bool` **dummy3**

Public Attributes inherited from [NtFx::NtPlugin](#)

- `std::array< LongRmsSensor< 250, 192 >, 2 > xRms`
Array of rms sensors for RMS meter.
- `std::array< Audio, nMetersMax > peakLevels`
Peak level to be displayed in the meters.
- `UiSpec uiSpec`
Specification for UI. Modify this to change the look of your plugin.
- `std::vector< KnobSpec > primaryKnobs`
vector of primary knobs. Add your number paramters to this to display them in the UI.
- `std::vector< KnobSpec > secondaryKnobs`
vector of secondary knobs.
- `std::vector< DropDownSpec > dropdowns`
vector of DropDowns to be displayed
- `std::vector< ToggleSpec > toggles`
vector of toggles to be displayed at the bottom of the UI.
- `std::vector< ToggleSetSpec > toggleSets`
vector of groups of toggles to be placed in the right side area.
- `std::vector< RadioButtonSetSpec > radioButtons`
vector of radioButtons to be displayed at the right of the UI.
- `std::vector< MeterSpec > meters`
List of all meters to be displayed in the UI.
- `signal_t tempo = 0`
Session tempo. Automatically updated by wrapper.
- `signal_t xSc`
Updated at base sample rate and can be used in [process\(\)](#) for exenal side chain input.
- `bool uiNeedsUpdate = false`
Set this to true if you want to force an update of the UI.
- `bool isInitialized = false`
True if plugin in fully initialized. Must be set by child plugin.

Public Attributes inherited from [NtFx::ComponentBase< \[Audio\]\(#\) >](#)

- `float fs`
Sample rate of component.

8.27.1 Member Function Documentation

8.27.1.1 `process()`

```
Audio ntCompressor::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< \[Audio\]\(#\) >](#).

8.27.1.2 reset()

```
void ntCompressor::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.27.1.3 update()

```
void ntCompressor::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

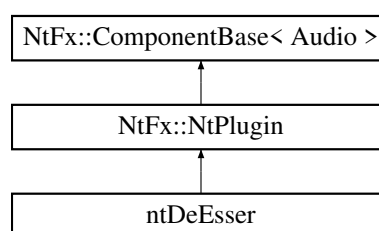
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- plugins/ntCompressor.h

8.28 ntDeEsser Struct Reference

Inheritance diagram for ntDeEsser:



Public Member Functions

- [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Member Functions inherited from [NtFx::NtPlugin](#)

- virtual void [onTempoChanged](#) () noexcept
Called by the wrapper whenever the tempo changes.
- signal_t * [getKnobValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- bool * [getToggleValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- int * [getDropDownValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void **setParameterActive** (std::string name, bool val)
- void **deactivateParameter** (std::string name)
- void **activateParameter** (std::string name)
- constexpr void **updateDefaults** () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggles. Should always be called as the last thing in the constructor of the derived plugin class.
- template<size_t idx, bool invert = false>
[Audio updatePeakLevel](#) (Audio val) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- [Audio getAndResetPeakLevel](#) (size_t idx) noexcept
Get and reset peak level for specified meter.
- [Audio getRms](#) (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Attributes

- [NtFx::Comp::PeakSideChainLinear](#) **sc**
- [NtFx::Biquad::EqBand](#) **scFlt**
- [NtFx::DynamicFilter::Shelf](#) **flt**
- [NtFx::Comp::ScSettings](#) **scSettings**
- signal_t **q** { 0.6 }
- signal_t **fc_hz** { 4e3 }
- bool **bypassEnable** { false }
- bool **scListenEnable** { false }

Public Attributes inherited from [NtFx::NtPlugin](#)

- std::array< [LongRmsSensor](#)< 250, 192 >, 2 > **xRms**
Array of rms sensors for RMS meter.
- std::array< [Audio](#), [nMetersMax](#) > **peakLevels**
Peak level to be displayed in the meters.
- [UiSpec](#) **uiSpec**
Specification for UI. Modify this to change the look of your plugin.
- std::vector< [KnobSpec](#) > **primaryKnobs**
vector of primary knobs. Add your number paramters to this to display them in the UI.
- std::vector< [KnobSpec](#) > **secondaryKnobs**
vector of secondary knobs.
- std::vector< [DropDownSpec](#) > **dropdowns**
vector of DropDowns to be displayed
- std::vector< [ToggleSpec](#) > **toggles**
vector of toggles to be displayed at the bottom of the UI.

- `std::vector< ToggleSetSpec > toggleSets`
vector of groups of toggles to be placed in the right side area.
- `std::vector< RadioButtonSetSpec > radioButtons`
vector of radioButtons to be displayed at the right of the UI.
- `std::vector< MeterSpec > meters`
List of all meters to be displayed in the UI.
- `signal_t tempo = 0`
Session tempo. Automatically updated by wrapper.
- `signal_t xSc`
Updated at base sample rate and can be used in [process\(\)](#) for exenal side chain input.
- `bool uiNeedsUpdate = false`
Set this to true if you want to force an update of the UI.
- `bool isInitialized = false`
True if plugin in fully initialized. Must be set by child plugin.

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- `float fs`
Sample rate of component.

8.28.1 Member Function Documentation

8.28.1.1 `process()`

```
Audio ntDeEsser::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.28.1.2 `reset()`

```
void ntDeEsser::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.28.1.3 update()

```
void ntDeEsser::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

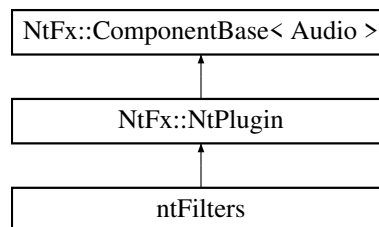
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- plugins/ntDeEsser.h

8.29 ntFilters Struct Reference

Inheritance diagram for ntFilters:



Public Member Functions

- [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Member Functions inherited from [NtFx::NtPlugin](#)

- virtual void [onTempoChanged](#) () noexcept
Called by the wrapper whenever the tempo changes.
- signal_t * [getKnobValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- bool * [getToggleValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- int * [getDropDownValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void **setParameterActive** (std::string name, bool val)
- void **deactivateParameter** (std::string name)
- void **activateParameter** (std::string name)
- constexpr void **updateDefaults** () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggels. Should always be called as the last thing in the constructor of the derived plugin class.
- template<size_t idx, bool invert = false>
[Audio updatePeakLevel](#) ([Audio](#) val) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- [Audio getAndResetPeakLevel](#) (size_t idx) noexcept
Get and reset peak level for specified meter.
- [Audio getRms](#) (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Attributes

- signal_t **fHpf** = 20
- signal_t **fLpf** = 20000
- signal_t **qHpf** = 0.707
- signal_t **qLpf** = 0.707
- Order **orderHpf**
- Order **orderLpf**
- bool **enableHpf** = true
- bool **enableLpf** = true
- [NtFx::FirstOrder::StereoFilter](#)< [NtFx::FirstOrder::Shape::hpf](#) > **firstOrderHpf**
- [NtFx::FirstOrder::StereoFilter](#)< [NtFx::FirstOrder::Shape::lpfZero](#) > **firstOrderLpf**
- [NtFx::Biquad::EqBand](#) **bqLpf0**
- [NtFx::Biquad::EqBand](#) **bqLpf1**
- [NtFx::Biquad::EqBand](#) **bqHpf0**
- [NtFx::Biquad::EqBand](#) **bqHpf1**

Public Attributes inherited from [NtFx::NtPlugin](#)

- std::array< [LongRmsSensor](#)< 250, 192 >, 2 > **xRms**
Array of rms sensors for RMS meter.
- std::array< [Audio](#), [nMetersMax](#) > **peakLevels**
Peak level to be displayed in the meters.
- [UiSpec](#) **uiSpec**
Specification for UI. Modify this to change the look of your plugin.
- std::vector< [KnobSpec](#) > **primaryKnobs**
vector of primary knobs. Add your number paramters to this to display them in the UI.

- `std::vector< KnobSpec > secondaryKnobs`
vector of secondary knobs.
- `std::vector< DropDownSpec > dropdowns`
vector of DropDowns to be displayed
- `std::vector< ToggleSpec > toggles`
vector of toggles to be displayed at the bottom of the UI.
- `std::vector< ToggleSetSpec > toggleSets`
vector of groups of toggles to be placed in the right side area.
- `std::vector< RadioButtonSetSpec > radioButtons`
vector of radioButtons to be displayed at the right of the UI.
- `std::vector< MeterSpec > meters`
List of all meters to be displayed in the UI.
- `signal_t tempo = 0`
Session tempo. Automatically updated by wrapper.
- `signal_t xSc`
Updated at base sample rate and can be used in `process()` for external side chain input.
- `bool uiNeedsUpdate = false`
Set this to true if you want to force an update of the UI.
- `bool isInitialized = false`
True if plugin is fully initialized. Must be set by child plugin.

Public Attributes inherited from `NtFx::ComponentBase< Audio >`

- `float fs`
Sample rate of component.

8.29.1 Member Function Documentation

8.29.1.1 `process()`

```
Audio ntFilters::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements `NtFx::ComponentBase< Audio >`.

8.29.1.2 `reset()`

```
void ntFilters::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.29.1.3 update()

```
void ntFilters::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

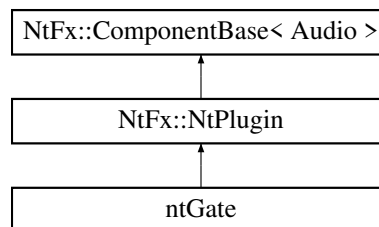
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- plugins/ntFilters.h

8.30 ntGate Struct Reference

Inheritance diagram for ntGate:



Public Member Functions

- [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Member Functions inherited from `NtFx::NtPlugin`

- virtual void `onTempoChanged` () noexcept
Called by the wrapper whenever the tempo changes.
- signal_t * `getKnobValuePtr` (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- bool * `getToggleValuePtr` (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- int * `getDropDownValuePtr` (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void `setParameterActive` (std::string name, bool val)
- void `deactivateParameter` (std::string name)
- void `activateParameter` (std::string name)
- constexpr void `updateDefaults` () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggles. Should always be called as the last thing in the constructor of the derived plugin class.
- template<size_t idx, bool invert = false>
`Audio updatePeakLevel` (Audio val) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- `Audio getAndResetPeakLevel` (size_t idx) noexcept
Get and reset peak level for specified meter.
- `Audio getRms` (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Attributes

- `NtFx::Gate::Sc` **sc**
- `NtFx::Gate::Sc` **scHf**
- `NtFx::DynamicFilter::Shelf` **flt**
- `NtFx::Biquad::EqBand` **hpf**
- `NtFx::Biquad::EqBand` **lpf**
- std::array< `Audio`, dLookaheadLen > **dLookahead**
- signal_t **ignoreThresh_db** { -20 }
- signal_t **ignoreThresh_lin** { 0.1 }
- signal_t **tlgnore_ms** { 6 }
- signal_t **ignorePeak** { 0 }
- signal_t **tLookahead_ms** { 0.2 }
- int **ignoreCount** { 0 }
- int **nIgnore** { 384 }
- int **scMode** { 0 }
- int **nLookahead** { 4 }
- int **iLookahead** { 0 }
- bool **bypassEnable** { false }
- bool **scListenEnable** { false }
- bool **hfAccelEnable** { false }
- bool **lookaheadEnable** { true }

Public Attributes inherited from [NtFx::NtPlugin](#)

- `std::array< LongRmsSensor< 250, 192 >, 2 > xRms`
Array of rms sensors for RMS meter.
- `std::array< Audio, nMetersMax > peakLevels`
Peak level to be displayed in the meters.
- `UiSpec uiSpec`
Specification for UI. Modify this to change the look of your plugin.
- `std::vector< KnobSpec > primaryKnobs`
vector of primary knobs. Add your number paramters to this to display them in the UI.
- `std::vector< KnobSpec > secondaryKnobs`
vector of secondary knobs.
- `std::vector< DropDownSpec > dropdowns`
vector of DropDowns to be displayed
- `std::vector< ToggleSpec > toggles`
vector of toggles to be displayed at the bottom of the UI.
- `std::vector< ToggleSetSpec > toggleSets`
vector of groups of toggles to be placed in the right side area.
- `std::vector< RadioButtonSetSpec > radioButtons`
vector of radioButtons to be displayed at the right of the UI.
- `std::vector< MeterSpec > meters`
List of all meters to be displayed in the UI.
- `signal_t tempo = 0`
Session tempo. Automatically updated by wrapper.
- `signal_t xSc`
Updated at base sample rate and can be used in [process\(\)](#) for exenal side chain input.
- `bool uiNeedsUpdate = false`
Set this to true if you want to force an update of the UI.
- `bool isInitialized = false`
True if plugin in fully initialized. Must be set by child plugin.

Public Attributes inherited from [NtFx::ComponentBase< \[Audio\]\(#\) >](#)

- `float fs`
Sample rate of component.

8.30.1 Member Function Documentation

8.30.1.1 `process()`

```
Audio ntGate::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< \[Audio\]\(#\) >](#).

8.30.1.2 reset()

```
void ntGate::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.30.1.3 update()

```
void ntGate::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

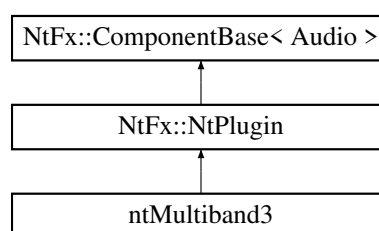
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- [plugins/ntGate.h](#)

8.31 ntMultiband3 Struct Reference

Inheritance diagram for ntMultiband3:



Public Member Functions

- [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.
- void [_updateMutes](#) ()

Public Member Functions inherited from NtFx::NtPlugin

- virtual void **onTempoChanged** () noexcept
Called by the wrapper whenever the tempo changes.
- signal_t * **getKnobValuePtr** (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- bool * **getToggleValuePtr** (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- int * **getDropDownValuePtr** (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void **setParameterActive** (std::string name, bool val)
- void **deactivateParameter** (std::string name)
- void **activateParameter** (std::string name)
- constexpr void **updateDefaults** () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggels. Should always be called as the last thing in the constructor of the derived plugin class.
- template<size_t idx, bool invert = false>
Audio updatePeakLevel (Audio val) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- **Audio getAndResetPeakLevel** (size_t idx) noexcept
Get and reset peak level for specified meter.
- **Audio getRms** (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Attributes

- signal_t **xOverLo_hz** { 200 }
- signal_t **xOverHi_hz** { 4000 }
- signal_t **ouputGain_db** { 0 }
- signal_t **ouputGain_lin** { 1 }
- bool **linkEnable** { false }
- bool **feedbackEnable** { false }
- bool **bypassEnable** { false }
- std::array< bool, Bands::n > **solos** = { false, false, false }
- std::array< bool, Bands::n > **mutesUi** = { false, false, false }
- std::array< bool, Bands::n > **mutes** = { false, false, false }
- std::array< NtFx::Comp::ScSettings, 3 > **scSettings**
- std::array< NtFx::Comp::PeakSideChainLinear, 3 > **sc**
- std::array< signal_t, Bands::n > **makeup_db** { 0, 0, 0 }
- std::array< signal_t, Bands::n > **makeup_lin** { 1, 1, 1 }
- std::array< Audio, Bands::n > **fbState** { 0, 0, 0 }
- const std::array< std::string, Bands::n > **BandNames** = { "High", "Mid", "Low" }
- NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::lpf > **loFlt**
- NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::hpf > **loMidFlt**
- NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::lpf > **hiMidFlt**
- NtFx::FirstOrder::StereoFilter< NtFx::FirstOrder::Shape::hpf > **hiFlt**

Public Attributes inherited from [NtFx::NtPlugin](#)

- `std::array< LongRmsSensor< 250, 192 >, 2 > xRms`
Array of rms sensors for RMS meter.
- `std::array< Audio, nMetersMax > peakLevels`
Peak level to be displayed in the meters.
- `UiSpec uiSpec`
Specification for UI. Modify this to change the look of your plugin.
- `std::vector< KnobSpec > primaryKnobs`
vector of primary knobs. Add your number paramters to this to display them in the UI.
- `std::vector< KnobSpec > secondaryKnobs`
vector of secondary knobs.
- `std::vector< DropDownSpec > dropdowns`
vector of DropDowns to be displayed
- `std::vector< ToggleSpec > toggles`
vector of toggles to be displayed at the bottom of the UI.
- `std::vector< ToggleSetSpec > toggleSets`
vector of groups of toggles to be placed in the right side area.
- `std::vector< RadioButtonSetSpec > radioButtons`
vector of radioButtons to be displayed at the right of the UI.
- `std::vector< MeterSpec > meters`
List of all meters to be displayed in the UI.
- `signal_t tempo = 0`
Session tempo. Automatically updated by wrapper.
- `signal_t xSc`
Updated at base sample rate and can be used in [process\(\)](#) for exenal side chain input.
- `bool uiNeedsUpdate = false`
Set this to true if you want to force an update of the UI.
- `bool isInitialized = false`
True if plugin in fully initialized. Must be set by child plugin.

Public Attributes inherited from [NtFx::ComponentBase< \[Audio\]\(#\) >](#)

- `float fs`
Sample rate of component.

8.31.1 Member Function Documentation

8.31.1.1 `process()`

```
Audio ntMultiband3::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< \[Audio\]\(#\) >](#).

8.31.1.2 reset()

```
void ntMultiband3::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.31.1.3 update()

```
void ntMultiband3::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

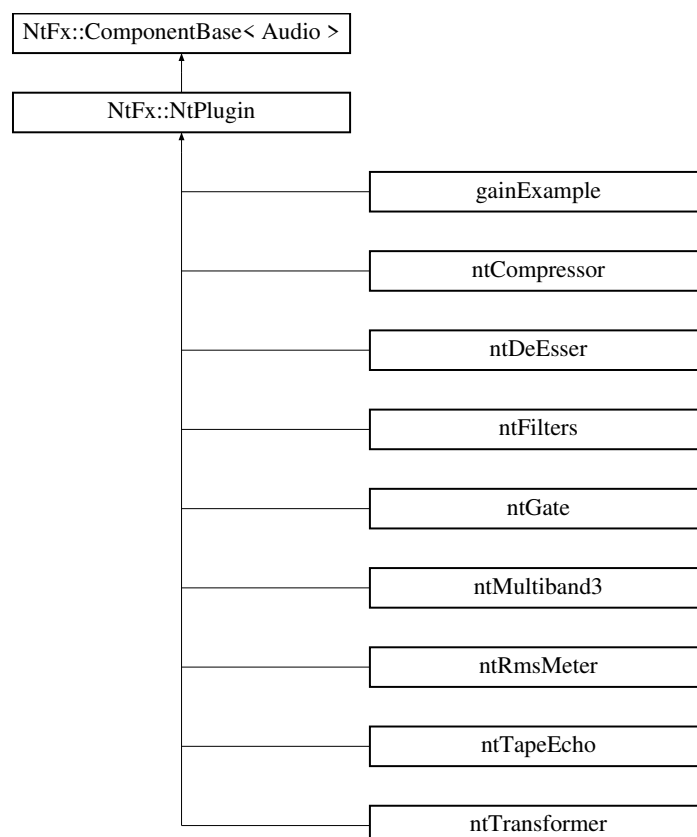
- plugins/ntMultiband3.h

8.32 NtFx::NtPlugin Struct Reference

True virtual base class for [NtPlugin](#). Inherit from this in order to make a stereo audio plugin with auto-generated UI.

```
#include <Plugin.h>
```

Inheritance diagram for NtFx::NtPlugin:



Public Member Functions

- virtual void **onTempoChanged** () noexcept
Called by the wrapper whenever the tempo changes.
- signal_t * **getKnobValuePtr** (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- bool * **getToggleValuePtr** (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- int * **getDropDownValuePtr** (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void **setParameterActive** (std::string name, bool val)
- void **deactivateParameter** (std::string name)
- void **activateParameter** (std::string name)
- constexpr void **updateDefaults** () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggles. Should always be called as the last thing in the constructor of the derived plugin class.
- template<size_t idx, bool invert = false>
Audio updatePeakLevel (Audio val) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- **Audio getAndResetPeakLevel** (size_t idx) noexcept
Get and reset peak level for specified meter.
- **Audio getRms** (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Member Functions inherited from **NtFx::ComponentBase**< **Audio** >

- virtual **Audio process** (Audio x) noexcept=0
Called for every sample as audio is processed.
- virtual void **update** () noexcept
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void **reset** (float fs) noexcept
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- std::array< **LongRmsSensor**< 250, 192 >, 2 > **xRms**
Array of rms sensors for RMS meter.
- std::array< **Audio**, nMetersMax > **peakLevels**
Peak level to be displayed in the meters.
- **UiSpec uiSpec**
Specification for UI. Modify this to change the look of your plugin.
- std::vector< **KnobSpec** > **primaryKnobs**
vector of primary knobs. Add your number paramters to this to display them in the UI.
- std::vector< **KnobSpec** > **secondaryKnobs**
vector of secondary knobs.
- std::vector< **DropDownSpec** > **dropdowns**
vector of DropDowns to be displayed
- std::vector< **ToggleSpec** > **toggles**
vector of toggles to be displayed at the bottom of the UI.
- std::vector< **ToggleSetSpec** > **toggleSets**

- vector of groups of toggles to be placed in the right side area.*
 - `std::vector< RadioButtonSetSpec > radioButtons`
 - vector of radioButtons to be displayed at the right of the UI.*
 - `std::vector< MeterSpec > meters`
 - List of all meters to be displayed in the UI.*
 - `signal_t tempo = 0`
 - Session tempo. Automatically updated by wrapper.*
 - `signal_t xSc`
 - Updated at base sample rate and can be used in [process\(\)](#) for external side chain input.*
 - `bool uiNeedsUpdate = false`
 - Set this to true if you want to force an update of the UI.*
 - `bool isInitialized = false`
 - True if plugin is fully initialized. Must be set by child plugin.*

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float **fs**
 - Sample rate of component.*

8.32.1 Detailed Description

True virtual base class for [NtPlugin](#). Inherit from this in order to make a stereo audio plugin with auto-generated UI.

Template Parameters

<i>signal</i> ↔ _t	Basic datatype for audio signal.
-----------------------	----------------------------------

8.32.2 Member Function Documentation

8.32.2.1 getAndResetPeakLevel()

```
Audio NtFx::NtPlugin::getAndResetPeakLevel (
    size_t idx) [inline], [noexcept]
```

Get and reset peak level for specified meter.

Parameters

<i>idx</i>	
------------	--

Returns

[Audio](#)<[signal_t](#)> Highest signal level since last call.

8.32.2.2 getDropDownValuePtr()

```
int * NtFx::NtPlugin::getDropDownValuePtr (
    std::string name) const [inline], [noexcept]
```

Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.

Parameters

<i>name</i>	Name of drop down to get value for.
-------------	-------------------------------------

8.32.2.3 getKnobValuePtr()

```
signal_t * NtFx::NtPlugin::getKnobValuePtr (
    std::string name) const [inline], [noexcept]
```

Used by the wrapper to get a pointer to a value based on the corresponding knob's name.

Parameters

<i>name</i>	Name of knob to get value for.
-------------	--------------------------------

8.32.2.4 getRms()

```
Audio NtFx::NtPlugin::getRms (
    size_t idx) [inline], [noexcept]
```

Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Parameters

<i>idx</i>	Index of meter to get RMS level for.
------------	--------------------------------------

Returns

Audio<signal_t> RMS level.

8.32.2.5 getToggleValuePtr()

```
bool * NtFx::NtPlugin::getToggleValuePtr (
    std::string name) const [inline], [noexcept]
```

Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.

Parameters

<i>name</i>	Name of toggle to get value for.
-------------	----------------------------------

8.32.2.6 onTempoChanged()

```
virtual void NtFx::NtPlugin::onTempoChanged () [inline], [virtual], [noexcept]
```

Called by the wrapper whenever the tempo changes.

Reimplemented in [ntTapeEcho](#).

8.32.2.7 updatePeakLevel()

```
template<size_t idx, bool invert = false>
Audio NtFx::NtPlugin::updatePeakLevel (
    Audio val) [inline], [noexcept]
```

Updates the peak level of a meter. This is a template so that we can do range checking at compile time.

Template Parameters

<i>idx</i>	Index of meter to update.
<i>invert</i>	If true, the smallest value will be selected.

Parameters

<i>val</i>	New value. If val has a larger magnitude than the current peak value, the current peak value will be overridden.
------------	--

Returns

8.32.3 Member Data Documentation

8.32.3.1 meters

```
std::vector<MeterSpec> NtFx::NtPlugin::meters
```

Initial value:

```
= {  
    { .name = "IN", .addRms = true },  
    { .name = "OUT", .hasScale = true, .addRms = true },  
}
```

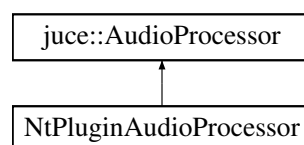
List of all meters to be displayed in the UI.

The documentation for this struct was generated from the following file:

- lib/[Plugin.h](#)

8.33 NtPluginAudioProcessor Struct Reference

Inheritance diagram for NtPluginAudioProcessor:



Public Member Functions

- void **prepareToPlay** (double sampleRate, int samplesPerBlock) override
- void **releaseResources** () override
- bool **isBusesLayoutSupported** (const BusesLayout &layouts) const override
- void **processBlock** (juce::AudioBuffer< float > &, juce::MidiBuffer &) override
- juce::AudioProcessorEditor * **createEditor** () override
- bool **hasEditor** () const override
- const juce::String **getName** () const override
- bool **acceptsMidi** () const override
- bool **producesMidi** () const override
- bool **isMidiEffect** () const override
- double **getTailLengthSeconds** () const override
- int **getNumPrograms** () override
- int **getCurrentProgram** () override
- void **setCurrentProgram** (int index) override
- const juce::String **getProgramName** (int index) override
- void **changeProgramName** (int index, const juce::String &newName) override
- void **getStateInformation** (juce::MemoryBlock &destData) override
- void **setStateInformation** (const void *data, int sizeInBytes) override
- template<typename T>
void **loadParameter** (std::vector< T > &v)
- void **loadRadioButtons** (std::vector< [NtFx::RadioButtonSetSpec](#) > &v)
- void **loadToggleSets** (std::vector< [NtFx::ToggleSetSpec](#) > &v)
- template<typename t_val, typename t_spec>
void **createParameters** (std::vector< t_spec > &vParams, juce::AudioProcessorValueTreeState::↳
ParameterLayout ¶meters, int &i)
- void **updateOversampling** (int mode=0)
- juce::AudioProcessorValueTreeState::ParameterLayout **createParameterLayout** ()

Public Attributes

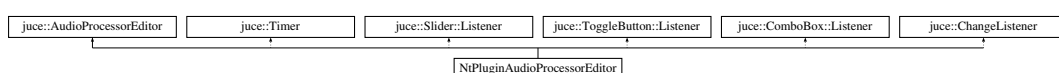
- [NtFx::TitleBarSpec](#) **titleBarSpec**
- float **fsBase** = 48000
- NTFX_PLUGIN **plug**
- [NtFx::Src::SampleRateConverter](#) **src**
- juce::AudioProcessorValueTreeState **paramLayout**
- bool **monoMode** { false }

The documentation for this struct was generated from the following files:

- JuceWrapper/PluginProcessor.h
- JuceWrapper/PluginProcessor.cpp

8.34 NtPluginAudioProcessorEditor Struct Reference

Inheritance diagram for NtPluginAudioProcessorEditor:



Public Member Functions

- **NtPluginAudioProcessorEditor** ([NtPluginAudioProcessor](#) &)
- virtual void **paint** (juce::Graphics &) override
- virtual void **resized** () override
- virtual void **sliderValueChanged** (juce::Slider *slider) override
- virtual void **buttonClicked** (juce::Button *button) override
- virtual void **changeListenerCallback** (juce::ChangeBroadcaster *source) override
- virtual void **comboBoxChanged** (juce::ComboBox *p_box) override
- virtual void **timerCallback** () override
- void **_initPrimaryKnob** ([NtFx::KnobSpec](#) &p_spec)
- void **_initSecondaryKnob** ([NtFx::KnobSpec](#) &p_spec)
- void **_initKnob** ([NtFx::KnobSpec](#) &p_spec, std::unique_ptr< juce::Slider > &p_slider, std::unique_ptr< juce::Label > &p_label)
- void **_initToggle** ([NtFx::ToggleSpec](#) &spec)
- void **__initToggle** ([NtFx::Toggle](#) *p_toggle, [NtFx::ToggleSpec](#) &spec)
- void **_initToggleGroup** ([NtFx::ToggleSetSpec](#) &spec)
- void **_initDropDown** ([NtFx::DropDownSpec](#) &p_spec, bool addToTitleBar=false)
- void **_initRadioButton** ([NtFx::RadioButtonSetSpec](#) &spec)
- void **_updateUi** ()
- void **_updateTitleBar** (juce::Rectangle< int > &area)
- void **_updateMeters** (juce::Rectangle< int > &area)
- void **_updateSecondaryKnobs** (juce::Rectangle< int > &area)
- void **_updatePrimaryKnobs** (juce::Rectangle< int > &area)
- void **_updateColours** ()
- void **_updateUiScale** ()
- void **_updateOversampling** ()
- void **_updateTheme** ()
- void **_placeSmallTogglesArea** (juce::Rectangle< int > &area)
- void **_placeBottomRow** (juce::Rectangle< int > &area)
- void **_placeDropdowns** (juce::Rectangle< int > &area, size_t columnWidth)
- void **_placeToggles** (juce::Rectangle< int > &area, size_t columnWidth)
- template<typename T>
void **_placeSmallToggles** (juce::Rectangle< int > &area, int size, std::vector< std::unique_ptr< juce::Label > > &labels, std::vector< std::unique_ptr< T > > &toggles)
- void **_initWindowSize** ()
- template<typename T, typename spec_t>
std::unique_ptr< T > **_makeSmallToggleSet** (spec_t &spec)
- void **_calcSliderRowsCols** (int nSliders, int &nRows, int &nColumns, int maxRows, int maxColumns)
- void **_makeGrid** (juce::Rectangle< int > &area, float nRows, float nCols, float colWidth, float rowHeight, float pad)
- std::unique_ptr< juce::Label > **_makeLabel** (const std::string name)

Public Attributes

- [NtPluginAudioProcessor](#) & **proc**
- [NtFx::MeterGroup](#) **meters**
- [NtFx::KnobLookAndFeel](#) **knobLookAndFeel**
- [NtFx::TitleBarLookAndFeel](#) **dropDownLookAndFeel**
- std::vector< std::unique_ptr< juce::Slider > > **primaryKnobs**
- std::vector< std::unique_ptr< juce::Label > > **primaryKnobLabels**
- std::vector< std::unique_ptr< juce::Slider > > **secondaryKnobs**
- std::vector< std::unique_ptr< juce::Label > > **secondaryKnobLabels**
- std::vector< std::unique_ptr< [NtFx::Toggle](#) > > **toggles**

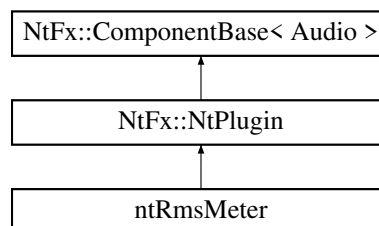
- `std::vector< std::unique_ptr< juce::ComboBox > >` **dropDowns**
- `std::vector< std::unique_ptr< juce::Label > >` **dropDownLabels**
- `std::vector< std::unique_ptr< NtFx::RadioButtonSet > >` **radioButtons**
- `std::vector< std::unique_ptr< juce::Label > >` **radioButtonLabels**
- `std::vector< std::unique_ptr< NtFx::ToggleSet > >` **toggleSets**
- `std::vector< std::unique_ptr< juce::Label > >` **toggleSetLabels**
- `std::vector< std::unique_ptr< juce::ComboBox > >` **titleBarDropDowns**
- `std::vector< std::unique_ptr< juce::Label > >` **titleBarDropDownLabels**
- `std::vector< std::unique_ptr< juce::AudioProcessorValueTreeState::SliderAttachment > >` **knob**↩
- `std::vector< std::unique_ptr< juce::AudioProcessorValueTreeState::ButtonAttachment > >` **toggle**↩
- `std::vector< std::unique_ptr< juce::AudioProcessorValueTreeState::ComboBoxAttachment > >` **drop**↩
- `std::vector< juce::Rectangle< int > >` **borderedAreas**
- `std::vector< juce::Rectangle< int > >` **grayAreas**
- float **unscaledWindowHeight** = 0
- float **unscaledWindowWidth** = 0
- float **uiScale** = 1
- float **titleBarScale** = 0.7
- float **pad** = 10
- bool **isInitialized** = false
- `juce::Label` **pluginNameLabel**

The documentation for this struct was generated from the following files:

- JuceWrapper/PluginEditor.h
- JuceWrapper/PluginEditor.cpp

8.35 ntRmsMeter Struct Reference

Inheritance diagram for ntRmsMeter:



Public Types

- enum **Meter** { **Peak** , **RMS** }

Public Member Functions

- virtual `Audio process (Audio x)` noexcept override
Called for every sample as audio is processed.
- virtual void `update ()` noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void `reset (float fs)` noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Member Functions inherited from [NtFx::NtPlugin](#)

- virtual void [onTempoChanged](#) () noexcept
Called by the wrapper whenever the tempo changes.
- signal_t * [getKnobValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- bool * [getToggleValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- int * [getDropDownValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void **setParameterActive** (std::string name, bool val)
- void **deactivateParameter** (std::string name)
- void **activateParameter** (std::string name)
- constexpr void **updateDefaults** () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggles. Should always be called as the last thing in the constructor of the derived plugin class.
- template<size_t idx, bool invert = false>
[Audio updatePeakLevel](#) ([Audio](#) val) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- [Audio getAndResetPeakLevel](#) (size_t idx) noexcept
Get and reset peak level for specified meter.
- [Audio getRms](#) (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Attributes

- [NtFx::LongRmsSensor](#) **msSensor**
- signal_t **decay_s** = 0.1
- signal_t **hold_s** = 2
- signal_t **tRms_ms** = 10

Public Attributes inherited from [NtFx::NtPlugin](#)

- std::array< [LongRmsSensor](#)< 250, 192 >, 2 > **xRms**
Array of rms sensors for RMS meter.
- std::array< [Audio](#), [nMetersMax](#) > **peakLevels**
Peak level to be displayed in the meters.
- [UiSpec](#) **uiSpec**
Specification for UI. Modify this to change the look of your plugin.
- std::vector< [KnobSpec](#) > **primaryKnobs**
vector of primary knobs. Add your number paramters to this to display them in the UI.
- std::vector< [KnobSpec](#) > **secondaryKnobs**
vector of secondary knobs.
- std::vector< [DropDownSpec](#) > **dropdowns**
vector of DropDowns to be displayed
- std::vector< [ToggleSpec](#) > **toggles**
vector of toggles to be displayed at the bottom of the UI.
- std::vector< [ToggleSetSpec](#) > **toggleSets**
vector of groups of toggles to be placed in the right side area.
- std::vector< [RadioButtonSetSpec](#) > **radioButtons**

vector of radioButtons to be displayed at the right of the UI.

- `std::vector< MeterSpec > meters`

List of all meters to be displayed in the UI.

- `signal_t tempo = 0`

Session tempo. Automatically updated by wrapper.

- `signal_t xSc`

Updated at base sample rate and can be used in [process\(\)](#) for external side chain input.

- `bool uiNeedsUpdate = false`

Set this to true if you want to force an update of the UI.

- `bool isInitialized = false`

True if plugin is fully initialized. Must be set by child plugin.

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- `float fs`

Sample rate of component.

8.35.1 Member Function Documentation

8.35.1.1 `process()`

```
virtual Audio ntRmsMeter::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.35.1.2 `reset()`

```
virtual void ntRmsMeter::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<code>fs</code>	Sample rate. If you need it for calculateCoeffs, store it.
-----------------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.35.1.3 update()

```
virtual void ntRmsMeter::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

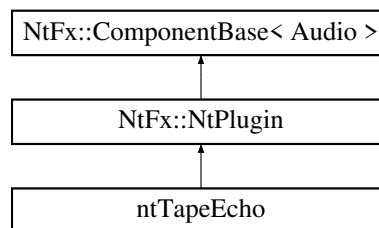
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- plugins/ntRmsMeter.h

8.36 ntTapeEcho Struct Reference

Inheritance diagram for ntTapeEcho:



Public Member Functions

- virtual [Audio process](#) ([Audio x](#)) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.
- virtual void [onTempoChanged](#) () noexcept override
Called by the wrapper whenever the tempo changes.

Public Member Functions inherited from [NtFx::NtPlugin](#)

- [signal_t *](#) [getKnobValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- [bool *](#) [getToggleValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- [int *](#) [getDropDownValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void [setParameterActive](#) (std::string name, bool val)
- void [deactivateParameter](#) (std::string name)
- void [activateParameter](#) (std::string name)
- constexpr void [updateDefaults](#) () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggles. Should always be called as the last thing in the constructor of the derived plugin class.
- [template<size_t idx, bool invert = false>](#)
[Audio updatePeakLevel](#) ([Audio val](#)) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- [Audio getAndResetPeakLevel](#) (size_t idx) noexcept
Get and reset peak level for specified meter.
- [Audio getRms](#) (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Attributes

- signal_t **tGui** = 0.5
- signal_t **fb_percent** = 20
- signal_t **modFreq** = 1.0
- signal_t **modPhase** = 0.0
- signal_t **clipG_db** = 0.0
- signal_t **mix_percent** = 100.0
- signal_t **tOffset** = 0.0
- signal_t **modDepth_percent** = 0.1
- signal_t **noise_db** = -100
- bool **sync** = false
- bool **mod** = true
- bool **clip** = true
- bool **doGlide** = true
- bool **bypass** = false
- SubDev **subDev** = SubDev::fourth
- signal_t **tempoScale** = 1
- [NtFx::Biquad::EqBand](#) **hpf**
- [NtFx::Biquad::EqBand](#) **lpf**
- [Audio](#) **fbState**
- std::array< [Audio](#), delayLineLength > **delayLine**
- signal_t **fb_lin** = 0.2
- signal_t **noise_lin** = 0
- signal_t **tGlide** = 0.36
- [NtFx::ExpGlider](#) **nDelay**
- size_t **iStore** = 0
- signal_t **aClip_lin** = 1
- size_t **timeCounter** = 0
- [NtFx::ExpGlider](#) **nOffset**
- signal_t **mix_lin** = 1
- [NtFx::ExpGlider](#) **modDepth**
- [NtFx::ExpGlider](#) **thetaMod**
- [NtFx::ExpGlider](#) **thetaModPhase**

Public Attributes inherited from [NtFx::NtPlugin](#)

- std::array< [LongRmsSensor](#)< 250, 192 >, 2 > **xRms**
Array of rms sensors for RMS meter.
- std::array< [Audio](#), nMetersMax > **peakLevels**
Peak level to be displayed in the meters.
- [UiSpec](#) **uiSpec**
Specification for UI. Modify this to change the look of your plugin.
- std::vector< [KnobSpec](#) > **primaryKnobs**
vector of primary knobs. Add your number paramters to this to display them in the UI.
- std::vector< [KnobSpec](#) > **secondaryKnobs**
vector of secondary knobs.
- std::vector< [DropDownSpec](#) > **dropdowns**
vector of DropDowns to be displayed
- std::vector< [ToggleSpec](#) > **toggles**
vector of toggles to be displayed at the bottom of the UI.
- std::vector< [ToggleSetSpec](#) > **toggleSets**

- vector of groups of toggles to be placed in the right side area.*
- `std::vector< RadioButtonSetSpec > radioButtons`
vector of radioButtons to be displayed at the right of the UI.
- `std::vector< MeterSpec > meters`
List of all meters to be displayed in the UI.
- `signal_t tempo = 0`
Session tempo. Automatically updated by wrapper.
- `signal_t xSc`
Updated at base sample rate and can be used in [process\(\)](#) for exenal side chain input.
- `bool uiNeedsUpdate = false`
Set this to true if you want to force an update of the UI.
- `bool isInitialized = false`
True if plugin in fully initialized. Must be set by child plugin.

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- `float fs`
Sample rate of component.

8.36.1 Member Function Documentation

8.36.1.1 onTempoChanged()

```
virtual void ntTapeEcho::onTempoChanged () [inline], [override], [virtual], [noexcept]
```

Called by the wrapper whenever the tempo changes.

Reimplemented from [NtFx::NtPlugin](#).

8.36.1.2 process()

```
virtual Audio ntTapeEcho::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.36.1.3 reset()

```
virtual void ntTapeEcho::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.36.1.4 update()

```
virtual void ntTapeEcho::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

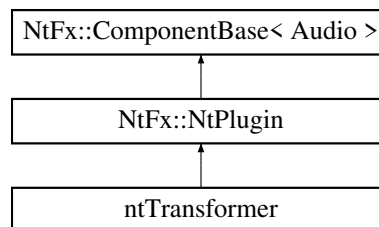
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- plugins/ntTapeEcho.h

8.37 ntTransformer Struct Reference

Inheritance diagram for ntTransformer:



Public Member Functions

- virtual [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Member Functions inherited from NtFx::NtPlugin

- virtual void [onTempoChanged](#) () noexcept
Called by the wrapper whenever the tempo changes.
- signal_t * [getKnobValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding knob's name.
- bool * [getToggleValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding toggle's name.
- int * [getDropDownValuePtr](#) (std::string name) const noexcept
Used by the wrapper to get a pointer to a value based on the corresponding drop down's name.
- void **setParameterActive** (std::string name, bool val)
- void **deactivateParameter** (std::string name)
- void **activateParameter** (std::string name)
- constexpr void **updateDefaults** () noexcept
Copies the values of all members in the derived plugin class to their respective knobs and toggles. Should always be called as the last thing in the constructor of the derived plugin class.
- template<size_t idx, bool invert = false>
[Audio updatePeakLevel](#) (Audio val) noexcept
Updates the peak level of a meter. This is a template so that we can do range checking at compile time.
- [Audio getAndResetPeakLevel](#) (size_t idx) noexcept
Get and reset peak level for specified meter.
- [Audio getRms](#) (size_t idx) noexcept
Get the RMS level of meter with index idx. RMS level is calculated from the peak level.

Public Attributes

- [NtFx::Biquad::EqBand](#) **bqHpf0**
- [NtFx::Transformer](#) **transformer**
- signal_t **drive_db** = -10
- signal_t **drive_lin** = 0.3
- bool **bypass** = false

Public Attributes inherited from NtFx::NtPlugin

- std::array< [LongRmsSensor](#)< 250, 192 >, 2 > **xRms**
Array of rms sensors for RMS meter.
- std::array< [Audio](#), [nMetersMax](#) > **peakLevels**
Peak level to be displayed in the meters.
- [UiSpec](#) **uiSpec**
Specification for UI. Modify this to change the look of your plugin.
- std::vector< [KnobSpec](#) > **primaryKnobs**
vector of primary knobs. Add your number paramters to this to display them in the UI.
- std::vector< [KnobSpec](#) > **secondaryKnobs**
vector of secondary knobs.
- std::vector< [DropDownSpec](#) > **dropdowns**
vector of DropDowns to be displayed
- std::vector< [ToggleSpec](#) > **toggles**
vector of toggles to be displayed at the bottom of the UI.
- std::vector< [ToggleSetSpec](#) > **toggleSets**
vector of groups of toggles to be placed in the right side area.
- std::vector< [RadioButtonSetSpec](#) > **radioButtons**

vector of radioButtons to be displayed at the right of the UI.

- `std::vector< MeterSpec > meters`

List of all meters to be displayed in the UI.

- `signal_t tempo = 0`

Session tempo. Automatically updated by wrapper.

- `signal_t xSc`

Updated at base sample rate and can be used in [process\(\)](#) for exenal side chain input.

- `bool uiNeedsUpdate = false`

Set this to true if you want to force an update of the UI.

- `bool isInitialized = false`

True if plugin in fully initialized. Must be set by child plugin.

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- `float fs`

Sample rate of component.

8.37.1 Member Function Documentation

8.37.1.1 `process()`

```
virtual Audio ntTransformer::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<code>x</code>	T Input
----------------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.37.1.2 `reset()`

```
virtual void ntTransformer::reset (  
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<code>fs</code>	Sample rate. If you need it for calculateCoeffs, store it.
-----------------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.37.1.3 update()

```
virtual void ntTransformer::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- `plugins/ntTransformer.h`

8.38 NtFx::OptionsSpec Struct Reference

Specification for drop down in UI.

```
#include <UiSpec.h>
```

Public Attributes

- `int * p_val { nullptr }`
- `std::string name`
Name of drop down, used for label and id.
- `std::vector< std::string > options`
Vector of options in the drop down.
- `int _defaultVal { 0 }`
Index of default option.
- `int _id { 0 }`

8.38.1 Detailed Description

Specification for drop down in UI.

The documentation for this struct was generated from the following file:

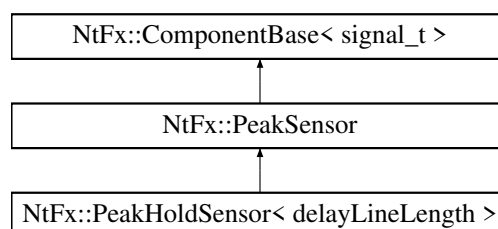
- `lib/UiSpec.h`

8.39 NtFx::PeakHoldSensor< delayLineLength > Struct Template Reference

Peak sensor with hold. Output will remain at the max value for `tHold_ms` milliseconds when after peak and the curve will be delayed by the same amount time when going down.

```
#include <PeakSensor.h>
```

Inheritance diagram for `NtFx::PeakHoldSensor< delayLineLength >`:



Public Member Functions

- virtual signal_t [process](#) (signal_t x) noexcept override
Processes signal with peak sensor with hold.
- virtual void [update](#) () noexcept override
Calculates coefficients.

Public Member Functions inherited from [NtFx::PeakSensor](#)

- virtual void [reset](#) (float fs) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- std::array< signal_t, delayLineLength > [_dl](#)
- signal_t [tHold_ms](#) { 10 }
- signal_t [_xMax](#) { 0 }
- int [_idx](#) { 0 }
- int [_nHold](#) { 0 }
- int [_holdCnt](#) { 0 }

Public Attributes inherited from [NtFx::PeakSensor](#)

- signal_t [tPeak_ms](#)
Time constant for peak detection in milliseconds.
- signal_t [_alpha](#) { 0 }
Smoothing factor for the peak detection.
- signal_t [_state](#) { 0 }
Internal state of the peak sensor.

Public Attributes inherited from [NtFx::ComponentBase< signal_t >](#)

- float [fs](#)
Sample rate of component.

Additional Inherited Members

Static Public Member Functions inherited from [NtFx::PeakSensor](#)

- static signal_t [_peakSensor](#) (signal_t alpha, signal_t &p_state, signal_t x)
Static method to apply the peak sensor algorithm.

8.39.1 Detailed Description

```
template<int delayLineLength = defaultPeakSensorDelayLineLength>
struct NtFx::PeakHoldSensor< delayLineLength >
```

Peak sensor with hold. Output will remain at the max value for tHold_ms miliseconds when after peak and the curve will be delayed by the same amount time when going down.

Template Parameters

<i>signal_t</i>	Audio signal type.
<i>delayLineLength</i>	Maximum delay time.

8.39.2 Member Function Documentation

8.39.2.1 process()

```
template<int delayLineLength = defaultPeakSensorDelayLineLength>
virtual signal_t NtFx::PeakHoldSensor< delayLineLength >::process (
    signal_t x) [inline], [override], [virtual], [noexcept]
```

Processes signal with peak sensor with hold.

Parameters

<i>x</i>	Input.
----------	--------

Returns

signal_t Sensor output.

Reimplemented from [NtFx::PeakSensor](#).

8.39.2.2 update()

```
template<int delayLineLength = defaultPeakSensorDelayLineLength>
virtual void NtFx::PeakHoldSensor< delayLineLength >::update () [inline], [override], [virtual],
[noexcept]
```

Calculates coefficients.

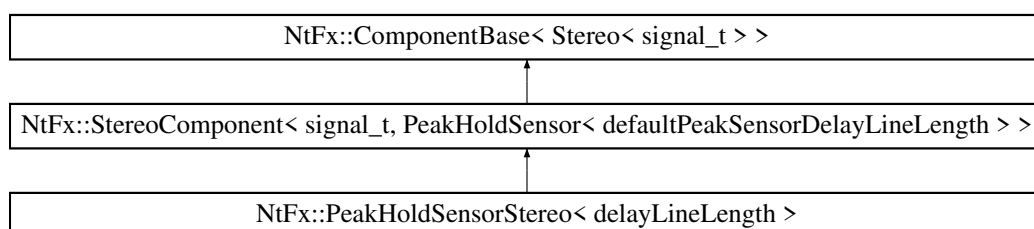
Reimplemented from [NtFx::PeakSensor](#).

The documentation for this struct was generated from the following file:

- lib/[PeakSensor.h](#)

8.40 NtFx::PeakHoldSensorStereo< delayLineLength > Struct Template Reference

Inheritance diagram for NtFx::PeakHoldSensorStereo< delayLineLength >:



Public Member Functions

- void [setT_ms](#) (signal_t t_ms)
Sets the time constant for peak detection in milliseconds.
- void [setTHold_ms](#) (signal_t tHold_ms)
Set the hold time in milliseconds.

Public Member Functions inherited from [NtFx::StereoComponent< signal_t, component_t >](#)

- virtual [Stereo< signal_t > process](#) ([Stereo< signal_t > x](#)) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float fs) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Additional Inherited Members

Public Attributes inherited from [NtFx::StereoComponent< signal_t, component_t >](#)

- component_t l
- component_t r

Public Attributes inherited from [NtFx::ComponentBase< Stereo< signal_t > >](#)

- float fs
Sample rate of component.

8.40.1 Member Function Documentation

8.40.1.1 [setT_ms\(\)](#)

```
template<int delayLineLength = defaultPeakSensorDelayLineLength>
void NtFx::PeakHoldSensorStereo< delayLineLength >::setT\_ms (
    signal_t t_ms) [inline]
```

Sets the time constant for peak detection in milliseconds.

This method updates the time constant for both the left and right channels.

Parameters

t_ms	The time constant in milliseconds.
----------------------	------------------------------------

8.40.1.2 [setTHold_ms\(\)](#)

```
template<int delayLineLength = defaultPeakSensorDelayLineLength>
void NtFx::PeakHoldSensorStereo< delayLineLength >::setTHold\_ms (
    signal_t tHold_ms) [inline]
```

Set the hold time in milliseconds.

Parameters

<code>tHold_ms</code>	Hold time.
-----------------------	------------

The documentation for this struct was generated from the following file:

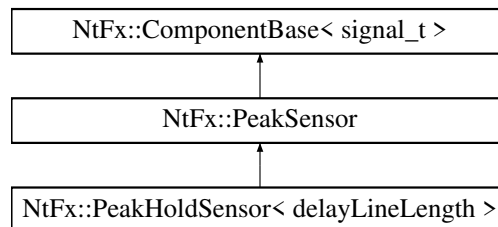
- lib/[PeakSensor.h](#)

8.41 NtFx::PeakSensor Struct Reference

Peak sensor component for audio processing.

```
#include <PeakSensor.h>
```

Inheritance diagram for NtFx::PeakSensor:



Public Member Functions

- virtual `signal_t` [process](#) (`signal_t x`) noexcept override
Processes the input signal to detect and track the peak amplitude.
- virtual void [update](#) () noexcept override
Updates the internal parameters of the peak sensor.
- virtual void [reset](#) (float `fs`) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Static Public Member Functions

- static `signal_t` [_peakSensor](#) (`signal_t alpha`, `signal_t &p_state`, `signal_t x`)
Static method to apply the peak sensor algorithm.

Public Attributes

- `signal_t` [tPeak_ms](#)
Time constant for peak detection in milliseconds.
- `signal_t` [_alpha](#) { 0 }
Smoothing factor for the peak detection.
- `signal_t` [_state](#) { 0 }
Internal state of the peak sensor.

Public Attributes inherited from `NtFx::ComponentBase< signal_t >`

- float `fs`
Sample rate of component.

8.41.1 Detailed Description

Peak sensor component for audio processing.

The `PeakSensor` component detects and tracks the peak amplitude of an audio signal. It uses an exponential decay to smooth the detected peaks, allowing for a configurable release time. This component is useful in audio effects that require peak detection, such as compressors, limiters, and more.

Template Parameters

<code>signal_t</code>	The type of the audio signal (e.g., float, double).
-----------------------	---

8.41.2 Member Function Documentation

8.41.2.1 `_peakSensor()`

```
signal_t NtFx::PeakSensor::_peakSensor (
    signal_t alpha,
    signal_t & p_state,
    signal_t x) [inline], [static]
```

Static method to apply the peak sensor algorithm.

This method implements the core peak sensor algorithm, which detects the peak amplitude of the input signal and applies an exponential decay to smooth the detected peaks.

Parameters

<code>alpha</code>	The smoothing factor for the peak detection.
<code>p_state</code>	Reference to the internal state of the peak sensor.
<code>x</code>	The input signal sample.

Returns

The detected peak amplitude.

8.41.2.2 `process()`

```
virtual signal_t NtFx::PeakSensor::process (
    signal_t x) [inline], [override], [virtual], [noexcept]
```

Processes the input signal to detect and track the peak amplitude.

This method applies the peak sensor algorithm to the input signal, updating the internal state and returning the detected peak amplitude.

Parameters

<i>x</i>	The input signal sample.
----------	--------------------------

Returns

The detected peak amplitude.

Implements [NtFx::ComponentBase< signal_t >](#).

Reimplemented in [NtFx::PeakHoldSensor< delayLineLength >](#).

8.41.2.3 reset()

```
virtual void NtFx::PeakSensor::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< signal_t >](#).

8.41.2.4 update()

```
virtual void NtFx::PeakSensor::update () [inline], [override], [virtual], [noexcept]
```

Updates the internal parameters of the peak sensor.

This method recalculates the smoothing factor (alpha) based on the current time constant (tPeak_ms) and sample rate (fs).

Reimplemented from [NtFx::ComponentBase< signal_t >](#).

Reimplemented in [NtFx::PeakHoldSensor< delayLineLength >](#).

8.41.3 Member Data Documentation**8.41.3.1 tPeak_ms**

```
signal_t NtFx::PeakSensor::tPeak_ms
```

Initial value:

```
{
    1
}
```

Time constant for peak detection in milliseconds.

The documentation for this struct was generated from the following file:

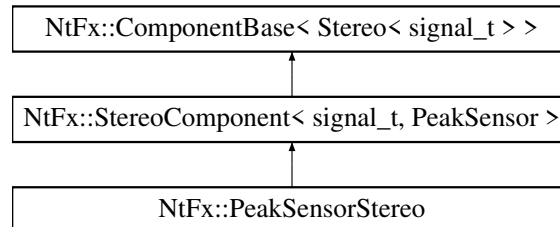
- [lib/PeakSensor.h](#)

8.42 NtFx::PeakSensorStereo Struct Reference

[Stereo](#) peak sensor component for audio processing.

```
#include <PeakSensor.h>
```

Inheritance diagram for NtFx::PeakSensorStereo:



Public Member Functions

- void [setT_ms](#) (signal_t t_ms)
Sets the time constant for peak detection in milliseconds.

Public Member Functions inherited from [NtFx::StereoComponent< signal_t, component_t >](#)

- virtual [Stereo< signal_t > process](#) ([Stereo< signal_t > x](#)) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float fs) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Additional Inherited Members

Public Attributes inherited from [NtFx::StereoComponent< signal_t, component_t >](#)

- component_t l
- component_t r

Public Attributes inherited from [NtFx::ComponentBase< Stereo< signal_t > >](#)

- float fs
Sample rate of component.

8.42.1 Detailed Description

[Stereo](#) peak sensor component for audio processing.

The [PeakSensorStereo](#) component is a stereo version of the [PeakSensor](#), allowing for independent peak detection on the left and right channels of an audio signal. This is useful in stereo audio effects where peak detection needs to be applied separately to each channel.

Template Parameters

<i>signal_t</i>	The type of the audio signal (e.g., float, double).
-----------------	---

8.42.2 Member Function Documentation

8.42.2.1 setT_ms()

```
void NtFx::PeakSensorStereo::setT_ms (
    signal_t t_ms) [inline]
```

Sets the time constant for peak detection in milliseconds.

This method updates the time constant for both the left and right channels.

Parameters

<i>t_ms</i>	The time constant in milliseconds.
-------------	------------------------------------

The documentation for this struct was generated from the following file:

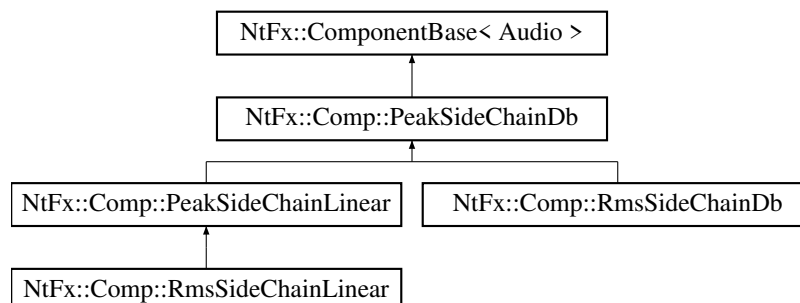
- lib/[PeakSensor.h](#)

8.43 NtFx::Comp::PeakSideChainDb Struct Reference

Peak-based side chain processor using dB domain.

```
#include <Comp.h>
```

Inheritance diagram for NtFx::Comp::PeakSideChainDb:



Public Member Functions

- [PeakSideChainDb](#) ([ScSettings](#) &[settings](#))
Constructor.
- virtual [Audio process](#) ([Audio](#) x) noexcept override
Process stereo audio signal.
- virtual void [update](#) () noexcept override
Update component coefficients.
- virtual void [reset](#) (float [fs](#)) noexcept override
Reset component with new sample rate.
- signal_t [_gainComputer_db](#) (signal_t x, signal_t &state) noexcept
Compute gain reduction in dB domain.

Public Attributes

- [PeakSensorStereo](#) [peakSensor](#)
Peak sensor for stereo signals.
- signal_t [alphaAtt](#) = signal_t(0)
Attack coefficient.
- signal_t [alphaRel](#) = signal_t(0)
Release coefficient.
- [ScSettings](#) & [settings](#)
Reference to side chain settings.
- [Audio stateFilter](#) = signal_t(0.0)
State filter for gain computation.

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float [fs](#)
Sample rate of component.

8.43.1 Detailed Description

Peak-based side chain processor using dB domain.

This component processes audio signals using peak detection and computes gain reduction in the dB domain. It includes attack and release time constants for dynamic control.

Template Parameters

signal_t	The signal type (e.g., float, double)
--------------------------	---------------------------------------

8.43.2 Constructor & Destructor Documentation

8.43.2.1 [PeakSideChainDb](#)()

```
NtFx::Comp::PeakSideChainDb::PeakSideChainDb (
    ScSettings & settings) [inline]
```

Constructor.

Parameters

<i>settings</i>	Reference to side chain settings
-----------------	----------------------------------

8.43.3 Member Function Documentation

8.43.3.1 `_gainComputer_db()`

```
signal_t NtFx::Comp::PeakSideChainDb::_gainComputer_db (  
    signal_t x,  
    signal_t & state) [inline], [noexcept]
```

Compute gain reduction in dB domain.

Parameters

<i>x</i>	Input signal level
<i>state</i>	Reference to state filter

Returns

Gain reduction factor

8.43.3.2 `process()`

```
virtual Audio NtFx::Comp::PeakSideChainDb::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Process stereo audio signal.

Parameters

<i>x</i>	Input stereo signal
----------	---------------------

Returns

Gain reduction in linear domain.

Implements [NtFx::ComponentBase< Audio >](#).

Reimplemented in [NtFx::Comp::PeakSideChainLinear](#), [NtFx::Comp::RmsSideChainDb](#), and [NtFx::Comp::RmsSideChainLinear](#).

8.43.3.3 `reset()`

```
virtual void NtFx::Comp::PeakSideChainDb::reset (  
    float fs) [inline], [override], [virtual], [noexcept]
```

Reset component with new sample rate.

Parameters

<i>fs</i>	Sample rate in Hz
-----------	-------------------

Reimplemented from [NtFx::ComponentBase< Audio >](#).

Reimplemented in [NtFx::Comp::RmsSideChainDb](#), and [NtFx::Comp::RmsSideChainLinear](#).

8.43.3.4 update()

```
virtual void NtFx::Comp::PeakSideChainDb::update () [inline], [override], [virtual], [noexcept]
```

Update component coefficients.

Reimplemented from [NtFx::ComponentBase< Audio >](#).

Reimplemented in [NtFx::Comp::PeakSideChainLinear](#), [NtFx::Comp::RmsSideChainDb](#), and [NtFx::Comp::RmsSideChainLinear](#).

The documentation for this struct was generated from the following file:

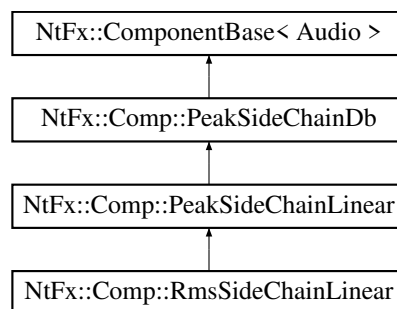
- lib/Comp.h

8.44 NtFx::Comp::PeakSideChainLinear Struct Reference

Peak-based side chain processor using linear domain.

```
#include <Comp.h>
```

Inheritance diagram for NtFx::Comp::PeakSideChainLinear:



Public Member Functions

- [PeakSideChainLinear](#) (ScSettings &settings)
Constructor.
- virtual [Audio process](#) (Audio x) noexcept override
Process stereo audio signal.
- virtual void [update](#) () noexcept override
Update component coefficients.
- [signal_t _gainComputer_lin](#) (signal_t x, signal_t &state) noexcept
Compute gain reduction in linear domain.

Public Member Functions inherited from NtFx::Comp::PeakSideChainDb

- [PeakSideChainDb](#) ([ScSettings](#) &[settings](#))
Constructor.
- virtual void [reset](#) (float [fs](#)) noexcept override
Reset component with new sample rate.
- signal_t [_gainComputer_db](#) (signal_t x, signal_t &state) noexcept
Compute gain reduction in dB domain.

Public Attributes

- signal_t [thresh_lin](#) = signal_t(1)
Threshold in linear domain.
- signal_t [ratio_lin](#) = signal_t(1)
Compression ratio in linear domain.
- signal_t [knee_lin](#) = signal_t(1)
Knee width in linear domain.

Public Attributes inherited from NtFx::Comp::PeakSideChainDb

- [PeakSensorStereo](#) [peakSensor](#)
Peak sensor for stereo signals.
- signal_t [alphaAtt](#) = signal_t(0)
Attack coefficient.
- signal_t [alphaRel](#) = signal_t(0)
Release coefficient.
- [ScSettings](#) & [settings](#)
Reference to side chain settings.
- [Audio](#) [stateFilter](#) = signal_t(0.0)
State filter for gain computation.

Public Attributes inherited from NtFx::ComponentBase< Audio >

- float [fs](#)
Sample rate of component.

8.44.1 Detailed Description

Peak-based side chain processor using linear domain.

This component processes audio signals using peak detection and computes gain reduction in the linear domain. It extends [PeakSideChainDb](#) with linear domain calculations.

Template Parameters

signal_t	The signal type (e.g., float, double)
--------------------------	---------------------------------------

8.44.2 Constructor & Destructor Documentation

8.44.2.1 PeakSideChainLinear()

```
NtFx::Comp::PeakSideChainLinear::PeakSideChainLinear (
    ScSettings & settings) [inline]
```

Constructor.

Parameters

<i>settings</i>	Reference to side chain settings
-----------------	----------------------------------

8.44.3 Member Function Documentation

8.44.3.1 `_gainComputer_lin()`

```
signal_t NtFx::Comp::PeakSideChainLinear::_gainComputer_lin (  
    signal_t x,  
    signal_t & state) [inline], [noexcept]
```

Compute gain reduction in linear domain.

Parameters

<i>x</i>	Input signal level
<i>state</i>	Reference to state filter

Returns

Gain reduction factor

8.44.3.2 `process()`

```
virtual Audio NtFx::Comp::PeakSideChainLinear::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Process stereo audio signal.

Parameters

<i>x</i>	Input stereo signal
----------	---------------------

Returns

Result of side chain. Multiply your input by this to apply side chain.

Reimplemented from [NtFx::Comp::PeakSideChainDb](#).

Reimplemented in [NtFx::Comp::RmsSideChainLinear](#).

8.44.3.3 update()

```
virtual void NtFx::Comp::PeakSideChainLinear::update () [inline], [override], [virtual],
[noexcept]
```

Update component coefficients.

Reimplemented from [NtFx::Comp::PeakSideChainDb](#).

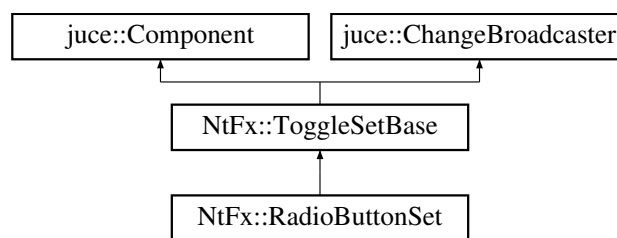
Reimplemented in [NtFx::Comp::RmsSideChainLinear](#).

The documentation for this struct was generated from the following file:

- lib/Comp.h

8.45 NtFx::RadioButtonSet Struct Reference

Inheritance diagram for NtFx::RadioButtonSet:



Public Member Functions

- **RadioButtonSet** ([RadioButtonSetSpec](#) &spec, [UiSpec](#) &uiSpec)
- virtual void [updateToggleStates](#) (int i) override

Public Member Functions inherited from [NtFx::ToggleSetBase](#)

- **ToggleSetBase** ([UiSpec](#) &uiSpec)
- std::unique_ptr< [Toggle](#) > **makeToggle** (std::string option, int i)
- void **resized** () override
- void **updateUi** ()

Public Attributes

- [RadioButtonSetSpec](#) & spec

Public Attributes inherited from [NtFx::ToggleSetBase](#)

- [UiSpec](#) & uiSpec
- std::vector< std::unique_ptr< [Toggle](#) > > toggles
- float uiScale = 1

Static Public Attributes

- static int **s_id** { 0 }

8.45.1 Member Function Documentation

8.45.1.1 updateToggleStates()

```
virtual void NtFx::RadioButtonSet::updateToggleStates (
    int i) [inline], [override], [virtual]
```

Reimplemented from [NtFx::ToggleSetBase](#).

The documentation for this struct was generated from the following files:

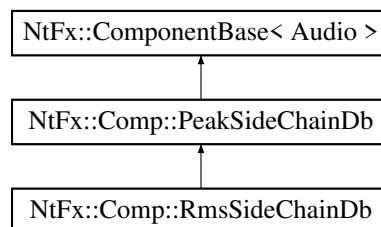
- JuceWrapper/RadioButton.h
- JuceWrapper/PluginEditor.cpp

8.46 NtFx::Comp::RmsSideChainDb Struct Reference

RMS-based side chain processor using dB domain.

```
#include <Comp.h>
```

Inheritance diagram for NtFx::Comp::RmsSideChainDb:



Public Member Functions

- **RmsSideChainDb** ([ScSettings](#) &[settings](#))
- virtual [Audio process](#) ([Audio x](#)) noexcept override
Process stereo audio signal.
- virtual void [update](#) () noexcept override
Update component parameters.
- virtual void [reset](#) (float [fs](#)) noexcept override
Reset component with new sample rate.

Public Member Functions inherited from NtFx::Comp::PeakSideChainDb

- [PeakSideChainDb](#) ([ScSettings](#) &[settings](#))
Constructor.
- [signal_t _gainComputer_db](#) ([signal_t](#) x, [signal_t](#) &state) noexcept
Compute gain reduction in dB domain.

Public Attributes

- [LongRmsSensor](#) **rmsSensor**
RMS sensor for stereo signals.

Public Attributes inherited from NtFx::Comp::PeakSideChainDb

- [PeakSensorStereo](#) **peakSensor**
Peak sensor for stereo signals.
- [signal_t](#) **alphaAtt** = [signal_t](#)(0)
Attack coefficient.
- [signal_t](#) **alphaRel** = [signal_t](#)(0)
Release coefficient.
- [ScSettings](#) & **settings**
Reference to side chain settings.
- [Audio](#) **stateFilter** = [signal_t](#)(0.0)
State filter for gain computation.

Public Attributes inherited from NtFx::ComponentBase< [Audio](#) >

- [float](#) **fs**
Sample rate of component.

8.46.1 Detailed Description

RMS-based side chain processor using dB domain.

This component processes audio signals using RMS detection and computes gain reduction in the dB domain. It extends [PeakSideChainDb](#) with RMS detection capabilities.

Template Parameters

signal_t	The signal type (e.g., float, double)
--------------------------	---------------------------------------

8.46.2 Member Function Documentation

8.46.2.1 process()

```
virtual Audio NtFx::Comp::RmsSideChainDb::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Process stereo audio signal.

Parameters

<i>x</i>	Input stereo signal
----------	---------------------

Returns

Processed stereo signal with gain reduction applied

Reimplemented from [NtFx::Comp::PeakSideChainDb](#).

8.46.2.2 reset()

```
virtual void NtFx::Comp::RmsSideChainDb::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Reset component with new sample rate.

Parameters

<i>fs</i>	Sample rate in Hz
-----------	-------------------

Reimplemented from [NtFx::Comp::PeakSideChainDb](#).

8.46.2.3 update()

```
virtual void NtFx::Comp::RmsSideChainDb::update () [inline], [override], [virtual], [noexcept]
```

Update component parameters.

Reimplemented from [NtFx::Comp::PeakSideChainDb](#).

The documentation for this struct was generated from the following file:

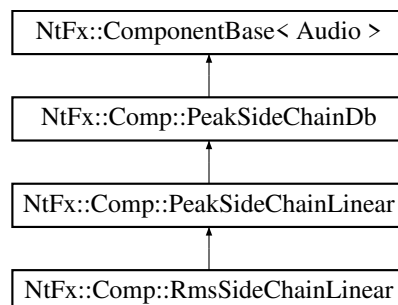
- lib/Comp.h

8.47 NtFx::Comp::RmsSideChainLinear Struct Reference

RMS-based side chain processor using linear domain.

```
#include <Comp.h>
```

Inheritance diagram for NtFx::Comp::RmsSideChainLinear:



Public Member Functions

- [RmsSideChainLinear](#) ([ScSettings](#) &[settings](#))
Constructor.
- virtual [Audio process](#) ([Audio](#) x) noexcept override
Process stereo audio signal.
- virtual void [update](#) () noexcept override
Update component parameters.
- virtual void [reset](#) (float [fs](#)) noexcept override
Reset component with new sample rate.

Public Member Functions inherited from [NtFx::Comp::PeakSideChainLinear](#)

- [PeakSideChainLinear](#) ([ScSettings](#) &[settings](#))
Constructor.
- signal_t [_gainComputer_lin](#) (signal_t x, signal_t &state) noexcept
Compute gain reduction in linear domain.

Public Member Functions inherited from [NtFx::Comp::PeakSideChainDb](#)

- [PeakSideChainDb](#) ([ScSettings](#) &[settings](#))
Constructor.
- signal_t [_gainComputer_db](#) (signal_t x, signal_t &state) noexcept
Compute gain reduction in dB domain.

Public Attributes

- [LongRmsSensor](#) [rmsSensor](#)
RMS sensor for stereo signals.

Public Attributes inherited from [NtFx::Comp::PeakSideChainLinear](#)

- signal_t [thresh_lin](#) = signal_t(1)
Threshold in linear domain.
- signal_t [ratio_lin](#) = signal_t(1)
Compression ratio in linear domain.
- signal_t [knee_lin](#) = signal_t(1)
Knee width in linear domain.

Public Attributes inherited from [NtFx::Comp::PeakSideChainDb](#)

- [PeakSensorStereo](#) [peakSensor](#)
Peak sensor for stereo signals.
- signal_t [alphaAtt](#) = signal_t(0)
Attack coefficient.
- signal_t [alphaRel](#) = signal_t(0)
Release coefficient.
- [ScSettings](#) & [settings](#)
Reference to side chain settings.
- [Audio stateFilter](#) = signal_t(0.0)
State filter for gain computation.

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float **fs**
Sample rate of component.

8.47.1 Detailed Description

RMS-based side chain processor using linear domain.

This component processes audio signals using RMS detection and computes gain reduction in the linear domain. It extends [PeakSideChainLinear](#) with RMS detection capabilities.

Template Parameters

<i>signal</i> ↔ _t	The signal type (e.g., float, double)
-----------------------	---------------------------------------

8.47.2 Constructor & Destructor Documentation

8.47.2.1 RmsSideChainLinear()

```
NtFx::Comp::RmsSideChainLinear::RmsSideChainLinear (
    ScSettings & settings) [inline]
```

Constructor.

Parameters

<i>settings</i>	Reference to side chain settings
-----------------	----------------------------------

8.47.3 Member Function Documentation

8.47.3.1 process()

```
virtual Audio NtFx::Comp::RmsSideChainLinear::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Process stereo audio signal.

Parameters

<i>x</i>	Input stereo signal
----------	---------------------

Returns

Processed stereo signal with gain reduction applied

Reimplemented from [NtFx::Comp::PeakSideChainLinear](#).

8.47.3.2 reset()

```
virtual void NtFx::Comp::RmsSideChainLinear::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Reset component with new sample rate.

Parameters

<i>fs</i>	Sample rate in Hz
-----------	-------------------

Reimplemented from [NtFx::Comp::PeakSideChainDb](#).

8.47.3.3 update()

```
virtual void NtFx::Comp::RmsSideChainLinear::update () [inline], [override], [virtual], [noexcept]
```

Update component parameters.

Reimplemented from [NtFx::Comp::PeakSideChainLinear](#).

The documentation for this struct was generated from the following file:

- lib/Comp.h

8.48 NtFx::Src::SampleRateConverter Struct Reference

Sample rate converter class.

```
#include <SampleRateConverter.h>
```

Public Member Functions

- [SampleRateConverter](#) (NtFx_PLUGIN &[plug](#))
Constructor.
- [Audio process](#) ([Audio](#) x)
Process audio samples through the sample rate converter.
- void **update** ()
Update coefficients based on current mode.
- void [reset](#) (float [fs](#))
Reset the sample rate converter and change the sample rate.

Public Attributes

- NtFx_PLUGIN & **plug**
Reference to the plugin.
- [State](#) **state**
State of the sample rate converter.
- [Coeffs](#) **coeffs**
Coefficients of the sample rate converter.
- [oversamplingMode](#) **mode**
Oversampling mode.
- float **fs**
Sampling rate.

8.48.1 Detailed Description

Sample rate converter class.

Template Parameters

<i>signal</i> ↔ _t	Type of signal samples
-----------------------	------------------------

8.48.2 Constructor & Destructor Documentation

8.48.2.1 SampleRateConverter()

```
NtFx::Src::SampleRateConverter::SampleRateConverter (
    NTFX_PLUGIN & plug) [inline]
```

Constructor.

Parameters

<i>plug</i>	Reference to the plugin
-------------	-------------------------

8.48.3 Member Function Documentation

8.48.3.1 process()

```
Audio NtFx::Src::SampleRateConverter::process (
    Audio x) [inline]
```

Process audio samples through the sample rate converter.

Parameters

<i>x</i>	Input audio samples
----------	---------------------

Returns

Processed audio samples

8.48.3.2 reset()

```
void NtFx::Src::SampleRateConverter::reset (
    float fs) [inline]
```

Reset the sample rate converter and change the sample rate.

Parameters

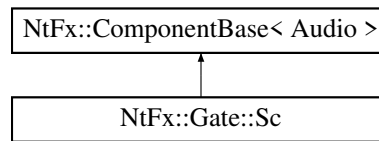
<i>fs</i>	Sampling rate
-----------	---------------

The documentation for this struct was generated from the following file:

- lib/[SampleRateConverter.h](#)

8.49 NtFx::Gate::Sc Struct Reference

Inheritance diagram for NtFx::Gate::Sc:



Public Member Functions

- virtual [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.
- signal_t [gateSc_db](#) (signal_t x)

Public Attributes

- [PeakSensor](#) **sensor**
- [ScSettings](#) **settings**
- State **state**
- signal_t **_slopeRel** { 0 }
- signal_t **_alphaAtt** { 0 }
- signal_t **_stateAtt** { 0 }
- signal_t **_stateRel** { 0 }
- int **_nHold** { 0 }
- int **_holdCount** { 0 }

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float **fs**
Sample rate of component.

8.49.1 Member Function Documentation

8.49.1.1 process()

```
virtual Audio NtFx::Gate::Sc::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

x	T Input
-----	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.49.1.2 reset()

```
virtual void NtFx::Gate::Sc::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.49.1.3 update()

```
virtual void NtFx::Gate::Sc::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- [lib/GateSc.h](#)

8.50 NtFx::Comp::ScSettings Struct Reference

Side chain settings structure.

```
#include <Comp.h>
```

Public Attributes

- `signal_t thresh_db` = `signal_t(0)`
Threshold in dB.
- `signal_t ratio_db` = `signal_t(2)`
Compression ratio.
- `signal_t knee_db` = `signal_t(12)`
Knee width in dB.
- `signal_t tAtt_ms` = `signal_t(1)`
Attack time in milliseconds.
- `signal_t tRel_ms` = `signal_t(100)`
Release time in milliseconds.
- `signal_t tRms_ms` = `signal_t(80)`
RMS time constant in milliseconds.
- `signal_t tPeak_ms` = `signal_t(20.0)`
Peak time constant in milliseconds.
- `bool linkEnable` = `false`

8.50.1 Detailed Description

Side chain settings structure.

Contains parameters for side chain processing including threshold, ratio, knee, attack time, release time, and time constants for RMS and peak detection.

Template Parameters

<code>signal_t</code>	The signal type (e.g., float, double)
-----------------------	---------------------------------------

The documentation for this struct was generated from the following file:

- `lib/Comp.h`

8.51 NtFx::Gate::ScSettings Struct Reference**Public Attributes**

- `signal_t thresh_db` { 0 }
- `signal_t range_db` { 0 }
- `signal_t tAtt_ms` { 0.01 }
- `signal_t tHold_ms` { 10 }
- `signal_t tRel_ms` { 200 }

The documentation for this struct was generated from the following file:

- `lib/GateSc.h`

8.52 NtFx::Biquad::Settings Struct Reference

Public Attributes

- Shape **shape** { Shape::bell }
- signal_t **fc_hz** { 1000.0 }
- signal_t **gain_db** { 0.0 }
- signal_t **q** { 0.707 }

The documentation for this struct was generated from the following file:

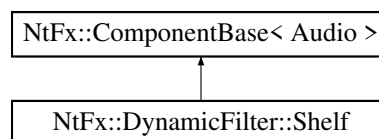
- lib/Biquad.h

8.53 NtFx::DynamicFilter::Shelf Struct Reference

An optimized dynamic shelving filter, which exposes both Q values. Feedback coeffs are calculated in update, while feed forward is calculated in process, so the gain of the filter can be updated while running, making the filter usefull for dynamic processing. Gain is in the linear domain for the same reason.

```
#include <DynamicFilter.h>
```

Inheritance diagram for NtFx::DynamicFilter::Shelf:



Public Member Functions

- virtual [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- [Biquad::EqBand6 _flt](#)
- signal_t **fc_hz** { 2000 }
Cutoff frequency.
- signal_t [gain_lin](#)
Gain in linear domain. Can be 0, creating a lpf.
- signal_t **q1** = 0.707
Q at the cutoff frequency.
- signal_t [q2](#)
Q where the response flattens (the other frequency).
- signal_t **_alphaSquared** { 0 }
- signal_t **_beta1** { 0 }
- signal_t **_beta2** { 0 }

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float **fs**
Sample rate of component.

8.53.1 Detailed Description

An optimized dynamic shelving filter, which exposes both Q values. Feedback coeffs are calculated in update, while feed forward is calculated in process, so the gain of the filter can be updated while running, making the filter usefull for dynamic processing. Gain is in the linear domain for the same reason.

Template Parameters

<i>signal</i> ↔	
<i>_t</i>	

8.53.2 Member Function Documentation

8.53.2.1 process()

```
virtual Audio NtFx::DynamicFilter::Shelf::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

x	T Input
---	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.53.2.2 reset()

```
virtual void NtFx::DynamicFilter::Shelf::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.53.2.3 update()

```
virtual void NtFx::DynamicFilter::Shelf::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.53.3 Member Data Documentation

8.53.3.1 gain_lin

```
signal_t NtFx::DynamicFilter::Shelf::gain_lin
```

Initial value:

```
{
    1
}
```

Gain in linear domain. Can be 0, creating a lpf.

8.53.3.2 q2

```
signal_t NtFx::DynamicFilter::Shelf::q2
```

Initial value:

```
=
    0.707
```

Q where the response flattens (the other frequency).

The documentation for this struct was generated from the following file:

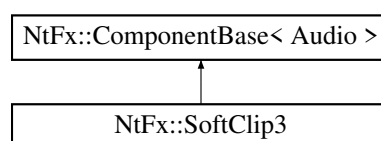
- [lib/DynamicFilter.h](#)

8.54 NtFx::SoftClip3 Class Reference

Third order soft clipper wrapped in a Component.

```
#include <SoftClip.h>
```

Inheritance diagram for NtFx::SoftClip3:



Additional Inherited Members

Public Member Functions inherited from [NtFx::ComponentBase< Audio >](#)

- virtual void [update](#) () noexcept
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float [fs](#)
Sample rate of component.

8.54.1 Detailed Description

Third order soft clipper wrapped in a Component.

Template Parameters

<i>signal</i> ↔ _t	Audio datatype.
-----------------------	-----------------

The documentation for this class was generated from the following file:

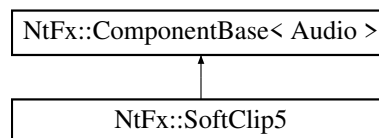
- lib/[SoftClip.h](#)

8.55 NtFx::SoftClip5 Class Reference

Fifth order soft clipper wrapped in a Component.

```
#include <SoftClip.h>
```

Inheritance diagram for NtFx::SoftClip5:



Additional Inherited Members

Public Member Functions inherited from [NtFx::ComponentBase< Audio >](#)

- virtual void [update](#) () noexcept
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float **fs**
Sample rate of component.

8.55.1 Detailed Description

Fifth order soft clipper wrapped in a Component.

Template Parameters

<i>signal</i> ↔ _t	Audio datatype.
-----------------------	-----------------

The documentation for this class was generated from the following file:

- lib/[SoftClip.h](#)

8.56 NtFx::Biquad::State Struct Reference

Public Attributes

- signal_t **x** [2] { 0, 0 }
- signal_t **y** [2] { 0, 0 }

The documentation for this struct was generated from the following file:

- lib/Biquad.h

8.57 NtFx::Src::State Struct Reference

[State](#) structure for sample rate converter.

```
#include <SampleRateConverter.h>
```

Public Attributes

- size_t **iStoreIn** { 0 }
Index for storing input samples.
- size_t **iStoreOut** { 0 }
Index for storing output samples.
- std::array< [Audio](#), [nDelayLine](#) *2 > **dlInterpolation**
Delay line for interpolation filter (at base sample rate).
- std::array< [Audio](#), [nDelayLine](#) *2 > **dlAntialiasing**
Delay line for antialiasing (stores processed samples at high sample rate).

8.57.1 Detailed Description

[State](#) structure for sample rate converter.

Template Parameters

<i>signal_t</i>	Type of signal samples
-----------------	------------------------

The documentation for this struct was generated from the following file:

- lib/[SampleRateConverter.h](#)

8.58 NtFx::Stereo< signal_t > Struct Template Reference

Wraps two values as a stereo signal. This allows us to do DSP math on both channels at the same time. The class overloads all the normal math functions. Please note that comparison operators are different for stereo signals and that $a < b$ not necessarily equals $b > a$ since ' $<$ ' compares the minimum values while ' $>$ ' compares maximums.

```
#include <Audio.h>
```

Public Member Functions

- **Stereo** (signal_t x)
- **Stereo** (signal_t l, signal_t r)
- **Stereo** ([Mono](#)< signal_t > l, [Mono](#)< signal_t > r)
- **Stereo**< signal_t > & **operator=** (const **Stereo**< signal_t > &x) noexcept
- **Stereo**< signal_t > & **operator=** (const signal_t &x) noexcept
- bool **operator==** (const **Stereo**< signal_t > &x) const noexcept
- bool **operator<** (const **Stereo**< signal_t > &x) const noexcept
- bool **operator>** (const **Stereo**< signal_t > &x) const noexcept
- bool **operator<=** (const **Stereo**< signal_t > &x) const noexcept
- bool **operator>=** (const **Stereo**< signal_t > &x) const noexcept
- bool **operator<** (const signal_t &x) const noexcept
- bool **operator>** (const signal_t &x) const noexcept
- bool **operator<=** (const signal_t &x) const noexcept
- bool **operator>=** (const signal_t &x) const noexcept
- **Stereo**< signal_t > & **operator*=** (const signal_t x) noexcept
- **Stereo**< signal_t > & **operator*=** (const **Stereo**< signal_t > x) noexcept
- **Stereo**< signal_t > & **operator+=** (const signal_t x) noexcept
- **Stereo**< signal_t > & **operator+=** (const **Stereo**< signal_t > x) noexcept
- **Stereo**< signal_t > **operator-** () const
- signal_t **avgSquared** () const noexcept

Square left and right channels and take the average.
- signal_t **absMax** () const noexcept

Returns the larges of left and right disregarding sign.
- signal_t **absMin** () const noexcept

Returns the smallest of left and right disregarding sign.
- **Stereo**< signal_t > **abs** () const noexcept

Returns the abs of both channels.

Public Attributes

- `signal_l { 0 }`
- `signal_r { 0 }`

8.58.1 Detailed Description

template<typename signal_t>
struct NtFx::Stereo< signal_t >

Wraps two values as a stereo signal. This allows us to do DSP math on both channels at the same time. The class overloads all the normal math functions. Please note that comparison operators are different for stereo signals and that `a < b` not necessarily equals `b > a` since '`<`' compares the minimum values while '`>`' compares maximums.

Template Parameters

<i>signal_t</i>	Audio datatype.
-----------------	-----------------

8.58.2 Member Function Documentation

8.58.2.1 abs()

```
template<typename signal_t>
Stereo< signal_t > NtFx::Stereo< signal_t >::abs () const [inline], [noexcept]
```

Returns the abs of both channels.

Returns

`Stereo<signal_t>`

8.58.2.2 absMax()

```
template<typename signal_t>
signal_t NtFx::Stereo< signal_t >::absMax () const [inline], [noexcept]
```

Returns the larges of left and right disregarding sign.

Returns

`signal_t`

8.58.2.3 absMin()

```
template<typename signal_t>
signal_t NtFx::Stereo< signal_t >::absMin () const [inline], [noexcept]
```

Returns the smallest of left and right disregarding sign.

Returns

signal_t

8.58.2.4 avgSquared()

```
template<typename signal_t>
signal_t NtFx::Stereo< signal_t >::avgSquared () const [inline], [noexcept]
```

Square left and right channels and take the average.

Returns

signal_t Result.

The documentation for this struct was generated from the following file:

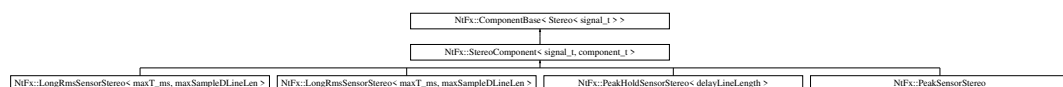
- lib/Audio.h

8.59 NtFx::StereoComponent< signal_t, component_t > Struct Template Reference

Wraps two objects of type Component<signal_t> as a Component<Audio> and calls through the both objects.

```
#include <Component.h>
```

Inheritance diagram for NtFx::StereoComponent< signal_t, component_t >:



Public Member Functions

- virtual Stereo< signal_t > [process](#) (Stereo< signal_t > x) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float fs) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- `component_t l`
- `component_t r`

Public Attributes inherited from [NtFx::ComponentBase< Stereo< signal_t > >](#)

- float `fs`
Sample rate of component.

8.59.1 Detailed Description

```
template<typename signal_t, typename component_t>
struct NtFx::StereoComponent< signal_t, component_t >
```

Wraps two objects of type `Component<signal_t>` as a `Component<Audio>` and calls through the both objects.

Template Parameters

<i>signal_t</i>	Datatype for signal. Must be a floating point type.
<i>component_t</i>	Type of Component to wrap.

8.59.2 Member Function Documentation**8.59.2.1 process()**

```
template<typename signal_t, typename component_t>
virtual Stereo< signal_t > NtFx::StereoComponent< signal_t, component_t >::process (
    Stereo< signal_t > x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<i>x</i>	T Input
----------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Stereo< signal_t > >](#).

8.59.2.2 reset()

```
template<typename signal_t, typename component_t>
virtual void NtFx::StereoComponent< signal_t, component_t >::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Stereo< signal_t > >](#).

8.59.2.3 update()

```
template<typename signal_t, typename component_t>
virtual void NtFx::StereoComponent< signal_t, component_t >::update () [inline], [override],
[virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

Reimplemented from [NtFx::ComponentBase< Stereo< signal_t > >](#).

The documentation for this struct was generated from the following file:

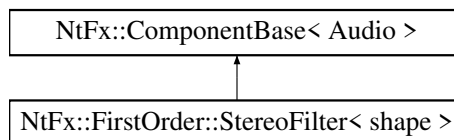
- [lib/Component.h](#)

8.60 NtFx::FirstOrder::StereoFilter< shape > Struct Template Reference

[Stereo](#) pair of first order filters.

```
#include <FirstOrder.h>
```

Inheritance diagram for NtFx::FirstOrder::StereoFilter< shape >:



Public Member Functions

- [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float [fs](#)) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.
- void [setFc](#) (signal_t fc) noexcept

Public Attributes

- [Filter](#)< shape > l
- [Filter](#)< shape > r

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float **fs**
Sample rate of component.

8.60.1 Detailed Description

```
template<Shape shape>
struct NtFx::FirstOrder::StereoFilter< shape >
```

[Stereo](#) pair of first order filters.

Template Parameters

<i>signal</i> ↔ _t	Audio datatype.
<i>shape</i>	Filter shape.

8.60.2 Member Function Documentation

8.60.2.1 process()

```
template<Shape shape>
Audio NtFx::FirstOrder::StereoFilter< shape >::process (
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<i>x</i>	T Input
----------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.60.2.2 reset()

```
template<Shape shape>
virtual void NtFx::FirstOrder::StereoFilter< shape >::reset (
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.60.2.3 update()

```
template<Shape shape>
virtual void NtFx::FirstOrder::StereoFilter< shape >::update () [inline], [override], [virtual],
[noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

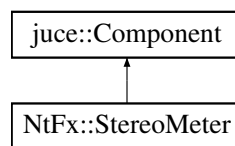
Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- lib/[FirstOrder.h](#)

8.61 NtFx::StereoMeter Struct Reference

Inheritance diagram for NtFx::StereoMeter:



Public Member Functions

- **StereoMeter** ([MeterSpec](#) &meterSpec, [UiSpec](#) &uiSpec)
- void **resized** () override
- void **updateUi** ()
- void **refresh** ([Audio](#) xPeak, [Audio](#) xRms)

Public Attributes

- [Meter](#) l
- [Meter](#) r
- [UiSpec](#) & spec
- juce::Label label
- int **fontSize** { 0 }
- float **uiScale** { 1 }
- bool **hasScale** { false }
- bool **onlyShowLeft** { false }

The documentation for this struct was generated from the following file:

- JuceWrapper/Meter.h

8.62 NtFx::Biquad::StereoState Struct Reference

Public Attributes

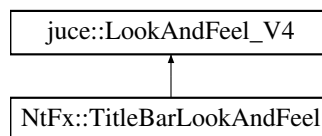
- [State l](#)
- [State r](#)

The documentation for this struct was generated from the following file:

- lib/Biquad.h

8.63 NtFx::TitleBarLookAndFeel Struct Reference

Inheritance diagram for NtFx::TitleBarLookAndFeel:



Public Member Functions

- juce::Font **getLabelFont** (juce::Label &l) override

Public Attributes

- float **fontSize** = 1
- float **uiScale** = 1
- uint32_t **backgroundColour** { 0xFFD3D3D3 }
- uint32_t **foregroundColour** { 0xFF000000 }

The documentation for this struct was generated from the following file:

- JuceWrapper/LookAndFeel.h

8.64 NtFx::TitleBarSpec Struct Reference

Specifies the layout of the title bar.

```
#include <UiSpec.h>
```

Public Attributes

- std::vector< [DropDownSpec](#) > **dropdowns**

8.64.1 Detailed Description

Specifies the layout of the title bar.

8.64.2 Member Data Documentation

8.64.2.1 dropdowns

```
std::vector<DropDownSpec> NtFx::TitleBarSpec::dropdowns
```

Initial value:

```
{
    {
        nullptr,
        "UI_Scale",
        { "50%", "75%", "100%", "125%", "150%", "175%", "200%" },
        2,
    },
    { nullptr, "Theme", { "Light", "Dark" }, 1 },
    {
        nullptr,
        "Oversampling",
        { "disable",

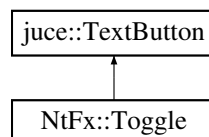
            "fir_2x_lq",
            "fir_4x_lq",
            "fir_8x_lq",
            "fir_2x_hq",
            "fir_4x_hq",
            "fir_8x_hq" },
        0,
    },
}
```

The documentation for this struct was generated from the following file:

- [lib/UiSpec.h](#)

8.65 NtFx::Toggle Class Reference

Inheritance diagram for NtFx::Toggle:



Public Member Functions

- **Toggle** (const juce::String &buttonName={ }, int _id=0)
- void **paintButton** (juce::Graphics &g, const bool isMouseOver, const bool isMouseDown) override

Public Attributes

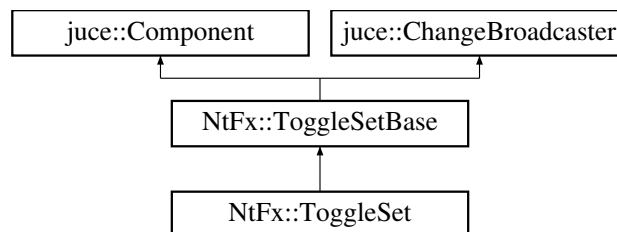
- `int _id { 0 }`
- `float fontSize { 10 }`
- `uint32_t colour { 0xFFFFFFFF }`

The documentation for this class was generated from the following file:

- `JuceWrapper/Toggle.h`

8.66 NtFx::ToggleSet Struct Reference

Inheritance diagram for `NtFx::ToggleSet`:



Public Member Functions

- **ToggleSet** ([ToggleSetSpec](#) spec, [UiSpec](#) &uiSpec)

Public Member Functions inherited from [NtFx::ToggleSetBase](#)

- **ToggleSetBase** ([UiSpec](#) &uiSpec)
- `std::unique_ptr< Toggle > makeToggle (std::string option, int i)`
- virtual void **updateToggleStates** (int i)
- void **resized** () override
- void **updateUi** ()

Public Attributes

- [ToggleSetSpec](#) spec

Public Attributes inherited from [NtFx::ToggleSetBase](#)

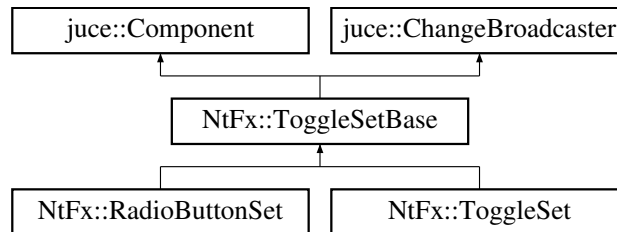
- [UiSpec](#) & uiSpec
- `std::vector< std::unique_ptr< Toggle > > toggles`
- `float uiScale = 1`

The documentation for this struct was generated from the following file:

- `JuceWrapper/RadioButton.h`

8.67 NtFx::ToggleSetBase Struct Reference

Inheritance diagram for NtFx::ToggleSetBase:



Public Member Functions

- **ToggleSetBase** ([UiSpec](#) &uiSpec)
- std::unique_ptr< [Toggle](#) > **makeToggle** (std::string option, int i)
- virtual void **updateToggleStates** (int i)
- void **resized** () override
- void **updateUi** ()

Public Attributes

- [UiSpec](#) & **uiSpec**
- std::vector< std::unique_ptr< [Toggle](#) > > **toggles**
- float **uiScale** = 1

The documentation for this struct was generated from the following file:

- JuceWrapper/RadioButton.h

8.68 NtFx::ToggleSetSpec Struct Reference

Public Attributes

- std::string **name**
- std::vector< [ToggleSpec](#) > **toggles**

The documentation for this struct was generated from the following file:

- lib/[UiSpec.h](#)

8.69 NtFx::ToggleSpec Struct Reference

Specification for toggles in UI.

```
#include <UiSpec.h>
```

Public Attributes

- `bool * p_val`
Pointer to value the knob represents. Used to bind to UI.
- `std::string name`
- `bool _defaultVal { false }`

8.69.1 Detailed Description

Specification for toggles in UI.

8.69.2 Member Data Documentation

8.69.2.1 name

```
std::string NtFx::ToggleSpec::name
```

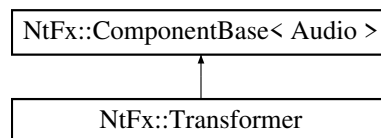
Name of knob, used for IDs and label in UI. Can't contain ' '. '_' is replaced with ' ' in UI labels.

The documentation for this struct was generated from the following file:

- [lib/UiSpec.h](#)

8.70 NtFx::Transformer Struct Reference

Inheritance diagram for NtFx::Transformer:



Public Member Functions

- virtual [Audio process](#) ([Audio](#) x) noexcept override
Called for every sample as audio is processed.
- virtual void [update](#) () noexcept override
Called when ever a parameter (knob or toggle) changes. Update your coefficients here.
- virtual void [reset](#) (float fs) noexcept override
Called when the plugin loads and when ever the samplerate or buffer size changes.

Public Attributes

- `signal_t fc_hz = 250`
- `signal_t lfCutoff_hz = 20`
- `signal_t gain_lin { 0 }`
- [NtFx::FirstOrder::StereoFilter](#) < [NtFx::FirstOrder::Shape::lpf](#) > **lpf**
- [NtFx::FirstOrder::StereoFilter](#) < [NtFx::FirstOrder::Shape::hpf](#) > **hpf**

Public Attributes inherited from [NtFx::ComponentBase< Audio >](#)

- float **fs**
Sample rate of component.

8.70.1 Member Function Documentation

8.70.1.1 `process()`

```
virtual Audio NtFx::Transformer::process (  
    Audio x) [inline], [override], [virtual], [noexcept]
```

Called for every sample as audio is processed.

Parameters

<i>x</i>	T Input
----------	---------

Returns

T output.

Implements [NtFx::ComponentBase< Audio >](#).

8.70.1.2 `reset()`

```
virtual void NtFx::Transformer::reset (  
    float fs) [inline], [override], [virtual], [noexcept]
```

Called when the plugin loads and when ever the samplerate or buffer size changes.

Parameters

<i>fs</i>	Sample rate. If you need it for calculateCoeffs, store it.
-----------	--

Reimplemented from [NtFx::ComponentBase< Audio >](#).

8.70.1.3 `update()`

```
virtual void NtFx::Transformer::update () [inline], [override], [virtual], [noexcept]
```

Called when ever a parameter (knob or toggle) changes. Update your coefficients here.

Reimplemented from [NtFx::ComponentBase< Audio >](#).

The documentation for this struct was generated from the following file:

- lib/Transformer.h

8.71 NtFx::UiSpec Struct Reference

General settings for to UI.

```
#include <UiSpec.h>
```

Public Attributes

- bool **includeMeters** { true }
Add meters to the UI.
- bool **includeTitleBar** { true }
Add title bar to UI.
- bool **includeSecondaryKnobs** { true }
Enable row below the main grid.
- uint32_t **foregroundColour** { 0xFFFFFFFF }
UI foreground colour in HEX.
- uint32_t **backgroundColour** { 0xFF000000 }
UI background colour in HEX.
- int **maxRows** { 3 }
The maximum number of rows in main knob grid.
- int **maxColumns** { 6 }
The maximum number of columns in main knob grid.
- float **defaultFontSize** { 16 }
Font size before scaling.
- float **labelHeight** { 20 }
Height of all text labels in the UI.
- float **toggleHeight** { 45 }
Height of toggle row at the bottom of the UI.
- float **radioButtonHeight** { 25 }
Height of each separate radio button.
- float **radioButtonAreaWidth** { 125 }
Width of the radio buttons ares.
- float **knobWidth** { 150 }
Width of primary knobs.
- float **knobHeight** { 200 }
Height or row of knobs in grid in UI.
- float **secondaryKnobWidth** { 75 }
Width of secondary knobs in UI.
- float **secondaryKnobHeight** { 115 }
Height of secondary knobs in UI.
- float **titleBarHeight** { 22 }
Height of title bar in pixels.
- int **meterHeight_dots** { 12 }
Number of dots in the meteres.
- int **meterWidth** { 35 }
Width of each meter in pixels.
- float **meterRefreshRate_hz** { 50 }
Refresh rate of meters in the UI.

8.71.1 Detailed Description

General settings for to UI.

The documentation for this struct was generated from the following file:

- [lib/UiSpec.h](#)

Chapter 9

File Documentation

9.1 LookAndFeel.h

```
00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  */
00017
00018 #pragma once
00019
00020 #include <stdint>
00021 #include <juce_audio_basics/juce_audio_basics.h>
00022 #include <juce_audio_devices/juce_audio_devices.h>
00023 #include <juce_audio_formats/juce_audio_formats.h>
00024 #include <juce_audio_plugin_client/juce_audio_plugin_client.h>
00025 #include <juce_audio_processors/juce_audio_processors.h>
00026 #include <juce_audio_processors_headless/juce_audio_processors_headless.h>
00027 #include <juce_audio_utils/juce_audio_utils.h>
00028 #include <juce_core/juce_core.h>
00029 #include <juce_data_structures/juce_data_structures.h>
00030 #include <juce_events/juce_events.h>
00031 #include <juce_graphics/juce_graphics.h>
00032 #include <juce_gui_basics/juce_gui_basics.h>
00033 #include <juce_gui_extra/juce_gui_extra.h>
00034
00035 namespace NtFx {
00036 // TODO: A prettier knob.
00037 // TODO: Make it a component instead like Toggle. Maybe...
00038 // TODO: OR build an NrPluginLookAndFeel that holds all of it.
00039 struct KnobLookAndFeel : public juce::LookAndFeel_V4 {
00040     float fontSize = 1;
00041     float uiScale = 1;
00042     // TODO: Use Juce' colour system instead. This is inconsistent.
00043     uint32_t backgroundColour { 0xFFD3D3D3 };
00044     uint32_t foregroundColour { 0xFF000000 };
00045
00046     juce::Font getLabelFont(juce::Label& l) override {
00047         return juce::Font(juce::FontOptions(this->fontSize));
00048     }
00049     void drawRotarySlider(juce::Graphics& g,
00050         int x,
00051         int y,
00052         int width,
00053         int height,
00054         float sliderPos,
00055         const float rotaryStartAngle,
00056         const float rotaryEndAngle,
00057         juce::Slider& slider) override {
```

```

00058     auto centreX = x + width / 2.0f;
00059     auto centreY = y + height / 2.0f;
00060     auto angle =
00061         rotaryStartAngle + sliderPos * (rotaryEndAngle - rotaryStartAngle);
00062     juce::Path knobPath;
00063     auto outerRadius = std::min(width / 2.5f, height / 2.5f) - 4.0f;
00064     knobPath.addPolygon({ 0, 0 }, 12, outerRadius);
00065     knobPath.applyTransform(
00066         juce::AffineTransform::rotation(angle).translated(centreX, centreY));
00067     auto color = this->foregroundColour;
00068     if (!slider.isEnabled()) { color &= 0x5FFFFFFF; }
00069     g.setColour(juce::Colour(color));
00070     g.strokePath(knobPath, juce::PathStrokeType(2 * this->uiScale));
00071     g.setColour(juce::Colour(this->backgroundColour));
00072     g.fillPath(knobPath);
00073     auto pointerLength = outerRadius * 0.5;
00074     auto pointerThickness = 4.0;
00075     juce::Path pointerPath;
00076     g.setColour(juce::Colour(color));
00077     pointerPath.addRectangle(-pointerThickness * 0.5f,
00078         -outerRadius,
00079         pointerThickness,
00080         pointerLength);
00081     pointerPath.applyTransform(
00082         juce::AffineTransform::rotation(angle).translated(centreX, centreY));
00083     g.fillPath(pointerPath);
00084 }
00085 };
00086 struct TitleBarLookAndFeel : public juce::LookAndFeel_V4 {
00087     float fontSize = 1;
00088     float uiScale = 1;
00089     uint32_t backgroundColour { 0xFFD3D3D3 };
00090     uint32_t foregroundColour { 0xFF000000 };
00091
00092     juce::Font getLabelFont(juce::Label& l) override {
00093         return juce::Font(juce::FontOptions(this->fontSize));
00094     }
00095 };
00096 } // namespace NtFx

```

9.2 Meter.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  */
00017
00018 #pragma once
00019
00020 #include "gcem.hpp"
00021 #include "juce_core/system/juce_PlatformDefs.h"
00022 #include "lib/Audio.h"
00023 #include "lib/PeakSensor.h"
00024 #include "lib/UiSpec.h"
00025 #include "lib/Utils.h"
00026
00027 #include <cstdint>
00028 #include <juce_audio_basics/juce_audio_basics.h>
00029 #include <juce_audio_devices/juce_audio_devices.h>
00030 #include <juce_audio_formats/juce_audio_formats.h>
00031 #include <juce_audio_plugin_client/juce_audio_plugin_client.h>
00032 #include <juce_audio_processors/juce_audio_processors.h>
00033 #include <juce_audio_processors_headless/juce_audio_processors_headless.h>
00034 #include <juce_audio_utils/juce_audio_utils.h>
00035 #include <juce_core/juce_core.h>
00036 #include <juce_data_structures/juce_data_structures.h>
00037 #include <juce_events/juce_events.h>
00038 #include <juce_graphics/juce_graphics.h>
00039 #include <juce_gui_basics/juce_gui_basics.h>
00040 #include <juce_gui_extra/juce_gui_extra.h>
00041

```

```

00042 #include <cstdint>
00043 #include <memory>
00044 #include <string>
00045 #include <vector>
00046
00047 namespace NtFx {
00048 struct Meter : public juce::Component {
00049     PeakHoldSensor<> peakSensor;
00050     MeterSpec& meterSpec;
00051     UiSpec& uiSpec;
00052     std::string label { "" };
00053     int pad { 10 };
00054     int dotDiameter { 0 };
00055     int dotDist { 0 };
00056     int nDots { 14 };
00057     int nActiveDotsPeak { 0 };
00058     float fractPeak { 0 };
00059     int nActiveDotsRms { 0 };
00060     float peakVal_lin { 0 };
00061     float rmsVal_lin { 0 };
00062     float dbPrDot { 0 };
00063     float opacity { 0.7f };
00064     bool isInitialized { false };
00065     int fontSize { 20 };
00066     int nHold_frames { 0 };
00067     int holdCounter_frames { 0 };
00068     float holdVal_db { 0 };
00069     int iHoldDot { 0 };
00070     bool hasScale { false };
00071
00072     Meter(MeterSpec& meterSpec, UiSpec& uiSpec)
00073         : meterSpec(meterSpec), uiSpec(uiSpec), nDots(uiSpec.meterHeight_dots) {
00074         // TODO: We need to be able to set these from the plugin.
00075         this->updateRelease(48000, 250, 100);
00076         this->refresh();
00077         this->isInitialized = true;
00078     };
00079     ~Meter() = default;
00080
00081     void paint(juce::Graphics& g) override {
00082         if (!this->isInitialized) { return; }
00083         if (this->getWidth() <= 0) { return; }
00084         int y = this->pad;
00085         g.setColour(juce::Colour(this->uiSpec.foregroundColour));
00086         g.setFont(this->fontSize);
00087         g.drawText(this->label,
00088             0,
00089             y,
00090             this->getWidth(),
00091             this->fontSize,
00092             juce::Justification::centred);
00093         for (size_t i = 0; i < this->nDots; i++) {
00094             int y;
00095             y = this->pad + (i + 1) * this->dotDist;
00096             // if (this->meterSpec.invert && i < 2) { continue; }
00097             g.setColour(juce::Colour(this->uiSpec.foregroundColour));
00098             g.drawEllipse(this->pad, y, this->dotDiameter, this->dotDiameter, 1);
00099             float fillPad = this->getWidth() * 4.0 / 35.0;
00100             float fillDiameter = this->dotDiameter - fillPad;
00101             if (fillDiameter < 0) { return; }
00102             float fillX = this->pad + fillPad / 2;
00103             float fillY = y + fillPad / 2;
00104
00105             // Filled regular dots.
00106             if ((!this->meterSpec.invert && i >= this->nActiveDotsPeak)
00107                 || (this->meterSpec.invert && i <= this->nActiveDotsPeak - 1)
00108                 && this->nActiveDotsPeak != 0) {
00109                 uint8_t opacity = 255.0f * this->opacity;
00110                 g.setColour(juce::Colour(
00111                     this->uiSpec.foregroundColour & 0x00FFFFFF | (opacity << 24)));
00112                 g.fillEllipse(fillX, fillY, fillDiameter, fillDiameter);
00113             }
00114
00115             // Fractional.
00116             if (this->meterSpec.invert && i == this->nActiveDotsPeak
00117                 && this->nActiveDotsPeak != 0) {
00118                 uint8_t opacity = 255.0f * (1 - this->fractPeak);
00119                 g.setColour(juce::Colour(
00120                     this->uiSpec.foregroundColour & 0x00FFFFFF | (opacity << 24)));
00121                 g.fillEllipse(fillX, fillY, fillDiameter, fillDiameter);
00122             }
00123             if (!this->meterSpec.invert && i == this->nActiveDotsPeak - 1) {
00124                 uint8_t opacity = 255.0f * this->fractPeak;
00125                 g.setColour(juce::Colour(
00126                     this->uiSpec.foregroundColour & 0x00FFFFFF | (opacity << 24)));
00127                 g.fillEllipse(fillX, fillY, fillDiameter, fillDiameter);
00128             }

```

```

00129
00130 // RMS overlay.
00131 if (!this->meterSpec.invert && i >= this->nActiveDotsRms) {
00132     uint8_t opacity = 255.0f * this->opacity;
00133     g.setColour(juce::Colour(
00134         this->uiSpec.foregroundColor & 0x00FFFFFF | (opacity << 24)));
00135     g.fillEllipse(fillX, fillY, fillDiameter, fillDiameter);
00136 }
00137
00138 // Peak hold ring.
00139 if (i == this->iHoldDot && this->meterSpec.hold_s
00140     && !(this->meterSpec.invert && i == 0)) {
00141     g.setColour(juce::Colour(
00142         this->uiSpec.foregroundColor)); // & 0x00FFFFFF | 0x8F000000));
00143     g.drawEllipse(fillX, fillY, fillDiameter, fillDiameter, 1);
00144 }
00145 }
00146 }
00147
00148 void resized() override { this->repaint(); }
00149
00150 void refresh(float xPeak, float xRms) {
00151     this->peakVal_lin = xPeak;
00152     this->rmsVal_lin = xRms;
00153     this->refresh();
00154 }
00155
00156 void refresh(bool repaint = true) {
00157     float ySens;
00158     ensureFinite(this->peakVal_lin);
00159     if (this->meterSpec.invert) {
00160         ySens = 1.0f - peakSensor.process(1.0f - this->peakVal_lin);
00161     } else {
00162         ySens = peakSensor.process(this->peakVal_lin);
00163     }
00164     float peak_db = NtFx::db(ySens);
00165     float rms_db = NtFx::db(this->rmsVal_lin);
00166
00167     if (peak_db < this->meterSpec.minVal_db) {
00168         peak_db = this->meterSpec.minVal_db;
00169     }
00170     if (peak_db > this->meterSpec.maxVal_db) {
00171         peak_db = this->meterSpec.maxVal_db;
00172     }
00173     if (rms_db < this->meterSpec.minVal_db) {
00174         rms_db = this->meterSpec.minVal_db;
00175     }
00176     if (rms_db > this->meterSpec.maxVal_db) {
00177         rms_db = this->meterSpec.maxVal_db;
00178     }
00179     this->nActiveDotsPeak = this->calcActiveDots(peak_db);
00180     this->nActiveDotsRms = this->calcActiveDots(rms_db);
00181     this->refreshPeakHold(peak_db);
00182     this->fractPeak = gcem::abs(peak_db + this->nActiveDotsPeak * this->dbPrDot)
00183         / this->dbPrDot;
00184     jassert(this->fractPeak <= 1 && this->fractPeak >= 0);
00185
00186     auto w = this->getWidth();
00187     if (!repaint || !w) { w = this->uiSpec.meterWidth; }
00188     this->pad = w * 10.0 / this->uiSpec.meterWidth;
00189     this->dotDiameter = w * 15.0 / this->uiSpec.meterWidth;
00190     this->dotDist = this->pad + this->dotDiameter;
00191     if (repaint) { this->repaint(); }
00192 }
00193
00194 void updateRelease(float fs, float t_ms, float tHold_ms = 100) {
00195     this->peakSensor.tPeak_ms = t_ms;
00196     this->peakSensor.tHold_ms = tHold_ms;
00197     this->peakSensor.reset(fs);
00198     this->nHold_frames =
00199         this->meterSpec.hold_s * this->uiSpec.meterRefreshRate_hz;
00200     this->dbPrDot =
00201         (this->meterSpec.maxVal_db - this->meterSpec.minVal_db) / this->nDots;
00202 }
00203
00204 int calcActiveDots(float peak_db) {
00205     int nActiveDots;
00206     nActiveDots =
00207         (peak_db + this->meterSpec.maxVal_db - this->meterSpec.minVal_db)
00208         / this->dbPrDot;
00209     return this->nDots - nActiveDots;
00210 }
00211
00212 void refreshPeakHold(float peak_db) {
00213     if ((!this->meterSpec.invert && peak_db > this->holdVal_db)
00214         || (this->meterSpec.invert && peak_db < this->holdVal_db)) {
00215         this->holdVal_db = peak_db;

```

```

00216         this->iHoldDot          = this->nActiveDotsPeak - 1;
00217         this->holdCounter_frames = 0;
00218         return;
00219     }
00220     this->holdCounter_frames++;
00221     if (this->holdCounter_frames > this->nHold_frames) {
00222         this->holdCounter_frames = 0;
00223         if (this->meterSpec.invert) {
00224             this->holdVal_db = this->meterSpec.maxVal_db;
00225         } else {
00226             this->holdVal_db = this->meterSpec.minVal_db;
00227         }
00228         this->iHoldDot = this->nActiveDotsPeak - 1;
00229     }
00230 }
00231 };
00232
00233 struct MeterScale : public juce::Component {
00234     Meter& meter;
00235     MeterScale(Meter& m) : meter(m) { }
00236     void paint(juce::Graphics& g) override {
00237         auto offset = this->meter.pad + this->meter.dotDist;
00238         g.setColour(juce::Colour(meter.uiSpec.foregroundColour));
00239         g.setFont(this->meter.fontSize);
00240         for (size_t i = 0; i < this->meter.nDots; i++) {
00241             auto y = i * this->meter.dotDist + offset;
00242             auto t = "- " + std::to_string(static_cast<int>(this->meter.dbPrDot * i));
00243             g.drawText(t, 0, y, 1000, 10, juce::Justification::left, false);
00244         }
00245     }
00246     void resized() override {
00247         this->meter.refresh();
00248         this->repaint();
00249     }
00250 };
00251
00252 struct StereoMeter : public juce::Component {
00253     Meter l;
00254     Meter r;
00255     UiSpec& spec;
00256     juce::Label label;
00257     int fontSize { 0 };
00258     float uiScale { 1 };
00259     bool hasScale { false };
00260     bool onlyShowLeft { false };
00261
00262     StereoMeter(MeterSpec& meterSpec, UiSpec& uiSpec)
00263         : l(meterSpec, uiSpec), r(meterSpec, uiSpec), spec(uiSpec),
00264         label(meterSpec.name, meterSpec.name) {
00265         this->addAndMakeVisible(this->label);
00266         this->addAndMakeVisible(this->l);
00267         this->addAndMakeVisible(this->r);
00268         this->l.label = "L";
00269         this->r.label = "R";
00270     }
00271     void resized() override {
00272         this->updateUi();
00273         this->repaint();
00274     }
00275     void updateUi() {
00276         this->l.fontSize = this->fontSize;
00277         this->r.fontSize = this->fontSize;
00278         auto area = getLocalBounds();
00279         if (this->onlyShowLeft) {
00280             this->l.setBounds(area);
00281             this->l.label = this->l.meterSpec.name;
00282             return;
00283         }
00284         auto labelArea =
00285             area.removeFromTop(this->l.uiSpec.labelHeight * this->uiScale);
00286         this->label.setFont(juce::FontOptions(this->fontSize));
00287         this->label.setBounds(labelArea);
00288         this->label.setJustificationType(juce::Justification::centredBottom);
00289         auto lArea = area.removeFromLeft(area.getWidth() / 2.0);
00290         this->l.setBounds(lArea);
00291         this->r.setBounds(area);
00292     }
00293     void refresh(Audio xPeak, Audio xRms) {
00294         this->l.refresh(xPeak.l, xRms.l);
00295         this->r.refresh(xPeak.r, xRms.r);
00296     }
00297 };
00298
00299 struct MeterGroup : public juce::Component {
00300     std::vector<std::unique_ptr<StereoMeter>> meters;
00301     std::vector<std::unique_ptr<MeterScale>> scales;
00302     int nChs { 2 };

```

```

00303 void resized() override {
00304     this->updateUi();
00305     this->repaint();
00306 }
00307 virtual void setFontSize(int size) {
00308     for (auto& m : this->meters) { m->fontSize = size; }
00309 }
00310 MeterGroup(UiSpec& uiSpec, std::vector<MeterSpec>& meterSpecs) {
00311     size_t i = 0;
00312     for (auto& spec : meterSpecs) {
00313         auto meter = std::make_unique<StereoMeter>(spec, uiSpec);
00314         this->addAndMakeVisible(meter.get());
00315         if (spec.hasScale) {
00316             meter->hasScale = true;
00317             auto scale = std::make_unique<MeterScale>(meter->l);
00318             this->addAndMakeVisible(scale.get());
00319             scales.push_back(std::move(scale));
00320         }
00321         meters.push_back(std::move(meter));
00322     }
00323 }
00324
00325 void refresh(size_t idx, Audio xPeak, Audio xRms) {
00326     this->meters[idx]->refresh(xPeak, xRms);
00327 }
00328 void updateUi() noexcept {
00329     auto area = this->getLocalBounds();
00330     auto totalWidth = area.getWidth();
00331     auto scaleWidth =
00332         totalWidth / (this->meters.size() * this->nChs + this->scales.size());
00333     auto meterWidth = scaleWidth * this->nChs;
00334     size_t iScale = 0;
00335     for (auto& m : this->meters) {
00336         m->setBounds(area.removeFromLeft(meterWidth));
00337         if (m->hasScale) {
00338             auto scaleArea = area.removeFromLeft(scaleWidth);
00339             scaleArea.removeFromTop(m->l.getLabel().getHeight());
00340             this->scales[iScale++]->setBounds(scaleArea);
00341         }
00342     }
00343 }
00344 void setUiScale(float uiScale) {
00345     for (auto& m : meters) { m->uiScale = uiScale; }
00346 }
00347 void setOnlyShowLeft(bool onlyShowLeft) {
00348     for (auto& m : meters) { m->onlyShowLeft = onlyShowLeft; }
00349     if (onlyShowLeft) { this->nChs = 1; }
00350 }
00351 void updateRelease(float fs) {
00352     for (auto& m : this->meters) {
00353         m->l.updateRelease(fs, m->l.meterSpec.decay_s * 1000);
00354         m->r.updateRelease(fs, m->r.meterSpec.decay_s * 1000);
00355     }
00356 }
00357 float getMinimalWidth() const noexcept {
00358     if (!this->meters.size()) { return 0; }
00359     return this->meters[0]->l.uiSpec.meterWidth
00360         * (this->meters.size() * this->nChs + this->scales.size());
00361 }
00362 float getMinimalHeight() const noexcept {
00363     if (!this->meters.size()) { return 0; }
00364     auto& m = this->meters[0]->l;
00365     m.refresh(false);
00366     return (m.uiSpec.labelHeight) * this->nChs + (m.nDots + 2) * m.dotDist
00367         + m.pad;
00368 }
00369 };
00370 } // namespace NtFx

```

9.3 PluginEditor.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.

```

```

00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  *
00017  * Modified from JUCE template.
00018  **/
00019
00020 #pragma once
00021
00022 #include "PluginProcessor.h"
00023
00024 #include "LookAndFeel.h"
00025 #include "Meter.h"
00026 #include "RadioButtons.h"
00027 #include "Toggle.h"
00028 #include "lib/UiSpec.h"
00029
00030 #include <juce_audio_basics/juce_audio_basics.h>
00031 #include <juce_audio_devices/juce_audio_devices.h>
00032 #include <juce_audio_formats/juce_audio_formats.h>
00033 #include <juce_audio_plugin_client/juce_audio_plugin_client.h>
00034 #include <juce_audio_processors/juce_audio_processors.h>
00035 #include <juce_audio_processors_headless/juce_audio_processors_headless.h>
00036 #include <juce_audio_utils/juce_audio_utils.h>
00037 #include <juce_core/juce_core.h>
00038 #include <juce_data_structures/juce_data_structures.h>
00039 #include <juce_events/juce_events.h>
00040 #include <juce_graphics/juce_graphics.h>
00041 #include <juce_gui_basics/juce_gui_basics.h>
00042 #include <juce_gui_extra/juce_gui_extra.h>
00043
00044 #include <cstdlib>
00045 #include <memory>
00046 #include <string>
00047 #include <vector>
00048
00049 #ifndef NTFX_PLUGIN
00050     #error NTFX_PLUGIN is not defined. Please add '-DNTFX_PLUGIN=[your plugin \
00051         name]' to cmake configure.
00052 #endif
00053
00054 struct NtPluginAudioProcessorEditor : public juce::AudioProcessorEditor,
00055                                     private juce::Timer,
00056                                     private juce::Slider::Listener,
00057                                     private juce::ToggleButton::Listener,
00058                                     private juce::ComboBox::Listener,
00059                                     private juce::ChangeListener {
00060
00061     NtPluginAudioProcessorEditor(NtPluginAudioProcessor&);
00062     ~NtPluginAudioProcessorEditor() override;
00063
00064     // "Public".
00065     virtual void paint(juce::Graphics&) override;
00066     virtual void resized() override;
00067
00068     // Callbacks.
00069     virtual void sliderValueChanged(juce::Slider* slider) override;
00070     virtual void buttonClicked(juce::Button* button) override;
00071     virtual void changeListenerCallback(juce::ChangeBroadcaster* source) override;
00072     virtual void comboBoxChanged(juce::ComboBox* p_box) override;
00073     virtual void timerCallback() override;
00074
00075     NtPluginAudioProcessor& proc;
00076     NtFx::MeterGroup meters;
00077     NtFx::KnobLookAndFeel knobLookAndFeel;
00078     NtFx::TitleBarLookAndFeel dropDownLookAndFeel;
00079
00080     std::vector<std::unique_ptr<juce::Slider> primaryKnobs;
00081     std::vector<std::unique_ptr<juce::Label> primaryKnobLabels;
00082     std::vector<std::unique_ptr<juce::Slider> secondaryKnobs;
00083     std::vector<std::unique_ptr<juce::Label> secondaryKnobLabels;
00084     std::vector<std::unique_ptr<NtFx::Toggle> toggles;
00085     std::vector<std::unique_ptr<juce::ComboBox> dropDowns;
00086     std::vector<std::unique_ptr<juce::Label> dropDownLabels;
00087     std::vector<std::unique_ptr<NtFx::RadioButtonSet> radioButtonSets;
00088     std::vector<std::unique_ptr<juce::Label> radioButtonLabels;
00089     std::vector<std::unique_ptr<NtFx::ToggleSet> toggleSets;
00090     std::vector<std::unique_ptr<juce::Label> toggleSetLabels;
00091     std::vector<std::unique_ptr<juce::ComboBox> titleBarDropDowns;
00092     std::vector<std::unique_ptr<juce::Label> titleBarDropDownLabels;
00093     std::vector<
00094         std::unique_ptr<juce::AudioProcessorValueTreeState::SliderAttachment>
00095         knobAttachments;
00096     std::vector<
00097         std::unique_ptr<juce::AudioProcessorValueTreeState::ButtonAttachment>
00098         toggleAttachments;
00099     std::vector<

```

```

00100         std::unique_ptr<juce::AudioProcessorValueTreeState::ComboBoxAttachment>
00101         dropDownAttachments;
00102
00103         std::vector<juce::Rectangle<int>> borderedAreas;
00104         std::vector<juce::Rectangle<int>> grayAreas;
00105
00106         float unscaledWindowHeight = 0;
00107         float unscaledWindowWidth = 0;
00108         float uiScale = 1;
00109         float titleBarScale = 0.7;
00110         float pad = 10;
00111         bool isInitialized = false;
00112
00113         juce::Label pluginNameLabel;
00114
00115         // Initializers.
00116         void _initPrimaryKnob(NtFx::KnobSpec& p_spec);
00117         void _initSecondaryKnob(NtFx::KnobSpec& p_spec);
00118         void _initKnob(NtFx::KnobSpec& p_spec,
00119             std::unique_ptr<juce::Slider>& p_slider,
00120             std::unique_ptr<juce::Label>& p_label);
00121         void _initToggle(NtFx::ToggleSpec& spec);
00122         void _initToggle(NtFx::Toggle* p_toggle, NtFx::ToggleSpec& spec);
00123         void _initToggleGroup(NtFx::ToggleSetSpec& spec);
00124         void _initDropDown(NtFx::DropDownSpec& p_spec, bool addToTitleBar = false);
00125         void _initRadioButton(NtFx::RadioButtonSetSpec& spec);
00126
00127         // UI update.
00128         void _updateUi();
00129         void _updateTitleBar(juce::Rectangle<int>& area);
00130         void _updateMeters(juce::Rectangle<int>& area);
00131         void _updateSecondaryKnobs(juce::Rectangle<int>& area);
00132         void _updatePrimaryKnobs(juce::Rectangle<int>& area);
00133         void _updateColours();
00134         void _updateUiScale();
00135         void _updateOversampling();
00136         void _updateTheme();
00137         void _placeSmallTogglesArea(juce::Rectangle<int>& area);
00138         void _placeBottomRow(juce::Rectangle<int>& area);
00139         void _placeDropdowns(juce::Rectangle<int>& area, size_t columnWidth);
00140         void _placeToggles(juce::Rectangle<int>& area, size_t columnWidth);
00141         template <typename T>
00142         void _placeSmallToggles(juce::Rectangle<int>& area,
00143             int size,
00144             std::vector<std::unique_ptr<juce::Label>& labels,
00145             std::vector<std::unique_ptr<T>& toggles);
00146         void _initWindowSize();
00147
00148         // Helpers.
00149         template <typename T, typename spec_t>
00150         std::unique_ptr<T> _makeSmallToggleSet(spec_t& spec);
00151         void _calcSliderRowsCols(
00152             int nSliders, int& nRows, int& nColumns, int maxRows, int maxColumns);
00153         void _makeGrid(juce::Rectangle<int>& area,
00154             float nRows,
00155             float nCols,
00156             float colWidth,
00157             float rowHeight,
00158             float pad);
00159         std::unique_ptr<juce::Label> _makeLabel(const std::string name);
00160         JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(NtPluginAudioProcessorEditor)
00161     };

```

9.4 PluginProcessor.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  *
00017  * Modified from JUCE template.
00018  */

```

```

00019 #pragma once
00020
00021 #define Q(x) #x
00022 #define QUOTE(x) Q(x)
00023
00024 // TODO: Pass version through to here.
00025 #if !JUCE_DONT_DECLARE_PROJECTINFO
00026 namespace ProjectInfo {
00027     const char* const projectName = QUOTE(NTFX_PLUGIN);
00028     const char* const companyName = "NtFx";
00029     const char* const versionString = "0.1.0";
00030     const int versionNumber = 0x100;
00031 }
00032 #endif
00033
00034 #include NTFX_PLUGIN_FILE
00035
00036 #include "lib/SampleRateConverter.h"
00037 #include "lib/UiSpec.h"
00038
00039 #include <juce_audio_basics/juce_audio_basics.h>
00040 #include <juce_audio_devices/juce_audio_devices.h>
00041 #include <juce_audio_formats/juce_audio_formats.h>
00042 #include <juce_audio_plugin_client/juce_audio_plugin_client.h>
00043 #include <juce_audio_processors/juce_audio_processors.h>
00044 #include <juce_audio_processors_headless/juce_audio_processors_headless.h>
00045 #include <juce_audio_utils/juce_audio_utils.h>
00046 #include <juce_core/juce_core.h>
00047 #include <juce_data_structures/juce_data_structures.h>
00048 #include <juce_events/juce_events.h>
00049 #include <juce_graphics/juce_graphics.h>
00050 #include <juce_gui_basics/juce_gui_basics.h>
00051 #include <juce_gui_extra/juce_gui_extra.h>
00052
00053 //=====
00054 struct NtPluginAudioProcessor : public juce::AudioProcessor {
00055     NtPluginAudioProcessor();
00056     ~NtPluginAudioProcessor() override;
00057     void prepareToPlay(double sampleRate, int samplesPerBlock) override;
00058     void releaseResources() override;
00059
00060 #ifndef JucePlugin_PreferredChannelConfigurations
00061     bool isBusesLayoutSupported(const BusesLayout& layouts) const override;
00062 #endif
00063
00064     void processBlock(juce::AudioBuffer<float>&, juce::MidiBuffer&) override;
00065
00066     juce::AudioProcessorEditor* createEditor() override;
00067     bool hasEditor() const override;
00068
00069     const juce::String getName() const override;
00070
00071     bool acceptsMidi() const override;
00072     bool producesMidi() const override;
00073     bool isMidiEffect() const override;
00074     double getTailLengthSeconds() const override;
00075
00076     int getNumPrograms() override;
00077     int getCurrentProgram() override;
00078     void setCurrentProgram(int index) override;
00079     const juce::String getProgramName(int index) override;
00080     void changeProgramName(int index, const juce::String& newName) override;
00081
00082     void getStateInformation(juce::MemoryBlock& destData) override;
00083     void setStateInformation(const void* data, int sizeInBytes) override;
00084
00085     template <typename T>
00086     void loadParameter(std::vector<T>& v);
00087     void loadRadioButtons(std::vector<NtFx::RadioButtonSetSpec>& v);
00088     void loadToggleSets(std::vector<NtFx::ToggleSetSpec>& v);
00089
00090     template <typename t_val, typename t_spec>
00091     void createParameters(std::vector<t_spec>& vParams,
00092         juce::AudioProcessorValueTreeState::ParameterLayout& parameters,
00093         int& i);
00094
00095     void updateOversampling(int mode = 0);
00096     juce::AudioProcessorValueTreeState::ParameterLayout createParameterLayout();
00097
00098     NtFx::TitleBarSpec titleBarSpec;
00099     float fsBase = 48000;
00100     NTFX_PLUGIN plug;
00101     NtFx::Src::SampleRateConverter src;
00102     juce::AudioProcessorValueTreeState paramLayout;
00103     bool monoMode { false };
00104
00105     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(NtPluginAudioProcessor)

```

```

00108 };
00109
00110 #undef Q
00111 #undef QUOTE

```

9.5 RadioButtons.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  */
00017
00018 #pragma once
00019
00020 #include <juce_audio_basics/juce_audio_basics.h>
00021 #include <juce_audio_devices/juce_audio_devices.h>
00022 #include <juce_audio_formats/juce_audio_formats.h>
00023 #include <juce_audio_plugin_client/juce_audio_plugin_client.h>
00024 #include <juce_audio_processors/juce_audio_processors.h>
00025 #include <juce_audio_processors_headless/juce_audio_processors_headless.h>
00026 #include <juce_audio_utils/juce_audio_utils.h>
00027 #include <juce_core/juce_core.h>
00028 #include <juce_data_structures/juce_data_structures.h>
00029 #include <juce_events/juce_events.h>
00030 #include <juce_graphics/juce_graphics.h>
00031 #include <juce_gui_basics/juce_gui_basics.h>
00032 #include <juce_gui_extra/juce_gui_extra.h>
00033 #include <memory>
00034 #include <string>
00035 #include <vector>
00036
00037 #include "Toggle.h"
00038 #include "lib/UiSpec.h"
00039
00040 namespace NtFx {
00041 struct ToggleSetBase : public juce::Component, public juce::ChangeBroadcaster {
00042     UiSpec& uiSpec;
00043     std::vector<std::unique_ptr<Toggle>> toggles;
00044
00045     float uiScale = 1;
00046     ToggleSetBase(UiSpec& uiSpec) : uiSpec(uiSpec) { }
00047     std::unique_ptr<Toggle> makeToggle(std::string option, int i) {
00048         auto p_toggle = std::make_unique<Toggle>(option, i);
00049         this->addAndMakeVisible(p_toggle.get());
00050         p_toggle->setButtonText(option);
00051         p_toggle->setClickingTogglesState(true);
00052         p_toggle->setToggleable(true);
00053         return std::move(p_toggle);
00054     }
00055
00056     virtual void updateToggleStates(int i) { };
00057
00058     void resized() override { this->updateUi(); }
00059
00060     void updateUi() {
00061         auto area = this->getLocalBounds();
00062         auto w = area.getWidth();
00063         auto h = area.getHeight();
00064         auto n = this->toggles.size();
00065         if (!w || !h || !n) { return; }
00066         float pad = 3 * this->uiScale;
00067         for (size_t i = 0; i < n; i++) {
00068             auto p_toggle = this->toggles[i].get();
00069             p_toggle->colour = uiSpec.foregroundColour;
00070             p_toggle->fontSize = this->uiSpec.defaultFontSize * this->uiScale;
00071             auto toggleArea =
00072                 area.removeFromTop(this->uiSpec.radioButtonHeight * this->uiScale);
00073             toggleArea.reduce(pad, pad);
00074             p_toggle->setBounds(toggleArea);
00075             this->updateToggleStates(i);
00076         }
00077     }
00078 }

```

```

00077     this->repaint();
00078 }
00079 };
00080
00081 struct ToggleSet : public ToggleSetBase {
00082     ToggleSetSpec spec;
00083     ToggleSet(ToggleSetSpec spec, UiSpec& uiSpec)
00084         : spec(spec), ToggleSetBase(uiSpec) {
00085         for (size_t i = 0; i < spec.toggles.size(); i++) {
00086             auto p_toggle = this->makeToggle(spec.toggles[i].name, i);
00087             auto pp_toggle = p_toggle.get();
00088             p_toggle->onClick = [this, pp_toggle]() {
00089                 DBG("OnClick toggle '" < pp_toggle->getName()
00090                     < "': " < int(pp_toggle->getToggleState()));
00091                 this->sendChangeMessage();
00092             };
00093             toggles.push_back(std::move(p_toggle));
00094         }
00095     }
00096     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(ToggleSet)
00097 };
00098
00099 struct RadioButtonSet : public ToggleSetBase {
00100     RadioButtonSetSpec& spec;
00101     static int s_id;
00102
00103     RadioButtonSet(RadioButtonSetSpec& spec, UiSpec& uiSpec)
00104         : spec(spec), ToggleSetBase(uiSpec) { //, val(spec._defaultVal) {
00105         for (size_t i = 0; i < spec.options.size(); i++) {
00106             auto option = spec.options[i];
00107             auto p_toggle = this->makeToggle(option, i);
00108             if (!this->spec._id) {
00109                 this->spec._id = ++s_id;
00110                 p_toggle->setRadioGroupId(
00111                     this->spec._id, juce::NotificationType::dontSendNotification);
00112             }
00113             auto pp_toggle = p_toggle.get();
00114             p_toggle->onClick = [this, pp_toggle]() {
00115                 if (!pp_toggle->getToggleState()) { return; }
00116                 *this->spec.p_val = pp_toggle->_id;
00117                 DBG("OnClick radiobutton '" < this->spec.name
00118                     < "', group ID: " < this->spec._id
00119                     < " val: " < *this->spec.p_val);
00120                 this->sendChangeMessage();
00121             };
00122             this->toggles.push_back(std::move(p_toggle));
00123         }
00124     }
00125
00126     virtual void updateToggleStates(int i) override {
00127         if (*this->spec.p_val != i) {
00128             this->toggles[i]->setToggleState(
00129                 false, juce::NotificationType::sendNotificationSync);
00130         } else {
00131             this->toggles[i]->setToggleState(
00132                 true, juce::NotificationType::sendNotificationSync);
00133         }
00134     }
00135
00136     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(RadioButtonSet)
00137 };
00138 }

```

9.6 Toggle.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  **/
00017
00018 #pragma once

```

```

00019
00020 #include "juce_core/system/juce_PlatformDefs.h"
00021 #include <juce_audio_basics/juce_audio_basics.h>
00022 #include <juce_audio_devices/juce_audio_devices.h>
00023 #include <juce_audio_formats/juce_audio_formats.h>
00024 #include <juce_audio_plugin_client/juce_audio_plugin_client.h>
00025 #include <juce_audio_processors/juce_audio_processors.h>
00026 #include <juce_audio_processors_headless/juce_audio_processors_headless.h>
00027 #include <juce_audio_utils/juce_audio_utils.h>
00028 #include <juce_core/juce_core.h>
00029 #include <juce_data_structures/juce_data_structures.h>
00030 #include <juce_events/juce_events.h>
00031 #include <juce_graphics/juce_graphics.h>
00032 #include <juce_gui_basics/juce_gui_basics.h>
00033 #include <juce_gui_extra/juce_gui_extra.h>
00034
00035 #include <cstdint>
00036
00037 namespace NtFx {
00038 class Toggle : public juce::TextButton {
00039 public:
00040     Toggle(const juce::String& buttonName = { }, int _id = 0)
00041         : juce::TextButton(buttonName, _id(_id)) { }
00042     int _id { 0 };
00043     float fontSize { 10 };
00044     uint32_t colour { 0xFFFFFFFF };
00045
00046     void paintButton(juce::Graphics& g,
00047                     const bool isMouseOver,
00048                     const bool isMouseDown) override {
00049         auto h = this->getHeight();
00050         auto w = this->getWidth();
00051         float outPadScale = 0.95;
00052         float dRing = h * outPadScale;
00053         auto x = (h - dRing) / 2;
00054         auto y = (h - dRing) / 2;
00055         g.setColour(juce::Colour(this->colour));
00056         juce::Rectangle<float> r1(x, y, dRing, dRing);
00057         g.drawEllipse(r1, h / 50.0);
00058         float inPadScale = outPadScale * 0.6;
00059         float dDot = h * inPadScale;
00060         float xDot = x + (dRing - dDot) / 2;
00061         float yDot = y + (dRing - dDot) / 2;
00062         juce::Rectangle<float> r2(xDot, yDot, dDot, dDot);
00063         if (this->getToggleState()) {
00064             g.setOpacity(0.7);
00065             g.fillEllipse(r2);
00066             g.setOpacity(1.0);
00067         }
00068         g.drawEllipse(r2, h / 100.0);
00069         x = (h + dRing) / 2 + h * 0.2;
00070         juce::Rectangle<float> r3(x, 0, 100, h);
00071         g.setFont(juce::FontOptions(this->fontSize));
00072         g.drawText(this->getButtonText(), r3, juce::Justification::centredLeft);
00073     }
00074
00075     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(Toggle)
00076 };
00077 }

```

9.7 Audio.h

```

00001 #pragma once
00002
00003
00004 #include "gcem.hpp"
00005 #include "lib/utils.h"
00006 #include <cstdint>
00007 #include <type_traits>
00008
00009 namespace NtFx {
00010
00011 template <typename signal_t>
00012 struct Stereo;
00013 template <typename signal_t>
00014 struct Mono {
00015     signal_t l { 0 };
00016     Mono() : l(0) { }
00017     Mono(signal_t v) : l(v) { }
00018     Mono(signal_t l, signal_t r) : l((l + r) / 2) { }
00019     Mono(Stereo<signal_t> x) : l(x.l, x.r) { };
00020     Mono<signal_t>& operator=(const Mono<signal_t>& x) noexcept {
00021         this->l = x.l;

```

```

00042     return *this;
00043 }
00044 Mono<signal_t>& operator=(const signal_t& x) noexcept {
00045     this->l = x;
00046     return *this;
00047 }
00048 bool operator==(const Mono<signal_t>& x) const noexcept {
00049     return (this->l == x.l);
00050 }
00051 bool operator<(const Mono<signal_t>& x) const noexcept {
00052     return this->absMin() < x.absMin();
00053 }
00054 bool operator>(const Mono<signal_t>& x) const noexcept {
00055     return this->absMax() > x.absMax();
00056 }
00057 bool operator<=(const Mono<signal_t>& x) const noexcept {
00058     return this->absMin() <= x.absMin();
00059 }
00060 bool operator>=(const Mono<signal_t>& x) const noexcept {
00061     return this->absMax() >= x.absMax();
00062 }
00063 bool operator<(const signal_t& x) const noexcept {
00064     return this->absMin() < x;
00065 }
00066 bool operator>(const signal_t& x) const noexcept {
00067     return this->absMax() > x;
00068 }
00069 bool operator<=(const signal_t& x) const noexcept {
00070     return this->absMin() <= x;
00071 }
00072 bool operator>=(const signal_t& x) const noexcept {
00073     return this->absMax() >= x;
00074 }
00075
00076 Mono<signal_t>& operator*=(const signal_t x) noexcept {
00077     this->l = this->l * x;
00078     return *this;
00079 }
00080 Mono<signal_t>& operator*=(const Mono<signal_t> x) noexcept {
00081     this->l = this->l * x.l;
00082     return *this;
00083 }
00084 Mono<signal_t>& operator+=(const signal_t x) noexcept {
00085     this->l = this->l + x;
00086     return *this;
00087 }
00088 Mono<signal_t>& operator+=(const Mono<signal_t> x) noexcept {
00089     this->l = this->l + x.l;
00090     return *this;
00091 }
00092 Mono<signal_t> operator-() const { return -this->l; }
00093 signal_t avgSquared() const noexcept { return this->l; }
00094
00105 signal_t absMax() const noexcept { return gcem::abs(this->l); }
00106
00112 signal_t absMin() const noexcept { return gcem::abs(this->l); }
00113
00119 Mono<signal_t> abs() const noexcept { return gcem::abs(this->l); }
00120 };
00121
00122 template <typename signal_t>
00123 static inline Mono<signal_t> operator+(
00124     const Mono<signal_t>& y, const signal_t& x) noexcept {
00125     return y.l + x;
00126 }
00127 template <typename signal_t>
00128 static inline Mono<signal_t> operator-(
00129     const Mono<signal_t>& y, const signal_t& x) noexcept {
00130     return y.l - x;
00131 }
00132 template <typename signal_t>
00133 static inline Mono<signal_t> operator*(
00134     const Mono<signal_t>& y, const signal_t& x) noexcept {
00135     return y.l * x;
00136 }
00137 template <typename signal_t>
00138 static inline Mono<signal_t> operator/(
00139     const Mono<signal_t>& y, const signal_t& x) noexcept {
00140     return y.l / x;
00141 }
00142 template <typename signal_t>
00143 static inline Mono<signal_t> operator+(
00144     const Mono<signal_t>& y, const int& x) noexcept {
00145     return y.l + x;
00146 }
00147 template <typename signal_t>
00148 static inline Mono<signal_t> operator-(

```

```

00149     const Mono<signal_t>& y, const int& x) noexcept {
00150     return y.l - x;
00151 }
00152 template <typename signal_t>
00153 static inline Mono<signal_t> operator*(
00154     const Mono<signal_t>& y, const int& x) noexcept {
00155     return y.l * x;
00156 }
00157 template <typename signal_t>
00158 static inline Mono<signal_t> operator/(
00159     const Mono<signal_t>& y, const int& x) noexcept {
00160     return y.l / x;
00161 }
00162 template <typename signal_t>
00163 static inline Mono<signal_t> operator+(
00164     const Mono<signal_t>& y, const Mono<signal_t>& x) noexcept {
00165     return y.l + x.l;
00166 }
00167 template <typename signal_t>
00168 static inline Mono<signal_t> operator-(
00169     const Mono<signal_t>& y, const Mono<signal_t>& x) noexcept {
00170     return y.l - x.l;
00171 }
00172 template <typename signal_t>
00173 static inline Mono<signal_t> operator*(
00174     const Mono<signal_t>& y, const Mono<signal_t>& x) noexcept {
00175     return y.l * x.l;
00176 }
00177 template <typename signal_t>
00178 static inline Mono<signal_t> operator/(
00179     const Mono<signal_t>& y, const Mono<signal_t>& x) noexcept {
00180     return y.l / x.l;
00181 }
00182 template <typename signal_t>
00183 static inline Mono<signal_t> operator+(
00184     const signal_t& x, const Mono<signal_t>& y) noexcept {
00185     return x + y.l;
00186 }
00187 template <typename signal_t>
00188 static inline Mono<signal_t> operator-(
00189     const signal_t& x, const Mono<signal_t>& y) noexcept {
00190     return x - y.l;
00191 }
00192 template <typename signal_t>
00193 static inline Mono<signal_t> operator*(
00194     const signal_t& x, const Mono<signal_t>& y) noexcept {
00195     return x * y.l;
00196 }
00197 template <typename signal_t>
00198 static inline Mono<signal_t> operator/(
00199     const signal_t& x, const Mono<signal_t>& y) noexcept {
00200     return x / y.l;
00201 }
00202 template <typename signal_t>
00203 static inline Mono<signal_t> operator*(
00204     const Mono<signal_t>& y, const size_t& x) noexcept {
00205     return y.l * signal_t(x);
00206 }
00207 template <typename signal_t>
00208 static inline Mono<signal_t> operator*(
00209     const size_t x, const Mono<signal_t>& y) noexcept {
00210     return y.l * signal_t(x);
00211 }
00212 template <typename signal_t>
00213 static inline void ensureFinite(
00214     Mono<signal_t>& x, signal_t def = signal_t(0)) noexcept {
00215     ensureFinite(x.l, def);
00216 }
00217
00227 template <typename signal_t>
00228 struct Stereo {
00229     signal_t l { 0 };
00230     signal_t r { 0 };
00231     Stereo() : l(0.0f), r(0.0f) {
00232         static_assert(
00233             std::is_floating_point_v<signal_t>, "Type must be floating point.");
00234     }
00235     Stereo(signal_t x) : l(x), r(x) {}
00236     Stereo(signal_t l, signal_t r) : l(l), r(r) {}
00237     Stereo(Mono<signal_t> l, Mono<signal_t> r) : l(l.l), r(r.l) {}
00238     Stereo<signal_t>& operator=(const Stereo<signal_t>& x) noexcept {
00239         this->l = x.l;
00240         this->r = x.r;
00241         return *this;
00242     }
00243     Stereo<signal_t>& operator=(const signal_t& x) noexcept {
00244         this->l = x;

```

```

00245     this->r = x;
00246     return *this;
00247 }
00248 bool operator==(const Stereo<signal_t>& x) const noexcept {
00249     return (this->l == x.l) && (this->r == x.r);
00250 }
00251 bool operator<(const Stereo<signal_t>& x) const noexcept {
00252     return this->absMin() < x.absMin();
00253 }
00254 bool operator>(const Stereo<signal_t>& x) const noexcept {
00255     return this->absMax() > x.absMax();
00256 }
00257 bool operator<=(const Stereo<signal_t>& x) const noexcept {
00258     return this->absMin() <= x.absMin();
00259 }
00260 bool operator>=(const Stereo<signal_t>& x) const noexcept {
00261     return this->absMax() >= x.absMax();
00262 }
00263 bool operator<(const signal_t& x) const noexcept {
00264     return this->absMin() < x;
00265 }
00266 bool operator>(const signal_t& x) const noexcept {
00267     return this->absMax() > x;
00268 }
00269 bool operator<=(const signal_t& x) const noexcept {
00270     return this->absMin() <= x;
00271 }
00272 bool operator>=(const signal_t& x) const noexcept {
00273     return this->absMax() >= x;
00274 }
00275
00276 Stereo<signal_t>& operator*=(const signal_t x) noexcept {
00277     this->l = this->l * x;
00278     this->r = this->r * x;
00279     return *this;
00280 }
00281 Stereo<signal_t>& operator*=(const Stereo<signal_t> x) noexcept {
00282     this->l = this->l * x.l;
00283     this->r = this->r * x.r;
00284     return *this;
00285 }
00286 Stereo<signal_t>& operator+=(const signal_t x) noexcept {
00287     this->l = this->l + x;
00288     this->r = this->r + x;
00289     return *this;
00290 }
00291 Stereo<signal_t>& operator+=(const Stereo<signal_t> x) noexcept {
00292     this->l = this->l + x.l;
00293     this->r = this->r + x.r;
00294     return *this;
00295 }
00296 Stereo<signal_t> operator-(const Stereo<signal_t>& x) const {
00297     return { -this->l, -this->r };
00298 }
00303 signal_t avgSquared() const noexcept {
00304     return (this->l * this->l + this->r * this->r) * 0.5f;
00305 }
00306
00312 signal_t absMax() const noexcept {
00313     return gcem::abs(this->l) > gcem::abs(this->r) ? this->l : this->r;
00314 }
00315
00321 signal_t absMin() const noexcept {
00322     return gcem::abs(this->l) < gcem::abs(this->r) ? this->l : this->r;
00323 }
00324
00330 Stereo<signal_t> abs() const noexcept {
00331     return { gcem::abs(this->l), gcem::abs(this->r) };
00332 }
00333 };
00334 template <typename signal_t>
00335 static inline Stereo<signal_t> operator+(
00336     const Stereo<signal_t>& y, const signal_t& x) noexcept {
00337     return { (y.l + x), (y.r + x) };
00338 }
00339 template <typename signal_t>
00340 static inline Stereo<signal_t> operator-(
00341     const Stereo<signal_t>& y, const signal_t& x) noexcept {
00342     return { (y.l - x), (y.r - x) };
00343 }
00344 template <typename signal_t>
00345 static inline Stereo<signal_t> operator*(
00346     const Stereo<signal_t>& y, const signal_t& x) noexcept {
00347     return { (y.l * x), (y.r * x) };
00348 }
00349 template <typename signal_t>
00350 static inline Stereo<signal_t> operator/(
00351     const Stereo<signal_t>& y, const signal_t& x) noexcept {

```

```

00352     return { (y.l / x), (y.r / x) };
00353 }
00354 template <typename signal_t>
00355 static inline Stereo<signal_t> operator+(
00356     const Stereo<signal_t>& y, const int& x) noexcept {
00357     return { (y.l + x), (y.r + x) };
00358 }
00359 template <typename signal_t>
00360 static inline Stereo<signal_t> operator-(
00361     const Stereo<signal_t>& y, const int& x) noexcept {
00362     return { (y.l - x), (y.r - x) };
00363 }
00364 template <typename signal_t>
00365 static inline Stereo<signal_t> operator*(
00366     const Stereo<signal_t>& y, const int& x) noexcept {
00367     return { (y.l * x), (y.r * x) };
00368 }
00369 template <typename signal_t>
00370 static inline Stereo<signal_t> operator/(
00371     const Stereo<signal_t>& y, const int& x) noexcept {
00372     return { (y.l / x), (y.r / x) };
00373 }
00374 template <typename signal_t>
00375 static inline Stereo<signal_t> operator+(
00376     const Stereo<signal_t>& y, const Stereo<signal_t>& x) noexcept {
00377     return { (y.l + x.l), (y.r + x.r) };
00378 }
00379 template <typename signal_t>
00380 static inline Stereo<signal_t> operator-(
00381     const Stereo<signal_t>& y, const Stereo<signal_t>& x) noexcept {
00382     return { (y.l - x.l), (y.r - x.r) };
00383 }
00384 template <typename signal_t>
00385 static inline Stereo<signal_t> operator*(
00386     const Stereo<signal_t>& y, const Stereo<signal_t>& x) noexcept {
00387     return { (y.l * x.l), (y.r * x.r) };
00388 }
00389 template <typename signal_t>
00390 static inline Stereo<signal_t> operator/(
00391     const Stereo<signal_t>& y, const Stereo<signal_t>& x) noexcept {
00392     return { (y.l / x.l), (y.r / x.r) };
00393 }
00394 template <typename signal_t>
00395 static inline Stereo<signal_t> operator+(
00396     const signal_t& x, const Stereo<signal_t>& y) noexcept {
00397     return { (x + y.l), (x + y.r) };
00398 }
00399 template <typename signal_t>
00400 static inline Stereo<signal_t> operator-(
00401     const signal_t& x, const Stereo<signal_t>& y) noexcept {
00402     return { (x - y.l), (x - y.r) };
00403 }
00404 template <typename signal_t>
00405 static inline Stereo<signal_t> operator*(
00406     const signal_t& x, const Stereo<signal_t>& y) noexcept {
00407     return { (x * y.l), (x * y.r) };
00408 }
00409 template <typename signal_t>
00410 static inline Stereo<signal_t> operator/(
00411     const signal_t& x, const Stereo<signal_t>& y) noexcept {
00412     return { (x / y.l), (x / y.r) };
00413 }
00414 template <typename signal_t>
00415 static inline Stereo<signal_t> operator*(
00416     const Stereo<signal_t>& y, const size_t& x) noexcept {
00417     return { (y.l * signal_t(x)), (y.r * signal_t(x)) };
00418 }
00419 template <typename signal_t>
00420 static inline Stereo<signal_t> operator*(
00421     const size_t x, const Stereo<signal_t>& y) noexcept {
00422     return { (y.l * signal_t(x)), (y.r * signal_t(x)) };
00423 }
00424
00432 template <typename signal_t>
00433 static inline void ensureFinite(
00434     Stereo<signal_t>& x, signal_t def = signal_t(0)) noexcept {
00435     ensureFinite(x.l, def);
00436     ensureFinite(x.r, def);
00437 }
00438 } // namespace NtFx
00439
00440 #ifndef NTFX_TESTING
00441 typedef double signal_t;
00442 #else
00443 typedef float signal_t;
00444 #endif
00445 using Audio = NtFx::Stereo<signal_t>;

```

9.8 Biquad.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  */
00017
00018 #pragma once
00019
00020 #include "lib/Audio.h"
00021 #include "lib/Component.h"
00022
00023 #include "gcem.hpp"
00024
00025 namespace NtFx {
00026 namespace Biquad {
00027     enum class Shape {
00028         unknown,
00029         bell,
00030         hiShelf,
00031         loShelf,
00032         notch,
00033         hpf,
00034         lpf,
00035         apf,
00036         none
00037     };
00038
00039     struct Settings {
00040         Shape shape { Shape::bell };
00041         signal_t fc_hz { 1000.0 };
00042         signal_t gain_db { 0.0 };
00043         signal_t q { 0.707 };
00044     };
00045
00046     struct Coeffs6 {
00047         signal_t b[3] { 1, 0, 0 };
00048         signal_t a[3] { 1, 0, 0 };
00049     };
00050
00051     struct Coeffs5 {
00052         signal_t b[3] { 1, 0, 0 };
00053         signal_t a[2] { 0, 0 };
00054     };
00055
00056     struct State {
00057         signal_t x[2] { 0, 0 };
00058         signal_t y[2] { 0, 0 };
00059     };
00060
00061     struct StereoState {
00062         State l;
00063         State r;
00064     };
00065
00066     struct Biquad6 {
00067         Coeffs6& coeffs;
00068         State state;
00069         Biquad6(Coeffs6& coeffs) : coeffs(coeffs) { }
00070         inline signal_t process(signal_t x) {
00071             signal_t y = (this->coeffs.b[0] * x + this->coeffs.b[1] * this->state.x[0]
00072                 + this->coeffs.b[2] * this->state.x[1]
00073                 - this->coeffs.a[1] * this->state.y[0]
00074                 - this->coeffs.a[2] * this->state.y[1])
00075                 / this->coeffs.a[0];
00076             this->state.y[1] = this->state.y[0];
00077             this->state.y[0] = y;
00078             this->state.x[1] = this->state.x[0];
00079             this->state.x[0] = x;
00080             return y;
00081         }
00082     };
00083
00084     inline static signal_t processBiquad5(

```

```

00085     signal_t x, Coeffs5& coeffs, State& state) noexcept {
00086     signal_t y = coeffs.b[0] * x + coeffs.b[1] * state.x[0]
00087         + coeffs.b[2] * state.x[1] - coeffs.a[0] * state.y[0]
00088         - coeffs.a[1] * state.y[1];
00089     state.y[1] = state.y[0];
00090     state.y[0] = y;
00091     state.x[1] = state.x[0];
00092     state.x[0] = x;
00093     return y;
00094 }
00095
00096 static inline Coeffs6 calcCoeffsBell(
00097     signal_t fs, signal_t fc_hz, signal_t q, signal_t a) {
00098     auto w0 = 2.0 * GCEM_PI * fc_hz / fs;
00099     auto cosW0 = gcem::cos(w0);
00100     auto alpha = gcem::sin(w0) / (2.0 * q);
00101     Coeffs6 c;
00102     c.b[0] = 1.0 + alpha * a;
00103     c.b[1] = -2.0 * cosW0;
00104     c.b[2] = 1.0 - alpha * a;
00105     c.a[0] = 1.0 + alpha / a;
00106     c.a[1] = -2.0 * cosW0;
00107     c.a[2] = 1.0 - alpha / a;
00108     return c;
00109 }
00110
00111 static inline Coeffs6 calcCoeffsLoShelf(
00112     signal_t fs, signal_t fc_hz, signal_t q, signal_t a) {
00113     auto w0 = 2.0 * GCEM_PI * fc_hz / fs;
00114     auto cosW0 = gcem::cos(w0);
00115     auto alpha = gcem::sin(w0) / (2.0 * q);
00116     Coeffs6 c;
00117     c.b[0] = a * ((a + 1.0) - (a - 1.0) * cosW0 + 2.0 * gcem::sqrt(a) * alpha);
00118     c.b[1] = 2.0 * a * ((a - 1.0) - (a + 1.0) * cosW0);
00119     c.b[2] = a * ((a + 1.0) - (a - 1.0) * cosW0 - 2.0 * gcem::sqrt(a) * alpha);
00120     c.a[0] = (a + 1.0) + (a - 1.0) * cosW0 + 2.0 * gcem::sqrt(a) * alpha;
00121     c.a[1] = -2.0 * ((a - 1.0) + (a + 1.0) * cosW0);
00122     c.a[2] = (a + 1.0) + (a - 1.0) * cosW0 - 2.0 * gcem::sqrt(a) * alpha;
00123     return c;
00124 }
00125
00126 static inline Coeffs6 calcCoeffsHiShelf(
00127     signal_t fs, signal_t fc_hz, signal_t q, signal_t a) {
00128     auto w0 = 2.0 * GCEM_PI * fc_hz / fs;
00129     auto cosW0 = gcem::cos(w0);
00130     auto alpha = gcem::sin(w0) / (2.0 * q);
00131     Coeffs6 c;
00132     c.b[0] = a * ((a + 1.0) + (a - 1.0) * cosW0 + 2.0 * gcem::sqrt(a) * alpha);
00133     c.b[1] = -2.0 * a * ((a - 1.0) + (a + 1.0) * cosW0);
00134     c.b[2] = a * ((a + 1.0) + (a - 1.0) * cosW0 - 2.0 * gcem::sqrt(a) * alpha);
00135     c.a[0] = (a + 1.0) - (a - 1.0) * cosW0 + 2.0 * gcem::sqrt(a) * alpha;
00136     c.a[1] = 2.0 * ((a - 1.0) - (a + 1.0) * cosW0);
00137     c.a[2] = (a + 1.0) - (a - 1.0) * cosW0 - 2.0 * gcem::sqrt(a) * alpha;
00138     return c;
00139 }
00140
00141 static inline Coeffs6 calcCoeffsHpF(signal_t fs, signal_t fc_hz, signal_t q) {
00142     auto w0 = 2.0 * GCEM_PI * fc_hz / fs;
00143     auto cosW0 = gcem::cos(w0);
00144     auto alpha = gcem::sin(w0) / (2.0 * q);
00145     Coeffs6 c;
00146     c.b[0] = (1.0 + cosW0) / 2;
00147     c.b[1] = -(1.0 + cosW0);
00148     c.b[2] = (1.0 + cosW0) / 2;
00149     c.a[0] = 1.0 + alpha;
00150     c.a[1] = -2.0 * cosW0;
00151     c.a[2] = 1.0 - alpha;
00152     return c;
00153 }
00154
00155 static inline Coeffs6 calcCoeffsLpF(signal_t fs, signal_t fc_hz, signal_t q) {
00156     auto w0 = 2.0 * GCEM_PI * fc_hz / fs;
00157     auto cosW0 = gcem::cos(w0);
00158     auto alpha = gcem::sin(w0) / (2.0 * q);
00159     Coeffs6 c;
00160     c.b[0] = (1.0 - cosW0) / 2;
00161     c.b[1] = 1.0 - cosW0;
00162     c.b[2] = (1.0 - cosW0) / 2;
00163     c.a[0] = 1.0 + alpha;
00164     c.a[1] = -2.0 * cosW0;
00165     c.a[2] = 1.0 - alpha;
00166     return c;
00167 }
00168
00169 static inline Coeffs6 calcCoeffsApF(signal_t fs, signal_t fc_hz, signal_t q) {
00170     auto w0 = 2.0 * GCEM_PI * fc_hz / fs;
00171     auto cosW0 = gcem::cos(w0);

```

```

00172     auto alpha = gcem::sin(w0) / (2.0 * q);
00173     Coeffs6 c;
00174     c.b[0] = 1.0 - alpha;
00175     c.b[1] = -2.0 * cosW0;
00176     c.b[2] = 1.0 + alpha;
00177     c.a[0] = 1.0 + alpha;
00178     c.a[1] = -2.0 * cosW0;
00179     c.a[2] = 1.0 - alpha;
00180     return c;
00181 }
00182
00183 static inline Coeffs6 calcCoeffsNotch(
00184     signal_t fs, signal_t fc_hz, signal_t q) {
00185     auto w0 = 2.0 * GCEM_PI * fc_hz / fs;
00186     auto cosW0 = gcem::cos(w0);
00187     auto alpha = gcem::sin(w0) / (2.0 * q);
00188     Coeffs6 c;
00189     c.b[0] = 1.0;
00190     c.b[1] = -2.0 * cosW0;
00191     c.b[2] = 1.0;
00192     c.a[0] = 1.0 + alpha;
00193     c.a[1] = -2.0 * cosW0;
00194     c.a[2] = 1.0 - alpha;
00195     return c;
00196 }
00197
00198 static inline Coeffs6 calcCoeffs6(
00199     Shape s, signal_t fs, signal_t fc_hz, signal_t q, signal_t a) {
00200     Coeffs6 c;
00201     switch (s) {
00202     case Shape::loShelf:
00203         c = calcCoeffsLoShelf(fs, fc_hz, q, a);
00204         break;
00205     case Shape::hiShelf:
00206         c = calcCoeffsHiShelf(fs, fc_hz, q, a);
00207         break;
00208     case Shape::bell:
00209         c = calcCoeffsBell(fs, fc_hz, q, a);
00210         break;
00211     case Shape::lpf:
00212         c = calcCoeffsLpf(fs, fc_hz, q);
00213         break;
00214     case Shape::hpf:
00215         c = calcCoeffsHpf(fs, fc_hz, q);
00216         break;
00217     case Shape::apf:
00218         c = calcCoeffsApf(fs, fc_hz, q);
00219         break;
00220     case Shape::notch:
00221         c = calcCoeffsNotch(fs, fc_hz, q);
00222         break;
00223     case Shape::none:
00224     case Shape::unknown:
00225     default:
00226         c = { 1.0, 0.0, 0.0, 1.0, 0.0, 0.0 };
00227         break;
00228     }
00229     return c;
00230 }
00231
00232 static inline Coeffs5 normalizeCoeffs(Coeffs6 coeffs6) {
00233     Coeffs5 coeffs5;
00234     coeffs5.b[0] = coeffs6.b[0] / coeffs6.a[0];
00235     coeffs5.b[1] = coeffs6.b[1] / coeffs6.a[0];
00236     coeffs5.b[2] = coeffs6.b[2] / coeffs6.a[0];
00237     coeffs5.a[0] = coeffs6.a[1] / coeffs6.a[0];
00238     coeffs5.a[1] = coeffs6.a[2] / coeffs6.a[0];
00239     return coeffs5;
00240 }
00241
00242 static inline Coeffs5 calcCoeffs5(
00243     Shape s, signal_t fs, signal_t fc_hz, signal_t q, signal_t a) {
00244     auto coeffs6 = calcCoeffs6(s, fs, fc_hz, q, a);
00245     return normalizeCoeffs(coeffs6);
00246 }
00247
00248 static inline Coeffs5 calcCoeffs5(Settings& settings, signal_t fs) {
00249     return calcCoeffs5(settings.shape,
00250         fs,
00251         settings.fc_hz,
00252         settings.q,
00253         gcem::pow(10, (settings.gain_db / 40)));
00254 }
00255
00256 static inline Coeffs6 calcCoeffs6(Settings& settings, signal_t fs) {
00257     return calcCoeffs6(settings.shape,
00258         fs,

```

```

00259         settings.fc_hz,
00260         settings.q,
00261         gcem::pow(10, (settings.gain_db / 40)));
00262     }
00263
00264     struct EqBandMono : public ComponentBase<Audio> {
00265         Settings settings;
00266         Coeffs5 coeffs;
00267         State state;
00268
00269         virtual Audio process(Audio x) noexcept override {
00270             return processBiquad5(x.l, this->coeffs, this->state);
00271         }
00272         virtual void update() noexcept override {
00273             this->coeffs = calcCoeffs5(this->settings, this->fs);
00274         }
00275         virtual void reset(float fs) noexcept override {
00276             this->fs = fs;
00277             this->state = { { 0, 0 }, { 0, 0 } };
00278             this->update();
00279         }
00280     };
00281
00282     struct EqBand6Stereo : public ComponentBase<Audio> {
00283         Biquad6 l;
00284         Biquad6 r;
00285         Settings settings;
00286         Coeffs6 coeffs;
00287         EqBand6Stereo() : l(coeffs), r(coeffs) { }
00288         virtual Audio process(Audio x) noexcept override {
00289             return { this->l.process(x.l), this->r.process(x.r) };
00290         }
00291         virtual void update() noexcept override {
00292             this->coeffs = calcCoeffs6(settings, this->fs);
00293         }
00294         virtual void reset(float fs) noexcept override {
00295             this->fs = fs;
00296             this->update();
00297         }
00298     };
00299
00300     struct EqBand6Mono : public ComponentBase<Audio> {
00301         Biquad6 l;
00302         Settings settings;
00303         Coeffs6 coeffs;
00304         EqBand6Mono() : l(coeffs) { }
00305         virtual Audio process(Audio x) noexcept override {
00306             return this->l.process(x.l);
00307         }
00308         virtual void update() noexcept override {
00309             this->coeffs = calcCoeffs6(settings, this->fs);
00310         }
00311         virtual void reset(float fs) noexcept override {
00312             this->fs = fs;
00313             this->update();
00314         }
00315     };
00316
00317     struct EqBandStereo : public ComponentBase<Audio> {
00318         Settings settings;
00319         Coeffs5 coeffs;
00320         State stateL;
00321         State stateR;
00322
00323         virtual Audio process(Audio x) noexcept override {
00324             return { processBiquad5(x.l, this->coeffs, this->stateL),
00325                     processBiquad5(x.r, this->coeffs, this->stateR) };
00326         }
00327         virtual void update() noexcept override {
00328             this->coeffs = calcCoeffs5(this->settings, this->fs);
00329         }
00330         virtual void reset(float fs) noexcept override {
00331             this->fs = fs;
00332             this->stateL = { { 0, 0 }, { 0, 0 } };
00333             this->stateR = { { 0, 0 }, { 0, 0 } };
00334             this->update();
00335         }
00336     };
00337
00338     using EqBand6 = EqBand6Stereo;
00339     using EqBand = EqBandStereo;
00340 } // namespace Biquad
00341 } // namespace NtFx
00342
00343 // #pragma once
00344
00345 // // Indexes for coeffs

```

```

00346 // constexpr int b0      = 0;
00347 // constexpr int b1      = 1;
00348 // constexpr int b2      = 2;
00349 // constexpr int a1      = 3;
00350 // constexpr int a2      = 4;
00351 // constexpr int minus1 = 0;
00352 // constexpr int minus2 = 1;
00353
00354 // template <typename T, bool t_32bitState = false> struct BiquadParameterSet {
00355 //     /**
00356 //      * @brief Coeffs are stored in the order { b0 a1 b1 a2 b2, padding * 3}
00357 //      *
00358 //      */
00359 //     T m_coeffs[8] { 1, 0, 0, 0, 0, 0, 0, 0 };
00360 // };
00361
00362 // /**
00363 //  * @brief Cascade of t_numStages biquad filters. Optimized to recycle feed
00364 //  * back state
00365 //  * of previous stage as feed forward state for the next stage in the cascade.
00366 //  * This
00367 //  * should decrease the amount of writes by half in the filter. To be
00368 //  * confirmed.
00369 //  *
00370 //  * @tparam T Data type of audio. Double or float.
00371 //  * @tparam t_numStages Number of stages.
00372 //  */
00373 // template <typename T, int t_numStages = 1> struct BiquadCascade {
00374 //     BiquadParameterSet<T> m_params[t_numStages];
00375
00376 //     /**
00377 //      * @brief Feed forward state for frist filter in chain.
00378 //      *
00379 //      */
00380 //     T m_xn[2];
00381
00382 //     /**
00383 //      * @brief Feed back state for each stage doubles as feed forward state for
00384 //      * the
00385 //      * next stage.
00386 //      *
00387 //      */
00388 //     T m_yn[2 * t_numStages];
00389
00390 //     /**
00391 //      * @brief Processes one sample with BiquadCascade.
00392 //      *
00393 //      * @param x Input sample.
00394 //      * @return T y Out put sample.
00395 //      */
00396 //     T processSample(T x) {
00397 //         // First stage uses the stored feed forward state m_xn.
00398 //         T acc = m_params[0].m_coeffs[b0] * x;
00399
00400 //         acc += m_params[0].m_coeffs[b1] * m_xn[minus1];
00401 //         acc += m_params[0].m_coeffs[b2] * m_xn[minus2];
00402 //         acc += m_params[0].m_coeffs[a1] * m_yn[minus1];
00403 //         acc += m_params[0].m_coeffs[a2] * m_yn[minus2];
00404
00405 //         // Store feed forward state for first stage.
00406 //         m_xn[minus2] = m_xn[minus1];
00407 //         m_xn[minus1] = x;
00408
00409 //         T xRemainingStages = acc;
00410
00411 //         // Process the remaining stages using feedback state from the previous
00412 //         // stage
00413 //         // for feed forward state.
00414 //         for (size_t i = 1; i < t_numStages; i++) {
00415 //             acc = m_params[i].m_coeffs[b0] * xRemainingStages;
00416
00417 //             acc += m_params[i].m_coeffs[b1] * m_yn[(i - 1) * 2 + minus1];
00418 //             acc += m_params[i].m_coeffs[b2] * m_yn[(i - 1) * 2 + minus2];
00419 //             acc += m_params[i].m_coeffs[a1] * m_yn[i * 2 + minus1];
00420 //             acc += m_params[i].m_coeffs[a2] * m_yn[i * 2 + minus2];
00421
00422 //             // Update feed back state for previous stage.
00423 //             m_yn[(i - 1) * 2 + minus2] = m_yn[(i - 1) * 2 + minus1];
00424 //             m_yn[(i - 1) * 2 + minus1] = xRemainingStages;
00425
00426 //             xRemainingStages = acc;
00427 //         }
00428
00429 //         // update feed back state for last stage.
00430 //         m_yn[(t_numStages - 1) * 2 + minus2] = m_yn[(t_numStages - 1) * 2 +
00431 //             minus1]; m_yn[(t_numStages - 1) * 2 + minus1] = acc; return acc;
00432 //     }

```

```

00433
00434 // /**
00435 //  * @brief Initialize cascade by resetting all states.
00436 //  *
00437 //  * @return error
00438 //  */
00439 // Error init() {
00440 //     for (size_t i = 0; i < t_numStages * 2; i++) {
00441 //         m_yn[i] = 0;
00442 //     }
00443 //     m_xn[0] = 0;
00444 //     m_xn[1] = 0;
00445
00446 //     return Error::noError;
00447 // }
00448
00449 // /**
00450 //  * @brief Set the Coeffs of biquad filter, with normalization and scaling.
00451 //  * This is the method to use in the general case and for any static
00452 //  * filter.
00453 //  *
00454 //  * @param[in] p_coeffs Pointer to unnormalized, floating point
00455 //  * coefficients.
00456 //  * @param coeffSet Number of stage in cascade to change coeffs for.
00457 //  * @return error
00458 //  */
00459 // template <typename T_coeffs>
00460 // Error setCoeffs(const BiquadCoeffs<T_coeffs>* const p_coeffs, int coeffSet
00461 // = 0) {
00462 //     if (coeffSet < 0 || coeffSet >= t_numStages) {
00463 //         return Error::outOfRangeError;
00464 //     }
00465
00466 //     // Copy the coeffs so that we don't mutate the original.
00467 //     BiquadCoeffs<T_coeffs> coeffs = *p_coeffs;
00468
00469 //     // Normalize coeffs with respect to a0.
00470 //     normalizeCoeffs(&coeffs);
00471
00472 //     m_params[coeffSet].m_coeffs[b0] = coeffs.m_b[0];
00473 //     m_params[coeffSet].m_coeffs[b1] = coeffs.m_b[1];
00474 //     m_params[coeffSet].m_coeffs[b2] = coeffs.m_b[2];
00475 //     m_params[coeffSet].m_coeffs[a1] = -coeffs.m_a[1];
00476 //     m_params[coeffSet].m_coeffs[a2] = -coeffs.m_a[2];
00477
00478 //     return Error::noError;
00479 // }
00480 // };
00481 // }

```

9.9 Comp.h

```

00001
00020
00021 #pragma once
00022
00023 #include "gcem.hpp"
00024 #include "lib/Audio.h"
00025 #include "lib/Component.h"
00026 #include "lib/PeakSensor.h"
00027 #include "lib/RmsSensor.h"
00028 #include "lib/Utils.h"
00029
00030 namespace NtFx {
00031 namespace Comp {
00041     struct ScSettings {
00042         signal_t thresh_db = signal_t(0);
00043         signal_t ratio_db = signal_t(2);
00044         signal_t knee_db = signal_t(12);
00045         signal_t tAtt_ms = signal_t(1);
00046         signal_t tRel_ms = signal_t(100);
00047         signal_t tRms_ms = signal_t(80);
00048         signal_t tPeak_ms = signal_t(20.0);
00049         bool linkEnable = false;
00050     };
00051
00061     struct PeakSideChainDb : public ComponentBase<Audio> {
00062         PeakSensorStereo peakSensor;
00063         signal_t alphaAtt = signal_t(0);
00064         signal_t alphaRel = signal_t(0);
00065
00066         ScSettings& settings;
00067         Audio stateFilter = signal_t(0.0);

```

```

00068
00073     PeakSideChainDb(ScSettings& settings) : settings(settings) { }
00074
00080     virtual Audio process(Audio x) noexcept override {
00081         auto ySens = this->peakSensor.process(x);
00082         ensureFinite(this->stateFilter);
00083         auto y = Audio({
00084             this->_gainComputer_db(ySens.l, this->stateFilter.l),
00085             this->_gainComputer_db(ySens.r, this->stateFilter.r),
00086         });
00087         if (this->settings.linkEnable) {
00088             auto _y = y.absMin();
00089             return { _y, _y };
00090         }
00091         return y;
00092     }
00093
00097     virtual void update() noexcept override {
00098         this->alphaAtt = gcem::exp(-2200.0 / (this->settings.tAtt_ms * this->fs));
00099         this->alphaRel = gcem::exp(-2200.0 / (this->settings.tRel_ms * this->fs));
00100         if (this->alphaRel < this->alphaAtt) { this->alphaRel = this->alphaAtt; }
00101         this->peakSensor.setT_ms(this->settings.tPeak_ms);
00102         this->peakSensor.update();
00103     }
00104
00109     virtual void reset(float fs) noexcept override {
00110         this->peakSensor.reset(fs);
00111         this->fs = fs;
00112         this->update();
00113     }
00114
00121     inline signal_t _gainComputer_db(signal_t x, signal_t& state) noexcept {
00122         signal_t x_db = db(x);
00123         signal_t y_db;
00124         if ((x_db - this->settings.thresh_db) > (this->settings.knee_db / 2)) {
00125             y_db = this->settings.thresh_db
00126                 + (x_db - this->settings.thresh_db) / this->settings.ratio_db;
00127         } else if ((x_db - this->settings.thresh_db)
00128                 < -(this->settings.knee_db / 2)) {
00129             y_db = x_db;
00130         } else {
00131             signal_t tmp =
00132                 (x_db - this->settings.thresh_db + this->settings.knee_db / 2);
00133             y_db = x_db
00134                 + (1 / this->settings.ratio_db - 1) * tmp * tmp
00135                 / (2 * this->settings.knee_db);
00136         }
00137         signal_t target = x_db - y_db;
00138         signal_t alpha = this->alphaRel;
00139         if (target > state) { alpha = this->alphaAtt; }
00140         signal_t yFilter = state * alpha + target * (1 - alpha);
00141         if (yFilter != yFilter) { yFilter = signal_t(0); }
00142         state = yFilter;
00143         return invDb(-yFilter);
00144     }
00145 };
00146
00156     struct PeakSideChainLinear : public PeakSideChainDb {
00157         signal_t thresh_lin = signal_t(1);
00158         signal_t ratio_lin = signal_t(1);
00159         signal_t knee_lin = signal_t(1);
00160
00165         PeakSideChainLinear(ScSettings& settings) : PeakSideChainDb(settings) { }
00166
00173         virtual Audio process(Audio x) noexcept override {
00174             auto ySens = this->peakSensor.process(x);
00175             ensureFinite(this->stateFilter);
00176             return {
00177                 this->_gainComputer_lin(ySens.l, this->stateFilter.l),
00178                 this->_gainComputer_lin(ySens.r, this->stateFilter.r),
00179             };
00180         }
00181
00185         virtual void update() noexcept override {
00186             this->PeakSideChainDb::update();
00187             this->thresh_lin = invDb(this->settings.thresh_db);
00188             this->knee_lin = invDb(this->settings.knee_db);
00189             const signal_t oneOverSqrt2 = 1.0 / gcem::sqrt(2.0);
00190             const signal_t tmp = oneOverSqrt2
00191                 - (this->settings.ratio_db - signal_t(3.0)) / signal_t(18.0);
00192             this->ratio_lin =
00193                 (signal_t(1.0) - signal_t(1.0) / this->settings.ratio_db)
00194                 * (oneOverSqrt2 - tmp * tmp * tmp * tmp * tmp);
00195         }
00196
00203         inline signal_t _gainComputer_lin(signal_t x, signal_t& state) noexcept {
00204             signal_t target;

```

```

00205     if (x < this->thresh_lin / this->knee_lin) {
00206         target = signal_t(0);
00207     } else if (x < this->thresh_lin) {
00208         target = (x / this->thresh_lin) * this->ratio_lin * this->thresh_lin
00209             / (this->knee_lin * x);
00210     } else {
00211         target = (x / this->thresh_lin) * this->ratio_lin;
00212     }
00213     signal_t alpha = this->alphaRel;
00214     if (target > state) { alpha = this->alphaAtt; }
00215     signal_t yFilter = state * alpha + target * (1 - alpha);
00216     state = yFilter;
00217     return signal_t(1.0) / (yFilter + 1);
00218 }
00219 };
00220
00230 struct RmsSideChainDb : public PeakSideChainDb {
00231     LongRmsSensor<> rmsSensor;
00232
00233     RmsSideChainDb(ScSettings& settings) : PeakSideChainDb(settings) { }
00234
00240     virtual Audio process(Audio x) noexcept override {
00241         auto ySens = rmsSensor.process(x);
00242
00243         ensureFinite(this->stateFilter);
00244         return {
00245             this->_gainComputer_db(ySens.l, this->stateFilter.l),
00246             this->_gainComputer_db(ySens.r, this->stateFilter.r),
00247         };
00248     }
00249
00253     virtual void update() noexcept override {
00254         this->rmsSensor.setT_ms(this->settings.tRms_ms);
00255         this->rmsSensor.update();
00256         this->PeakSideChainDb::update();
00257     }
00258
00263     virtual void reset(float fs) noexcept override {
00264         this->rmsSensor.reset(fs);
00265         this->PeakSideChainDb::reset(fs);
00266     }
00267 };
00268
00278 struct RmsSideChainLinear : public PeakSideChainLinear {
00279     LongRmsSensor<> rmsSensor;
00280
00285     RmsSideChainLinear(ScSettings& settings) : PeakSideChainLinear(settings) { }
00286
00292     virtual Audio process(Audio x) noexcept override {
00293         auto ySens = rmsSensor.process(x);
00294         ensureFinite(this->stateFilter);
00295         return {
00296             this->_gainComputer_lin(ySens.l, this->stateFilter.l),
00297             this->_gainComputer_lin(ySens.r, this->stateFilter.r),
00298         };
00299     }
00300
00304     virtual void update() noexcept override {
00305         this->rmsSensor.setT_ms(this->settings.tRms_ms);
00306         this->rmsSensor.update();
00307         this->PeakSideChainLinear::update();
00308     }
00309
00314     virtual void reset(float fs) noexcept override {
00315         this->rmsSensor.reset(fs);
00316         this->PeakSideChainLinear::reset(fs);
00317     }
00318 };
00319 } // namespace NtFx
00320 }

```

9.10 lib/Component.h File Reference

Base class for Audio components. Defines the general interface of the NTplugin framework.

```
#include "lib/Audio.h"
```

Classes

- struct `NtFx::ComponentBase< T >`
Base class for audio components in `NtPlugin`. All processors adhere to this interface.
- struct `NtFx::StereoComponent< signal_t, component_t >`
Wraps two objects of type `Component<signal_t>` as a `Component<Audio>` and calls through the both objects.
- struct `NtFx::MonoComponent< signal_t, component_t >`
Wraps two objects of type `Component<signal_t>` as a `Component<Audio>` and calls through the both objects.

Namespaces

- namespace `NtFx`
An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Typedefs

- `template<typename signal_t, typename component_t>`
using `NtFx::AudioComponent` = `StereoComponent<signal_t, component_t>`

9.10.1 Detailed Description

Base class for Audio components. Defines the general interface of the NTplugin framework.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

You are free to download, build and use this code for commercial purposes. Just don't resell it or a build of it, modified or otherwise.

9.11 Component.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00027
00028 #include "lib/Audio.h"
00029
00030 namespace NtFx {
00037 template <typename T>
00038 struct ComponentBase {
00043     float fs { 48e3 };
00044
00051     virtual T process(T x) noexcept = 0;
00052
00058     virtual void update() noexcept { }
00059
00066     virtual void reset(float fs) noexcept {
00067         this->fs = fs;
00068         this->update();
00069     };
00070 };
00071
00079 template <typename signal_t, typename component_t>
00080 struct StereoComponent : public ComponentBase<Stereo<signal_t> > {
00081     component_t l;
00082     component_t r;
00083
00084     virtual Stereo<signal_t> process(Stereo<signal_t> x) noexcept override {
00085         return { this->l.process(x.l), this->r.process(x.r) };
00086     }
00087
00088     virtual void update() noexcept override {
00089         this->l.update();
00090         this->r.update();
00091     }
00092
00093     virtual void reset(float fs) noexcept override {
00094         this->l.reset(fs);
00095         this->r.reset(fs);
00096     }
00097 };
00098
00106 template <typename signal_t, typename component_t>
00107 struct MonoComponent : public ComponentBase<Mono<signal_t> > {
00108     component_t l;
00109     virtual Mono<signal_t> process(Mono<signal_t> x) noexcept override {
00110         return this->l.process(x.l);
00111     }
00112
00113     virtual void update() noexcept override { this->l.update(); }
00114     virtual void reset(float fs) noexcept override { this->l.reset(fs); }
00115 };
00116
00117 template <typename signal_t, typename component_t>
00118 using AudioComponent = StereoComponent<signal_t, component_t>;
00119 } // namespace NtFx

```

9.12 lib/ComponentTest.h File Reference

Test of audio components.

```

#include "lib/Audio.h"
#include "lib/Component.h"
#include <algorithm>
#include <cstring>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <memory>
#include <sstream>

```

```
#include <stdlib.h>
#include <string>
#include <vector>
```

Classes

- struct [NtFx::ComponentTestSet](#)
Set of a number of tests. Represents a full test file and this a test of a Component in the library or an [NtPlugin](#).
- struct [NtFx::ComponentTest](#)
Processes a vector of test data with audio component and compares to expected results. Total number of tests and number of succesful test are stored as statics in order to make statistics.

Namespaces

- namespace [NtFx](#)
An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Macros

- `#define _NTFX_QUOTE(str)`
- `#define _NTFX_EXPAND_AND_QUOTE(str)`
- `#define _NTFX_ADD_TEST_IMPL(object, stimuli)`
- `#define NTFX_ADD_TEST(object, stimuli)`
Adds a new test to ComponentTest.
- `#define _NTFX_TEST_BEGIN_IMPL`
- `#define NTFX_TEST_BEGIN _NTFX_TEST_BEGIN_IMPL`
Add this to the beginning of you test suite to instantiate the needed statics of ComponentTest. Sucks, I know, but that's what C++ demands.
- `#define _NTFX_TEST_STARTED`
- `#define _NTFX_TEST_IMPL()`
- `#define NTFX_TEST()`
- `#define NTFX_RUN_TESTS()`
Runs all tests added to test set.

Functions

- `constexpr auto testFileName (std::string_view fileName)`

Variables

- `constexpr char NtFx::SEPARATOR = ','`

9.12.1 Detailed Description

Test of audio components.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

A unit test framework for NtPlugin library components and plugins. Works in conjunction with 'testWrapper/test.py', which builds and runs the tests. Tests are specified in 'testWrapper/tests' in the form of cpp files named after the Component to test appended '_test.cpp'. Inputs are stored in 'testWrapper/in' and results are stored in 'testWrapper/out'. 'in' is under git source control since it contains expected test vectors along with stimuli. Input and output files must contain two columns of floating point numbers.

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

You are free to download, build and use this code for commercial purposes. Just don't resell it or a build of it, modified or otherwise.

9.12.2 Macro Definition Documentation

9.12.2.1 _NTFX_ADD_TEST_IMPL

```
#define _NTFX_ADD_TEST_IMPL(  
    object,  
    stimuli)
```

Value:

```
componentTestSet.addTest(object, _NTFX_EXPAND_AND_QUOTE(object), { stimuli })
```

9.12.2.2 _NTFX_EXPAND_AND_QUOTE

```
#define _NTFX_EXPAND_AND_QUOTE(  
    str)
```

Value:

```
_NTFX_QUOTE(str)
```

9.12.2.3 _NTFX_QUOTE

```
#define _NTFX_QUOTE(  
    str)
```

Value:

```
#str
```

9.12.2.4 _NTFX_TEST_BEGIN_IMPL

```
#define _NTFX_TEST_BEGIN_IMPL
```

Value:

```
auto componentTestSet =  
    NtFx::ComponentTestSet(std::string(testFileName(__FILE__)));
```

9.12.2.5 _NTFX_TEST_IMPL

```
#define _NTFX_TEST_IMPL()
```

Value:

```
int main()
```

9.12.2.6 NTFX_ADD_TEST

```
#define NTFX_ADD_TEST(  
    object,  
    stimuli)
```

Value:

```
_NTFX_ADD_TEST_IMPL(object, stimuli)
```

Adds a new test to ComponentTest.

Parameters

<i>object</i>	Object derived from Component class that need to be tested. Note that the object is NOT copied and must stay in scope until the test has run.
---------------	---

9.12.2.7 NTFX_RUN_TESTS

```
#define NTFX_RUN_TESTS()
```

Value:

```
componentTestSet.runAllTests()
```

Runs all tests added to test set.

9.12.2.8 NTFX_TEST

```
#define NTFX_TEST()
```

Value:

```
_NTFX_TEST_IMPL()
```

9.13 ComponentTest.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00035
00036 #include "lib/Audio.h"
00037 #include "lib/Component.h"
00038 #include <algorithm>
00039 #include <cstring>
00040 #include <filesystem>
00041 #include <fstream>
00042 #include <iomanip>
00043 #include <iostream>
00044 #include <memory>
00045 #include <sstream>
00046 #include <stdlib.h>
00047 #include <string>
00048 #include <vector>
00049
00050 constexpr auto testFileName(std::string_view fileName) {
00051     auto _begin =
00052         fileName.find("tests/") + std::char_traits<char>::length("tests/");
00053     auto n
00054         = fileName.length();
00055     auto baseName
00056         = fileName.substr(_begin, n);
00057     auto _end
00058         = baseName.find(".cpp");
00059     auto baseNameNoExt
00060         = baseName.substr(0, _end);
00061     _end
00062         = baseNameNoExt.find("_test");
00063     baseNameNoExt
00064         = baseNameNoExt.substr(0, _end);
00065     return baseNameNoExt;
00066 }
00067
00068 #define _NTFX_QUOTE(str) #str
00069 #define _NTFX_EXPAND_AND_QUOTE(str) _NTFX_QUOTE(str)
00070
00071 #define _NTFX_ADD_TEST_IMPL(object, stimuli) \
00072     componentTestSet.addTest(object, _NTFX_EXPAND_AND_QUOTE(object), { stimuli })
00073
00074 #define NTFX_ADD_TEST(object, stimuli) _NTFX_ADD_TEST_IMPL(object, stimuli)
00075
00076 #define _NTFX_TEST_BEGIN_IMPL \
00077     auto componentTestSet = \
00078         NtFx::ComponentTestSet(std::string(testFileName(__FILE__)));
00079
00080 #ifndef _NTFX_TEST_STARTED
00081     #define NTFX_TEST_BEGIN _NTFX_TEST_BEGIN_IMPL
00082     #define _NTFX_TEST_STARTED
00083 #else
00084     #define NTFX_TEST_BEGIN
00085 #endif
00086
00087 #define _NTFX_TEST_IMPL() int main()
00088 #define NTFX_TEST() _NTFX_TEST_IMPL()
00089
00090 #define NTFX_RUN_TESTS() componentTestSet.runAllTests()
00091
00092 namespace NtFx {
00093     static const std::vector<std::string> STIMULI_NAMES { "impulse",
00094         "syncSweep",
00095         "linearSweep",
00096         "dynamic_alternating",
00097         "dynamic_matched" };
00098
00099     constexpr char SEPARATOR = '.';
00100
00101     struct ComponentTest;
00102
00103     struct ComponentTestSet {
00104         int nTests;
00105         int nSuccessful;
00106     };
00107 }
```

```

00123     std::vector<std::unique_ptr<ComponentTest>>
00124         tests;
00125     std::string name;
00126
00127     ComponentTestSet(std::string name) : name(name) { }
00128
00129     bool getResults() {
00130         std::cout << std::fixed << std::setprecision(2);
00131         if (nSuccessful == this->nTests) {
00132             std::cout << "\033[32m";
00133         } else {
00134             std::cout << "\033[31m";
00135         }
00136         std::cout << "Ran a total of " << this->nTests << " test on "
00137             << this->tests.size() << " objects. " << this->nSuccessful
00138             << " succeeded. ("
00139             << 100.0 * double(this->nSuccessful) / double(this->nTests)
00140             << "%)."
00141             << "\033[0m" << "\n";
00142         auto f = std::ofstream("testWrapper/out/results.txt", std::ios_base::app);
00143         f << this->name << ", " << this->nTests << ", " << this->tests.size() << ", "
00144             << this->nSuccessful << "\n";
00145         return nSuccessful == nTests;
00146     }
00147
00148     bool addTest(ComponentBase<Stereo<signal_t>>& componentObj,
00149         std::string objName,
00150         std::vector<std::string> stimuli);
00151
00152     int runAllTests();
00153 };
00154
00155 struct ComponentTest {
00156     ComponentTestSet& owner;
00157     const std::string objName;
00158     ComponentBase<Stereo<signal_t>>& cut;
00159     std::vector<std::string> activeStimuli = STIMULI_NAMES;
00160
00161     ComponentTest(ComponentTestSet& owner,
00162         std::string objName,
00163         ComponentBase<Stereo<signal_t>>& cut,
00164         std::vector<std::string> stimuli = { })
00165         : owner(owner), objName(objName), cut(cut) {
00166         if (stimuli.empty()) { return; }
00167         for (auto& stimulus : stimuli) {
00168             if (std::find(STIMULI_NAMES.begin(), STIMULI_NAMES.end(), stimulus)
00169                 == STIMULI_NAMES.end()) {
00170                 std::cout << "Stimuli name '" << stimulus << "' not accepted.";
00171                 this->activeStimuli.clear();
00172                 return;
00173             }
00174         }
00175         this->activeStimuli = stimuli;
00176     }
00177
00178     bool run(std::string stimulus) {
00179         if (!this->stimulusIsActive(stimulus)) { return true; }
00180         this->owner.nTests++;
00181         auto x = this->_readInput(stimulus);
00182         std::vector<Stereo<signal_t>> y;
00183         this->cut.reset(NTFX_FS);
00184         for (auto _x : x) { y.push_back(cut.process(_x)); }
00185         const auto yPath = "testWrapper/out/" + this->owner.name + SEPARATOR
00186             + this->objName + SEPARATOR + stimulus + SEPARATOR + "result.txt";
00187         std::ofstream yFile(yPath);
00188         yFile << std::fixed << std::setprecision(16);
00189         for (auto _y : y) { yFile << _y.l << " " << _y.r << "\n"; }
00190         auto success = this->_compareExpected(stimulus, y);
00191         if (success) {
00192             std::cout << "\033[32m";
00193             this->owner.nSuccessful++;
00194         } else {
00195             std::cout << "\033[31m";
00196         }
00197         std::cout << "Test '" << stimulus << "' for object '" << this->objName
00198             << "' in file '" << this->owner.name << "'";
00199         std::cout << (success ? " passed." : " failed.") << "\033[0m" << "\n";
00200         return success;
00201     }
00202
00203     std::vector<Stereo<signal_t>> _readInput(std::string stimulus) {
00204         auto xPath = "testWrapper/in/" + stimulus + ".txt";
00205         if (!std::filesystem::exists(xPath)) {
00206             std::cout << "Input file '" << xPath << "' not found. Aborting test."
00207                 << "\n";
00208             return { };
00209         }
00210         std::fstream xFile(xPath);

```

```

00259     std::vector<Stereo<signal_t>> x;
00260     signal_t l, r;
00261     while (xFile >> l >> r) { x.push_back({ l, r }); }
00262     return x;
00263 }
00264
00265 bool _stimulusIsActive(std::string stimulus) {
00266     if (std::find(
00267         this->activeStimuli.begin(), this->activeStimuli.end(), stimulus)
00268         == this->activeStimuli.end()) {
00269         return false;
00270     }
00271     return true;
00272 }
00273
00274 std::vector<Stereo<signal_t>> _readExpected(std::string stimulus) {
00275     auto expPath = "testWrapper/in/" + this->owner.name + SEPARATOR
00276         + this->objName + SEPARATOR + stimulus + SEPARATOR + "expected.txt";
00277     bool expFileExists = std::filesystem::exists(expPath);
00278     if (!expFileExists) {
00279         std::cout << "File with expected results not found at path '" + expPath
00280             + "'. Skipping comparison."
00281             << "\n";
00282         return { };
00283     }
00284     std::vector<Stereo<signal_t>> e;
00285     std::fstream eFile(expPath);
00286     std::string line;
00287     while (std::getline(eFile, line)) {
00288         std::istringstream iss(line);
00289         float l, r;
00290         if (iss >> l >> r) {
00291             e.push_back({ l, r });
00292         } else {
00293             e.push_back({ 0, 0 });
00294         }
00295     }
00296     return e;
00297 }
00298
00299 bool _compareExpected(
00300     std::string stimulus, const std::vector<Stereo<signal_t>>& y) {
00301     auto e = this->_readExpected(stimulus);
00302     if (e.size() != y.size()) {
00303         std::cout << "Expected result has different length than result. Aborting."
00304             << " e: " << e.size() << ", y: " << y.size() << "\n";
00305         return false;
00306     }
00307     signal_t acceptedDiff = 0.0001;
00308     for (size_t i = 0; i < y.size(); i++) {
00309         auto diff = gcem::abs(y[i] - e[i]);
00310         if (diff > acceptedDiff) {
00311             std::cout << this->owner.name << "." << this->objName << "." << stimulus
00312                 << ":" << " output: {" << y[i].l << ", " << y[i].r
00313                 << "}, expected: {" << e[i].l << ", " << e[i].r
00314                 << "}, at index: " << i << ". Diff: {" << diff.l << ", "
00315                 << diff.r << "}. " << "\n";
00316             return false;
00317         }
00318     }
00319     return true;
00320 }
00321 };
00322
00323 bool ComponentTestSet::addTest(ComponentBase<Stereo<signal_t>>& componentObj,
00324     std::string objName,
00325     std::vector<std::string> stimuli) {
00326     this->tests.push_back(std::make_unique<NtFx::ComponentTest>(
00327         *this, objName, componentObj, stimuli));
00328     return true;
00329 }
00330
00331 int ComponentTestSet::runAllTests() {
00332     for (auto& stimulus : STIMULI_NAMES) {
00333         for (size_t i = 0; i < tests.size(); i++) { tests[i]->run(stimulus); }
00334     }
00335     return !this->getResults();
00336 }
00337 }

```

9.14 lib/DynamicFilter.h File Reference

Dynamic biquad filter.

```
#include "gcem.hpp"
#include "lib/Audio.h"
#include "lib/Biquad.h"
#include "lib/Component.h"
```

Classes

- struct [NtFx::DynamicFilter::Shelf](#)

An optimized dynamic shelving filter, which exposes both Q values. Feedback coeffs are calculated in update, while feed forward is calculated in process, so the gain of the filter can be updated while running, making the filter usefull for dynamic processing. Gain is in the linear domain for the same reason.

Namespaces

- namespace [NtFx](#)

An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

9.14.1 Detailed Description

Dynamic biquad filter.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

You are free to download, build and use this code for commercial purposes. Just don't resell it or a build of it, modified or otherwise.

9.15 DynamicFilter.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00026
00027 #include "gcem.hpp"
00028 #include "lib/Audio.h"
00029 #include "lib/Biquad.h"
00030 #include "lib/Component.h"
00031
00032 namespace NtFx {
00033 namespace DynamicFilter {
00043     struct Shelf : public ComponentBase<Audio> {
00044         Biquad::EqBand6 _flt;
00045         signal_t fc_hz { 2000 };
00046         signal_t gain_lin {
00047             1
00048         };
00049         signal_t q1 = 0.707;
00050         signal_t q2 =
00051             0.707;
00052         signal_t _alphaSquared { 0 };
00053         signal_t _betat { 0 };
00054         signal_t _beta2 { 0 };
00055
00056         virtual Audio process(Audio x) noexcept override {
00057             auto A4
00058                 = signal_t(4) * gain_lin;
00059             auto z
00060                 = this->_betat * gcem::sqrt(this->gain_lin);
00061             this->_flt.coeffs.b[0] = this->_alphaSquared + z + A4;
00062             this->_flt.coeffs.b[1] = signal_t(2) * (this->_alphaSquared - A4);
00063             this->_flt.coeffs.b[2] = this->_alphaSquared - z + A4;
00064             return _flt.process(x);
00065         }
00066
00067         virtual void update() noexcept override {
00068             auto alpha
00069                 = signal_t(2) * GCEM_PI * this->fc_hz / this->fs;
00070             this->_alphaSquared = alpha * alpha;
00071             this->_betat = signal_t(2) * alpha / this->q1;
00072             this->_beta2 = signal_t(2) * alpha / this->q2;
00073             this->_flt.coeffs.a[0] = this->_alphaSquared + this->_betat + 4;
00074             this->_flt.coeffs.a[1] = signal_t(2) * this->_alphaSquared - 8;
00075             this->_flt.coeffs.a[2] = this->_alphaSquared - this->_betat + 4;
00076         }
00077
00078         virtual void reset(float fs) noexcept override {
00079             this->fs = fs;
00080             this->_flt.reset(fs);
00081             this->update();
00082         }
00083     };
00084 }

```

9.16 lib/FirstOrder.h File Reference

First order high- and low pass filters.

```

#include "lib/Audio.h"
#include "lib/Component.h"
#include <math.h>

```

Classes

- struct [NtFx::FirstOrder::Filter< shape >](#)
A single channel first order filter.
- struct [NtFx::FirstOrder::StereoFilter< shape >](#)
Stereo pair of first order filters.

Namespaces

- namespace `NtFx`

An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Enumerations

- enum class `NtFx::FirstOrder::Shape` { `none` , `lpf` , `lpfZero` , `hpf` }

Filter shape for first order filters. 'none' denoted bypass and 'lpfZero' has a zero added at Nyquist.

9.16.1 Detailed Description

First order high- and low pass filters.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.17 FirstOrder.h

[Go to the documentation of this file.](#)

```
00001
00022
00023 #include "lib/Audio.h"
00024 #include "lib/Component.h"
00025 #include <math.h>
00026
00027 namespace NtFx {
00028 namespace FirstOrder {
00034 enum class Shape {
00035     none,
00036     lpf,
00037     lpfZero,
00038     hpf,
00039 };
00040
00047 template <Shape shape>
00048 struct Filter : public ComponentBase<signal_t> {
00049     signal_t fc_hz = 1000;
00050     signal_t _a = 0;
00051     signal_t _yn1 = 0;
00052     signal_t _xn1 = 0;
```

```

00053
00054     virtual signal_t process(signal_t x) noexcept override {
00055         signal_t y;
00056         if constexpr (shape == Shape::none) {
00057             return x;
00058         } else if constexpr (shape == Shape::lpf) {
00059             y = this->_a * x + (1 - this->_a) * _ynl;
00060         } else if constexpr (shape == Shape::hpf) {
00061             y = this->_a * (this->_ynl + x - this->_xn1);
00062         } else {
00063             y = signal_t(0.5) * this->_a * (x + _xn1) + (1 - this->_a) * _ynl;
00064         }
00065         this->_xn1 = x;
00066         this->_ynl = y;
00067         return y;
00068     }
00069
00070     virtual void update() noexcept override {
00071         if (this->fs <= 0) { return; }
00072         signal_t z = 2 * GCEM_PI * this->fc_hz / this->fs;
00073         if constexpr (shape == Shape::hpf) {
00074             this->_a = 1.0 / (z + 1.0);
00075         } else {
00076             this->_a = z / (z + 1.0);
00077         }
00078     }
00079
00080     virtual void reset(float fs) noexcept override {
00081         this->fs = fs;
00082         this->_xn1 = 0;
00083         this->_ynl = 0;
00084         this->update();
00085     }
00086 };
00087
00094 template <Shape shape>
00095 struct StereoFilter : public ComponentBase<Audio> {
00096     Filter<shape> l;
00097     Filter<shape> r;
00098
00099     Audio process(Audio x) noexcept override {
00100         return { l.process(x.l), r.process(x.r) };
00101     }
00102
00103     virtual void update() noexcept override {
00104         l.update();
00105         r.update();
00106     }
00107
00108     virtual void reset(float fs) noexcept override {
00109         l.reset(fs);
00110         r.reset(fs);
00111     }
00112
00113     void setFc(signal_t fc) noexcept {
00114         l.fc_hz = fc;
00115         r.fc_hz = fc;
00116     }
00117 };
00118 }
00119 }

```

9.18 lib/GateSc.h File Reference

Gate side chain.

```

#include "lib/Audio.h"
#include "lib/Component.h"
#include "lib/PeakSensor.h"
#include "lib/utils.h"

```

Classes

- struct [NtFx::Gate::ScSettings](#)
- struct [NtFx::Gate::Sc](#)

Namespaces

- namespace [NtFx](#)

An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Enumerations

- enum **State** { open , hold , release , closed }

9.18.1 Detailed Description

Gate side chain.

Author

Niels Thøgersen

Copyright

Copyright (C) 2026 Niels Thøgersen, NTIyd

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.19 GateSc.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00021
00022 #include "lib/Audio.h"
00023 #include "lib/Component.h"
00024 #include "lib/PeakSensor.h"
00025 #include "lib/Utils.h"
00026
00027 namespace NtFx {
00028 namespace Gate {
00029
00030 struct ScSettings {
00031     signal_t thresh_db { 0 };
00032     signal_t range_db { 0 };
00033     signal_t tAtt_ms { 0.01 };
00034     signal_t tHold_ms { 10 };
00035     signal_t tRel_ms { 200 };
00036 };
00037
00038 enum State { open, hold, release, closed };
00039
00040 struct Sc : public ComponentBase<Audio> {
00041     PeakSensor sensor;
00042     ScSettings settings;
00043     State state;
```

```

00044     signal_t _slopeRel { 0 };
00045     signal_t _alphaAtt { 0 };
00046     signal_t _stateAtt { 0 };
00047     signal_t _stateRel { 0 };
00048     int _nHold { 0 };
00049     int _holdCount { 0 };
00050
00051     virtual Audio process(Audio x) noexcept override {
00052         auto xSc = x.absMax();
00053         auto y = this->gateSc_db(xSc);
00054         return { y, y };
00055     }
00056
00057     virtual void update() noexcept override {
00058         this->_alphaAtt = gcem::exp(-2200 / (this->settings.tAtt_ms * this->fs));
00059         this->_slopeRel = this->settings.range_db * signal_t(20)
00060             / (this->settings.range_db * this->settings.tRel_ms * signal_t(0.001)
00061                 * this->fs
00062                 + 1e-20);
00063         this->_nHold = gcem::round(this->settings.tHold_ms * 0.001 * this->fs);
00064         this->sensor.update();
00065     }
00066
00067     virtual void reset(float fs) noexcept override {
00068         this->fs = fs;
00069         _stateAtt = -100;
00070         _stateRel = -100;
00071         this->sensor.tPeak_ms = 1;
00072         this->sensor.reset(fs);
00073         this->update();
00074     }
00075
00076     signal_t gateSc_db(signal_t x) {
00077         auto ySens { this->sensor.process(x) };
00078         auto x_db = NtFx::db(ySens);
00079         signal_t target_db { -1e-20 };
00080         if (x_db > this->settings.thresh_db) {
00081             this->state = State::open;
00082             this->_holdCount = this->_nHold;
00083         } else {
00084             target_db = this->settings.range_db;
00085         }
00086         if (this->_stateRel >= target_db) {
00087             if (this->_holdCount > 0) {
00088                 this->state = State::hold;
00089                 this->_holdCount--;
00090             } else {
00091                 this->state = State::release;
00092                 this->_stateRel -= this->_slopeRel;
00093             }
00094         } else {
00095             this->state = State::closed;
00096             this->_stateRel = target_db;
00097         }
00098         ensureFinite(this->_stateAtt);
00099         auto y_db = this->_alphaAtt * this->_stateAtt
00100             + (1 - this->_alphaAtt) * this->_stateRel;
00101         this->_stateAtt = y_db;
00102         return NtFx::invDb(y_db);
00103     }
00104 };
00105 }
00106 }

```

9.20 Generator.h

```

00001 #include "lib/Audio.h"
00002 #include "lib/Component.h"
00003 #include "lib/Utils.h"
00004
00005 namespace NtFx {
00006     namespace Generator {
00007         struct Noise : public ComponentBase<Audio> {
00008             virtual Audio process(Audio x) noexcept override {
00009                 return { rand<signal_t>(), rand<signal_t>() };
00010             }
00011         };
00012         // TODO: Sine, square, saw and so on.
00013     }
00014 }

```

9.21 lib/Glider.h File Reference

Glider smooths out changes in parameters.

```
#include "gcem.hpp"
```

Classes

- struct [NtFx::ExpGlider](#)
- struct [NtFx::LinGlider](#)

Glides member 'pr' linearly towards 'ui'.

Namespaces

- namespace [NtFx](#)

An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

9.21.1 Detailed Description

Glider smooths out changes in parameters.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.22 Glider.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00023
00024 #include "gcem.hpp"
00025
00026 // TODO: Make this a Component.
00032 namespace NtFx {
00033
00034 struct ExpGlider {
00035     signal_t ui = 0;
00036     signal_t pr = 0;
00037     signal_t _a = 0;
00038
00044     ExpGlider(signal_t def = 0) : ui(def), pr(def) { }
00045
00051     inline signal_t process() noexcept {
00052         this->pr += this->_a * (this->ui - this->pr);
00053         return this->pr;
00054     }
00055
00062     inline void update(signal_t fs, signal_t tSmooth) noexcept {
00063         if (tSmooth <= 0) {
00064             this->_a = 0;
00065             this->pr = this->ui;
00066             return;
00067         }
00068         this->_a = 1.0 - gcem::exp(-1.0 / (fs * tSmooth));
00069     }
00070 };
00071
00077 struct LinGlider {
00078     signal_t ui = 0.0;
00079     signal_t pr = 0.0;
00080     signal_t s = 0.0;
00081
00087     LinGlider(signal_t def = 0) : ui(def), pr(def) { }
00088
00094     inline signal_t process() noexcept {
00095         if (this->ui == this->pr) { return this->pr; }
00096         if (this->pr < this->ui) {
00097             this->pr += s;
00098         } else {
00099             this->pr -= s;
00100         }
00101         if (gcem::abs(this->ui - this->pr) < gcem::abs(this->s)) {
00102             this->pr = this->ui;
00103         }
00104         return this->pr;
00105     }
00106
00113     inline void update(signal_t fs, signal_t tSmooth) {
00114         if (tSmooth <= 0) {
00115             this->pr = this->ui;
00116             return;
00117         }
00118         this->s = gcem::abs(this->ui - this->pr) / (tSmooth * fs);
00119     }
00120 };
00121 }

```

9.23 lib/PeakSensor.h File Reference

Peak sensor component for audio processing. Detect and tracks the peak amplitude of an audio signal. The peak.

```

#include "gcem.hpp"
#include "lib/Audio.h"
#include "lib/Component.h"
#include "lib/utils.h"
#include <array>
#include <cstdint>

```

Classes

- struct [NtFx::PeakSensor](#)
Peak sensor component for audio processing.
- struct [NtFx::PeakSensorStereo](#)
Stereo peak sensor component for audio processing.
- struct [NtFx::PeakHoldSensor< delayLineLength >](#)
Peak sensor with hold. Output will remain at the max value for tHold_ms milliseconds when after peak and the curve will be delayed by the same amount time when going down.
- struct [NtFx::PeakHoldSensorStereo< delayLineLength >](#)

Namespaces

- namespace [NtFx](#)
An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Variables

- constexpr int [NtFx::defaultPeakSensorDelayLineLength](#) = 192 * 10 * 8

9.23.1 Detailed Description

Peak sensor component for audio processing. Detect and tracks the peak amplitude of an audio signal. The peak.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.24 PeakSensor.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00024
00025 #include "gcm.hpp"
00026 #include "lib/Audio.h"
00027 #include "lib/Component.h"
00028 #include "lib/Utils.h"
00029 #include <array>
00030 #include <cstdlib>
00031
00032 namespace NtFx {
00033
00034 struct PeakSensor : public ComponentBase<signal_t> {
00045     signal_t tPeak_ms {
00046         1
00047     };
00048     signal_t _alpha { 0 };
00049     signal_t _state { 0 };
00050
00060     virtual signal_t process(signal_t x) noexcept override {
00061         return this->_peakSensor(this->_alpha, this->_state, x);
00062     }
00063
00070     virtual void update() noexcept override {
00071         this->_alpha = gcm::exp(-2200.0 / (this->tPeak_ms * this->fs));
00072     }
00073
00074     virtual void reset(float fs) noexcept override {
00075         this->fs = fs;
00076         this->_alpha = 0;
00077         this->_state = 0;
00078         this->update();
00079     }
00080
00093     static inline signal_t _peakSensor(
00094         signal_t alpha, signal_t& p_state, signal_t x) {
00095         auto xAbs = gcm::abs(x);
00096         signal_t sensRelease = alpha * p_state + (1 - alpha) * xAbs;
00097         signal_t ySens = gcm::max(xAbs, sensRelease);
00098         ensureFinite(ySens);
00099         p_state = ySens;
00100         return ySens;
00101     }
00102 };
00103
00114 struct PeakSensorStereo : public AudioComponent<signal_t, PeakSensor> {
00122     void setT_ms(signal_t t_ms) {
00123         this->l.tPeak_ms = t_ms;
00124         this->r.tPeak_ms = t_ms;
00125     }
00126 };
00127
00128 constexpr int defaultPeakSensorDelayLineLength = 192 * 10 * 8;
00129
00138 template <int delayLineLength = defaultPeakSensorDelayLineLength>
00139 struct PeakHoldSensor : public PeakSensor {
00140     std::array<signal_t, delayLineLength> _dl;
00141     signal_t tHold_ms { 10 };
00142     signal_t _xMax { 0 };
00143     int _idx { 0 };
00144     int _nHold { 0 };
00145     int _holdCnt { 0 };
00146
00153     virtual signal_t process(signal_t x) noexcept override {
00154         auto _x = this->_peakSensor(this->_alpha, this->_state, x);
00155
00156         this->_dl[this->_idx++] = _x;
00157         if (this->_idx >= delayLineLength) { this->_idx = 0; }
00158         if (_x >= this->_xMax) {
00159             this->_holdCnt = 0;
00160             this->_xMax = _x;
00161         } else if (this->_holdCnt < this->_nHold) {
00162             this->_holdCnt++;
00163         } else {
00164             auto readIdx = this->_idx - _nHold - 1;
00165             if (readIdx < 0) { readIdx += delayLineLength; }
00166             this->_xMax = this->_dl[readIdx];
00167         }
00168         return this->_xMax;
00169     }
00170
00175     virtual void update() noexcept override {

```

```

00176     this->_nHold = this->tHold_ms * this->fs / 1000;
00177     this->_nHold =
00178         (this->_nHold >= delayLineLength ? delayLineLength - 1 : this->_nHold);
00179     this->PeakSensor::update();
00180 }
00181 };
00182
00183 template <int delayLineLength = defaultPeakSensorDelayLineLength>
00184 struct PeakHoldSensorStereo
00185     : public AudioComponent<signal_t, PeakHoldSensor<delayLineLength> > {
00193     void setT_ms(signal_t t_ms) {
00194         this->l.tPeak_ms = t_ms;
00195         this->r.tPeak_ms = t_ms;
00196     }
00197
00203     void setTHold_ms(signal_t tHold_ms) {
00204         this->l.tHold_ms = tHold_ms;
00205         this->r.tHold_ms = tHold_ms;
00206     }
00207 };
00208 }

```

9.25 lib/Plugin.h File Reference

Base class for NTplugin framework.

```

#include "lib/Audio.h"
#include "lib/Component.h"
#include "lib/RmsSensor.h"
#include "lib/UiSpec.h"
#include <array>
#include <cstdint>
#include <string>
#include <vector>

```

Classes

- struct [NtFx::NtPlugin](#)

True virtual base class for [NtPlugin](#). Inherit from this in order to make a stereo audio plugin with auto-generated UI.

Namespaces

- namespace [NtFx](#)

An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

9.25.1 Detailed Description

Base class for NTplugin framework.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.26 Plugin.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00023
00024 #include "lib/Audio.h"
00025 #include "lib/Component.h"
00026 #include "lib/RmsSensor.h"
00027 #include "lib/UiSpec.h"
00028
00029 #include <array>
00030 #include <cstdlib>
00031 #include <string>
00032 #include <vector>
00033
00034 namespace NtFx {
00035
00042 struct NtPlugin : public ComponentBase<Audio> {
00047     std::array<LongRmsSensor<250, 192>, 2> xRms;
00048
00052     std::array<Audio, nMetersMax> peakLevels;
00053
00058     UiSpec uiSpec;
00059
00064     std::vector<KnobSpec> primaryKnobs;
00065
00069     std::vector<KnobSpec> secondaryKnobs;
00070
00074     std::vector<DropDownSpec> dropdowns;
00075
00079     std::vector<ToggleSpec> toggles;
00080
00085     std::vector<ToggleSetSpec> toggleSets;
00086
00090     std::vector<RadioButtonSetSpec> radioButtons;
00091
00095     std::vector<MeterSpec> meters = {
00096         { .name = "IN", .addRms = true },
00097         { .name = "OUT", .hasScale = true, .addRms = true },
00098     };
00099
00103     signal_t tempo = 0;
00104
00109     signal_t xSc;
00110
00114     bool uiNeedsUpdate = false;
00115
00119     bool isInitialized = false;
00120
00125     virtual void onTempoChanged() noexcept { }
00126
00133     signal_t* getKnobValuePtr(std::string name) const noexcept {
00134         for (auto param : this->primaryKnobs) {
00135             if (param.name == name) { return param.p_val; }
00136         }
00137         for (auto param : this->secondaryKnobs) {
00138             if (param.name == name) { return param.p_val; }
00139         }

```

```

00140     return nullptr;
00141 }
00142
00149 bool* getToggleValuePtr(std::string name) const noexcept {
00150     for (auto param : this->toggles) {
00151         if (param.name == name) { return param.p_val; }
00152     }
00153     return nullptr;
00154 }
00155
00162 int* getDropDownValuePtr(std::string name) const noexcept {
00163     for (auto dropdown : this->dropdowns) {
00164         if (dropdown.name == name) { return dropdown.p_val; }
00165     }
00166     return nullptr;
00167 }
00168
00169 void setParameterActive(std::string name, bool val) {
00170     for (auto& param : this->primaryKnobs) {
00171         if (param.name == name) {
00172             param.isActive = val;
00173             this->uiNeedsUpdate = true;
00174             return;
00175         }
00176     }
00177     for (auto& param : this->secondaryKnobs) {
00178         if (param.name == name) {
00179             param.isActive = val;
00180             this->uiNeedsUpdate = true;
00181             return;
00182         }
00183     }
00184     // TODO: isActive field for all param types. INCLUDES fixing editor.
00185     // for (auto& param : this->toggles) {
00186     //     if (param.name == name) {
00187     //         param.isActive = false;
00188     //         this->uiNeedsUpdate = true;
00189     //         return;
00190     //     }
00191     // }
00192     // for (auto& param : this->dropdowns) {
00193     //     if (param.name == name) {
00194     //         param.isActive = false;
00195     //         this->uiNeedsUpdate = true;
00196     //         return;
00197     //     }
00198     // }
00199     // for (auto& param : this->radioButtons) {
00200     //     if (param.name == name) {
00201     //         param.isActive = false;
00202     //         this->uiNeedsUpdate = true;
00203     //         return;
00204     //     }
00205     // }
00206     // for (auto& param : this->toggleSets) {
00207     //     if (param.name == name) {
00208     //         param.isActive = false;
00209     //         this->uiNeedsUpdate = true;
00210     //         return;
00211     //     }
00212     // }
00213 }
00214
00215 void deactivateParameter(std::string name) {
00216     this->setParameterActive(name, false);
00217 }
00218
00219 void activateParameter(std::string name) {
00220     this->setParameterActive(name, true);
00221 }
00222
00229 constexpr void updateDefaults() noexcept {
00230     for (auto& k : this->primaryKnobs) {
00231         if (k.p_val) { k._defaultVal = *k.p_val; }
00232         if (k.logScale) { k.setLogScale(); }
00233     }
00234     for (auto& k : this->secondaryKnobs) {
00235         if (k.p_val) { k._defaultVal = *k.p_val; }
00236         if (k.logScale) { k.setLogScale(); }
00237     }
00238     for (auto& t : this->toggles) {
00239         if (t.p_val) { t._defaultVal = *t.p_val; }
00240     }
00241     for (auto& t : this->dropdowns) {
00242         if (t.p_val) { t._defaultVal = *t.p_val; }
00243     }
00244     for (auto& t : this->radioButtons) {

```

```

00245         if (t.p_val) { t._defaultVal = *t.p_val; }
00246     }
00247     for (auto& s : this->toggleSets) {
00248         for (auto& t : s.toggles) {
00249             if (t.p_val) { t._defaultVal = *t.p_val; }
00250         }
00251     }
00252     this->isInitialized = true;
00253 }
00254
00255 template <size_t idx, bool invert = false>
00256 Audio updatePeakLevel(Audio val) noexcept {
00257     static_assert(idx < nMetersMax, "Meter index is out of bounds.");
00258     if constexpr (invert) {
00259         if (val < this->peakLevels[idx]) { this->peakLevels[idx] = val; }
00260     } else {
00261         if (val > this->peakLevels[idx]) { this->peakLevels[idx] = val; }
00262     }
00263     return val;
00264 }
00265
00266 Audio getAndResetPeakLevel(size_t idx) noexcept {
00267     signal_t def = signal_t(0);
00268     if (idx >= this->meters.size()) { return def; }
00269     if (this->meters[idx].invert) { def = signal_t(1); }
00270     ensureFinite(this->peakLevels[idx]);
00271     auto tmp = this->peakLevels[idx];
00272     this->peakLevels[idx] = signal_t(def);
00273     return tmp;
00274 }
00275
00276 Audio getRms(size_t idx) noexcept {
00277     if (idx >= 2) { return signal_t(0); }
00278     return this->xRms[idx].getRms();
00279 }
00280 };
00281 }

```

9.27 lib/RmsSensor.h File Reference

RMS sensor for audio. Accumulates in multiple stages, with a one millisecond accumulator followed by a user controllable accumulator. This means that the output has a real world sample rate of 1 kHz.

```

#include "lib/Audio.h"
#include "lib/Component.h"
#include <algorithm>
#include <array>

```

Classes

- struct [NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >](#)
RMS (Root Mean Square) sensor component for audio signal processing.
- struct [NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >](#)
Stereo RMS sensor component.

Namespaces

- namespace [NtFx](#)
An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Typedefs

- template<int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
using **NtFx::LongRmsSensor** = [LongRmsSensorStereo](#)<maxT_ms, maxSampleDLineLen>

9.27.1 Detailed Description

RMS sensor for audio. Accumulates in multiple stages, with a one millisecond accumulator followed by a user controllable accumulator. This means that the output has a real world sample rate of 1 kHz.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.28 RmsSensor.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00025
00026 #include "lib/Audio.h"
00027 #include "lib/Component.h"
00028 #include <algorithm>
00029 #include <array>
00030
00031 namespace NtFx {
00032
00047 template <typename signal_t,
00048     int maxT_ms = 1000,
00049     int maxSampleDLineLen = 192 * 8>
00050 struct LongRmsSensorMono : public ComponentBase<Mono<signal_t>> {
00051     std::array<signal_t, maxSampleDLineLen> sampleDLine;
00052     std::array<signal_t, maxT_ms> msDLine;
00053     signal_t sampleAccum { 0 };
00054     signal_t msAccum { 0 };
00055     int sampleIdx { 0 };
00056     int msIdx { 0 };
00057     bool resetAccums { false };
00058     int msDLineLen { maxT_ms };
00059     int sampleDLineLen { 48 };
00060
00070     virtual Mono<signal_t> process(Mono<signal_t> x) noexcept override {
00071         this->processDelayLine(x.l);
00072         return this->getRms();
00073     }
00074
00083     void processDelayLine(signal_t x) noexcept {
00084         auto x2 = x * x;
00085         if (x2 != x2) { x2 = signal_t(0.0); }
00086         this->sampleAccum += x2 - this->sampleDLine[this->sampleIdx];
00087         this->sampleDLine[this->sampleIdx] = x2;
00088         if (++this->sampleIdx < this->sampleDLineLen) { return; }
00089         this->sampleIdx = 0;
00090         this->msAccum += sampleAccum - this->msDLine[this->msIdx];
00091         this->msDLine[this->msIdx] = sampleAccum;
00092         if (++this->msIdx >= this->msDLineLen) { this->msIdx = 0; }
```

```

00093     }
00094
00101     virtual void update() noexcept override {
00102         if (this->resetAccums) {
00103             this->sampleIdx = 0;
00104             this->msIdx = 0;
00105             this->sampleAccum = 0;
00106             this->msAccum = 0;
00107             std::fill(this->sampleDLine.begin(), this->sampleDLine.end(), 0);
00108             std::fill(this->msDLine.begin(), this->msDLine.end(), 0);
00109             this->resetAccums = false;
00110         }
00111     }
00112
00121     virtual void reset(float fs) noexcept override {
00122         this->fs = fs;
00123         this->sampleDLineLen = fs / 1000;
00124         this->resetAccums = true;
00125         this->update();
00126     }
00127
00136     signal_t getRms() const noexcept {
00137         signal_t y = gcem::sqrt(signal_t(2.0) * this->msAccum
00138             / signal_t(this->sampleDLineLen * this->msDLineLen));
00139         if (y != y) { y = signal_t(0.0); }
00140         return y;
00141     }
00142
00151     void setT_ms(int t_ms) {
00152         if (t_ms == this->msDLineLen) { return; }
00153         this->msDLineLen = t_ms;
00154         this->resetAccums = true;
00155         this->update();
00156     }
00157 };
00158
00171 template <int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
00172 struct LongRmsSensorStereo
00173     : public AudioComponent<signal_t,
00174         LongRmsSensorMono<signal_t, maxT_ms, maxSampleDLineLen> {
00182     void setT_ms(int t_ms) {
00183         this->l.setT_ms(t_ms);
00184         this->r.setT_ms(t_ms);
00185     }
00194     Audio getRms() const noexcept {
00195         return { this->l.getRms(), this->r.getRms() };
00196     }
00197 };
00198 template <int maxT_ms = 1000, int maxSampleDLineLen = 192 * 8>
00199 using LongRmsSensor = LongRmsSensorStereo<maxT_ms, maxSampleDLineLen>;
00200 }

```

9.29 lib/SampleRateConverter.h File Reference

Sample rate converter using FIR filtering for oversampling.

```

#include "lib/Audio.h"
#include "lib/utils.h"
#include "lib/windowFunctions.h"
#include <algorithm>
#include <array>
#include <cstdint>

```

Classes

- struct [NtFx::Src::State](#)
State structure for sample rate converter.
- struct [NtFx::Src::Coeffs](#)
Coefficients structure for sample rate converter.
- struct [NtFx::Src::SampleRateConverter](#)
Sample rate converter class.

Namespaces

- namespace [NtFx](#)

An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Enumerations

- enum [NtFx::Src::oversamplingMode](#) : int {
 disable = 1 , **fir_2x_lq** , **fir_4x_lq** , **fir_8x_lq** ,
 fir_2x_hq , **fir_4x_hq** , **fir_8x_hq** , **end** = fir_8x_hq }

Oversampling mode enumeration.

Variables

- constexpr int **NtFx::Src::nDelayLine** = 192
Delay line size for interpolation and antialiasing.
- constexpr int **NtFx::Src::oversamplingFirMultHq** = 24
Oversampling FIR multiplier for high quality mode.
- constexpr int **NtFx::Src::oversamplingFirMultLq** = 12
Oversampling FIR multiplier for low quality mode.
- constexpr int **NtFx::Src::fc** = 22e3
Cutoff frequency for anti-aliasing filter.

9.29.1 Detailed Description

Sample rate converter using FIR filtering for oversampling.

Author

Niels Thøgersen

Copyright

Copyright (C) 2026 Niels Thøgersen, NTlyd

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.30 SampleRateConverter.h

[Go to the documentation of this file.](#)

```

00001
00020
00021 #pragma once
00022
00023 #include NTFX_PLUGIN_FILE
00024
00025 #include "lib/Audio.h"
00026 #include "lib/Utils.h"
00027 #include "lib/windowFunctions.h"
00028 #include <algorithm>
00029 #include <array>
00030 #include <cstdlib>
00031
00032 namespace NtFx {
00033 namespace Src {
00037     constexpr int nDelayLine = 192;
00041     constexpr int oversamplingFirMultHq = 24;
00045     constexpr int oversamplingFirMultLq = 12;
00049     constexpr int fc = 22e3;
00053     enum oversamplingMode : int {
00054         disable = 1,
00055         // iir_2x,
00056         // iir_4x,
00057         // iir_8x,
00058         fir_2x_lq,
00059         fir_4x_lq,
00060         fir_8x_lq,
00061         fir_2x_hq,
00062         fir_4x_hq,
00063         fir_8x_hq,
00064         end = fir_8x_hq
00065     };
00070     struct State {
00074         size_t iStoreIn { 0 };
00078         size_t iStoreOut { 0 };
00082         std::array<Audio, nDelayLine * 2> dlInterpolation;
00087         std::array<Audio, nDelayLine * 2> dlAntialiasing;
00088     };
00093     struct Coeffs {
00097         bool disable = false;
00101         size_t osFactor = 1;
00105         size_t osFirLenMult = 12;
00109         size_t n = 12;
00113         signal_t fsHi = 48000;
00117         std::array<signal_t, nDelayLine> b;
00118     };
00123     struct SampleRateConverter {
00127         NTFX_PLUGIN& plug;
00131         State state;
00135         Coeffs coeffs;
00139         oversamplingMode mode;
00143         float fs;
00144
00149         SampleRateConverter(NTFX_PLUGIN& plug) : plug(plug) { }
00150
00156         Audio process(Audio x) {
00157             ensureFinite(x);
00158             if (this->coeffs.disable) {
00159                 auto y = this->plug.process(x);
00160                 ensureFinite(y);
00161                 return y;
00162             }
00163             this->state.dlInterpolation[this->state.iStoreIn] = x;
00164             this->state.dlInterpolation[this->state.iStoreIn + nDelayLine] = x;
00165             if (++this->state.iStoreIn >= nDelayLine) { this->state.iStoreIn = 0; }
00166             auto iReadIn = this->state.iStoreIn + nDelayLine;
00167             for (size_t i = 0; i < this->coeffs.osFactor; i++) {
00168                 Audio accum;
00169                 for (size_t j = 0; j < this->coeffs.osFirLenMult; j++) {
00170                     accum += this->coeffs.b[j * this->coeffs.osFactor + i]
00171                         * this->state.dlInterpolation[iReadIn - j];
00172                 }
00173                 auto xProc = accum * this->coeffs.osFactor;
00174                 auto yProc = this->plug.process(xProc);
00175                 this->state.dlAntialiasing[this->state.iStoreOut] = yProc;
00176                 this->state.dlAntialiasing[this->state.iStoreOut + nDelayLine] = yProc;
00177                 if (++this->state.iStoreOut >= nDelayLine) {
00178                     this->state.iStoreOut = 0;
00179                 }
00180             }
00181             auto iReadOut = this->state.iStoreOut + nDelayLine
00182                 - this->coeffs.osFactor * this->coeffs.osFirLenMult;

```

```

00183     Audio accum;
00184     for (size_t i = 0; i < this->coeffs.osFactor * this->coeffs.osFirLenMult;
00185         i++) {
00186         accum += this->coeffs.b[i] * this->state.dlAntialiasing[iReadOut + i];
00187     }
00188     ensureFinite(accum);
00189     return accum;
00190 }
00191
00192 inline void update() {
00193     switch (this->mode) {
00194     // TODO: IIR oversampling
00195     // case iir_2x:
00196     //     this->coeffs.osFactor = 2;
00197     //     break;
00198     // case iir_4x:
00199     //     this->coeffs.osFactor = 4;
00200     //     break;
00201     // case iir_8x:
00202     //     this->coeffs.osFactor = 8;
00203     //     break;
00204     case fir_2x_lq:
00205         this->coeffs.osFactor = 2;
00206         this->coeffs.osFirLenMult = oversamplingFirMultLq;
00207         this->coeffs.disable = false;
00208         break;
00209     case fir_4x_lq:
00210         this->coeffs.osFactor = 4;
00211         this->coeffs.osFirLenMult = oversamplingFirMultLq;
00212         this->coeffs.disable = false;
00213         break;
00214     case fir_8x_lq:
00215         this->coeffs.osFactor = 8;
00216         this->coeffs.osFirLenMult = oversamplingFirMultLq;
00217         this->coeffs.disable = false;
00218         break;
00219     case fir_2x_hq:
00220         this->coeffs.osFactor = 2;
00221         this->coeffs.osFirLenMult = oversamplingFirMultHq;
00222         this->coeffs.disable = false;
00223         break;
00224     case fir_4x_hq:
00225         this->coeffs.osFactor = 4;
00226         this->coeffs.osFirLenMult = oversamplingFirMultHq;
00227         this->coeffs.disable = false;
00228         break;
00229     case fir_8x_hq:
00230         this->coeffs.osFactor = 8;
00231         this->coeffs.osFirLenMult = oversamplingFirMultHq;
00232         this->coeffs.disable = false;
00233         break;
00234     default:
00235     case disable:
00236         this->coeffs.osFactor = 1;
00237         this->coeffs.osFirLenMult = 1;
00238         this->coeffs.disable = true;
00239     }
00240     this->coeffs.fsHi = this->fs * this->coeffs.osFactor;
00241     this->coeffs.n = this->coeffs.osFactor * this->coeffs.osFirLenMult;
00242     if (!this->coeffs.disable) {
00243         auto b = windowMethod<signal_t>(fc, this->coeffs.n, this->coeffs.fsHi);
00244         std::fill(this->coeffs.b.begin(), this->coeffs.b.end(), 0.0);
00245         for (size_t i = 0; i < this->coeffs.n; i++) {
00246             this->coeffs.b[i] = b[i];
00247         }
00248     }
00249 }
00250
00251 inline void reset(float fs) {
00252     this->fs = fs;
00253     this->state.iStoreIn = 0;
00254     this->state.iStoreOut = 0;
00255     std::fill(this->state.dlAntialiasing.begin(),
00256              this->state.dlAntialiasing.end(),
00257              0.0);
00258     std::fill(this->state.dlInterpolation.begin(),
00259              this->state.dlInterpolation.end(),
00260              0.0);
00261     std::fill(this->coeffs.b.begin(), this->coeffs.b.end(), 0.0);
00262     this->update();
00263 }
00264 };
00265 }
00266 }

```

9.31 lib/SoftClip.h File Reference

Soft clippers for audio processing. Calculates coeffs at compile time and applies polynomials to signals.

```
#include "lib/Audio.h"
#include "lib/Component.h"
#include <array>
```

Classes

- class [NtFx::SoftClip3](#)
Third order soft clipper wrapped in a Component.
- class [NtFx::SoftClip5](#)
Fifth order soft clipper wrapped in a Component.

Namespaces

- namespace [NtFx](#)
An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Functions

- `template<typename signal_t, size_t N>`
`constexpr std::array< signal_t, N+1 > NtFx::\_calculateSoftClipCoeffs () noexcept`
Calculates coefficients for symmetrical soft clipper at compile time.

Variables

- `constexpr std::array< signal_t, 4 > NtFx::\_coeffsSeventh`
- `constexpr std::array< signal_t, 3 > NtFx::\_coeffsFifth`
- `constexpr std::array< signal_t, 2 > NtFx::\_coeffsThird`

9.31.1 Detailed Description

Soft clippers for audio processing. Calculates coeffs at compile time and applies polynomials to signals.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.32 SoftClip.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00024
00025 #include "lib/Audio.h"
00026 #include "lib/Component.h"
00027 #include <array>
00028
00029 namespace NtFx {
00030
00039 template <typename signal_t, size_t N>
00040 constexpr inline std::array<signal_t, N + 1>
00041 _calculateSoftClipCoeffs() noexcept {
00042     std::array<signal_t, N + 1> a_n;
00043     for (int n = 0; n < N + 1; n++) {
00044         a_n[n] = gcem::pow(-1, n) * gcem::tgamma((2 * N + 1) + 1)
00045             / (gcem::pow(4, N) * gcem::tgamma(N + 1) * (2 * n + 1)
00046                 * gcem::tgamma(n + 1) * gcem::tgamma(N - n + 1));
00047     }
00048     return a_n;
00049 }
00050
00051 constexpr std::array<signal_t, 4> _coeffsSeventh =
00052     _calculateSoftClipCoeffs<signal_t, 3>();
00053
00054 constexpr std::array<signal_t, 3> _coeffsFifth =
00055     _calculateSoftClipCoeffs<signal_t, 2>();
00056
00057 constexpr std::array<signal_t, 2> _coeffsThird =
00058     _calculateSoftClipCoeffs<signal_t, 1>();
00059
00067 static inline signal_t softClip5thMono(signal_t x) noexcept {
00068     signal_t x_ = x / _coeffsFifth[0];
00069     if (x_ > 1.0) { return signal_t(1.0); }
00070     if (x_ < -1.0) { return signal_t(-1.0); }
00071     auto x3 = x_ * x_ * x_;
00072     auto x5 = x3 * x_ * x_;
00073     return x + _coeffsFifth[1] * x3 + _coeffsFifth[2] * x5;
00074 }
00075
00084 static inline Audio softClip5thStereo(Audio x) noexcept {
00085     return { softClip5thMono(x.l), softClip5thMono(x.r) };
00086 }
00087
00095 static inline signal_t softClip3rdMono(signal_t x) {
00096     if (x > 1.0) { return signal_t(1.0); }
00097     if (x < -1.0) { return signal_t(-1.0); }
00098     auto x_ = x / _coeffsThird[0];
00099     auto x3 = x_ * x_ * x_;
00100     return x - _coeffsThird[1] * x3;
00101 }
00102
00111 static inline Audio softClip3rdStereo(Audio x) {
00112     return { softClip3rdMono(x.l), softClip3rdMono(x.r) };
00113 }
00114
00120 class SoftClip3 : public ComponentBase<Audio> {
00121     virtual Audio process(Audio x) noexcept override {
00122         return softClip3rdStereo(x);
00123     }
00124 };
00125
00131 class SoftClip5 : public ComponentBase<Audio> {
00132     virtual Audio process(Audio x) noexcept override {
00133         return softClip5thStereo(x);
00134     }
00135 };
00136 } // namespace NtFx

```

9.33 Transformer.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.

```

```

00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  **/
00017
00018 #pragma once
00019
00020 #include "lib/Audio.h"
00021 #include "lib/Component.h"
00022 #include "lib/FirstOrder.h"
00023 #include "lib/SoftClip.h"
00024
00025 namespace NtFx {
00026
00027 struct Transformer : public ComponentBase<Audio> {
00028     signal_t fc_hz      = 250;
00029     signal_t lfCutoff_hz = 20;
00030     signal_t gain_lin { 0 };
00031     NtFx::FirstOrder::StereoFilter<NtFx::FirstOrder::Shape::lpf> lpf;
00032     NtFx::FirstOrder::StereoFilter<NtFx::FirstOrder::Shape::hpf> hp;
00033     virtual Audio process(Audio x) noexcept override {
00034         auto yShelf = x + this->lpf.process(x) * this->gain_lin;
00035         auto yClip  = NtFx::softClip5thStereo(yShelf);
00036         auto yHp    = this->hp.process(yClip);
00037         return yHp;
00038     }
00039     virtual void update() noexcept override {
00040         if (this->fc_hz < 20) { this->fc_hz = 20; }
00041         this->gain_lin = this->fc_hz / this->lfCutoff_hz - 1;
00042         this->hp.setFc(fc_hz);
00043         this->lpf.update();
00044         this->hp.update();
00045     }
00046     virtual void reset(float fs) noexcept override {
00047         this->fs = fs;
00048         this->lpf.setFc(this->lfCutoff_hz);
00049         this->lpf.reset(fs);
00050         this->hp.reset(fs);
00051         this->update();
00052     }
00053 };
00054 }

```

9.34 lib/UiSpec.h File Reference

Specification for plugin UI.

```

#include "gcm.hpp"
#include <stdint>
#include <string>
#include <vector>

```

Classes

- struct [NtFx::KnobSpec](#)
Specification for a knob.
- struct [NtFx::ToggleSpec](#)
Specification for toggles in UI.
- struct [NtFx::ToggleSetSpec](#)
- struct [NtFx::MeterSpec](#)
Specification for meters.
- struct [NtFx::OptionsSpec](#)
Specification for drop down in UI.

- struct [NtFx::TitleBarSpec](#)
Specifies the layout of the title bar.
- struct [NtFx::UiSpec](#)
General settings for to UI.

Namespaces

- namespace [NtFx](#)
An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

Typedefs

- typedef [OptionsSpec](#) [NtFx::DropDownSpec](#)
- typedef [OptionsSpec](#) [NtFx::RadioButtonSetSpec](#)

Variables

- constexpr int [NtFx::nMetersMax](#) = 8

9.34.1 Detailed Description

Specification for plugin UI.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

Copyright (C) 2026 Niels Thøgersen, NTlyd

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.35 UiSpec.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00025
00026 #include "gcem.hpp"
00027 #include <cstdint>
00028 #include <string>
00029 #include <vector>
00030
00031 namespace NtFx {
00032 struct KnobSpec {
00033     signal_t* p_val { nullptr };
00034     // TODO: allow spaces in names and replace the other way around.
00035     std::string name { "" };
00036     std::string suffix { "" };
00037     signal_t minVal { 0.0 };
00038     signal_t maxVal { 1.0 };
00039     bool logScale { false };
00040     bool isActive { true };
00041     signal_t midPoint { 0.0 };
00042     signal_t _defaultVal;
00043     void setLogScale() {
00044         this->midPoint = gcem::sqrt(this->minVal * this->maxVal);
00045     }
00046 };
00047
00048 struct ToggleSpec {
00049     bool* p_val;
00050     std::string name;
00051     bool _defaultVal { false };
00052 };
00053
00054 static inline ToggleSpec makeTmpToggle(
00055     std::string name, std::string option, std::string prefix) {
00056     return { nullptr, prefix + ":" + name + ":" + option };
00057 }
00058
00059 struct ToggleSetSpec {
00060     std::string name;
00061     std::vector<ToggleSpec> toggles;
00062 };
00063
00064 constexpr int nMetersMax = 8;
00065
00066 struct MeterSpec {
00067     std::string name { "" };
00068     float maxVal_db { 0.0 };
00069     float minVal_db { -36.0 };
00070     bool invert { false };
00071     bool hasScale { false };
00072     float decay_s { 0.25 };
00073     float hold_s { 2 };
00074     bool addRms = false;
00075 };
00076
00077 struct OptionsSpec {
00078     int* p_val { nullptr };
00079     std::string name;
00080     std::vector<std::string> options;
00081     int _defaultVal { 0 };
00082     int _id { 0 };
00083 };
00084
00085 typedef OptionsSpec DropDownSpec;
00086 typedef OptionsSpec RadioButtonSetSpec;
00087
00088 struct TitleBarSpec {
00089     std::vector<DropDownSpec> dropdowns {
00090         {
00091             nullptr,
00092             "UI_Scale",
00093             { "50%", "75%", "100%", "125%", "150%", "175%", "200%" },
00094             2,
00095         },
00096         { nullptr, "Theme", { "Light", "Dark" }, 1 },
00097         {
00098             nullptr,
00099             "Oversampling",
00100             { "disable",
00101               // TODO: IIR oversampling
00102               // "iir_2x",
00103               // "iir_4x",
00104               // "iir_8x",

```

```

00132         "fir_2x_lq",
00133         "fir_4x_lq",
00134         "fir_8x_lq",
00135         "fir_2x_hq",
00136         "fir_4x_hq",
00137         "fir_8x_hq" },
00138     0,
00139 },
00140 };
00141 };
00142
00147 struct UiSpec {
00148     bool includeMeters { true };
00149     bool includeTitleBar { true };
00150     bool includeSecondaryKnobs { true };
00151     uint32_t foregroundColour { 0xFFFFFFFF };
00152     uint32_t backgroundColour { 0xFF000000 };
00153     int maxRows { 3 };
00154     int maxColumns { 6 };
00155     float defaultFontSize { 16 };
00156     float labelHeight { 20 };
00157     float toggleHeight { 45 };
00158     float radioButtonHeight { 25 };
00159     float radioButtonAreaWidth { 125 };
00160     float knobWidth { 150 };
00161     float knobHeight { 200 };
00162     float secondaryKnobWidth { 75 };
00163     float secondaryKnobHeight { 115 };
00164     float titleBarHeight { 22 };
00165     int meterHeight_dots { 12 };
00166     int meterWidth { 35 };
00167     float meterRefreshRate_hz { 50 };
00168 };
00169 } // namespace NtFx

```

9.36 utils.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTllyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  */
00017
00018 #pragma once
00019 #include "gcem.hpp"
00020 #include <climits>
00021 #include <cmath>
00022 #include <cstdlib>
00023 #include <limits>
00024 #include <vector>
00025
00026 namespace NtFx {
00027
00035 template <typename signal_t>
00036 static inline signal_t db(signal_t x) noexcept {
00037     return signal_t(20.0) * gcem::log10(gcem::abs(x));
00038 }
00039
00047 template <typename signal_t>
00048 static inline signal_t invDb(signal_t x) noexcept {
00049     return gcem::pow(signal_t(10.0), x * signal_t(0.05));
00050 }
00051
00052 // #define NTFX_OPTIMIZE_ENSURE_FINITE
00053
00061 template <typename signal_t>
00062 static inline void ensureFinite(
00063     signal_t& x, signal_t def = signal_t(0)) noexcept {
00064     #ifdef NTFX_OPTIMIZE_ENSURE_FINITE
00065         signal_t y = def;
00066         if (x == x) {
00067             if (gcem::abs(x) > std::numeric_limits<signal_t>::max()) { x = y; }

```

```

00068     } else {
00069         x = y;
00070     }
00071 #else
00072     if (!std::isfinite(x)) { x = def; }
00073 #endif
00074 }
00075
00076 template <typename T>
00077 static inline std::vector<T> zeros(size_t n) {
00078     return std::vector<T>(n, 0.0);
00079 }
00080
00081 template <typename T>
00082 static inline T saw(T x) {
00083     const T alpha = 2 / GCEM_PI;
00084
00085     T x_ = gcem::fmod(x, T(2.0) * GCEM_PI);
00086     x_ = (x_ < 0 ? x_ + 2 * GCEM_PI : x_);
00087     T y;
00088     if (x_ < 0.5 * GCEM_PI) {
00089         y = x_ * alpha;
00090     } else if (x_ < 1.5 * GCEM_PI) {
00091         y = -x_ * alpha + 2;
00092     } else {
00093         y = x_ * alpha - 4;
00094     }
00095     return y;
00096 }
00097
00098 static inline uint64_t _KISS() noexcept {
00099     static uint64_t x = 123456789, y = 362436000, z = 521288629, c = 7654321, t;
00100     x = 69069 * x + 12345;
00101     y ^= y << 13;
00102     y ^= y >> 17;
00103     y ^= y << 5;
00104     t = 698769069ULL * z + c;
00105     c = t >> 32;
00106     return x + y + (z = t);
00107 }
00108
00109 template <typename T>
00110 static inline T rand() noexcept {
00111     const uint64_t uintMax = std::numeric_limits<uint64_t>::max();
00112     return T(_KISS() - (uintMax >> 1)) / T(uintMax) * 2 - 1;
00113 }
00114
00115 }
00116
00117 }
00118
00119 }
00120
00121 }
00122
00123 }
00124
00125 }
00126
00127 }
00128
00129 }
00130
00131 }
00132
00133 }
00134
00135 }
00136
00137 }

```

9.37 lib/windowFunctions.h File Reference

Various usefull math functions. Mainly concerning FIR fitlers. This code is modified from <https://github.com/sidneycadot/WindowFunctions.git>.

```

#include "gcem.hpp"
#include <cassert>
#include <cmath>
#include <complex>
#include <vector>

```

Namespaces

- namespace [NtFx](#)

An exponential glider. Member value 'ui' is set from the UI side of the plugin and 'pr' is read from the processing side.

9.37.1 Detailed Description

Various usefull math functions. Mainly concerning FIR fitlers. This code is modified from <https://github.com/sidneycadot/WindowFunctions.git>.

Author

Niels Thøgersen (niels.thoegersen@gmail.com)

Copyright

Copyright (c) 2026

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.38 windowFunctions.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00024
00025 #include "gcem.hpp"
00026 #include <cassert>
00027 #include <cmath>
00028 #include <complex>
00029 #include <vector>
00030
00031 namespace NtFx {
00032
00088 template <typename T>
00089 inline static std::vector<T> cosineWindow(
00090     size_t n, const T* coeffs, size_t nCoeffs, bool sflag) {
00091     std::vector<T> w;
00092     if (n == 1) {
00093         w.push_back(1.0);
00094     } else {
00095         const size_t wlength = sflag ? (n - 1) : n;
00096         for (size_t i = 0; i < n; ++i) {
00097             T wi = 0.0;
00098             for (size_t j = 0; j < nCoeffs; ++j) {
00099                 wi += coeffs[j] * cos(i * j * 2.0 * GCEM_PI / wlength);
00100             }
00101             w.push_back(wi);
00102         }
00103     }
00104     return w;
00105 }
00106
00107 template <typename T>
00108 inline static std::vector<T> rectwin(size_t n) noexcept {
00109     std::vector<T> w;
00110     for (size_t i = 0; i < n; ++i) { w.push_back(1.0); }
00111     return w;
00112 }
00113
00114 template <typename T>
00115 inline static std::vector<T> hanning(size_t n, bool sflag = false) noexcept {
```

```

00116     const T coeff[2] = { 0.5, -0.5 };
00117     return cosineWindow<T>(n, coeff, sizeof(coeff) / sizeof(T), sflag);
00118 }
00119
00120 template <typename T>
00121 inline static std::vector<T> hamming(size_t n, bool sflag = false) noexcept {
00122     const T coeff[2] = { 0.54, -0.46 };
00123     return cosineWindow<T>(n, coeff, sizeof(coeff) / sizeof(T), sflag);
00124 }
00125
00126 template <typename T>
00127 inline static std::vector<T> blackman(size_t n, bool sflag = false) noexcept {
00128     const T coeff[3] = { 0.42, -0.5, 0.08 };
00129     return cosineWindow<T>(n, coeff, sizeof(coeff) / sizeof(T), sflag);
00130 }
00131
00132 template <typename T>
00133 inline static std::vector<T> blackmanharris(
00134     size_t n, bool sflag = false) noexcept {
00135     const T coeff[4] = { 0.35875, -0.48829, 0.14128, -0.01168 };
00136     return cosineWindow<T>(n, coeff, sizeof(coeff) / sizeof(T), sflag);
00137 }
00138
00139 template <typename T>
00140 inline static std::vector<T> nuttallwin(size_t n, bool sflag = false) noexcept {
00141     const T coeff[4] = { 0.3635819, -0.4891775, 0.1365995, -0.0106411 };
00142     return cosineWindow<T>(n, coeff, sizeof(coeff) / sizeof(T), sflag);
00143 }
00144
00145 template <typename T>
00146 inline static std::vector<T> nuttallwin_octave(
00147     size_t n, bool sflag = false) noexcept {
00148     const T coeff[4] = { 0.355768, -0.487396, 0.144232, -0.012604 };
00149     return cosineWindow<T>(n, coeff, sizeof(coeff) / sizeof(T), sflag);
00150 }
00151
00152 template <typename T>
00153 inline static std::vector<T> flattopwin(size_t n, bool sflag = false) noexcept {
00154     const T coeff[5] = {
00155         0.21557895, -0.41663158, 0.277263158, -0.083578947, 0.006947368
00156     };
00157     return cosineWindow<T>(n, coeff, sizeof(coeff) / sizeof(T), sflag);
00158 }
00159
00160 template <typename T>
00161 inline static std::vector<T> flattopwin_octave(
00162     size_t n, bool sflag = false) noexcept {
00163     const T coeff[5] = { 1.0 / 4.6402,
00164         -1.93 / 4.6402,
00165         1.29 / 4.6402,
00166         -0.388 / 4.6402,
00167         0.0322 / 4.6402 };
00170     return cosineWindow<T>(n, coeff, sizeof(coeff) / sizeof(T), sflag);
00171 }
00172
00173 template <typename T>
00174 inline static std::vector<T> triang(size_t n) noexcept {
00175     std::vector<T> w;
00176     const size_t denominator = (n % 2 != 0) ? (n + 1) : n;
00177     for (size_t i = 0; i < n; ++i) {
00178         w.push_back(1.0 - std::abs(2.0 * i - (n - 1)) / denominator);
00179     }
00180     return w;
00181 }
00182
00183 template <typename T>
00184 inline static std::vector<T> bartlett(size_t n) noexcept {
00185     std::vector<T> w;
00186     if (n == 1) {
00187         w.push_back(1.0);
00188     } else {
00189         const size_t denominator = (n - 1);
00190         for (size_t i = 0; i < n; ++i) {
00191             w.push_back(1.0 - std::abs(2.0 * i - (n - 1)) / denominator);
00192         }
00193     }
00194     return w;
00195 }
00196
00197 template <typename T>
00198 inline static std::vector<T> barthannwin(size_t n) noexcept {
00199     std::vector<T> w;
00200     if (n == 1) {
00201         w.push_back(1.0);
00202     }

```

```

00203     } else {
00204         for (size_t i = 0; i < n; ++i) {
00205             const T x = std::abs(i / (n - 1.0) - 0.5);
00206             w.push_back(0.62 - 0.48 * x + 0.38 * cos(2.0 * GCEM_PI * x));
00207         }
00208     }
00209     return w;
00210 }
00211
00212 template <typename T>
00213 inline static std::vector<T> bohmanwin(size_t n) noexcept {
00214     std::vector<T> w;
00215     if (n == 1) {
00216         w.push_back(1.0);
00217     } else {
00218         for (size_t i = 0; i < n; ++i) {
00219             const T x = std::abs(2.0 * i - (n - 1)) / (n - 1);
00220             w.push_back((1.0 - x) * cos(GCEM_PI * x) + sin(GCEM_PI * x) / GCEM_PI);
00221         }
00222     }
00223     return w;
00224 }
00225
00226 template <typename T>
00227 inline static std::vector<T> parzenwin(size_t n) noexcept {
00228     std::vector<T> w;
00229     if (n == 1) {
00230         w.push_back(1.0);
00231     } else {
00232         for (size_t i = 0; i < n; ++i) {
00233             const T x = std::abs(2.0 * i - (n - 1)) / n;
00234             const T y = 1.0 - x;
00235             w.push_back(
00236                 std::min(1.0 - 6.0 * x * x + 6.0 * x * x * x, 2.0 * y * y * y));
00237         }
00238     }
00239     return w;
00240 }
00241
00242 template <typename T>
00243 inline static std::vector<T> gausswin(size_t n, T alpha) noexcept {
00244     std::vector<T> w;
00245     if (n == 1) {
00246         w.push_back(1.0);
00247     } else {
00248         for (size_t i = 0; i < n; ++i) {
00249             const T x = std::abs(2.0 * i - (n - 1)) / (n - 1);
00250             const T ax = alpha * x;
00251             const T ax_squared = ax * ax;
00252             w.push_back(std::exp(-0.5 * ax_squared));
00253         }
00254     }
00255     return w;
00256 }
00257
00258 template <typename T>
00259 inline static std::vector<T> tukeywin(size_t n, T r) noexcept {
00260     std::vector<T> w;
00261     if (n == 1) {
00262         w.push_back(1.0);
00263     } else {
00264         r = std::max(0.0, std::min(1.0, r));
00265         for (size_t i = 0; i < n; ++i) {
00266             w.push_back(
00267                 (cos(std::max(std::abs((T)i - (n - 1) / 2.0) * (2.0 / (n - 1) / r)
00268                     - (1.0 / r - 1.0),
00269                     0.0)
00270                     * GCEM_PI)
00271                 + 1.0)
00272                 / 2.0);
00273         }
00274     }
00275     return w;
00276 }
00277
00278 template <typename T>
00279 static inline T sq(T x) noexcept {
00280     return x * x;
00281 }
00282
00283 template <typename T>
00284 inline static std::vector<T> taylorwin(size_t n, size_t nbar, T sll) noexcept {
00285     std::vector<T> w;
00286     w.reserve(n);
00287     const T amplification = pow(10.0, -sll / 20.0);
00288     const T a = acosh(amplification) / GCEM_PI;
00289     const T a2 = sq(a);

```

```

00290     const T sp2          = sq(nbar) / (a2 + sq(nbar - 0.5));
00291     for (size_t i = 0; i < n; ++i) { w.push_back(1.0); }
00292     for (size_t m = 1; m < nbar; ++m) {
00293         T numerator      = 1.0;
00294         T denominator    = 1.0;
00295         for (size_t i = 1; i < nbar; ++i) {
00296             numerator *= (1.0 - sq(m) / (sp2 * (a2 + sq(i - 0.5))));
00297             if (i != m) { denominator *= (1.0 - static_cast<T>(sq(m)) / sq(i)); }
00298         }
00299         const T Fm = -(numerator / denominator);
00300         for (size_t i = 0; i < n; ++i) {
00301             const T x = 2 * GCEM_PI * (i + 0.5) / n;
00302             w[i] += Fm * cos(m * x);
00303         }
00304     }
00305     return w;
00306 }
00307
00308 template <typename T>
00309 static T chbev1(T x, const T* coeff, size_t n) {
00310     T b0 = 0.0;
00311     T b1 = 0.0;
00312     T b2;
00313     for (size_t i = 0; i < n; ++i) {
00314         b2 = b1;
00315         b1 = b0;
00316         b0 = x * b1 - b2 + coeff[i];
00317     }
00318     return 0.5 * (b0 - b2);
00319 }
00320
00321 template <typename T>
00322 static T besseli0(T x) {
00323     const T A[30] = { -4.41534164647933937950e-18,
00324         3.33079451882223809783e-17,
00325         -2.43127984654795469359e-16,
00326         1.71539128555513303061e-15,
00327         -1.16853328779934516808e-14,
00328         7.67618549860493561688e-14,
00329         -4.85644678311192946090e-13,
00330         2.95505266312963983461e-12,
00331         -1.72682629144155570723e-11,
00332         9.67580903537323691224e-11,
00333         -5.18979560163526290666e-10,
00334         2.65982372468238665035e-9,
00335         -1.30002500998624804212e-8,
00336         6.04699502254191894932e-8,
00337         -2.67079385394061173391e-7,
00338         1.11738753912010371815e-6,
00339         -4.41673835845875056359e-6,
00340         1.64484480707288970893e-5,
00341         -5.75419501008210370398e-5,
00342         1.88502885095841655729e-4,
00343         -5.76375574538582365885e-4,
00344         1.63947561694133579842e-3,
00345         -4.32430999505057594430e-3,
00346         1.05464603945949983183e-2,
00347         -2.37374148058994688156e-2,
00348         4.93052842396707084878e-2,
00349         -9.49010970480476444210e-2,
00350         1.71620901522208775349e-1,
00351         -3.04682672343198398683e-1,
00352         6.76795274409476084995e-1 };
00353
00354     const T B[30] = { -7.23318048787475395456e-18,
00355         -4.83050448594418207126e-18,
00356         4.46562142029675999901e-17,
00357         3.46122286769746109310e-17,
00358         -2.82762398051658348494e-16,
00359         -3.42548561967721913462e-16,
00360         1.77256013305652638360e-15,
00361         3.81168066935262242075e-15,
00362         -9.55484669882830764870e-15,
00363         -4.15056934728722208663e-14,
00364         1.54008621752140982691e-14,
00365         3.85277838274214270114e-13,
00366         7.18012445138366623367e-13,
00367         -1.79417853150680611778e-12,
00368         -1.32158118404477131188e-11,
00369         -3.14991652796324136454e-11,
00370         1.18891471078464383424e-11,
00371         4.94060238822496958910e-10,
00372         3.39623202570838634515e-9,
00373         2.26666899049817806459e-8,
00374         2.04891858946906374183e-7,
00375         2.89137052083475648297e-6,
00376         6.88975834691682398426e-5,

```

```

00377     3.36911647825569408990e-3,
00378     8.04490411014108831608e-1 };
00379
00380 x = std::abs(x);
00381 if (x <= 8.0) {
00382     return std::exp(x) * chbev1<T>(x / 2.0 - 2.0, A, 30);
00383 } else {
00384     return std::exp(x) * chbev1<T>(32.0 / x - 2.0, B, 25) / sqrt(x);
00385 }
00386 }
00387
00388 template <typename T>
00389 inline static std::vector<T> kaiser(size_t n, T beta) noexcept {
00390     std::vector<T> w;
00391     if (n == 1) {
00392         w.push_back(1.0);
00393     } else {
00394         for (size_t i = 0; i < n; ++i) {
00395             const T x = (2.0 * i - (n - 1)) / (n - 1);
00396             w.push_back(bessel_i0(beta * sqrt(1.0 - x * x)) / bessel_i0(beta));
00397         }
00398     }
00399     return w;
00400 }
00401
00402 inline static size_t bitreverse(size_t n, size_t size) noexcept {
00403     size_t ri = 0;
00404     while (size != 1) {
00405         ri *= 2;
00406         ri |= (n & 1);
00407         n >>= 1;
00408         size /= 2;
00409     }
00410     return ri;
00411 }
00412
00413 template <typename T>
00414 inline static void fft_radix2(std::complex<T>* z, size_t size) noexcept {
00415     size_t num_subffts = size / 2;
00416     size_t size_subfft = 2;
00417     auto ww = new std::complex<T>[size / 2];
00418
00419     for (size_t i = 0; i < size / 2; ++i) {
00420         ww[i] = std::exp(static_cast<T>(-2.0) * static_cast<T>(GCEM_PI)
00421             * std::complex<T>(0.0, 1.0) * static_cast<T>(i) / static_cast<T>(size));
00422     }
00423
00424     for (size_t i = 0; i < size; ++i) {
00425         const size_t ri = bitreverse(i, size);
00426         if (i < ri) {
00427             const std::complex<T> temp = z[i];
00428             z[i] = z[ri];
00429             z[ri] = temp;
00430         }
00431     }
00432
00433     while (num_subffts != 0) {
00434         for (size_t i = 0; i < num_subffts; ++i) {
00435             size_t subfft_offset = size_subfft * i;
00436             for (size_t j = 0; j < size_subfft / 2; ++j) {
00437                 size_t target1 = subfft_offset + j;
00438                 size_t target2 = subfft_offset + j + size_subfft / 2;
00439                 size_t left = target1;
00440                 size_t right = target2;
00441                 const size_t ww_index = (j * num_subffts);
00442                 const std::complex<T> w = ww[ww_index];
00443                 const std::complex<T> zleft = z[left];
00444                 const std::complex<T> wzright = w * z[right];
00445                 z[target1] = zleft + wzright;
00446                 z[target2] = zleft - wzright;
00447             }
00448         }
00449         num_subffts /= 2;
00450         size_subfft *= 2;
00451     }
00452     delete[] ww;
00453 }
00454
00455 template <typename T>
00456 inline static void czt(std::complex<T>* z,
00457     size_t n,
00458     std::complex<T>* ztrans,
00459     size_t m,
00460     std::complex<T> w,
00461     std::complex<T> a) noexcept {
00462     size_t fft_size = 1;
00463     while (fft_size < n + m - 1) { fft_size *= 2; }

```

```

00464     auto zz = new std::complex<T>[fft_size];
00465     for (size_t k = 0; k < fft_size; ++k) {
00466         if (k < n) {
00467             auto w1 =
00468                 std::pow(w, static_cast<std::complex<T>>(0.5) * static_cast<T>(k * k))
00469                 / std::pow(a, static_cast<T>(k));
00470             zz[k] = w1 * z[k];
00471         } else {
00472             zz[k] = 0;
00473         }
00474     }
00475     fft_radix2(zz, fft_size);
00476     auto w2 = new std::complex<T>[fft_size];
00477     for (size_t k = 0; k < fft_size; ++k) {
00478         if (k < n + m - 1) {
00479             const int kshift = k - (n - 1);
00480             w2[k] = std::pow(w,
00481                 static_cast<std::complex<T>>(-0.5) * static_cast<T>(kshift * kshift));
00482         } else {
00483             w2[k] = 0;
00484         }
00485     }
00486     fft_radix2(w2, fft_size);
00487     for (size_t k = 0; k < fft_size; ++k) { zz[k] *= w2[k]; }
00488     fft_radix2(zz, fft_size);
00489     for (size_t k = 0; k < fft_size; ++k) { zz[k] /= fft_size; }
00490     for (size_t k = 1; k < fft_size - k; ++k) {
00491         const size_t kswap = fft_size - k;
00492         const std::complex<T> temp = zz[k];
00493         zz[k] = zz[kswap];
00494         zz[kswap] = temp;
00495     }
00496 }
00497
00498     for (size_t k = 0; k < m; ++k) {
00499         const std::complex<T> w3 =
00500             std::pow(w, static_cast<std::complex<T>>(0.5) * static_cast<T>(k * k));
00501         ztrans[k] = w3 * zz[n - 1 + k];
00502     }
00503     delete[] zz;
00504     delete[] w2;
00505 }
00506
00507 template <typename T>
00508 inline static void czt_fft(std::complex<T>* z, size_t size) noexcept {
00509     if (size == 0) { return; }
00510     size_t sz = size;
00511     while (sz % 2 == 0) { sz /= 2; }
00512     if (sz == 1) {
00513         fft_radix2(z, size);
00514     } else {
00515         const std::complex<T> w =
00516             std::exp(static_cast<T>(-2.0) * static_cast<T>(GCEM_PI)
00517                 * std::complex<T>(0.0, 1.0) / static_cast<std::complex<T>>(size));
00518         const std::complex<T> a = 1;
00519         czt(z, size, z, size, w, a);
00520     }
00521 }
00522
00523 template <typename T>
00524 inline static void fft(
00525     std::complex<T>* z, size_t size, bool inv = false) noexcept {
00526     czt_fft(z, size);
00527     if (inv) {
00528         for (size_t k = 0; k < size; ++k) { z[k] /= size; }
00529         for (size_t k = 1; k < size - k; ++k) {
00530             const size_t kswap = size - k;
00531             const std::complex<T> temp = z[k];
00532             z[k] = z[kswap];
00533             z[kswap] = temp;
00534         }
00535     }
00536 }
00537
00538 template <typename T>
00539 inline static std::vector<std::complex<T>> fft(
00540     const std::vector<T> x, bool invert = false) {
00541     auto n = x.size();
00542     auto z = new std::complex<T>[n];
00543     for (size_t i = 0; i < n; i++) { z[i] = x[i]; }
00544     fft(z, n, invert);
00545     std::vector<std::complex<T>> y;
00546     for (size_t i = 0; i < n; i++) { y.push_back(z[i]); }
00547     delete[] z;
00548     return y;
00549 }
00550

```

```

00551 template <typename T>
00552 inline static std::vector<T> chebwin(size_t n, T r) noexcept {
00553     std::vector<T> w;
00554     w.reserve(n);
00555     if (n == 1) {
00556         w[0] = 1.0;
00557     } else {
00558         const size_t order = n - 1;
00559         const T amplification = pow(10.0, std::abs(r) / 20.0);
00560         const T beta = cosh(acosh(amplification) / order);
00561         std::complex<T> p[n];
00562         if (n % 2 != 0) {
00563             for (size_t i = 0; i < n; ++i) {
00564                 const T x = beta * cos(GCEM_PI * i / n);
00565                 if (x > 1.0) {
00566                     p[i] = cosh(order * acosh(x));
00567                 } else if (x < -1.0) {
00568                     p[i] = cosh(order * acosh(-x));
00569                 } else {
00570                     p[i] = cos(order * acos(x));
00571                 }
00572             }
00573             czt_fft(p, n);
00574             const size_t h = (n - 1) / 2;
00575             for (size_t i = 0; i < n; ++i) {
00576                 const size_t j = (i <= h) ? (h - i) : (i - h);
00577                 w[i] = std::real(p[j]);
00578             }
00579         } else {
00580             for (size_t i = 0; i < n; ++i) {
00581                 const T x = beta * cos(GCEM_PI * i / n);
00582                 const std::complex<T> z = std::exp(GCEM_PI * std::complex<T>(0.0, 1.0)
00583                     * static_cast<T>(i) / static_cast<T>(n));
00584                 if (x > 1) {
00585                     p[i] = z * cosh(order * acosh(x));
00586                 } else if (x < -1) {
00587                     p[i] = -z * cosh(order * acosh(-x));
00588                 } else {
00589                     p[i] = z * cos(order * acos(x));
00590                 }
00591             }
00592             czt_fft(p, n);
00593             const size_t h = n / 2;
00594             for (size_t i = 0; i < n; ++i) {
00595                 const size_t j = (i < h) ? (h - i) : (i - h + 1);
00596                 w[i] = std::real(p[j]);
00597             }
00598         }
00599         T maxw = w[0];
00600         for (size_t i = 1; i < n; ++i) { maxw = std::max(maxw, w[i]); }
00601         for (size_t i = 0; i < n; ++i) { w[i] /= maxw; }
00602     }
00603     return w;
00604 }
00605
00606 template <typename T>
00607 static inline std::vector<T> generateFreqRespFft(T fc, size_t n, T fs) {
00608     std::vector<T> ft(n, 0.0);
00609     auto hzPrBin = fs / static_cast<T>(n);
00610     std::fill(ft.begin(), ft.begin() + std::ceil(fc / hzPrBin), 1);
00611     std::fill(ft.begin() + std::ceil((fs - fc) / hzPrBin), ft.end(), 1);
00612     return ft;
00613 }
00614
00615 template <typename T>
00616 inline static std::vector<T> windowMethod(T fc, size_t n, T fs) {
00617     auto fResp = generateFreqRespFft(fc, n, fs);
00618     auto tmp = fft<T>(fResp);
00619     std::vector<T> tResp;
00620     for (auto v : tmp) { tResp.push_back(std::real(v) / static_cast<T>(n)); }
00621     std::vector<T> firstHalf(tResp.begin(), tResp.begin() + tResp.size() / 2);
00622     std::vector<T> secondHalf(tResp.begin() + tResp.size() / 2, tResp.end());
00623     secondHalf.insert(secondHalf.end(), firstHalf.begin(), firstHalf.end());
00624     auto bFull = secondHalf;
00625     assert(bFull.size() == n);
00626     auto hanningWindow = hanning<T>(n);
00627     for (size_t i = 0; i < n; ++i) { bFull[i] *= hanningWindow[i]; }
00628     return bFull;
00629 }
00630 } // namespace NtFx

```

9.39 gainExample.h

```
00001 /*
```

```

00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  *
00017  * You are free to download, build and use this code for commercial
00018  * purposes. Just don't resell it or a build of it, modified or otherwise.
00019  **/
00020
00021 #pragma once
00022
00023 #include "lib/Audio.h"
00024 #include "lib/Plugin.h"
00025
00026 // Make a class the inherits from Plugin. This must be a template so we can keep
00027 // the whole thing in headers and swap the signal data type. I prefer using
00028 // structs so I don't need to consider private/public.
00029
00030 struct gainExample : public NtFx::NtPlugin {
00031     // Make some variables.
00032     signal_t gain_db { 0 };
00033     signal_t gain_lin { 1 };
00034
00035     // Write the constructor.
00036     gainExample() {
00037
00038         // Add a knob.
00039         this->primaryKnobs.push_back({
00040             .p_val = &this->gain_db,
00041             .name = "Gain",
00042             .suffix = " dB",
00043             .minVal = -24,
00044             .maxVal = 24,
00045         });
00046
00047         // Change the default background colours.
00048         this->uiSpec.backgroundColour = 0xFFFFFFFF;
00049         this->uiSpec.foregroundColour = 0xFF000000;
00050
00051         // We don't need that big a window for just one knob.
00052
00053         // Let's make the meter smaller.
00054         this->uiSpec.meterHeight_dots = 8;
00055
00056         // Add two meters.
00057         this->meters.push_back({ .name = "IN" });
00058         this->meters.push_back({ .name = "OUT", .hasScale = true });
00059
00060         // Always remember to update defaults. This will load the member values into
00061         // the UI and session storage.
00062         this->updateDefaults();
00063     }
00064
00065     // Override the process method.
00066     Audio process(Audio x) noexcept override {
00067
00068         // Update the input meter.
00069         this->template updatePeakLevel<0>(x);
00070
00071         // Calculate gain
00072         Audio y = x * this->gain_lin;
00073
00074         // Update output meter.
00075         this->template updatePeakLevel<1>(y);
00076         return y;
00077     }
00078
00079     // Override the process method.
00080     void update() noexcept override {
00081
00082         // Calculate the gain in the linear domain.
00083         this->gain_lin = NtFx::invDb(this->gain_db);
00084     }
00085
00086     // Override the reset method.
00087     void reset(float fs) noexcept override {
00088         this->fs = fs;

```

```

00089     this->update();
00090 }
00091 };

```

9.40 ntCompressor.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  *
00017  * You are free to download, build and use this code for commercial
00018  * purposes. Just don't resell it or a build of it, modified or otherwise.
00019  **/
00020
00021 #pragma once
00022
00023 #include "lib/Audio.h"
00024 #include "lib/Biquad.h"
00025 #include "lib/Comp.h"
00026 #include "lib/Plugin.h"
00027 #include "lib/SoftClip.h"
00028 #include "lib/Utils.h"
00029
00030 enum scMode { feedForward = 0, feedback, external };
00031
00032 struct ntCompressor : public NtFx::NtPlugin {
00033     NtFx::Comp::ScSettings scSettings;
00034     NtFx::Comp::PeakSideChainDb peakScDb;
00035     NtFx::Comp::PeakSideChainLinear peakScLin;
00036     NtFx::Comp::RmsSideChainDb rmsScDb;
00037     NtFx::Comp::RmsSideChainLinear rmsScLin;
00038     NtFx::Biquad::EqBand hpf;
00039     NtFx::Biquad::EqBand boost;
00040
00041     signal_t makeup_db { signal_t(0.0) };
00042     signal_t mix_percent { signal_t(100.0) };
00043     int linEnable { 0 };
00044     int rmsEnable { 0 };
00045     int scMode { scMode::feedForward };
00046     bool scListenEnable { false };
00047     bool clip { true };
00048     bool bypassEnable { false };
00049     signal_t mix_lin { signal_t(1.0) };
00050     signal_t makeup_lin { signal_t(1.0) };
00051     Audio fbState { signal_t(0.0) };
00052
00053     bool dummy1;
00054     bool dummy2;
00055     bool dummy3;
00056
00057     ntCompressor()
00058         : peakScDb(scSettings), peakScLin(scSettings), rmsScDb(scSettings),
00059           rmsScLin(scSettings) {
00060         this->primaryKnobs = {
00061             {
00062                 .p_val = &this->scSettings.thresh_db,
00063                 .name = "Threshold",
00064                 .suffix = " dB",
00065                 .minVal = -60.0,
00066                 .maxVal = 0.0,
00067             },
00068             {
00069                 .p_val = &this->scSettings.ratio_db,
00070                 .name = "Ratio",
00071                 .suffix = "",
00072                 .minVal = 1.0,
00073                 .maxVal = 20.0,
00074                 .midPoint = 2.0,
00075             },
00076             {
00077                 .p_val = &this->scSettings.tAtt_ms,

```

```

00078         .name   = "Attack",
00079         .suffix  = " ms",
00080         .minVal  = 0.01,
00081         .maxVal  = 50.0,
00082     },
00083     {
00084         .p_val   = &this->scSettings.tRel_ms,
00085         .name    = "Release",
00086         .suffix  = " ms",
00087         .minVal  = 10.0,
00088         .maxVal  = 1000.0,
00089     },
00090     {
00091         .p_val   = &this->makeup_db,
00092         .name    = "Makeup",
00093         .suffix  = " dB",
00094         .minVal  = 0.0,
00095         .maxVal  = 24.0,
00096     },
00097 };
00098
00099 this->secondaryKnobs = {
00100     {
00101         .p_val   = &this->scSettings.knee_db,
00102         .name    = "Knee",
00103         .suffix  = " dB",
00104         .minVal  = 0.0,
00105         .maxVal  = 24.0,
00106     },
00107     {
00108         .p_val   = &this->scSettings.tRms_ms,
00109         .name    = "RMS_time",
00110         .suffix  = " ms",
00111         .minVal  = 1.0,
00112         .maxVal  = 250.0,
00113     },
00114     {
00115         .p_val   = &this->hpf.settings.fc_hz,
00116         .name    = "SC_HPF",
00117         .suffix  = " Hz",
00118         .minVal  = 20.0,
00119         .maxVal  = 2000.0,
00120         .midPoint = 200.0,
00121     },
00122     {
00123         .p_val   = &this->boost.settings.gain_db,
00124         .name    = "SC_Boost",
00125         .suffix  = " dB",
00126         .minVal  = 0.0,
00127         .maxVal  = 24.0,
00128     },
00129     {
00130         .p_val   = &this->mix_percent,
00131         .name    = "Mix",
00132         .suffix  = " %",
00133         .minVal  = 0.0,
00134         .maxVal  = 100.0,
00135     },
00136 };
00137
00138 this->toggles = {
00139     { .p_val = &this->scSettings.linkEnable, .name = "Link" },
00140     { .p_val = &this->scListenEnable, .name = "SC_Listen" },
00141     { .p_val = &this->clip, .name = "Softclip" },
00142     { .p_val = &this->bypassEnable, .name = "Bypass" },
00143 };
00144
00145 this->radioButtons.push_back({
00146     .p_val   = (int*)&this->linEnable,
00147     .name    = "Domain",
00148     .options = { "dB", "Linear" },
00149 });
00150 this->radioButtons.push_back({
00151     .p_val   = (int*)&this->rmsEnable,
00152     .name    = "Sensor",
00153     .options = { "Peak", "RMS" },
00154 });
00155 this->radioButtons.push_back({
00156     .p_val   = (int*)&this->scMode,
00157     .name    = "Side_Chain",
00158     .options = { "Normal", "Feedback", "External" },
00159 });
00160
00161 this->meters = {
00162     { .name = "IN", .addRms = true },
00163     { .name = "OUT", .hasScale = true, .addRms = true },
00164     { .name = "GR", .invert = true, .hasScale = true },

```

```

00165     };
00166     this->hpf.settings.fc_hz = 20;
00167     this->boost.settings.fc_hz = 3000.0;
00168     this->hpf.settings.shape = NtFx::Biquad::Shape::hpf;
00169     this->boost.settings.shape = NtFx::Biquad::Shape::bell;
00170     this->updateDefaults();
00171 }
00172
00173 Audio process(Audio x) noexcept override {
00174     this->template updatePeakLevel<0>(x);
00175     if (this->bypassEnable) {
00176         this->template updatePeakLevel<1>(x);
00177         return x;
00178     }
00179     NtFx::ensureFinite(x);
00180     NtFx::ensureFinite(this->fbState);
00181     Audio xHpf = x;
00182     if (this->scMode == scMode::feedback) {
00183         xHpf = this->fbState;
00184     } else if (this->scMode == scMode::external) {
00185         xHpf = this->xSc;
00186     }
00187
00188     Audio xBoost = hpf.process(xHpf);
00189     Audio xSc = boost.process(xBoost);
00190
00191     Audio gr;
00192     if (this->linEnable) {
00193         if (this->rmsEnable) {
00194             gr = rmsScLin.process(xSc);
00195         } else {
00196             gr = peakScLin.process(xSc);
00197         }
00198     } else {
00199         if (this->rmsEnable) {
00200             gr = rmsScDb.process(xSc);
00201         } else {
00202             gr = peakScDb.process(xSc);
00203         }
00204     }
00205     this->template updatePeakLevel<2, true>(gr);
00206     NtFx::ensureFinite(gr, signal_t(1.0));
00207     Audio yComp = x * gr;
00208     this->fbState = yComp;
00209     auto xClip = yComp * this->makeup_lin;
00210     auto xMix = xClip;
00211     if (this->clip) { xMix = NtFx::softClip5thStereo(xClip); }
00212     auto y = this->mix_lin * xMix + (1 - this->mix_lin) * x;
00213     this->template updatePeakLevel<1>(y);
00214     if (this->scListenEnable) { return xSc; }
00215     return y;
00216 }
00217
00218 void update() noexcept override {
00219     if (this->rmsEnable) {
00220         this->activateParameter("RMS");
00221     } else {
00222         this->deactivateParameter("RMS");
00223     }
00224     this->hpf.update();
00225     this->boost.update();
00226     this->peakScDb.update();
00227     this->peakScLin.update();
00228     this->rmsScDb.update();
00229     this->rmsScLin.update();
00230     this->makeup_lin = NtFx::invDb(this->makeup_db);
00231     this->mix_lin = this->mix_percent / 100.0;
00232 }
00233
00234 void reset(float fs) noexcept override {
00235     this->fs = fs;
00236     this->fbState = signal_t(0);
00237     this->peakScDb.reset(this->fs);
00238     this->peakScLin.reset(this->fs);
00239     this->rmsScDb.reset(this->fs);
00240     this->rmsScLin.reset(this->fs);
00241     this->hpf.reset(this->fs);
00242     this->boost.reset(this->fs);
00243     this->update();
00244 }
00245 };

```

9.41 ntDeEsser.h

```

00001 #pragma once
00002
00003 #include "lib/Audio.h"
00004 #include "lib/Biquad.h"
00005 #include "lib/Comp.h"
00006 #include "lib/DynamicFilter.h"
00007 #include "lib/Plugin.h"
00008
00009 struct ntDeEsser : public NtFx::NtPlugin {
00010     NtFx::Comp::PeakSideChainLinear sc;
00011     NtFx::Biquad::EqBand scFlt;
00012     NtFx::DynamicFilter::Shelf flt;
00013     NtFx::Comp::ScSettings scSettings;
00014     signal_t q { 0.6 };
00015     signal_t fc_hz { 4e3 };
00016     bool bypassEnable { false };
00017     bool scListenEnable { false };
00018     ntDeEsser() : sc(scSettings) {
00019         this->primaryKnobs = {
00020             {
00021                 .p_val = &this->fc_hz,
00022                 .name = "Frequency",
00023                 .suffix = " Hz",
00024                 .minVal = 20,
00025                 .maxVal = 20e3,
00026                 .logScale = true,
00027             },
00028             {
00029                 .p_val = &this->scSettings.thresh_db,
00030                 .name = "Threshold",
00031                 .suffix = " dB",
00032                 .minVal = -60,
00033                 .maxVal = 0,
00034             },
00035         };
00036         this->secondaryKnobs = {
00037             {
00038                 .p_val = &this->q,
00039                 .name = "Q",
00040                 .minVal = 0.5,
00041                 .maxVal = 1,
00042             },
00043             // {
00044             //     .p_val = &this->scSettings.ratio_db,
00045             //     .name = "Ratio",
00046             //     .suffix = "",
00047             //     .minVal = 1.0,
00048             //     .maxVal = 20.0,
00049             //     .midPoint = 2.0,
00050             // },
00051             // {
00052             //     .p_val = &this->scSettings.tAtt_ms,
00053             //     .name = "Attack",
00054             //     .suffix = " ms",
00055             //     .minVal = 0.01,
00056             //     .maxVal = 50.0,
00057             // },
00058             {
00059                 .p_val = &this->scSettings.tRel_ms,
00060                 .name = "Release",
00061                 .suffix = " ms",
00062                 .minVal = 1.0,
00063                 .maxVal = 100.0,
00064             },
00065         };
00066         this->toggles = {
00067             { .p_val = &this->bypassEnable, .name = "Bypass" },
00068             { .p_val = &this->scListenEnable, .name = "SC_Listen" },
00069         };
00070         this->meters.push_back({ .name = "GR", .invert = true });
00071         this->scSettings.ratio_db = 10;
00072         this->scSettings.knee_db = 3;
00073         this->scSettings.tAtt_ms = 0.01;
00074         this->updateDefaults();
00075     }
00076
00077     Audio process(Audio x) noexcept override {
00078         this->template updatePeakLevel<0>(x);
00079         if (this->bypassEnable) {
00080             this->template updatePeakLevel<1>(x);
00081             return x;
00082         }
00083         auto yScFlt = scFlt.process(x);
00084         auto ySc = sc.process(yScFlt);

```

```

00085     this->flt.gain_lin = ySc.absMin();
00086     auto y             = flt.process(x);
00087     this->template updatePeakLevel<1>(y);
00088     this->template updatePeakLevel<2, true>(ySc);
00089     if (this->scListenEnable) { return yScFlt; }
00090     return y;
00091 }
00092
00093 void update() noexcept override {
00094     this->scFlt.settings.shape = NtFx::Biquad::Shape::hpf;
00095     this->scFlt.settings.q     = this->q;
00096     this->flt.q1               = this->q;
00097     this->flt.q2               = this->q;
00098     this->scFlt.settings.fc_hz = this->fc_hz;
00099     this->flt.fc_hz           = this->fc_hz;
00100     // this->scSettings.tAtt_ms = signal_t(1e3) / this->fc_hz;
00101     // this->scSettings.tRel_ms = this->scSettings.tAtt_ms * 10;
00102     this->sc.update();
00103     this->scFlt.update();
00104     // Don't update flt. It's done in process.
00105 }
00106
00107 void reset(float fs) noexcept override {
00108     this->sc.reset(fs);
00109     this->scFlt.reset(fs);
00110     this->flt.reset(fs);
00111     this->fs = fs;
00112     this->update();
00113 }
00114 };

```

9.42 ntFilters.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  *
00017  * You are free to download, build and use this code for commercial
00018  * purposes. Just don't resell it or a build of it, modified or otherwise.
00019  *
00020  */
00021
00022 #pragma once
00023
00024 #include "lib/Audio.h"
00025 #include "lib/Biquad.h"
00026 #include "lib/FirstOrder.h"
00027 #include "lib/Plugin.h"
00028
00029 enum Order : int {
00030     first,
00031     second,
00032     third,
00033     fourth,
00034 };
00035
00036 struct ntFilters : public NtFx::NtPlugin {
00037     signal_t fHpf = 20;
00038     signal_t fLpf = 20000;
00039     signal_t qHpf = 0.707;
00040     signal_t qLpf = 0.707;
00041     Order orderHpf;
00042     Order orderLpf;
00043     bool enableHpf = true;
00044     bool enableLpf = true;
00045     NtFx::FirstOrder::StereoFilter<NtFx::FirstOrder::Shape::hpf> firstOrderHpf;
00046     NtFx::FirstOrder::StereoFilter<NtFx::FirstOrder::Shape::lpfZero>
00047         firstOrderLpf;
00048     NtFx::Biquad::EqBand bqLpf0;
00049     NtFx::Biquad::EqBand bqLpf1;
00050     NtFx::Biquad::EqBand bqHpf0;

```

```

00051 NtFx::Biquad::EqBand bqHpfl;
00052 ntFilters() {
00053
00054     this->primaryKnobs = {
00055         {
00056             .p_val    = &this->fHpfl,
00057             .name     = "HPF",
00058             .suffix   = " Hz",
00059             .minVal   = 20,
00060             .maxVal   = 20e3,
00061             .midPoint = 2e3,
00062         },
00063         {
00064             .p_val    = &this->qHpfl,
00065             .name     = "Q_HP",
00066             .suffix   = "",
00067             .minVal   = 0.1,
00068             .maxVal   = 2,
00069         },
00070         {
00071             .p_val    = &this->fLpf,
00072             .name     = "LPF",
00073             .suffix   = " Hz",
00074             .minVal   = 20,
00075             .maxVal   = 20e3,
00076             .midPoint = 2e3,
00077         },
00078         {
00079             .p_val    = &this->qLpf,
00080             .name     = "Q_LPF",
00081             .suffix   = "",
00082             .minVal   = 0.1,
00083             .maxVal   = 2,
00084         },
00085     };
00086     this->dropdowns = {
00087         {
00088             (int*)&this->orderHpfl,
00089             "HPF_Order",
00090             {
00091                 "first",
00092                 "second",
00093                 "third",
00094                 "fourth",
00095             },
00096             0,
00097         },
00098         {
00099             (int*)&this->orderLpf,
00100             "LPF_Order",
00101             {
00102                 "first",
00103                 "second",
00104                 "third",
00105                 "fourth",
00106             },
00107             0,
00108         },
00109     };
00110     this->toggles = {
00111         { &this->enableHpfl, "HPF_on" },
00112         { &this->enableLpf, "LPF_on" },
00113     };
00114     this->meters = {
00115         { .name = "IN", .addRms = true },
00116         { .name = "OUT", .hasScale = true, .addRms = true },
00117     };
00118     this->updateDefaults();
00119 }
00120
00121 Audio process(Audio x) noexcept override {
00122     auto xBqHpfl0 = x;
00123     if ((this->orderHpfl + 1) % 2) { xBqHpfl0 = this->firstOrderHpfl.process(x); }
00124     auto yBqHpfl0 = this->bqHpfl0.process(xBqHpfl0);
00125     auto yBqHpfl1 = this->bqHpfl1.process(yBqHpfl0);
00126
00127     auto xLpf = yBqHpfl1;
00128     if (!this->enableLpf) { xLpf = x; }
00129     auto xBqLpf0 = xLpf;
00130     if ((this->orderLpf + 1) % 2) {
00131         xBqLpf0 = this->firstOrderLpf.process(xLpf);
00132     }
00133     auto yBqLpf0 = this->bqLpf0.process(xBqLpf0);
00134     auto yBqLpf1 = this->bqLpf1.process(yBqLpf0);
00135
00136     auto y = yBqLpf1;
00137     if (!this->enableLpf) { y = xLpf; }

```

```

00138     this->template updatePeakLevel<0>(x);
00139     this->template updatePeakLevel<1>(y);
00140     return y;
00141 }
00142
00143 void update() noexcept override {
00144     this->primaryKnobs[1].isActive = true;
00145     this->primaryKnobs[3].isActive = true;
00146     this->firstOrderHpF.setFc(fHpF);
00147     this->firstOrderHpF.update();
00148     if (this->orderHpF == Order::fourth) {
00149         this->bqHpF0.settings.q = gcem::sqrt(this->qHpF);
00150         this->bqHpF1.settings.q = gcem::sqrt(this->qHpF);
00151         this->bqHpF0.settings.shape = NtFx::Biquad::Shape::hpF;
00152         this->bqHpF1.settings.shape = NtFx::Biquad::Shape::hpF;
00153     } else if (this->orderHpF == Order::first) {
00154         this->bqHpF0.settings.shape = NtFx::Biquad::Shape::none;
00155         this->bqHpF1.settings.shape = NtFx::Biquad::Shape::none;
00156         this->primaryKnobs[1].isActive = false;
00157     } else {
00158         this->bqHpF0.settings.q = this->qHpF;
00159         this->bqHpF0.settings.shape = NtFx::Biquad::Shape::hpF;
00160         this->bqHpF1.settings.shape = NtFx::Biquad::Shape::none;
00161     }
00162     this->bqHpF0.settings.fc_hz = fHpF;
00163     this->bqHpF1.settings.fc_hz = fHpF;
00164     this->bqHpF0.update();
00165     this->bqHpF1.update();
00166
00167     this->firstOrderLpF.setFc(fLpF);
00168     this->firstOrderLpF.update();
00169     if (this->orderLpF == Order::fourth) {
00170         this->bqLpF0.settings.q = gcem::sqrt(this->qLpF);
00171         this->bqLpF1.settings.q = gcem::sqrt(this->qLpF);
00172         this->bqLpF0.settings.shape = NtFx::Biquad::Shape::lpF;
00173         this->bqLpF1.settings.shape = NtFx::Biquad::Shape::lpF;
00174     } else if (this->orderLpF == Order::first) {
00175         this->bqLpF0.settings.shape = NtFx::Biquad::Shape::none;
00176         this->bqLpF1.settings.shape = NtFx::Biquad::Shape::none;
00177         this->primaryKnobs[3].isActive = false;
00178     } else {
00179         this->bqLpF0.settings.q = this->qLpF;
00180         this->bqLpF0.settings.shape = NtFx::Biquad::Shape::lpF;
00181         this->bqLpF1.settings.shape = NtFx::Biquad::Shape::none;
00182     }
00183     this->bqLpF0.settings.fc_hz = fLpF;
00184     this->bqLpF1.settings.fc_hz = fLpF;
00185     this->bqLpF0.update();
00186     this->bqLpF1.update();
00187     this->uiNeedsUpdate = true;
00188 }
00189
00190 void reset(float fs) noexcept override {
00191     this->fs = fs;
00192     this->firstOrderHpF.reset(fs);
00193     this->firstOrderLpF.reset(fs);
00194     this->bqHpF0.reset(fs);
00195     this->bqHpF1.reset(fs);
00196     this->bqLpF0.reset(fs);
00197     this->bqLpF1.reset(fs);
00198     this->update();
00199 }
00200 };

```

9.43 plugins/ntGate.h File Reference

Niose gate.

```

#include "gcem.hpp"
#include "lib/Audio.h"
#include "lib/Biquad.h"
#include "lib/DynamicFilter.h"
#include "lib/GateSc.h"
#include "lib/Plugin.h"
#include "lib/Utils.h"
#include <array>

```

Classes

- struct [ntGate](#)

Enumerations

- enum **ScMode** { **internal** , **external** , **ignore** }

Variables

- constexpr int **dlLookaheadLen** = 192 * 8 * 10

9.43.1 Detailed Description

Niose gate.

Author

Niels Thøgersen

Copyright

Copyright (C) 2026 Niels Thøgersen, NTIyd

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

9.44 ntGate.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00021
00022 #include "gcem.hpp"
00023 #include "lib/Audio.h"
00024 #include "lib/Biquad.h"
00025 #include "lib/DynamicFilter.h"
00026 #include "lib/GateSc.h"
00027 #include "lib/Plugin.h"
00028 #include "lib/utils.h"
00029 #include <array>
00030
00031 enum ScMode { internal, external, ignore };
00032
00033 constexpr int dlLookaheadLen = 192 * 8 * 10;
00034
00035 struct ntGate : public NtFx::NtPlugin {
00036     NtFx::Gate::Sc sc;
```

```

00037 NtFx::Gate::Sc scHf;
00038 NtFx::DynamicFilter::Shelf flt;
00039 NtFx::Biquad::EqBand hpf;
00040 NtFx::Biquad::EqBand lpf;
00041 std::array<Audio, dlLookaheadLen> dlLookahead;
00042 signal_t ignoreThresh_db { -20 };
00043 signal_t ignoreThresh_lin { 0.1 };
00044 signal_t tIgnore_ms { 6 };
00045 signal_t ignorePeak { 0 };
00046 signal_t tLookahead_ms { 0.2 };
00047 int ignoreCount { 0 };
00048 int nIgnore { 384 };
00049 int scMode { 0 };
00050 int nLookahead { 4 };
00051 int iLookahead { 0 };
00052 bool bypassEnable { false };
00053 bool scListenEnable { false };
00054 bool hfAccelEnable { false };
00055 bool lookaheadEnable { true };
00056
00057 ntGate() {
00058     this->primaryKnobs = {
00059         {
00060             .p_val = &this->sc.settings.thresh_db,
00061             .name = "Threshold",
00062             .suffix = " dB",
00063             .minVal = -60,
00064             .maxVal = 0,
00065         },
00066         {
00067             .p_val = &this->sc.settings.range_db,
00068             .name = "Range",
00069             .suffix = " dB",
00070             .minVal = -60,
00071             .maxVal = -0.0,
00072         },
00073         {
00074             .p_val = &this->sc.settings.tAtt_ms,
00075             .name = "Attack",
00076             .suffix = " ms",
00077             .minVal = 0.01,
00078             .maxVal = 50.0,
00079         },
00080         {
00081             .p_val = &this->sc.settings.tHold_ms,
00082             .name = "Hold",
00083             .suffix = " ms",
00084             .minVal = 0.01,
00085             .maxVal = 1000.0,
00086         },
00087         {
00088             .p_val = &this->sc.settings.tRel_ms,
00089             .name = "Release",
00090             .suffix = " ms",
00091             .minVal = 10.0,
00092             .maxVal = 1000.0,
00093         },
00094     };
00095     this->hpf.settings.fc_hz = 20;
00096     this->lpf.settings.fc_hz = 20e3;
00097     this->secondaryKnobs = {
00098         {
00099             .p_val = &this->hpf.settings.fc_hz,
00100             .name = "SC_HPF",
00101             .suffix = " Hz",
00102             .minVal = 20,
00103             .maxVal = 20e3,
00104             .logScale = true,
00105         },
00106         {
00107             .p_val = &this->lpf.settings.fc_hz,
00108             .name = "SC_LPF",
00109             .suffix = " Hz",
00110             .minVal = 20,
00111             .maxVal = 20e3,
00112             .logScale = true,
00113         },
00114         {
00115             .p_val = &this->flt.fc_hz,
00116             .name = "Xover",
00117             .suffix = " Hz",
00118             .minVal = 200,
00119             .maxVal = 20e3,
00120             .logScale = true,
00121             .isActive = false,
00122         },
00123     };

```

```

00124         .p_val      = &this->scHf.settings.tHold_ms,
00125         .name       = "HF_Hold",
00126         .suffix     = " ms",
00127         .minVal     = 0.1,
00128         .maxVal     = 1000,
00129         .isActive   = false,
00130     },
00131     {
00132         .p_val      = &this->scHf.settings.tRel_ms,
00133         .name       = "HF_Release",
00134         .suffix     = " ms",
00135         .minVal     = 0.1,
00136         .maxVal     = 1000,
00137         .isActive   = false,
00138     },
00139     {
00140         .p_val      = &this->ignoreThresh_db,
00141         .name       = "Ignore_Sens",
00142         .suffix     = " dB",
00143         .minVal     = -80,
00144         .maxVal     = 0,
00145         .isActive   = false,
00146     },
00147     {
00148         .p_val      = &this->tLookahead_ms,
00149         .name       = "Lookahead",
00150         .suffix     = " ms",
00151         .minVal     = 0,
00152         .maxVal     = 10,
00153     },
00154 };
00155 this->toggles = {
00156     { .p_val = &this->scListenEnable, .name = "SC_Listen" },
00157     { .p_val = &this->hfAccelEnable, .name = "Dual_Band" },
00158     // { .p_val = &this->lookaheadEnable, .name = "Lookahead" },
00159     { .p_val = &this->bypassEnable, .name = "Bypass" },
00160 };
00161 this->radioButtons = {
00162     {
00163         .p_val      = (int*)&this->scMode,
00164         .name       = "Side_Chain",
00165         .options    = { "Internal", "External", "Ignore" },
00166     },
00167 };
00168 this->meters = {
00169     { .name = "IN", .addRms = true },
00170     { .name = "OUT", .hasScale = true, .addRms = true },
00171     { .name = "GR", .invert = true },
00172     { .name = "HF GR", .invert = true, .hasScale = true },
00173 };
00174 this->updateDefaults();
00175 }
00176
00177 Audio process(Audio x) noexcept override {
00178     this->dlLookahead[this->iLookahead++] = x;
00179     if (this->iLookahead >= dlLookaheadLen) { this->iLookahead = 0; }
00180     this->template updatePeakLevel<0>(x);
00181     if (this->bypassEnable) {
00182         this->template updatePeakLevel<1>(x);
00183         this->template updatePeakLevel<2>(1);
00184         return x;
00185     }
00186     auto xSc = x;
00187     if (this->scMode == ScMode::external) {
00188         xSc = this->xSc;
00189     } else if (this->scMode == ScMode::ignore) {
00190         auto xAbs = gcem::abs(this->xSc);
00191         if (xAbs >= this->ignorePeak) {
00192             this->ignorePeak = xAbs;
00193             this->ignoreCount = 0;
00194         }
00195         if (this->ignoreCount++ > this->nIgnore) { this->ignorePeak = 0; }
00196         if (this->ignorePeak > this->ignoreThresh_lin
00197             && this->sc.state != NtFx::Gate::State::open) {
00198             xSc = x / signal_t(4);
00199         }
00200     }
00201     NtFx::ensureFinite(xSc);
00202     auto yHpf = this->hpf.process(xSc);
00203     auto yLpf = this->lpf.process(yHpf);
00204     if (this->scListenEnable) {
00205         this->template updatePeakLevel<1>(yLpf);
00206         return yLpf;
00207     }
00208     auto gr = this->sc.process(yLpf);
00209     auto grHf = gr;
00210     auto xGr = x;

```

```

00211     if (this->lookaheadEnable) {
00212         // TODO: delayline class.
00213         auto i = this->iLookahead - this->nLookahead;
00214         if (i < 0) { i += dlLookaheadLen; }
00215         xGr = this->dlLookahead[i];
00216     }
00217     auto y = xGr * gr;
00218     if (this->hfAccelEnable) {
00219         grHf = this->scHf.process(yLpf);
00220         auto A = grHf / gr;
00221         flt.gain_lin = A.absMax();
00222         y = this->flt.process(xGr) * gr;
00223     }
00224     this->template updatePeakLevel<1>(y);
00225     this->template updatePeakLevel<2, true>(gr);
00226     this->template updatePeakLevel<3, true>(grHf);
00227     return y;
00228 }
00229
00230 void update() noexcept override {
00231     if (this->hfAccelEnable) {
00232         this->activateParameter("Xover");
00233         this->activateParameter("HF_Hold");
00234         this->activateParameter("HF_Release");
00235     } else {
00236         this->deactivateParameter("Xover");
00237         this->deactivateParameter("HF_Hold");
00238         this->deactivateParameter("HF_Release");
00239     }
00240     if (this->scMode == ScMode::ignore) {
00241         this->activateParameter("Ignore_Sens");
00242     } else {
00243         this->deactivateParameter("Ignore_Sens");
00244     }
00245     this->lookaheadEnable = (this->tLookahead_ms > 0);
00246     this->nLookahead = this->tLookahead_ms * 0.001 * this->fs;
00247     this->nIgnore = gcem::round(this->tIgnore_ms * 0.001 * this->fs);
00248     this->ignoreThresh_lin = NtFx::invDb(this->ignoreThresh_db);
00249     this->flt.q1 = 0.6;
00250     this->flt.q2 = 0.6;
00251     this->flt.update();
00252     this->lpf.update();
00253     this->hpf.update();
00254     this->sc.update();
00255     this->scHf.settings.thresh_db = this->sc.settings.thresh_db;
00256     this->scHf.settings.range_db = this->sc.settings.range_db;
00257     this->scHf.settings.tAtt_ms = this->sc.settings.tAtt_ms;
00258     if (this->scHf.settings.tHold_ms > this->sc.settings.tHold_ms) {
00259         this->scHf.settings.tHold_ms = this->sc.settings.tHold_ms;
00260     }
00261     if (this->scHf.settings.tRel_ms > this->sc.settings.tRel_ms) {
00262         this->scHf.settings.tRel_ms = this->sc.settings.tRel_ms;
00263     }
00264     this->scHf.update();
00265 }
00266
00267 void reset(float fs) noexcept override {
00268     this->fs = fs;
00269     this->sc.reset(fs);
00270     this->scHf.reset(fs);
00271     this->hpf.settings.shape = NtFx::Biquad::Shape::hpf;
00272     this->hpf.reset(fs);
00273     this->lpf.settings.shape = NtFx::Biquad::Shape::lpf;
00274     this->lpf.reset(fs);
00275     this->flt.reset(fs);
00276     this->update();
00277 }
00278 };

```

9.45 ntMultiband3.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.

```

```

00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  *
00017  * You are free to download, build and use this code for commercial
00018  * purposes. Just don't resell it or a build of it, modified or otherwise.
00019  **/
00020
00026
00027 #pragma once
00028
00029 #include "lib/Audio.h"
00030 #include "lib/Comp.h"
00031 #include "lib/FirstOrder.h"
00032 #include "lib/Plugin.h"
00033 #include "lib/Utils.h"
00034 #include <array>
00035 #include <string>
00036
00037 enum Bands { hi, mid, lo, n };
00038
00039 struct ntMultiband3 : public NtFx::NtPlugin {
00040     signal_t xOverLo_hz { 200 };
00041     signal_t xOverHi_hz { 4000 };
00042     signal_t outputGain_db { 0 };
00043     signal_t outputGain_lin { 1 };
00044     bool linkEnable { false };
00045     bool feedbackEnable { false };
00046     bool bypassEnable { false };
00047     // bool noise { false };
00048
00049     std::array<bool, Bands::n> solos = { false, false, false };
00050     std::array<bool, Bands::n> mutesUi = { false, false, false };
00051     std::array<bool, Bands::n> mutes = { false, false, false };
00052
00053     std::array<NtFx::Comp::ScSettings, 3> scSettings;
00054     std::array<NtFx::Comp::PeakSideChainLinear, 3> sc;
00055     std::array<signal_t, Bands::n> makeup_db { 0, 0, 0 };
00056     std::array<signal_t, Bands::n> makeup_lin { 1, 1, 1 };
00057     std::array<Audio, Bands::n> fbState { 0, 0, 0 };
00058     const std::array<std::string, Bands::n> BandNames = { "High", "Mid", "Low" };
00059     NtFx::FirstOrder::StereoFilter<NtFx::FirstOrder::Shape::lpf> loFilt;
00060     NtFx::FirstOrder::StereoFilter<NtFx::FirstOrder::Shape::hpf> loMidFilt;
00061     NtFx::FirstOrder::StereoFilter<NtFx::FirstOrder::Shape::lpf> hiMidFilt;
00062     NtFx::FirstOrder::StereoFilter<NtFx::FirstOrder::Shape::hpf> hiFilt;
00063     ntMultiband3() : sc { scSettings[0], scSettings[1], scSettings[2] } {
00064         this->uiSpec.maxColumns = 5;
00065         this->uiSpec.maxRows = Bands::n;
00066
00067         for (size_t i = 0; i < Bands::n; i++) {
00068             this->primaryKnobs.push_back({
00069                 .p_val = &this->scSettings[i].thresh_db,
00070                 .name = this->BandNames[i] + "_Threshold",
00071                 .suffix = " dB",
00072                 .minVal = -60,
00073                 .maxVal = 0,
00074             });
00075             this->primaryKnobs.push_back({
00076                 .p_val = &this->scSettings[i].ratio_db,
00077                 .name = this->BandNames[i] + "_Ratio",
00078                 .suffix = "",
00079                 .minVal = 1.0,
00080                 .maxVal = 20.0,
00081                 .midPoint = 2.0,
00082             });
00083             this->primaryKnobs.push_back({
00084                 .p_val = &this->scSettings[i].tAtt_ms,
00085                 .name = this->BandNames[i] + "_Attack",
00086                 .suffix = " ms",
00087                 .minVal = 0.01,
00088                 .maxVal = 50.0,
00089                 .midPoint = 5,
00090             });
00091             this->primaryKnobs.push_back({
00092                 .p_val = &this->scSettings[i].tRel_ms,
00093                 .name = this->BandNames[i] + "_Release",
00094                 .suffix = " ms",
00095                 .minVal = 10.0,
00096                 .maxVal = 1000.0,
00097                 .midPoint = 100.0,
00098             });
00099             this->primaryKnobs.push_back({
00100                 .p_val = &this->makeup_db[i],
00101                 .name = this->BandNames[i] + "_Makeup",
00102                 .suffix = " dB",
00103                 .minVal = 0.0,

```

```

00105         .maxVal = 24.0,
00106     });
00107 }
00108 this->secondaryKnobs = {
00109     {
00110         .p_val = &this->xOverLo_hz,
00111         .name = "Lo_Xover",
00112         .suffix = " Hz",
00113         .minVal = 20,
00114         .maxVal = 2000,
00115         .midPoint = 200,
00116     },
00117     {
00118         .p_val = &this->xOverHi_hz,
00119         .name = "Hi_Xover",
00120         .suffix = " Hz",
00121         .minVal = 200,
00122         .maxVal = 20000,
00123         .midPoint = 2000,
00124     },
00125     {
00126         .p_val = &this->ouputGain_db,
00127         .name = "Out",
00128         .suffix = " dB",
00129         .minVal = -12,
00130         .maxVal = 12,
00131     },
00132 };
00133 this->toggles = {
00134     { &this->linkEnable, "Link" },
00135     { &this->feedbackEnable, "Feedback" },
00136     { &this->bypassEnable, "Bypass" },
00137     // { &this->noise, "Noise" },
00138 };
00139 this->toggleSets = {
00140     { "Solo", { } },
00141     { "Mute", { } },
00142 };
00143 for (size_t i = 0; i < Bands::n; i++) {
00144     this->toggleSets[0].toggles.push_back({
00145         .p_val = &this->solos[i],
00146         .name = BandNames[i],
00147     });
00148     this->toggleSets[1].toggles.push_back({
00149         .p_val = &this->mutesUi[i],
00150         .name = BandNames[i],
00151     });
00152 }
00153 this->meters = {
00154     { .name = "IN", .decay_s = 0.75, .addRms = true },
00155     { .name = "OUT", .hasScale = true, .decay_s = 0.75, .addRms = true },
00156 };
00157 for (int i = Bands::n - 1; i >= 0; i--) {
00158     this->meters.push_back({ .name = this->BandNames[i], .invert = true });
00159 }
00160 this->meters[Bands::n - 1 + 2].hasScale = true;
00161 this->uiSpec.meterHeight_dots = 25;
00162 for (auto& m : this->meters) { m.minVal_db = -50; }
00163 this->updateDefaults();
00164 }
00165
00166 Audio process(Audio x) noexcept override {
00167     if (this->bypassEnable) { return x; }
00168     std::array<Audio, 3> xComp;
00169     xComp[Bands::hi] = this->hiFilt.process(x);
00170     auto xLoMidFilt = this->hiMidFilt.process(x);
00171     xComp[Bands::mid] = this->loMidFilt.process(xLoMidFilt);
00172     xComp[Bands::lo] = this->loFilt.process(x);
00173     std::array<Audio, 3> xSc;
00174     if (this->feedbackEnable) {
00175         for (size_t i = 0; i < Bands::n; i++) { xSc[i] = this->fbState[i]; }
00176     } else {
00177         for (size_t i = 0; i < Bands::n; i++) { xSc[i] = xComp[i]; }
00178     }
00179     std::array<Audio, Bands::n> gr;
00180     for (size_t i = 0; i < Bands::n; i++) {
00181         gr[i] = this->sc[i].process(xSc[i]);
00182         if (this->linkEnable) { gr[i] = gr[i].absMin(); }
00183     }
00184     Audio yComp;
00185     for (size_t i = 0; i < Bands::n; i++) {
00186         auto tmp = xComp[i] * gr[i];
00187         this->fbState[i] = tmp;
00188         if (!this->mutes[i]) { yComp += tmp * this->makeup_lin[i]; }
00189     }
00190     auto y = yComp * this->ouputGain_lin;
00191     this->template updatePeakLevel<0>(x);

```

```

00192     this->template updatePeakLevel<1>(y);
00193     this->template updatePeakLevel<2, true>(gr[Bands::lo]);
00194     this->template updatePeakLevel<3, true>(gr[Bands::mid]);
00195     this->template updatePeakLevel<4, true>(gr[Bands::hi]);
00196     // if (this->noise) { return NtFx::rand() * 0.125; }
00197     return y;
00198 }
00199
00200 void update() noexcept override {
00201     this->hiFilt.setFc(this->xOverHi_hz);
00202     this->hiMidFilt.setFc(this->xOverHi_hz);
00203     this->loMidFilt.setFc(this->xOverLo_hz);
00204     this->loFilt.setFc(this->xOverLo_hz);
00205     this->hiFilt.update();
00206     this->hiMidFilt.update();
00207     this->loMidFilt.update();
00208     this->loFilt.update();
00209     this->ouputGain_lin = NtFx::invDb(this->ouputGain_db);
00210     for (size_t i = 0; i < Bands::n; i++) {
00211         this->makeup_lin[i] = NtFx::invDb(makeup_db[i]);
00212         this->sc[i].update();
00213     }
00214     this->_updateMutes();
00215 }
00216
00217 void reset(float fs) noexcept override {
00218     this->fs = fs;
00219     this->hiFilt.reset(this->fs);
00220     this->hiMidFilt.reset(this->fs);
00221     this->loMidFilt.reset(this->fs);
00222     this->loFilt.reset(this->fs);
00223     for (size_t i = 0; i < Bands::n; i++) { this->sc[i].reset(this->fs); }
00224     for (size_t i = 0; i < Bands::n; i++) { this->makeup_db[i] = 0; }
00225     for (size_t i = 0; i < Bands::n; i++) { this->makeup_lin[i] = 1; }
00226     for (size_t i = 0; i < Bands::n; i++) { this->fbState[i] = 0; }
00227     this->update();
00228 }
00229
00230 void _updateMutes() {
00231     for (size_t i = 0; i < Bands::n; i++) { this->mutes[i] = this->mutesUi[i]; }
00232     for (size_t i = 0; i < Bands::n; i++) {
00233         if (!this->solos[i]) { continue; }
00234         for (size_t j = 0; j < Bands::n; j++) {
00235             if (i != j && !this->solos[j]) { this->mutes[j] = true; }
00236         }
00237     }
00238 }
00239 };

```

9.46 ntRmsMeter.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTlyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  */
00017
00018 #pragma once
00019
00020 #include "lib/Audio.h"
00021 #include "lib/Plugin.h"
00022 #include "lib/RmsSensor.h"
00023
00024 struct ntRmsMeter : public NtFx::NtPlugin {
00025     enum Meter {
00026         Peak,
00027         RMS,
00028     };
00029
00030     NtFx::LongRmsSensor<> msSensor;
00031     signal_t decay_s = 0.1;
00032     signal_t hold_s = 2;

```

```

00033     signal_t tRms_ms = 10;
00034
00035     ntRmsMeter() {
00036         this->meters = {
00037             { .name = "Peak", .minVal_db = -50, .addRms = true },
00038             { .name = "RMS", .minVal_db = -50, .hasScale = true },
00039         };
00040         this->primaryKnobs = {
00041             { &this->decay_s, "Decay", " s", 0, 1 },
00042             { &this->hold_s, "Hold", " s", 0, 10 },
00043             { &this->tRms_ms, "RMS_Time", " ms", 1, 1000 },
00044         };
00045         this->uiSpec.meterHeight_dots = 25;
00046
00047         this->uiSpec.maxColumns = 1;
00048         this->updateDefaults();
00049     }
00050
00051     virtual Audio process(Audio x) noexcept override {
00052         this->template updatePeakLevel<Peak>(x);
00053         this->template updatePeakLevel<RMS>(msSensor.process(x));
00054         return x;
00055     }
00056
00057     virtual void update() noexcept override {
00058         for (auto& m : this->meters) { m.decay_s = this->decay_s; }
00059         for (auto& m : this->meters) { m.hold_s = this->hold_s; }
00060         this->msSensor.setT_ms(this->tRms_ms);
00061         this->uiNeedsUpdate = true;
00062     }
00063
00064     virtual void reset(float fs) noexcept override {
00065         this->fs = fs;
00066         this->msSensor.reset(fs);
00067         this->update();
00068     }
00069 };

```

9.47 ntTapeEcho.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTllyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  *
00017  * You are free to download, build and use this code for commercial
00018  * purposes. Just don't resell it or a build of it, modified or otherwise.
00019  */
00020
00021 #pragma once
00022
00023 #include "gcm.h"
00024 #include "lib/Audio.h"
00025 #include "lib/Biquad.h"
00026 #include "lib/Glider.h"
00027 #include "lib/Plugin.h"
00028 #include "lib/SoftClip.h"
00029 #include "lib/Utils.h"
00030 #include <algorithm>
00031 #include <array>
00032 #include <stddef>
00033
00034 enum SubDev : int {
00035     half,
00036     fourth,
00037     eighth_dot,
00038     eighth,
00039     sixteenth_dot,
00040     sixteenth,
00041 };
00042 // 192 kHz * 2 seconds * 8 x oversampling * 2 stores pr sample * 2 for mod (the
00043 // last doesn't need to be that big).

```

```

00044 // TODO: Calculate space for mod.
00045 // TODO: Dynamic allocation of delayline.
00046 constexpr int delayLineLength = 192e3 * 2 * 8 * 2 * 2;
00047
00048 struct ntTapeEcho : public NtFx::NtPlugin {
00049     signal_t tGui          = 0.5;
00050     signal_t fb_percent    = 20;
00051     signal_t modFreq       = 1.0;
00052     signal_t modPhase      = 0.0;
00053     signal_t clipG_db      = 0.0;
00054     signal_t mix_percent   = 100.0;
00055     signal_t tOffset       = 0.0;
00056     signal_t modDepth_percent = 0.1;
00057     signal_t noise_db      = -100;
00058     bool sync              = false;
00059     bool mod                = true;
00060     bool clip               = true;
00061     bool doGlide            = true;
00062     bool bypass             = false;
00063     SubDev subDev           = SubDev::fourth;
00064     signal_t tempoScale     = 1;
00065     NtFx::Biquad::EqBand hpf;
00066     NtFx::Biquad::EqBand lpf;
00067     Audio fbState;
00068     std::array<Audio, delayLineLength> delayLine;
00069     signal_t fb_lin        = 0.2;
00070     signal_t noise_lin     = 0;
00071     signal_t tGlide        = 0.36;
00072     // NtFx::LinGlider nDelay;
00073     NtFx::ExpGlider nDelay;
00074     size_t iStore          = 0;
00075     signal_t aClip_lin     = 1;
00076     size_t timeCounter     = 0;
00077     NtFx::ExpGlider nOffset;
00078     // TODO: Mix should have -3 dB law.
00079     signal_t mix_lin       = 1;
00080     NtFx::ExpGlider modDepth;
00081     // TODO: How in the world do we get mod to glide gracefully?
00082     NtFx::ExpGlider thetaMod;
00083     NtFx::ExpGlider thetaModPhase;
00084
00085     ntTapeEcho() : modDepth(0.1) {
00086         this->primaryKnobs = {
00087             { &this->tGui, "Time", " s", 0.02, 2 },
00088             { &this->fb_percent, "Feedback", " %", 0, 200 },
00089             { &this->clipG_db, "Drive", " dB", -20, 20 },
00090             { &this->hpf.settings.fc_hz, "HPF", " Hz", 20, 2000, true },
00091             { &this->lpf.settings.fc_hz, "LPF", " Hz", 200, 20000, true },
00092         };
00093         this->secondaryKnobs = {
00094             { &this->hpf.settings.q, "Q_HP", "", 0.5, 2, true },
00095             { &this->lpf.settings.q, "Q_LP", "", 0.5, 2, true },
00096             { &this->modFreq, "Mod_Freq", " Hz", 0.1, 10, true },
00097             { &this->modDepth_percent, "Mod_Depth", " %", 0.01, 1, true },
00098             { &this->modPhase, "Mod_Phase", " deg", 0, 180 },
00099             { &this->tOffset, "Offset", " ms", 0, 50 },
00100             { &this->noise_db, "Noise", " dB", -100, 0, true },
00101             { &this->mix_percent, "Mix", " %", 0, 100 },
00102             { &this->tGlide, "Glide_Time", " s", 0.0, 1 },
00103         };
00104
00105         this->dropdowns = {
00106             {
00107                 .p_val = (int*)&this->subDev,
00108                 .name = "Subdivision",
00109                 .options = {
00110                     "half",
00111                     "fourth",
00112                     "eighth_dot",
00113                     "eighth",
00114                     "sixteenth_dot",
00115                     "sixteenth",
00116                 },
00117                 ._defaultVal = 0,
00118             },
00119         };
00120         this->toggles = {
00121             { &this->sync, "Sync" },
00122             { &this->mod, "Mod" },
00123             { &this->clip, "Softclip" },
00124             { &this->doGlide, "Glide" },
00125             { &this->bypass, "Bypass" },
00126         };
00127         this->meters = { { "IN" }, { .name = "OUT", .hasScale = true } };
00128         this->lpf.settings.shape = NtFx::Biquad::Shape::lpf;
00129         this->hpf.settings.shape = NtFx::Biquad::Shape::hpf;
00130         this->lpf.settings.fc_hz = 20e3;

```

```

00131     this->hpf.settings.fc_hz = 20;
00132     this->updateDefaults();
00133 }
00134
00135 virtual Audio process(Audio x) noexcept override {
00136     this->nDelay.process();
00137     this->modDepth.process();
00138     this->thetaMod.process();
00139     this->thetaModPhase.process();
00140     this->nOffset.process();
00141     auto xNoisy = x + NtFx::rand<signal_t>() * this->noise_lin;
00142     NtFx::ensureFinite(xNoisy);
00143     NtFx::ensureFinite(this->fbState);
00144     this->iStore++;
00145     if (this->iStore >= delayLineLength) { this->iStore = 0; }
00146     this->delayLine[this->iStore] = xNoisy + this->fb_lin * this->fbState;
00147
00148     int nModL = this->nDelay.pr;
00149     int nModR = this->nDelay.pr + this->nOffset.pr;
00150     if (this->mod) {
00151         auto modSawL = NtFx::saw(this->thetaMod.pr * this->timeCounter);
00152         auto modSawR = NtFx::saw(
00153             this->thetaMod.pr * this->timeCounter + this->thetaModPhase.pr);
00154         nModL = gcem::round(modSawL * this->nDelay.pr * this->modDepth.pr)
00155             + this->nDelay.pr;
00156         nModR = gcem::round(modSawR * this->nDelay.pr * this->modDepth.pr)
00157             + this->nDelay.pr + this->nOffset.pr;
00158         this->timeCounter++;
00159     }
00160
00161     // TODO: DelayLine class.
00162     int iLoadL = this->iStore - gcem::round(nModL);
00163     if (iLoadL < 0) { iLoadL += delayLineLength; }
00164     int iLoadR = this->iStore - gcem::round(nModR);
00165     if (iLoadR < 0) { iLoadR += delayLineLength; }
00166     Audio yDelay = {
00167         this->delayLine[iLoadL].l,
00168         this->delayLine[iLoadR].r,
00169     };
00170     auto yFbClip =
00171         NtFx::softClip3rdStereo(yDelay * this->aClip_lin) / aClip_lin;
00172     NtFx::ensureFinite(yFbClip);
00173     auto yHp = hpf.process(yFbClip);
00174     auto yLp = lpf.process(yHp);
00175     this->fbState = yLp;
00176     auto yOutClip = yLp;
00177     if (this->clip) { yOutClip = NtFx::softClip5thStereo(yLp); }
00178     auto y = (signal_t(1.0) - this->mix_lin) * x + this->mix_lin * yOutClip;
00179     // TODO: Make this a member.
00180     y *= (2 - gcem::abs(this->mix_lin * 2 - 1));
00181     this->template updatePeakLevel<0>(x);
00182     if (this->bypass) {
00183         this->template updatePeakLevel<1>(x);
00184         return x;
00185     }
00186     this->template updatePeakLevel<1>(y);
00187     return y;
00188 }
00189
00190 virtual void update() noexcept override {
00191     this->hpf.update();
00192     this->lpf.update();
00193     this->nOffset.ui = gcem::round(this->tOffset / 1000 * this->fs);
00194     this->aClip_lin = NtFx::invDb(this->clipG_db);
00195     this->mix_lin = this->mix_percent / 100;
00196     this->fb_lin = this->fb_percent / 100;
00197     this->noise_lin = NtFx::invDb(this->noise_db);
00198     this->modDepth.ui = this->modDepth_percent / 100;
00199     switch (this->subDev) {
00200     case SubDev::half:
00201         this->tempoScale = 2;
00202         break;
00203     case SubDev::fourth:
00204         this->tempoScale = 1;
00205         break;
00206     case SubDev::eighth_dot:
00207         this->tempoScale = 0.5 * 1.5;
00208         break;
00209     case SubDev::eighth:
00210         this->tempoScale = 0.5;
00211         break;
00212     case SubDev::sixteenth_dot:
00213         this->tempoScale = 0.25 * 1.5;
00214         break;
00215     case SubDev::sixteenth:
00216         this->tempoScale = 0.25;
00217         break;

```

```

00218     }
00219     this->thetaMod.ui      = 2.0 * GCEM_PI * this->modFreq / this->fs;
00220     this->thetaModPhase.ui = this->modPhase * GCEM_PI / 180;
00221     this->onTempoChanged();
00222     signal_t tGlide = 0.0;
00223     if (this->doGlide) { tGlide = this->tGlide; }
00224     this->modDepth.update(this->fs, tGlide);
00225     // TODO: fix mod glider.
00226     // this->thetaMod.update(this->fs, tGlide);
00227     this->thetaMod.update(this->fs, 0);
00228     this->thetaModPhase.update(this->fs, tGlide);
00229     this->nOffset.update(this->fs, tGlide);
00230     this->nDelay.update(this->fs, tGlide);
00231 }
00232
00233 virtual void reset(float fs) noexcept override {
00234     this->fs = fs;
00235     std::fill(this->delayLine.begin(), this->delayLine.end(), 0);
00236     this->hpf.reset(this->fs);
00237     this->lpf.reset(this->fs);
00238     this->update();
00239 }
00240
00241 virtual void onTempoChanged() noexcept override {
00242     if (this->sync && this->tempo) {
00243         this->tGui      = 60 / this->tempo * this->tempoScale;
00244         this->primaryKnobs[0].isActive = false;
00245     } else {
00246         this->primaryKnobs[0].isActive = true;
00247     }
00248     this->uiNeedsUpdate = true;
00249     this->nDelay.ui      = gcem::round(this->tGui * this->fs);
00250 }
00251 };

```

9.48 ntTransformer.h

```

00001 /*
00002  * Copyright (C) 2026 Niels Thøgersen, NTllyd
00003  *
00004  * This program is free software: you can redistribute it and/or modify it under
00005  * the terms of the GNU Affero General Public License as published by the Free
00006  * Software Foundation, either version 3 of the License, or (at your option) any
00007  * later version.
00008  *
00009  * This program is distributed in the hope that it will be useful, but WITHOUT
00010  * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00011  * FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more
00012  * details.
00013  *
00014  * You should have received a copy of the GNU Affero General Public License
00015  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00016  *
00017  * You are free to download, build and use this code for commercial
00018  * purposes. Just don't resell it or a build of it, modified or otherwise.
00019  */
00020
00021 #pragma once
00022
00023 #include "lib/Audio.h"
00024 #include "lib/Biquad.h"
00025 #include "lib/Plugin.h"
00026 #include "lib/Transformer.h"
00027 #include "lib/Utils.h"
00028
00029 struct ntTransformer : public NtFx::NtPlugin {
00030     NtFx::Biquad::EqBand bqHpf0;
00031     NtFx::Transformer transformer;
00032     signal_t drive_db = -10;
00033     signal_t drive_lin = 0.3;
00034     bool bypass = false;
00035     ntTransformer() {
00036         this->primaryKnobs = {
00037             { &this->drive_db, "Drive", " dB", -24, 24 },
00038         };
00039         this->toggles = { { &this->bypass, "Bypass" } };
00040     }
00041     virtual Audio process(Audio x) noexcept override {
00042         this->template updatePeakLevel<0>(x);
00043         if (this->bypass) {
00044             this->template updatePeakLevel<1>(x);
00045             return x;
00046         }
00047     }

```

```
00047     auto xTrans = this->bqHpf0.process(x);
00048     auto y =
00049         this->transformer.process(xTrans * this->drive_lin) / this->drive_lin;
00050     this->template updatePeakLevel<1>(y);
00051     return y;
00052 }
00053 virtual void update() noexcept override {
00054     this->drive_lin = NtFx::invDb(this->drive_db);
00055     this->transformer.update();
00056     this->bqHpf0.update();
00057 }
00058 virtual void reset(float fs) noexcept override {
00059     this->fs = fs;
00060     this->transformer.reset(fs);
00061     this->bqHpf0.settings.fc_hz = 40;
00062     this->bqHpf0.settings.q = 1.1;
00063     this->bqHpf0.settings.shape = NtFx::Biquad::Shape::hpf;
00064     this->bqHpf0.reset(fs);
00065     this->update();
00066 }
00067 };
```


Index

- [_NTFX_ADD_TEST_IMPL](#)
 - [ComponentTest.h, 192](#)
 - [_NTFX_EXPAND_AND_QUOTE](#)
 - [ComponentTest.h, 192](#)
 - [_NTFX_QUOTE](#)
 - [ComponentTest.h, 192](#)
 - [_NTFX_TEST_BEGIN_IMPL](#)
 - [ComponentTest.h, 193](#)
 - [_NTFX_TEST_IMPL](#)
 - [ComponentTest.h, 193](#)
 - [_calculateSoftClipCoeffs](#)
 - [NtFx, 35](#)
 - [_coeffsFifth](#)
 - [NtFx, 35](#)
 - [_coeffsSeventh](#)
 - [NtFx, 35](#)
 - [_coeffsThird](#)
 - [NtFx, 35](#)
 - [_gainComputer_db](#)
 - [NtFx::Comp::PeakSideChainDb, 129](#)
 - [_gainComputer_lin](#)
 - [NtFx::Comp::PeakSideChainLinear, 132](#)
 - [_peakSensor](#)
 - [NtFx::PeakSensor, 124](#)
- [abs](#)
 - [NtFx::Mono< signal_t >, 84](#)
 - [NtFx::Stereo< signal_t >, 150](#)
- [absMax](#)
 - [NtFx::Mono< signal_t >, 84](#)
 - [NtFx::Stereo< signal_t >, 150](#)
- [absMin](#)
 - [NtFx::Mono< signal_t >, 85](#)
 - [NtFx::Stereo< signal_t >, 150](#)
- [acceptLatestResult](#)
 - [test, 47](#)
- [addNewPluginId](#)
 - [ntPlugin, 37](#)
- [addRms](#)
 - [NtFx::MeterSpec, 83](#)
- [addTest](#)
 - [NtFx::ComponentTestSet, 59](#)
- [avgSquared](#)
 - [NtFx::Mono< signal_t >, 85](#)
 - [NtFx::Stereo< signal_t >, 151](#)
- [build](#)
 - [ntPlugin, 37](#)
- [CATEGORY_MAP](#)
 - [ntPlugin, 40](#)
- [clean](#)
 - [test, 47](#)
- [ComponentTest](#)
 - [NtFx::ComponentTest, 57](#)
- [ComponentTest.h](#)
 - [_NTFX_ADD_TEST_IMPL, 192](#)
 - [_NTFX_EXPAND_AND_QUOTE, 192](#)
 - [_NTFX_QUOTE, 192](#)
 - [_NTFX_TEST_BEGIN_IMPL, 193](#)
 - [_NTFX_TEST_IMPL, 193](#)
 - [NTFX_ADD_TEST, 193](#)
 - [NTFX_RUN_TESTS, 193](#)
 - [NTFX_TEST, 193](#)
- [ComponentTestSet](#)
 - [NtFx::ComponentTestSet, 59](#)
- [configure](#)
 - [ntPlugin, 37](#)
- [createParser](#)
 - [ntPlugin, 38](#)
 - [package, 41](#)
 - [test, 47](#)
- [dropdowns](#)
 - [NtFx::TitleBarSpec, 157](#)
- [ExpGlider](#)
 - [NtFx::ExpGlider, 66](#)
- [gain_lin](#)
 - [NtFx::DynamicFilter::Shelf, 146](#)
- [gainExample, 69](#)
 - [process, 71](#)
 - [reset, 71](#)
 - [update, 71](#)
- [generateBurstSine](#)
 - [test, 47](#)
- [generateImpulse](#)
 - [test, 48](#)
- [generateLinearSweep](#)
 - [test, 48](#)
- [generateSyncSweep](#)
 - [test, 48](#)
- [generateTestVectors](#)
 - [test, 49](#)
- [getAndResetPeakLevel](#)
 - [NtFx::NtPlugin, 105](#)
- [getDropDownValuePtr](#)
 - [NtFx::NtPlugin, 105](#)
- [getKnobValuePtr](#)

- NtFx::NtPlugin, 106
- getResults
 - NtFx::ComponentTestSet, 59
- getRms
 - NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >, 77
 - NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >, 80
 - NtFx::NtPlugin, 106
- getToggleValuePtr
 - NtFx::NtPlugin, 106
- JuceWrapper/LookAndFeel.h, 165
- JuceWrapper/Meter.h, 166
- JuceWrapper/PluginEditor.h, 170
- JuceWrapper/PluginProcessor.h, 172
- JuceWrapper/RadioButton.h, 174
- JuceWrapper/Toggle.h, 175
- lib/Audio.h, 176
- lib/Biquad.h, 181
- lib/Comp.h, 186
- lib/Component.h, 188, 190
- lib/ComponentTest.h, 190, 194
- lib/DynamicFilter.h, 196, 198
- lib/FirstOrder.h, 198, 199
- lib/GateSc.h, 200, 201
- lib/Generator.h, 202
- lib/Glider.h, 203, 204
- lib/PeakSensor.h, 204, 206
- lib/Plugin.h, 207, 208
- lib/RmsSensor.h, 210, 211
- lib/SampleRateConverter.h, 212, 214
- lib/SoftClip.h, 216, 217
- lib/Transformer.h, 217
- lib/UiSpec.h, 218, 220
- lib/Utils.h, 221
- lib/windowFunctions.h, 222, 223
- LinGlider
 - NtFx::LinGlider, 74
- main
 - ntPlugin, 38
 - package, 41
 - test, 49
- makeDistributionXml
 - package, 42
- makeInstallerPkg
 - package, 42
- makePluginPkg
 - package, 42
- makePluginPkgs
 - package, 42
- meters
 - NtFx::NtPlugin, 107
- name
 - NtFx::KnobSpec, 73
 - NtFx::ToggleSpec, 160
- newPlugin
 - ntPlugin, 38
- newPluginTest
 - ntPlugin, 38
- nMetersMax
 - NtFx, 36
- notarize
 - package, 43
- notarizePlugins
 - package, 43
- ntCompressor, 88
 - process, 90
 - reset, 90
 - update, 91
- ntDeEsser, 91
 - process, 93
 - reset, 93
 - update, 94
- ntFilters, 94
 - process, 96
 - reset, 96
 - update, 97
- NtFx, 33
 - _calculateSoftClipCoeffs, 35
 - _coeffsFifth, 35
 - _coeffsSeventh, 35
 - _coeffsThird, 35
 - nMetersMax, 36
- NtFx::Biquad::Biquad6, 53
- NtFx::Biquad::Coeffs5, 54
- NtFx::Biquad::Coeffs6, 54
- NtFx::Biquad::EqBand6Mono, 60
 - process, 60
 - reset, 61
 - update, 61
- NtFx::Biquad::EqBand6Stereo, 61
 - process, 62
 - reset, 62
 - update, 63
- NtFx::Biquad::EqBandMono, 63
 - process, 63
 - reset, 64
 - update, 64
- NtFx::Biquad::EqBandStereo, 64
 - process, 65
 - reset, 65
 - update, 66
- NtFx::Biquad::Settings, 144
- NtFx::Biquad::State, 148
- NtFx::Biquad::StereoState, 156
- NtFx::Comp::PeakSideChainDb, 127
 - _gainComputer_db, 129
 - PeakSideChainDb, 128
 - process, 129
 - reset, 129
 - update, 130
- NtFx::Comp::PeakSideChainLinear, 130
 - _gainComputer_lin, 132

- PeakSideChainLinear, 131
- process, 132
- update, 132
- NtFx::Comp::RmsSideChainDb, 134
 - process, 135
 - reset, 136
 - update, 136
- NtFx::Comp::RmsSideChainLinear, 136
 - process, 138
 - reset, 138
 - RmsSideChainLinear, 138
 - update, 139
- NtFx::Comp::ScSettings, 142
- NtFx::ComponentBase< T >, 54
 - process, 55
 - reset, 55
 - update, 56
- NtFx::ComponentTest, 56
 - ComponentTest, 57
 - run, 57
- NtFx::ComponentTestSet, 58
 - addTest, 59
 - ComponentTestSet, 59
 - getResults, 59
 - runAllTests, 59
- NtFx::DynamicFilter::Shelf, 144
 - gain_lin, 146
 - process, 145
 - q2, 146
 - reset, 145
 - update, 145
- NtFx::ExpGlider, 66
 - ExpGlider, 66
 - process, 67
 - update, 67
- NtFx::FirstOrder::Filter< shape >, 67
 - process, 68
 - reset, 68
 - update, 69
- NtFx::FirstOrder::StereoFilter< shape >, 153
 - process, 154
 - reset, 154
 - update, 155
- NtFx::Gate::Sc, 141
 - process, 141
 - reset, 142
 - update, 142
- NtFx::Gate::ScSettings, 143
- NtFx::Generator::Noise, 87
 - process, 88
- NtFx::KnobLookAndFeel, 72
- NtFx::KnobSpec, 72
 - name, 73
 - setLogScale, 73
- NtFx::LinGlider, 74
 - LinGlider, 74
 - process, 75
 - update, 75
- NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >, 75
 - getRms, 77
 - process, 77
 - processDelayLine, 77
 - reset, 78
 - setT_ms, 78
 - update, 78
- NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >, 79
 - getRms, 80
 - setT_ms, 80
- NtFx::Meter, 80
- NtFx::MeterGroup, 81
- NtFx::MeterScale, 82
- NtFx::MeterSpec, 83
 - addRms, 83
- NtFx::Mono< signal_t >, 84
 - abs, 84
 - absMax, 84
 - absMin, 85
 - avgSquared, 85
- NtFx::MonoComponent< signal_t, component_t >, 85
 - process, 86
 - reset, 86
 - update, 87
- NtFx::NtPlugin, 103
 - getAndResetPeakLevel, 105
 - getDropDownValuePtr, 105
 - getKnobValuePtr, 106
 - getRms, 106
 - getToggleValuePtr, 106
 - meters, 107
 - onTempoChanged, 106
 - updatePeakLevel, 106
- NtFx::OptionsSpec, 119
- NtFx::PeakHoldSensor< delayLineLength >, 119
 - process, 121
 - update, 121
- NtFx::PeakHoldSensorStereo< delayLineLength >, 121
 - setT_ms, 122
 - setTHold_ms, 122
- NtFx::PeakSensor, 123
 - _peakSensor, 124
 - process, 124
 - reset, 125
 - tPeak_ms, 125
 - update, 125
- NtFx::PeakSensorStereo, 126
 - setT_ms, 127
- NtFx::RadioButtonSet, 133
 - updateToggleStates, 134
- NtFx::SoftClip3, 146
- NtFx::SoftClip5, 147
- NtFx::Src::Coeffs, 53
- NtFx::Src::SampleRateConverter, 139
 - process, 140
 - reset, 140

- SampleRateConverter, 140
- NtFx::Src::State, 148
- NtFx::Stereo< signal_t >, 149
 - abs, 150
 - absMax, 150
 - absMin, 150
 - avgSquared, 151
- NtFx::StereoComponent< signal_t, component_t >, 151
 - process, 152
 - reset, 152
 - update, 153
- NtFx::StereoMeter, 155
- NtFx::TitleBarLookAndFeel, 156
- NtFx::TitleBarSpec, 156
 - dropdowns, 157
- NtFx::Toggle, 157
- NtFx::ToggleSet, 158
- NtFx::ToggleSetBase, 159
- NtFx::ToggleSetSpec, 159
- NtFx::ToggleSpec, 159
 - name, 160
- NtFx::Transformer, 160
 - process, 161
 - reset, 161
 - update, 161
- NtFx::UiSpec, 162
- NTFX_ADD_TEST
 - ComponentTest.h, 193
- NTFX_RUN_TESTS
 - ComponentTest.h, 193
- NTFX_TEST
 - ComponentTest.h, 193
- ntGate, 97
 - process, 99
 - reset, 99
 - update, 100
- ntMultiband3, 100
 - process, 102
 - reset, 102
 - update, 103
- NtPlugin, 1
- ntPlugin, 36
 - addNewPluginId, 37
 - build, 37
 - CATEGORY_MAP, 40
 - configure, 37
 - createParser, 38
 - main, 38
 - newPlugin, 38
 - newPluginTest, 38
 - process, 39
 - readPluginIds, 39
 - runAuVal, 39
 - runCTest, 39
 - storeArtifacts, 39
- NtPlugin License, 11
- NtPluginAudioProcessor, 107
- NtPluginAudioProcessorEditor, 108
- ntRmsMeter, 110
 - process, 112
 - reset, 112
 - update, 112
- ntTapeEcho, 113
 - onTempoChanged, 115
 - process, 115
 - reset, 115
 - update, 116
- ntTransformer, 116
 - process, 118
 - reset, 118
 - update, 118
- onTempoChanged
 - NtFx::NtPlugin, 106
 - ntTapeEcho, 115
- package, 40
 - createParser, 41
 - main, 41
 - makeDistributionXml, 42
 - makeInstallerPkg, 42
 - makePluginPkg, 42
 - makePluginPkgs, 42
 - notarize, 43
 - notarizePlugins, 43
 - readPlugins, 43
 - SECRETS, 45
 - sign, 44
 - staple, 44
 - staplePlugins, 44
 - TARGET_EXT_MAP, 45
 - validateInstaller, 44
 - zipPackage, 45
- PeakSideChainDb
 - NtFx::Comp::PeakSideChainDb, 128
- PeakSideChainLinear
 - NtFx::Comp::PeakSideChainLinear, 131
- plotDynamic
 - test, 49
- plotFrequencyDomain
 - test, 50
- plotImpulse
 - test, 50
- plotSpectrum
 - test, 50
- plotSweeps
 - test, 51
- plotTimeDomain
 - test, 51
- plugins/gainExample.h, 229
- plugins/ntCompressor.h, 231
- plugins/ntDeEsser.h, 234
- plugins/ntFilters.h, 235
- plugins/ntGate.h, 237, 238
- plugins/ntMultiband3.h, 241
- plugins/ntRmsMeter.h, 244

- plugins/ntTapeEcho.h, 245
- plugins/ntTransformer.h, 248
- process
 - gainExample, 71
 - ntCompressor, 90
 - ntDeEsser, 93
 - ntFilters, 96
 - NtFx::Biquad::EqBand6Mono, 60
 - NtFx::Biquad::EqBand6Stereo, 62
 - NtFx::Biquad::EqBandMono, 63
 - NtFx::Biquad::EqBandStereo, 65
 - NtFx::Comp::PeakSideChainDb, 129
 - NtFx::Comp::PeakSideChainLinear, 132
 - NtFx::Comp::RmsSideChainDb, 135
 - NtFx::Comp::RmsSideChainLinear, 138
 - NtFx::ComponentBase< T >, 55
 - NtFx::DynamicFilter::Shelf, 145
 - NtFx::ExpGlider, 67
 - NtFx::FirstOrder::Filter< shape >, 68
 - NtFx::FirstOrder::StereoFilter< shape >, 154
 - NtFx::Gate::Sc, 141
 - NtFx::Generator::Noise, 88
 - NtFx::LinGlider, 75
 - NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >, 77
 - NtFx::MonoComponent< signal_t, component_t >, 86
 - NtFx::PeakHoldSensor< delayLineLength >, 121
 - NtFx::PeakSensor, 124
 - NtFx::Src::SampleRateConverter, 140
 - NtFx::StereoComponent< signal_t, component_t >, 152
 - NtFx::Transformer, 161
 - ntGate, 99
 - ntMultiband3, 102
 - ntPlugin, 39
 - ntRmsMeter, 112
 - ntTapeEcho, 115
 - ntTransformer, 118
- processDelayLine
 - NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >, 77
- q2
 - NtFx::DynamicFilter::Shelf, 146
- readPluginIds
 - ntPlugin, 39
- readPlugins
 - package, 43
- reset
 - gainExample, 71
 - ntCompressor, 90
 - ntDeEsser, 93
 - ntFilters, 96
 - NtFx::Biquad::EqBand6Mono, 61
 - NtFx::Biquad::EqBand6Stereo, 62
 - NtFx::Biquad::EqBandMono, 64
 - NtFx::Biquad::EqBandStereo, 65
 - NtFx::Comp::PeakSideChainDb, 129
 - NtFx::Comp::RmsSideChainDb, 136
 - NtFx::Comp::RmsSideChainLinear, 138
 - NtFx::ComponentBase< T >, 55
 - NtFx::DynamicFilter::Shelf, 145
 - NtFx::ExpGlider, 67
 - NtFx::FirstOrder::Filter< shape >, 68
 - NtFx::FirstOrder::StereoFilter< shape >, 154
 - NtFx::Gate::Sc, 142
 - NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >, 78
 - NtFx::MonoComponent< signal_t, component_t >, 86
 - NtFx::PeakSensor, 125
 - NtFx::Src::SampleRateConverter, 140
 - NtFx::StereoComponent< signal_t, component_t >, 152
 - NtFx::Transformer, 161
 - ntGate, 99
 - ntMultiband3, 102
 - ntRmsMeter, 112
 - ntTapeEcho, 115
 - ntTransformer, 118
 - RmsSideChainLinear
 - NtFx::Comp::RmsSideChainLinear, 138
 - run
 - NtFx::ComponentTest, 57
 - test, 51
 - runAllTests
 - NtFx::ComponentTestSet, 59
 - runAuVal
 - ntPlugin, 39
 - runCtest
 - ntPlugin, 39
 - runTests
 - test, 52
 - SampleRateConverter
 - NtFx::Src::SampleRateConverter, 140
 - SECRETS
 - package, 45
 - setLogScale
 - NtFx::KnobSpec, 73
 - setT_ms
 - NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >, 78
 - NtFx::LongRmsSensorStereo< maxT_ms, maxSampleDLineLen >, 80
 - NtFx::PeakHoldSensorStereo< delayLineLength >, 122
 - NtFx::PeakSensorStereo, 127
 - setTHold_ms
 - NtFx::PeakHoldSensorStereo< delayLineLength >, 122
 - sign
 - package, 44
 - staple
 - package, 44
 - staplePlugins
 - package, 44

- STIMULI
 - test, [52](#)
- storeArtifacts
 - ntPlugin, [39](#)
- TARGET_EXT_MAP
 - package, [45](#)
- test, [46](#)
 - acceptLatestResult, [47](#)
 - clean, [47](#)
 - createParser, [47](#)
 - generateBurstSine, [47](#)
 - generateImpulse, [48](#)
 - generateLinearSweep, [48](#)
 - generateSyncSweep, [48](#)
 - generateTestVectors, [49](#)
 - main, [49](#)
 - plotDynamic, [49](#)
 - plotFrequencyDomain, [50](#)
 - plotImpulse, [50](#)
 - plotSpectrum, [50](#)
 - plotSweeps, [51](#)
 - plotTimeDomain, [51](#)
 - run, [51](#)
 - runTests, [52](#)
 - STIMULI, [52](#)
- tPeak_ms
 - NtFx::PeakSensor, [125](#)
- update
 - gainExample, [71](#)
 - ntCompressor, [91](#)
 - ntDeEsser, [94](#)
 - ntFilters, [97](#)
 - NtFx::Biquad::EqBand6Mono, [61](#)
 - NtFx::Biquad::EqBand6Stereo, [63](#)
 - NtFx::Biquad::EqBandMono, [64](#)
 - NtFx::Biquad::EqBandStereo, [66](#)
 - NtFx::Comp::PeakSideChainDb, [130](#)
 - NtFx::Comp::PeakSideChainLinear, [132](#)
 - NtFx::Comp::RmsSideChainDb, [136](#)
 - NtFx::Comp::RmsSideChainLinear, [139](#)
 - NtFx::ComponentBase< T >, [56](#)
 - NtFx::DynamicFilter::Shelf, [145](#)
 - NtFx::ExpGlider, [67](#)
 - NtFx::FirstOrder::Filter< shape >, [69](#)
 - NtFx::FirstOrder::StereoFilter< shape >, [155](#)
 - NtFx::Gate::Sc, [142](#)
 - NtFx::LinGlider, [75](#)
 - NtFx::LongRmsSensorMono< signal_t, maxT_ms, maxSampleDLineLen >, [78](#)
 - NtFx::MonoComponent< signal_t, component_t >, [87](#)
 - NtFx::PeakHoldSensor< delayLineLength >, [121](#)
 - NtFx::PeakSensor, [125](#)
 - NtFx::StereoComponent< signal_t, component_t >, [153](#)
 - NtFx::Transformer, [161](#)
 - ntGate, [100](#)
 - ntMultiband3, [103](#)
 - ntRmsMeter, [112](#)
 - ntTapeEcho, [116](#)
 - ntTransformer, [118](#)
 - updatePeakLevel
 - NtFx::NtPlugin, [106](#)
 - updateToggleStates
 - NtFx::RadioButtonSet, [134](#)
 - validateInstaller
 - package, [44](#)
 - zipPackage
 - package, [45](#)